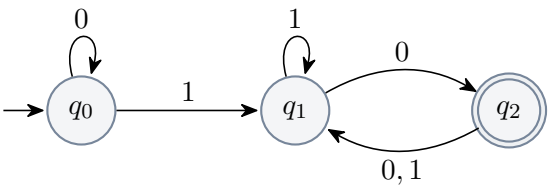
جزوه اتوماتا

نظریه زبان‌ها و ماشین‌ها

Theory of Formal Languages and Automata



بر اساس اسلایدهای درس دکتر مهدی دولتی

دانشکده مهندسی کامپیوتر، دانشگاه صنعتی شریف

از زبان‌های منظم تا ماشین تورینگ، تصمیم‌ناپذیری و NP-تمامیت

همراه با اثبات کامل قضیه‌ها به زبان ساده

پیش‌گفتار

این جزوه برای درس «نظریه زبان‌ها و ماشین‌ها» نوشته شده است. پایه آن اسلایدهای درس دکتر مهدی دولتی در دانشگاه صنعتی شریف است. ترتیب مطالب تقریباً همان ترتیب جلسه‌های درس است، اما مطالب هر جلسه به شکل یک فصل منظم با تعریف، قضیه، اثبات و مثال بازنویسی شده‌اند تا جزوه مثل یک کتاب خوانده شود، نه مثل مجموعه‌ای از اسلایدهای پشت سر هم.

این جزوه برای چه کسی است؟

برای خواندن این جزوه لازم نیست از قبل چیزی درباره اتوماتا بدانید. فقط کمی آشنایی با منطق و مجموعه‌ها کافی است و همان‌ها هم در فصل‌های ۱ و ۲ مرور شده‌اند. هر مفهوم جدید اول با یک توضیح شهودی و مثال معرفی می‌شود و بعد تعریف دقیق آن می‌آید.

درباره اثبات‌ها

هر جا در اسلایدها فقط «ایده اثبات» آمده بود یا جای اثبات خالی مانده بود (مثلاً با جمله‌هایی مثل «Formal correctness proof?» یا «Exercise»)، اثبات کامل آن در این جزوه نوشته شده است. اثبات‌ها با زبانی روان نوشته شده‌اند: اول می‌گوییم چه چیزی را می‌خواهیم نشان دهیم، بعد قدم‌به‌قدم جلو می‌رویم. اگر اثباتی طولانی است، قبل از آن یک «ایده اثبات» آمده تا نقشه کلی را ببینید.

راهنمای کادرها

مطالب جزوه در کادرهای رنگی جدا شده‌اند تا راحت‌تر مرور شوند:

تعریف ۱.۰ (نمونه)

کادر آبی برای **تعریف‌ها** است. هر چیزی که باید دقیق بدانید در این کادرهاست.

قضیه ۲.۰ (نمونه)

کادر بنفش برای **قضیه‌ها** است. کادرهای **لم** و **نتیجه** هم رنگ‌های نزدیک به این دارند.

ایده اثبات. این کادر خط‌چین، نقشه کلی اثبات را قبل از جزئیات نشان می‌دهد.

اثبات. اثبات‌ها با یک نوار خاکستری در کنار متن مشخص شده‌اند و با علامت ■ تمام می‌شوند.

■

مثال ۳.۰ (نمونه). مثال‌ها با نوار سبز مشخص شده‌اند.

نکته. نکته‌های مهم و دام‌های رایج در این کادر نارنجی آمده‌اند.

تعریف‌ها، قضیه‌ها و مثال‌های هر فصل با یک شمارنده مشترک شماره‌گذاری شده‌اند؛ مثلاً «قضیه ۳.۴» یعنی سومین کادر شماره‌دار فصل ۴.

نمادها و قراردادها

- اصطلاح‌های مهم در اولین بار با معادل انگلیسی آمده‌اند، مثل **زبان منظم** (Regular Language). در انتهای جزوه یک واژه‌نامه کوتاه هم هست.
- ε رشته تهی است، Σ الفبای ورودی است و $\sqcup$ نماد خانه خالی نوار ماشین تورینگ است.
- $\langle O \rangle$ کدگذاری (نمایش به صورت رشته) شیء O است.
- در شکل‌ها، حالت شروع با یک پیکان ورودی و حالت پذیرش با دایره دوخطی نشان داده شده است.

منابع

مرجع اصلی درس کتاب Introduction to the Theory of Computation نوشته مایکل سیپسر است. بخش‌هایی از درس (فرم‌های نرمال، گرامرهای منظم، CYK و PDAهای قطعی) از کتاب An Introduction to Formal Languages and Automata نوشته پیتر لینز است. مثال‌های فصل لم تزریق هم از اسلایدهای Costas Busch آمده‌اند.

ساختار جزوه

جزوه پنج بخش دارد که تقریباً به ترتیب از ماشین ضعیف‌تر به قوی‌تر جلو می‌روند:

۱. **پیش‌نیازها:** منطق، مجموعه، تابع، گراف، رشته، زبان و روش‌های اثبات.
۲. **زبان‌های منظم:** ماشین‌های متناهی قطعی و غیرقطعی، عبارات منظم، گرامرهای منظم و لم تزریق.
۳. **زبان‌های مستقل از متن:** گرامرهای مستقل از متن، فرم‌های نرمال، الگوریتم CYK، ماشین‌های پشته‌ای و لم تزریق برای این زبان‌ها.
۴. **محاسبه‌پذیری:** ماشین تورینگ، تز چرچ–تورینگ، تصمیم‌ناپذیری، کاهش و مسئله تناظر پست.
۵. **پیچیدگی:** کلاس‌های P و NP و NP-تمامیت.

فهرست مطالب

پیش‌گفتار آ

اول پیش‌نیازهای ریاضی ۱

۱ مقدمات ریاضی: منطق، مجموعه، تابع و گراف ۲

۱.۱ منطق بولی ۲

۱.۱.۱ عملگرهای بولی ۲

۲.۱.۱ سورها ۳

۲.۱ مجموعه‌ها ۳

۱.۲.۱ عملیات روی مجموعه‌ها ۴

۲.۲.۱ مجموعه توانی و قضیه کانتور ۴

۳.۱ دنباله‌ها، چندتایی‌ها و ضرب دکارتی ۵

۴.۱ توابع و روابط ۶

۱.۴.۱ مقایسه اندازه مجموعه‌ها ۶

۲.۴.۱ توابع چندمتغیره، گزاره‌نما و رابطه ۷

۳.۴.۱ انواع رابطه دوتایی ۸

۵.۱ گراف‌ها ۹

۲ رشته‌ها، زبان‌ها و روش‌های اثبات ۱۱

۱.۲ الفبا، رشته و زبان ۱۱

۱.۱.۲ ترتیب رشته‌ها ۱۲

۲.۱.۲ زبان‌ها ۱۲

۲.۲ تعریف، قضیه و اثبات ۱۳

۱.۲.۲ چطور اثبات پیدا کنیم؟ ۱۳

۲.۲.۲ چطور اثبات بنویسیم؟ ۱۳

۳.۲ انواع روش‌های اثبات ۱۴

۱.۳.۲ اثبات مستقیم و غیرمستقیم ۱۴

۲.۳.۲ اثبات ساختنی ۱۴
۳.۳.۲ برهان خلف ۱۵
۴.۳.۲ استقرا ۱۵

دوم زبان‌های منظم **۱۸**

۳ ماشین‌های متناهی قطعی **۱۹**
۱.۳ ایده ماشین متناهی ۱۹
۲.۳ نمودار حالت و تعریف صوری ۲۰
۳.۳ تعریف صوری محاسبه ۲۲
۴.۳ زبان‌های منظم و طراحی DFA ۲۳
۵.۳ عملیات منظم و بستاری ۲۴
۱.۵.۳ بستاری تحت متمم ۲۴
۲.۵.۳ بستاری تحت اجتماع و اشتراک ۲۵
۳.۵.۳ الحاق: نیاز به ابزاری جدید ۲۶

۴ نامعینی و ماشین‌های متناهی غیرقطعی **۲۸**
۱.۴ ایده نامعینی ۲۸
۱.۱.۴ ماشین غیرقطعی می‌تواند «حدس بزند» ۲۹
۲.۴ تعریف صوری NFA ۳۰
۳.۴ هم‌ارزی NFA و DFA ۳۱

۵ ویژگی‌های بستاری زبان‌های منظم **۳۵**
۱.۵ اجتماع ۳۵
۲.۵ الحاق ۳۶
۳.۵ ستاره ۳۷
۴.۵ معکوس ۳۸
۵.۵ پیشوند، پسوند و خارج‌قسمت ۳۸

۶ عبارات منظم **۴۱**
۱.۶ تعریف و مثال ۴۱
۱.۱.۶ اتحادهای ساده ۴۲
۲.۶ هم‌ارزی عبارات منظم و ماشین‌های متناهی ۴۳

۱.۲.۶ از عبارت منظم به NFA . . . ۴۳
۲.۲.۶ از DFA به عبارت منظم . . . ۴۴

۷ گرامرهای منظم ۴۸
۱.۷ گرامر چیست؟ . . . ۴۸
۲.۷ گرامرهای خطی راست و چپ . . . ۴۸
۳.۷ هر گرامر خطی راست زبانی منظم تولید می‌کند . . . ۴۹
۴.۷ هر زبان منظم یک گرامر خطی راست دارد . . . ۵۰
۵.۷ گرامرهای خطی چپ . . . ۵۲

۸ زبان‌های نامنظم و لم تزریق ۵۳
۱.۸ محدودیت ماشین‌های متناهی . . . ۵۳
۲.۸ لم تزریق . . . ۵۳
۳.۸ روش استفاده از لم تزریق . . . ۵۵
۴.۸ مثال‌ها . . . ۵۵
۵.۸ استفاده از ویژگی‌های بستاری . . . ۵۷
۶.۸ یک زبان تزریق‌پذیر که منظم نیست . . . ۵۷
۷.۸ ویژگی‌های بستاری زبان‌های نامنظم . . . ۵۸

سوم زبان‌های مستقل از متن ۶۰

۹ گرامرهای مستقل از متن ۶۱
۱.۹ گرامر و اشتقاق . . . ۶۱
۲.۹ تعریف صوری . . . ۶۳
۳.۹ ابهام . . . ۶۴

۱۰ ساده‌سازی گرامرها و فرم‌های نرمال ۶۷
۱.۱۰ یک قاعده جایگزینی مفید . . . ۶۷
۲.۱۰ حذف متغیرها و قاعده‌های بی‌فایده . . . ۶۸
۳.۱۰ حذف قاعده‌های ε . . . ۶۹
۴.۱۰ حذف قاعده‌های واحد . . . ۷۰
۵.۱۰ فرم نرمال چامسکی . . . ۷۱
۶.۱۰ فرم نرمال گرایباخ . . . ۷۳

۱۱ عضویت، تجزیه و الگوریتم CYK **۷۴**

۱.۱۱ صورت جمله‌ای، عضویت و تجزیه . . . ۷۴

۲.۱۱ تجزیه با جست‌وجوی کامل . . . ۷۴

۳.۱۱ گرامرهای ساده . . . ۷۶

۴.۱۱ الگوریتم CYK . . . ۷۶

۱۲ ماشین‌های پشته‌ای **۷۸**

۱.۱۲ ایده ماشین پشته‌ای . . . ۷۸

۱.۱.۱۲ گذارها . . . ۷۸

۲.۱۲ تعریف صوری . . . ۷۹

۳.۱۲ یک مثال . . . ۸۰

۴.۱۲ هم‌ارزی ماشین‌های پشته‌ای و گرامرهای مستقل از متن . . . ۸۱

۱.۴.۱۲ از گرامر به ماشین پشته‌ای . . . ۸۱

۲.۴.۱۲ از ماشین پشته‌ای به گرامر . . . ۸۳

۱۳ ویژگی‌های بستاری زبان‌های مستقل از متن **۸۶**

۱.۱۳ اجتماع، الحاق و ستاره . . . ۸۶

۲.۱۳ پیشوند . . . ۸۷

۳.۱۳ معکوس و پسوند . . . ۸۸

۴.۱۳ اشتراک با یک زبان منظم . . . ۸۹

۵.۱۳ عملیاتی که تحت آن‌ها بسته نیستند . . . ۸۹

۱۴ لم تزریق برای زبان‌های مستقل از متن **۹۱**

۱.۱۴ صورت لم . . . ۹۱

۲.۱۴ مثال‌ها . . . ۹۲

۱۵ ماشین‌های پشته‌ای قطعی و گرامرهای LL **۹۴**

۱.۱۵ چند ویژگی تصمیم‌پذیر . . . ۹۴

۲.۱۵ ماشین پشته‌ای قطعی . . . ۹۴

۳.۱۵ نامعینی در ماشین‌های پشته‌ای قدرت اضافه می‌کند . . . ۹۵

۴.۱۵ گرامرهای زبان‌های مستقل از متن قطعی . . . ۹۷

۵.۱۵ تصویر کلی . . . ۹۸

چهارم محاسبه‌پذیری **۱۰۰**

۱۶ ماشین تورینگ **۱۰۱**

۱.۱۶ ماشین تورینگ چیست؟ . . . ۱۰۱

۲.۱۶ مثال: سه دسته نماد برابر . . . ۱۰۲

۳.۱۶ تعریف صوری . . . ۱۰۳

۴.۱۶ محاسبه ماشین تورینگ . . . ۱۰۳

۵.۱۶ زبان‌های تشخیص‌پذیر و تصمیم‌پذیر . . . ۱۰۴

۶.۱۶ مثال: توان‌های دو . . . ۱۰۵

۱۷ گونه‌های ماشین تورینگ **۱۰۷**

۱.۱۷ هم‌ارزی و ماشین با حرکت «ماندن» . . . ۱۰۷

۲.۱۷ ماشین تورینگ چندنواره . . . ۱۰۷

۳.۱۷ ماشین تورینگ غیرقطعی . . . ۱۰۹

۴.۱۷ شمارنده‌ها . . . ۱۱۰

۵.۱۷ مدل‌های دیگر . . . ۱۱۱

۱۸ تز چرچ–تورینگ و زبان‌های تصمیم‌پذیر **۱۱۲**

۱.۱۸ الگوریتم چیست؟ . . . ۱۱۲

۱.۱.۱۸ مسئله دهم هیلبرت . . . ۱۱۲

۲.۱.۱۸ تز چرچ–تورینگ . . . ۱۱۲

۳.۱.۱۸ مسئله هیلبرت به زبان این درس . . . ۱۱۳

۲.۱۸ سطوح توصیف ماشین تورینگ . . . ۱۱۳

۳.۱۸ زبان‌های تصمیم‌پذیر درباره زبان‌های منظم . . . ۱۱۴

۴.۱۸ زبان‌های تصمیم‌پذیر درباره زبان‌های مستقل از متن . . . ۱۱۶

۱۹ تصمیم‌ناپذیری **۱۱۹**

۱.۱۹ مسئله پذیرش برای ماشین تورینگ . . . ۱۱۹

۲.۱۹ روش قطری‌سازی . . . ۱۱۹

۳.۱۹ زبان‌هایی که تشخیص‌پذیر نیستند . . . ۱۲۱

۴.۱۹ اثبات تصمیم‌ناپذیری مسئله پذیرش . . . ۱۲۲

۵.۱۹ یک زبان تشخیص‌ناپذیر . . . ۱۲۳

۲۰ کاهش‌پذیری **۱۲۵**

۱.۲۰ ایده کاهش . . . ۱۲۵

۲.۲۰ مسئله توقف . . . ۱۲۵

۳.۲۰ تهی بودن زبان یک ماشین . . . ۱۲۶

۴.۲۰ آیا زبان یک ماشین منظم است؟ . . . ۱۲۷

۵.۲۰ هم‌ارزی دو ماشین . . . ۱۲۸

۶.۲۰ تاریخچه محاسبه و ماشین‌های خطی‌کران‌دار . . . ۱۲۸

۷.۲۰ گرامرهایی که همه رشته‌ها را تولید می‌کنند . . . ۱۳۱

۲۱ مسئله تناظر پست و کاهش نگاشتی **۱۳۳**

۱.۲۱ مسئله تناظر پست . . . ۱۳۳

۲.۲۱ کاهش نگاشتی . . . ۱۳۶

پنجم پیچیدگی محاسبه **۱۳۹**

۲۲ پیچیدگی زمانی و کلاس P **۱۴۰**

۱.۲۲ اندازه‌گیری زمان . . . ۱۴۰

۱.۱.۲۲ نمادهای O بزرگ و o کوچک . . . ۱۴۱

۲.۱.۲۲ تحلیل الگوریتم . . . ۱۴۲

۲.۲۲ رابطه پیچیدگی مدل‌های محاسباتی . . . ۱۴۳

۳.۲۲ کلاس P . . . ۱۴۴

۲۳ کلاس NP **۱۴۷**

۱.۲۳ یک مثال: مسیر همیلتونی . . . ۱۴۷

۲.۲۳ وارسی‌کننده و کلاس NP . . . ۱۴۸

۳.۲۳ پرسش P در برابر NP . . . ۱۵۰

۲۴ تمامیت-NP **۱۵۱**

۱.۲۴ کاهش‌پذیری در زمان چندجمله‌ای . . . ۱۵۱

۲.۲۴ تمامیت-NP . . . ۱۵۲

۳.۲۴ مسئله صدق‌پذیری . . . ۱۵۳

۴.۲۴ 3SAT . . . ۱۵۵

واژه‌نامه **۱۵۸**

بخش اول

پیش‌نیازهای ریاضی

۱ فصل

مقدمات ریاضی: منطق، مجموعه، تابع و گراف

در این درس با «ماشین»هایی سروکار داریم که رشته‌ها را می‌خوانند و درباره آن‌ها تصمیم می‌گیرند. برای اینکه بتوانیم درباره این ماشین‌ها دقیق حرف بزنیم و قضیه ثابت کنیم، به چند ابزار ساده ریاضی نیاز داریم. این فصل همین ابزارها را مرور می‌کند. اگر این مطالب را از درس ریاضیات گسسته به خاطر دارید، می‌توانید سریع از آن عبور کنید.

۱.۱ منطق بولی

منطق بولی یک دستگاه ریاضی است که فقط دو مقدار دارد: **درست** (True) و **نادرست** (False). معمولاً درست را با 1 و نادرست را با 0 نشان می‌دهیم. این دو مقدار می‌توانند هر دوگانه‌ای را مدل کنند: ولتاژ بالا و پایین، گزاره درست و غلط، یا جواب «بله» و «نه». منطق بولی پایه طراحی مدارهای دیجیتال و کامپیوترهاست.

۱.۱.۱ عملگرهای بولی

با عملگرهای بولی گزاره‌های ساده را به هم وصل می‌کنیم و گزاره‌های پیچیده‌تر می‌سازیم. اگر P و Q دو گزاره باشند (به آن‌ها **عملوند** می‌گوییم)، عملگرهای اصلی این‌ها هستند:

- **نقیض** (NOT) با نماد $\neg$: مقدار را برعکس می‌کند.
- **عطف** (AND) با نماد $\wedge$: فقط وقتی درست است که هر دو عملوند درست باشند.
- **فصل** (OR) با نماد $\vee$: وقتی درست است که دست‌کم یکی از عملوندها درست باشد.
- **یای انحصاری** (XOR) با نماد $\oplus$: وقتی درست است که دقیقاً یکی از عملوندها درست باشد.
- **هم‌ارزی** با نماد $\leftrightarrow$: وقتی درست است که دو عملوند هم‌مقدار باشند.
- **استلزام** با نماد $\rightarrow$: «اگر P آنگاه Q». فقط در یک حالت نادرست است: وقتی P درست و Q نادرست باشد.

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \oplus Q$	$P \leftrightarrow Q$	$P \rightarrow Q$
0	0	1	0	0	0	1	1
0	1	1	0	1	1	0	1
1	0	0	0	1	1	0	0
1	1	0	1	1	0	1	1

همه عملگرها را می‌توان فقط با AND و NOT نوشت:

$$P \vee Q \equiv \neg(\neg P \wedge \neg Q), \qquad P \to Q \equiv \neg P \vee Q,$$
$$P \leftrightarrow Q \equiv (P \to Q) \wedge (Q \to P), \qquad P \oplus Q \equiv \neg(P \leftrightarrow Q).$$

قوانین پخش‌پذیری هم مثل ضرب و جمع در حساب کار می‌کنند، با این تفاوت که هر دو جهت برقرارند:

$$P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R), \qquad P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R).$$

۲.۱.۱ سورها

گاهی می‌خواهیم بگوییم یک ویژگی «برای همه» برقرار است یا «دست‌کم برای یکی». برای این کار از **سورها** استفاده می‌کنیم:

- **سور عمومی** $\forall$ («برای هر»): مثلاً $\forall x \ \big(P(x) \to Q(x)\big)$ یعنی هر x که ویژگی P را دارد، ویژگی Q را هم دارد.
- **سور وجودی** $\exists$ («وجود دارد»): مثلاً $\exists x \ \big(P(x) \to Q(x)\big)$.
- سورها را می‌توان ترکیب کرد: $\forall x \, \exists y \ P(y) > Q(x)$ یعنی برای هر x یک y پیدا می‌شود که $P(y) > Q(x)$.

نقیض سورها قاعده ساده‌ای دارد: $\forall$ به $\exists$ تبدیل می‌شود و برعکس، و نقیض به داخل می‌رود. این قاعده در فصل‌های بعد (مثلاً در لم تزریق) بارها به کار می‌آید:

$$\neg\big(\forall x \ P(x) \to Q(x)\big) \equiv \exists x \ \neg\big(P(x) \to Q(x)\big) \equiv \exists x \ \neg\big(\neg P(x) \vee Q(x)\big) \equiv \exists x \ \big(P(x) \wedge \neg Q(x)\big),$$
$$\neg\big(\exists x \ P(x) \to Q(x)\big) \equiv \forall x \ \neg\big(P(x) \to Q(x)\big) \equiv \forall x \ \big(P(x) \wedge \neg Q(x)\big),$$
$$\neg\big(\forall x \, \exists y \ P(y) > Q(x)\big) \equiv \exists x \, \forall y \ \neg\big(P(y) > Q(x)\big) \equiv \exists x \, \forall y \ P(y) \le Q(x).$$

۲.۱ مجموعه‌ها

تعریف ۱.۱ (مجموعه)

مجموعه (Set) گروهی از اشیاء است. به این اشیاء **عضو** یا **عنصر** مجموعه می‌گوییم. ترتیب و تکرار عضوها در مجموعه مهم نیست.

سه راه برای تعریف یک مجموعه داریم:

۱. **فهرست کردن عضوها:** $A = \{1, 2, 3\}$.
۲. **استفاده از سه‌نقطه** برای مجموعه‌های نامتناهی: $\mathbb{N} = \{1, 2, 3, \dots\}$ و $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$.
۳. **بیان یک قاعده** (نماد سازنده مجموعه): $B = \{n \mid n = m^2$ و m یک عدد صحیح است$\}$.

چند مفهوم پایه:

- **عضویت:** $1 \in \{1, 2, 3\}$ و $4 \notin \{1, 2, 3\}$.
- **اندازه** (Cardinality): تعداد عضوهای مجموعه، مثلاً $|A| = 3$.
- **زیرمجموعه:** $A \subseteq B$ یعنی هر عضو A عضو B هم هست: $a \in A \to a \in B$.
- **زیرمجموعه سره:** $A \subsetneq B$ یعنی $A \subseteq B$ و $A \neq B$.

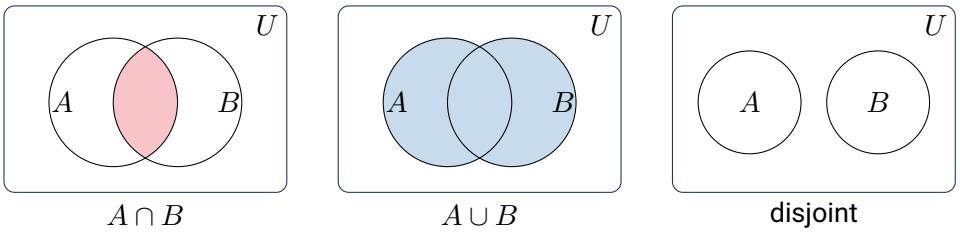
- **چندمجموعه** (Multiset): مجموعه‌ای که در آن تعداد تکرار مهم است؛ مثلاً $\{5\}$ و $\{5,5\}$ به عنوان چندمجموعه با هم فرق دارند، ولی به عنوان مجموعه یکی‌اند.
- **مجموعه تهی** $\emptyset$ یا $\{\}$ هیچ عضوی ندارد. مجموعه **تک‌عضوی** مثل $\{10\}$ و **زوج نامرتب** مثل $\{10,30\}$.

۱.۲.۱ عملیات روی مجموعه‌ها

فرض کنید همه مجموعه‌ها زیرمجموعه یک **مجموعه مرجع** U باشند.

- **اجتماع:** $A \cup B = \{x \mid x \in A \text{ یا } x \in B\}$.
- **اشتراک:** $A \cap B = \{x \mid x \in A \text{ و } x \in B\}$.
- **تفاضل:** $B - A = \{x \mid x \notin A \text{ و } x \in B\}$.
- **متمم:** $\overline{A} = U - A$.

دو مجموعه را **مجزا** می‌گوییم اگر $A \cap B = \emptyset$. برای نمایش این عملیات از **نمودار ون** استفاده می‌شود:



چند اتحاد مفید درباره مجموعه تهی:

$$A \cup \emptyset = A, \quad A \cap \emptyset = \emptyset, \quad A - \emptyset = A, \quad \emptyset - A = \emptyset, \quad \overline{\emptyset} = U.$$

و **قوانین دمورگان**:

$$\overline{A \cup B} = \overline{A} \cap \overline{B}, \qquad \overline{A \cap B} = \overline{A} \cup \overline{B}.$$

(اثبات قانون اول را در فصل بعد به عنوان نمونه‌ای از یک اثبات کامل می‌بینیم.)

۲.۲.۱ مجموعه توانی و قضیه کانتور

تعریف ۲.۱ (مجموعه توانی)

مجموعه توانی (Power Set) مجموعه A، که با $\mathcal{P}(A)$ یا 2^A نشان داده می‌شود، مجموعه همه زیرمجموعه‌های A است. پس عضوهای آن خودشان مجموعه‌اند.

مثال ۳.۱. اگر $A = \{a,b,c\}$ باشد:

$$2^A = \{\emptyset, \{a\}, \{b\}, \{c\}, \{a,b\}, \{a,c\}, \{b,c\}, \{a,b,c\}\}.$$

دقت کنید که همیشه $\emptyset \in 2^A$ و $A \in 2^A$.

برای مجموعه متناهی A داریم $|2^A| = 2^{|A|}$. دلیلش ساده است: برای ساختن یک زیرمجموعه، درباره هر عضو A یک تصمیم دوتایی می‌گیریم: «باشد» یا «نباشد». با $|A|$ تصمیم مستقل، $2^{|A|}$ زیرمجموعه به دست می‌آید.

قضیه زیر نشان می‌دهد که حتی برای مجموعه‌های نامتناهی، مجموعه توانی همیشه «بزرگ‌تر» است. (معنای دقیق مقایسه اندازه مجموعه‌های نامتناهی را در بخش ۱.۴.۱ می‌بینیم.)

قضیه ۴.۱ (کانتور)

برای هر مجموعه A داریم $|A| < |\mathcal{P}(A)|$.

اثبات. باید دو چیز را نشان دهیم: $|A| \leq |\mathcal{P}(A)|$ و $|A| \neq |\mathcal{P}(A)|$.

بخش اول ساده است: تابع $a \mapsto \{a\}$ هر عضو A را به یک زیرمجموعه تک‌عضوی می‌برد و یک‌به‌یک است. پس A با زیرمجموعه‌ای از $\mathcal{P}(A)$ هم‌اندازه است.

برای بخش دوم با برهان خلف فرض کنید تابعی یک‌به‌یک و پوشا $f : A \to \mathcal{P}(A)$ وجود دارد. پس هر عضو a به یک زیرمجموعه $f(a)$ نظیر شده است. مجموعه زیر را تعریف کنید:

$$D = \{a \in A \mid a \notin f(a)\}.$$

یعنی D شامل عضوهایی است که در زیرمجموعه نظیر خودشان نیستند. D یک زیرمجموعه از A است و چون f پوشاست، عضوی مثل d هست که $f(d) = D$. حالا می‌پرسیم: آیا $d \in D$؟

- اگر $d \in D$، طبق تعریف D باید $d \notin f(d) = D$. تناقض.
- اگر $d \notin D$، یعنی $d \notin f(d)$، پس طبق تعریف D باید $d \in D$. باز هم تناقض.

هر دو حالت به تناقض می‌رسد. پس چنین f ای وجود ندارد و $|A| < |\mathcal{P}(A)|$.

■

نکته. این روش اثبات «قطری‌سازی» نام دارد: D طوری ساخته شده که با هر $f(a)$ دست‌کم در یک نقطه (خود a) فرق داشته باشد. در فصل ۱۹ همین ایده را برای اثبات وجود مسائل حل‌نشدنی به کار می‌بریم.

۳.۱ دنباله‌ها، چندتایی‌ها و ضرب دکارتی

دنباله فهرستی از اشیاء با ترتیب مشخص است، مثل $D = (5, 3, 13)$. برخلاف مجموعه، در دنباله ترتیب و تکرار مهم است: $(5, 3) \neq (3, 5)$. دنباله متناهی را **چندتایی** (Tuple) می‌نامیم. دنباله‌ای با k عضو یک **k-تایی** است و به 2-تایی **زوج مرتب** می‌گوییم.

تعریف ۵.۱ (ضرب دکارتی)

ضرب دکارتی (Cartesian Product) دو مجموعه A و B مجموعه همه زوج‌های مرتب (a, b) است که $a \in A$ و $b \in B$:

$$A \times B = \{(a, b) \mid a \in A,\ b \in B\}.$$

به طور کلی $A_1 \times \cdots \times A_k = \{(a_1, \ldots, a_k) \mid a_i \in A_i\}$ و $A^k = \underbrace{A \times \cdots \times A}_{k}$.

مثال ۶.۱. اگر $A = \{1, 2\}$ و $B = \{x, y, z\}$:

$$A \times B = \{(1, x), (1, y), (1, z), (2, x), (2, y), (2, z)\},$$
$$A \times B \times A = \{(1, x, 1), (1, x, 2), (1, y, 1), (1, y, 2), (1, z, 1), (1, z, 2),$$
$$(2, x, 1), (2, x, 2), (2, y, 1), (2, y, 2), (2, z, 1), (2, z, 2)\}.$$

۴.۱ توابع و روابط

تعریف ۷.۱ (تابع)

تابع (Function) شیئی است که یک رابطه ورودی–خروجی برقرار می‌کند: به هر ورودی a دقیقاً یک خروجی $f(a) = b$ نسبت می‌دهد. می‌گوییم f، a را به b **می‌نگارد**. مجموعه ورودی‌های ممکن **دامنه** و مجموعه خروجی‌های ممکن **برد** نام دارد و می‌نویسیم $f : D \to R$.

مثلاً تابع جمع $add : \mathbb{Z} \times \mathbb{Z} \to \mathbb{Z}$ و تابع قدرمطلق $abs : \mathbb{Z} \to \mathbb{Z}$. به عنوان مثالی دیگر، اگر $\mathbb{Z}_m = \{0, 1, \ldots, m-1\}$ باشد، تابع $f : \mathbb{Z}_5 \to \mathbb{Z}_5$ با ضابطه $f(n) = (n+1) \bmod 5$ همان جدول زیر است:

n	0	1	2	3	4
$f(n)$	1	2	3	4	0

تعریف ۸.۱ (انواع تابع)

تابع $f : A \to B$ را

- **یک‌به‌یک** (Injective) می‌گوییم اگر ورودی‌های متفاوت را به خروجی‌های متفاوت ببرد: $a_1 \neq a_2 \to f(a_1) \neq f(a_2)$.
- **پوشا** (Onto) می‌گوییم اگر همه عضوهای برد را بپوشاند: برای هر $b \in B$ عضوی مثل a باشد که $f(a) = b$.
- **تناظر یک‌به‌یک** یا **دوسویی** (Bijection) می‌گوییم اگر هم یک‌به‌یک و هم پوشا باشد. چنین تابعی وارون‌پذیر است.

مثال ۹.۱. تابع $f(n) = (n+1) \bmod 5$ بالا دوسویی است. اما تابع $g : \mathbb{Z}_5 \to \mathbb{Z}_5$ با جدول $g(0) = 1,\ g(1) = 2,\ g(2) = 1,\ g(3) = 2,\ g(4) = 1$ نه یک‌به‌یک است (چون $g(0) = g(2)$) و نه پوشا (هیچ ورودی‌ای به 0 نمی‌رود). اگر اندازه دامنه و برد متناهی و متفاوت باشد، مثل $f : \{0, 1, 2, 3\} \to \{0, 1, 2, 3, 4\}$، تابع هرگز نمی‌تواند دوسویی باشد.

۱.۴.۱ مقایسه اندازه مجموعه‌ها

برای مجموعه‌های متناهی، مقایسه اندازه یعنی شمردن. اما مجموعه‌های نامتناهی را نمی‌توان شمرد. ایده کانتور این بود که اندازه‌ها را با **جفت کردن عضوها** مقایسه کنیم.

تعریف ۱۰.۱ (هم‌اندازگی)

- $|A| = |B|$ اگر و تنها اگر یک تابع دوسویی $f : A \to B$ وجود داشته باشد.
- $|A| \leq |B|$ اگر زیرمجموعه‌ای مثل $C \subseteq B$ باشد که $|C| = |A|$.
- $|A| < |B|$ اگر $|A| \leq |B|$ و $|A| \neq |B|$.

تعریف ۱۱.۱ (متناهی و شمارا)

- A **متناهی** است اگر عدد صحیح $n \geq 0$ باشد که $|A| = |\{1, 2, \ldots, n\}|$. در غیر این صورت **نامتناهی** است.
- A **شمارای نامتناهی** (Denumerable) است اگر با $\mathbb{N}$ تناظر یک‌به‌یک داشته باشد.
- A **شمارا** (Countable) است اگر متناهی یا شمارای نامتناهی باشد، و **ناشمارا** است اگر شمارا نباشد.

شمارا بودن یعنی بتوانیم عضوها را در یک فهرست (دنباله) بنویسیم طوری که هر عضو بالاخره در جایی از فهرست بیاید.

مثال ۱۲.۱ (مجموعه‌های شمارا و ناشمارا).

- اعداد طبیعی $1, 2, 3, \ldots$ شمارا هستند.
- اعداد صحیح هم شمارا هستند. کافی است آن‌ها را به این ترتیب فهرست کنیم: $0, 1, -1, 2, -2, 3, -3, \ldots$.
- زوج‌های مرتب اعداد طبیعی شمارا هستند: آن‌ها را بر اساس مجموع دو مؤلفه فهرست می‌کنیم: اول $(1, 1)$، بعد $(1, 2), (2, 1)$، بعد $(1, 3), (2, 2), (3, 1)$ و همین‌طور تا آخر.
- اعداد گویای بین صفر و یک شمارا هستند: آن‌ها را بر اساس مخرج فهرست می‌کنیم و تکراری‌ها را حذف می‌کنیم: $\frac{1}{2}, \frac{1}{3}, \frac{2}{3}, \frac{1}{4}, \frac{3}{4}, \frac{1}{5}, \ldots$
- اما بازه $[0, 1]$، مجموعه $\mathcal{P}(\mathbb{N})$ و مجموعه همه توابع $f : \mathbb{N} \to \{0, 1\}$ **ناشمارا** هستند. اثبات این‌ها با قطری‌سازی است (قضیه ۲.۲.۱ و فصل ۱۹).

کمی تاریخ. اندازه یک مجموعه متناهی همان تعداد عضوهای آن است. برای مجموعه‌های نامتناهی از **اعداد اصلی ترامتناهی** استفاده می‌شود. اولین آن‌ها $\aleph_0 := |\mathbb{N}|$ است. کانتور در سال ۱۸۷۴ نشان داد $c = |\mathbb{R}| = 2^{\aleph_0} > \aleph_0$. **فرضیه پیوستار** می‌گوید هیچ مجموعه‌ای با اندازه‌ای بین $\aleph_0$ و c وجود ندارد. این اولین مسئله از ۲۳ مسئله هیلبرت (سال ۱۹۰۰) بود. گودل و کوهن نشان دادند این فرضیه در دستگاه اصول موضوعه ZFC نه قابل اثبات است و نه قابل رد.

۲.۴.۱ توابع چندمتغیره، گزاره‌نما و رابطه

تابعی که دامنه‌اش $A_1 \times \cdots \times A_k$ باشد، یک k-تایی $(a_1, \ldots, a_k)$ می‌گیرد. به k **آریتی** و به هر a_i یک **آرگومان** می‌گوییم. تابع با $k = 1$ **یکانی** و با $k = 2$ **دوتایی** است. تابع دوتایی را می‌توان به شکل پیشوندی $(add(a, b))$ یا میانوندی $(a + b)$ نوشت.

تعریف ۱۳.۱ (گزاره‌نما و رابطه)

- **گزاره‌نما** یا **ویژگی** (Predicate) تابعی است که بردش $\{\text{True}, \text{False}\}$ است. مثلاً $even(3) = \text{False}$ و $even(6) = \text{True}$.

- **رابطه k-تایی** روی A یک گزاره‌نماست که دامنه‌اش A^k باشد. رابطه دوتایی را معمولاً میانوندی می‌نویسیم، مثل $a < b$ یا $x = y$.

وقتی می‌نویسیم aRb منظور این است که $aRb = \text{True}$.

هر گزاره‌نما $P : D \to \{\text{True}, \text{False}\}$ را می‌توان با یک مجموعه نشان داد: $S = \{a \in D \mid P(a) = \text{True}\}$. اگر D از متن معلوم باشد، همان S کافی است. این ایده بعداً خیلی مهم می‌شود: در این درس «مسئله‌ها» را با «مجموعه‌ها» (زبان‌ها) نشان می‌دهیم.

مثال ۱۴.۱ (سنگ، کاغذ، قیچی). رابطه $beats$ («می‌برد») را می‌توان با جدول یا با مجموعه نشان داد:

beats	Scissors	Paper	Stone
Scissors	False	True	False
Paper	False	False	True
Stone	True	False	False

که معادل مجموعه $\{(\text{Scissors}, \text{Paper}), (\text{Paper}, \text{Stone}), (\text{Stone}, \text{Scissors})\}$ است.

۳.۴.۱ انواع رابطه دوتایی

رابطه دوتایی R روی مجموعه A را

۱. **بازتابی** می‌گوییم اگر $\forall x \in A : \ xRx$،

۲. **متقارن** می‌گوییم اگر $\forall x, y \in A : \ xRy \to yRx$،

۳. **پادمتقارن** می‌گوییم اگر $\forall x, y \in A : \ (xRy \wedge yRx) \to x = y$،

۴. **متعدی** می‌گوییم اگر $\forall x, y, z \in A : \ (xRy \wedge yRz) \to xRz$.

تعریف ۱۵.۱ (رابطه هم‌ارزی و ترتیب جزئی)

رابطه‌ای که بازتابی، متقارن و متعدی باشد **رابطه هم‌ارزی** (Equivalence Relation) است. رابطه‌ای که بازتابی، پادمتقارن و متعدی باشد **رابطه ترتیب جزئی** (Partial Order) است.

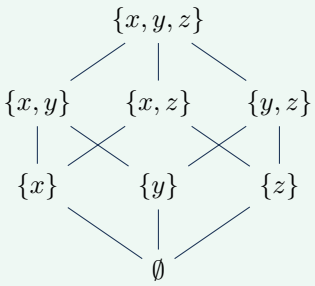
مثال ۱۶.۱ (همنهشتی به پیمانه ۷). روی $\mathbb{N}$ تعریف کنید $i \equiv_7 j$ اگر $i - j$ مضربی از 7 باشد. این رابطه هم‌ارزی است:

۱. **بازتابی:** $i - i = 0$ مضرب 7 است.

۲. **متقارن:** اگر $i - j$ مضرب 7 باشد، $j - i = -(i - j)$ هم مضرب 7 است.

۳. **متعدی:** اگر $i \equiv_7 j$ و $j \equiv_7 k$، آنگاه $i - k = (i - j) + (j - k)$ مجموع دو مضرب 7 است و خودش مضرب 7 است.

مثال ۱۷.۱ (رابطه زیرمجموعه). رابطه $\subseteq$ روی زیرمجموعه‌های $\{x, y, z\}$ یک ترتیب جزئی است: هر مجموعه زیرمجموعه خودش است (بازتابی)؛ اگر $A \subseteq B$ و $B \subseteq A$ آنگاه $A = B$ (پادمتقارن)؛ و اگر $A \subseteq B \subseteq C$ آنگاه $A \subseteq C$ (متعدی). به آن «جزئی» می‌گوییم چون بعضی جفت‌ها مثل $\{x\}$ و $\{y\}$ قابل مقایسه نیستند.

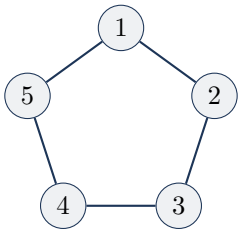


۵.۱ گراف‌ها

تعریف ۱۸.۱ (گراف)

گراف (Graph) بی‌جهت $G = (V, E)$ مجموعه‌ای از نقطه‌ها (**رأس‌ها**، V) است که بعضی از آن‌ها با خط (**یال‌ها**، E) به هم وصل شده‌اند. در گراف بی‌جهت، (i, j) و (j, i) یک یال‌اند، پس یال را با زوج نامرتب $\{i, j\}$ هم نشان می‌دهیم. **درجه** یک رأس تعداد یال‌های متصل به آن است.

در حالت عادی بین هر دو رأس حداکثر یک یال داریم. یالی که یک رأس را به خودش وصل کند **طوقه** نام دارد. برای مثال گراف زیر $\big(\{1, 2, 3, 4, 5\}, \{(1, 2), (2, 3), (3, 4), (4, 5), (5, 1)\}\big)$ است:



مثال‌های واقعی: رأس‌ها شهرها و یال‌ها جاده‌های بین آن‌ها؛ یا رأس‌ها افراد و یال‌ها دوستی‌ها. اگر روی رأس‌ها یا یال‌ها برچسب (مثلاً طول جاده) بگذاریم، **گراف برچسب‌دار** داریم.

تعریف ۱۹.۱ (مفاهیم پایه گراف)

- **زیرگراف:** زیرگراف H از G است اگر رأس‌های H زیرمجموعه رأس‌های G باشند و یال‌های H همان یال‌های G روی آن رأس‌ها باشند.
- **مسیر:** دنباله‌ای از رأس‌ها که رأس‌های متوالی با یال به هم وصل‌اند. **مسیر ساده** رأس تکراری ندارد.
- **همبند:** گرافی که بین هر دو رأس آن مسیری وجود دارد.
- **دور:** مسیری که از یک رأس شروع و به همان رأس ختم می‌شود. **دور ساده** دست‌کم سه رأس دارد و فقط

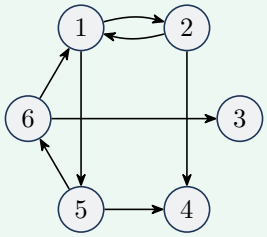
رأس اول و آخرش تکراری است.

- **درخت:** گراف همبندی که دور ساده ندارد. رأس با درجه یک **برگ** است و گاهی یک رأس را به عنوان **ریشه** انتخاب می‌کنیم.

تعریف ۲۰.۱ (گراف جهت‌دار)

در **گراف جهت‌دار** به جای خط، پیکان داریم و یال از i به j با زوج مرتب (i,j) نشان داده می‌شود. تعداد پیکان‌های خارج‌شونده از یک رأس **درجه خروجی** و تعداد پیکان‌های واردشونده **درجه ورودی** آن است. **مسیر جهت‌دار** مسیری است که همه پیکان‌هایش در جهت حرکت باشند. گراف جهت‌دار **قویاً همبند** است اگر از هر رأس به هر رأس دیگر مسیر جهت‌دار وجود داشته باشد.

مثال ۲۱.۱. گراف جهت‌دار $\big(\{1,\dots,6\},\{(1,2),(1,5),(2,1),(2,4),(5,4),(5,6),(6,1),(6,3)\}\big)$:



هر رابطه دوتایی R روی مجموعه D را می‌توان با یک گراف جهت‌دار $G=(D,E)$ نشان داد که $E=\{(x,y)\mid xRy\}$.
مثلاً گراف رابطه $beats$ یک دور جهت‌دار سه‌رأسی است: یالی از قیچی به کاغذ، از کاغذ به سنگ و از سنگ به قیچی.

جمع‌بندی فصل

- عملگرهای بولی، جدول درستی آن‌ها و قاعده نقیض سورها را دیدیم.
- با مجموعه، مجموعه توانی و ضرب دکارتی آشنا شدیم و قضیه کانتور را با قطری‌سازی ثابت کردیم.
- اندازه مجموعه‌های نامتناهی را با تابع دوسویی مقایسه می‌کنیم؛ مجموعه‌ها شمارا یا ناشمارا هستند.
- رابطه هم‌ارزی (بازتابی، متقارن، متعدی) و ترتیب جزئی (بازتابی، پادمتقارن، متعدی) را تعریف کردیم.
- مفاهیم پایه گراف‌های بی‌جهت و جهت‌دار را مرور کردیم.

۲ فصل

رشته‌ها، زبان‌ها و روش‌های اثبات

همه ماشین‌هایی که در این درس می‌بینیم ورودی خود را به شکل یک **رشته** می‌گیرند. مجموعه‌ای از رشته‌ها را **زبان** می‌نامیم. در نیمه اول این فصل این دو مفهوم را دقیق تعریف می‌کنیم. در نیمه دوم روش‌های اصلی اثبات را مرور می‌کنیم، چون بخش بزرگی از این درس ثابت کردن قضیه‌ها درباره زبان‌ها و ماشین‌هاست.

۱.۲ الفبا، رشته و زبان

تعریف ۱.۲ (الفبا و رشته)

- **الفبا** (Alphabet) هر مجموعه **متناهی و ناتهی** است. الفبا را معمولاً با Σ یا Γ نشان می‌دهیم و به عضوهای آن **نماد** می‌گوییم.
- **رشته** (String) روی یک الفبا، دنباله‌ای **متناهی** از نمادهای آن الفباست. نمادها را بدون فاصله و ویرگول پشت هم می‌نویسیم.

مثال ۲.۲. $\Sigma_1 = \{0,1\}$، $\Sigma_2 = \{a,b,c,\dots,z\}$ و $\Gamma = \{0,1,x,y,z\}$ سه الفبا هستند. رشته 01001 روی Σ_1 و رشته $abracadabra$ روی Σ_2 است.

چند مفهوم پایه درباره رشته‌ها:

- **طول:** اگر $w = w_1 w_2 \cdots w_n$ باشد $(w_i \in \Sigma)$، طول آن $|w| = n$ است.
- **رشته تهی** ε رشته‌ای با طول صفر است. (رشته تهی همان نقش عدد صفر در جمع را دارد.)
- **معکوس:** $w^R = w_n w_{n-1} \cdots w_1$. مثلاً $abb^R = bba$، $a^R = a$ و $\varepsilon^R = \varepsilon$.
- **زیررشته:** z زیررشته w است اگر z به صورت **پیوسته** درون w بیاید. مثلاً cad زیررشته $abracadabra$ است، 0100 زیررشته 001000 است، اما 0000 زیررشته 001000 نیست.
- **الحاق:** اگر $x = x_1 \cdots x_m$ و $y = y_1 \cdots y_n$، آنگاه $xy = x_1 \cdots x_m y_1 \cdots y_n$. داریم $|xy| = |x| + |y|$ و $\varepsilon x = x\varepsilon = x$. همچنین $x^k = \underbrace{xx\cdots x}_{k}$، $x^0 = \varepsilon$ و $|x^k| = k|x|$.
- **پیشوند:** x پیشوند y است اگر رشته‌ای مثل z باشد که $xz = y$. اگر علاوه بر این $x \neq y$، به x **پیشوند سره** می‌گوییم. **پسوند** هم به همین شکل تعریف می‌شود: x پسوند y است اگر $zx = y$.

مثال ۳.۲. پیشوندهای $aaba$ عبارت‌اند از ε، a، aa، aab و $aaba$. به طور کلی رشته‌ای به طول n دقیقاً $n+1$ پیشوند دارد (پیشوندهایی به طول $0,1,\dots,n$). تنها پیشوند ε خودش است.

۱.۱.۲ ترتیب رشته‌ها

ترتیب لغت‌نامه‌ای همان ترتیب واژه‌ها در فرهنگ لغت است. اما در این درس معمولاً از **ترتیب رشته‌ای** (Shortlex Order) استفاده می‌کنیم: رشته‌های کوتاه‌تر جلوتر می‌آیند و رشته‌های هم‌طول به ترتیب لغت‌نامه‌ای مرتب می‌شوند:

$$(\varepsilon,\ 0,\ 1,\ 00,\ 01,\ 10,\ 11,\ 000,\ \dots).$$

نکته. چرا ترتیب لغت‌نامه‌ای کافی نیست؟ در ترتیب لغت‌نامه‌ای روی $\{0,1\}$ فهرست با $\varepsilon, 0, 00, 000, \dots$ شروع می‌شود و هیچ‌وقت به رشته 1 نمی‌رسد. اما در ترتیب رشته‌ای هر رشته بالاخره در یک جای مشخص فهرست قرار می‌گیرد. از این ویژگی بعداً برای فهرست کردن همه رشته‌ها استفاده می‌کنیم (مثلاً برای اثبات شمارا بودن Σ^*).

۲.۱.۲ زبان‌ها

تعریف ۴.۲ (زبان)

زبان (Language) هر مجموعه‌ای از رشته‌هاست. Σ^* مجموعه همه رشته‌ها روی Σ (از جمله ε) و $\Sigma^+ = \Sigma^* - \{\varepsilon\}$ است. پس هر زبان روی Σ زیرمجموعه‌ای از Σ^* است. زبانی که هیچ عضوش پیشوند سره عضو دیگری نباشد **بی‌پیشوند** (Prefix-free) نام دارد.

مثال ۵.۲. زبان‌های متناهی: $L_1 = \emptyset$، $L_2 = \{\varepsilon\}$ و $L_3 = \{a, aa, aba\}$. زبان‌های نامتناهی: $L_4 = \Sigma^*$، $L_5 = \Sigma^+$ و

$$L_6 = \{a^n b^n \mid n \geq 0\} = \{\varepsilon, ab, aabb, aaabbb, \dots\}.$$

دقت کنید! $\emptyset$ و $\{\varepsilon\}$ یکی نیستند! $\emptyset$ هیچ رشته‌ای ندارد، اما $\{\varepsilon\}$ یک عضو دارد: رشته تهی.

تعریف ۶.۲ (عملیات روی زبان‌ها)

فرض کنید L, L_1, L_2 زبان‌هایی روی Σ باشند.

- **اجتماع و اشتراک:** چون زبان‌ها مجموعه‌اند، همان اجتماع و اشتراک مجموعه‌ها.
- **متمم:** $\overline{L} = \Sigma^* - L$.
- **معکوس:** $L^R = \{w^R \mid w \in L\}$.
- **الحاق:** $L_1 \circ L_2 = \{xy \mid x \in L_1,\ y \in L_2\}$.
- **ستاره کلینی:** $L^* = \{x_1 x_2 \cdots x_k \mid k \geq 0,\ x_i \in L\}$. چون $k = 0$ مجاز است، همیشه $\varepsilon \in L^*$.

جدول زیر سه مفهوم اصلی را مقایسه می‌کند:

	می‌تواند تهی باشد؟	می‌تواند نامتناهی باشد؟
الفبا	×	×
رشته	✓	×
زبان	✓	✓

۲.۲ تعریف، قضیه و اثبات

تعریف ۷.۲ (واژه‌های پایه ریاضی)

- **تعریف** شیئی را که به کار می‌بریم و نماد آن را توضیح می‌دهد. تعریف باید دقیق باشد، چه برای مفهومی ساده مثل «مجموعه» و چه برای مفهومی پیچیده مثل «امنیت».
- **گزاره ریاضی** ویژگی‌ای از یک شیء را بیان می‌کند که ممکن است درست یا نادرست باشد، اما نباید مبهم باشد.
- **اثبات** استدلالی منطقی و قانع‌کننده است که درستی یک گزاره را بدون هیچ شکی نشان می‌دهد.
- **قضیه** گزاره‌ای است که درستی آن اثبات شده است. قضیه‌های کمکی را **لم** و قضیه‌هایی را که به‌سادگی از قضیه دیگری نتیجه می‌شوند **نتیجه** می‌نامیم.

۱.۲.۲ چطور اثبات پیدا کنیم؟

پیدا کردن اثبات همیشه ساده نیست، اما این راهکارها کمک می‌کنند:

۱. نمادها را کامل بفهمید و گزاره را با کلمات خودتان بازنویسی کنید.

۲. گزاره را به بخش‌های کوچک‌تر بشکنید:

- «P اگر و تنها اگر Q» یعنی $P \leftrightarrow Q$ و دو جهت دارد: جهت رفت $P \to Q$ و جهت برگشت $Q \to P$.
- «مجموعه‌های A و B برابرند» یعنی هر عضو A در B است و هر عضو B در A است.

۳. دنبال مثال نقض بگردید. اگر پیدا نشد، معمولاً دلیلش شما را به اثبات می‌رساند.

۴. حالت‌های ساده‌تر را امتحان کنید.

مثال ۸.۲. گزاره: «مجموع درجه‌های همه رأس‌های یک گراف بی‌جهت عددی زوج است.» با امتحان چند گراف کوچک (بدون یال: مجموع 0؛ یک یال: 2؛ دو یال: 4؛ سه یال: 6) می‌بینیم که **هر بار یک یال اضافه می‌کنیم، مجموع دقیقاً دو واحد زیاد می‌شود**. همین مشاهده ما را به اثبات (قضیه ۲.۲.۲) می‌رساند.

۲.۲.۲ چطور اثبات بنویسیم؟

یک اثبات خوب دنباله‌ای از جمله‌هاست که هر کدام به‌طور منطقی از قبلی‌ها نتیجه می‌شود. دقیق باشید، مرتب بنویسید و از شکل و متن ساده کمک بگیرید. اگر اثبات طولانی است، اول یک طرح کلی بدهید و بعد جزئیات را بنویسید. خلاصه: **مرتب و مختصر**.

قضیه ۹.۲ (دمورگان)

برای هر دو مجموعه A و B داریم $\overline{A \cup B} = \overline{A} \cap \overline{B}$.

اثبات. برابری دو مجموعه را با اثبات دو جهت نشان می‌دهیم.
جهت رفت ($\overline{A \cup B} \subseteq \overline{A} \cap \overline{B}$): فرض کنید $x \in \overline{A \cup B}$. پس $x \notin A \cup B$؛ یعنی x نه در A است و نه در B. پس $x \in \overline{A}$ و $x \in \overline{B}$، یعنی $x \in \overline{A} \cap \overline{B}$.

جهت برگشت ($\overline{A} \cap \overline{B} \subseteq \overline{A \cup B}$): فرض کنید $x \in \overline{A} \cap \overline{B}$. پس $x \notin A$ و $x \notin B$. بنابراین x در اجتماع آن‌ها هم نیست: $x \notin A \cup B$، یعنی $x \in \overline{A \cup B}$.

■

قضیه ۱۰.۲

در هر گراف بی‌جهت G مجموع درجه‌های همه رأس‌ها عددی زوج است.

اثبات. فرض کنید $G = (V, E)$ و $d(v)$ درجه رأس v باشد. هر یال $\{u, v\} \in E$ دقیقاً یک واحد به $d(u)$ و یک واحد به $d(v)$ اضافه می‌کند. پس هر یال دقیقاً 2 واحد در مجموع درجه‌ها سهم دارد و

$$\sum_{v \in V} d(v) = \sum_{\{u,v\} \in E} (1 + 1) = 2|E|,$$

که عددی زوج است.

■

۳.۲ انواع روش‌های اثبات

یک اثبات ممکن است از چند روش مختلف در بخش‌های مختلفش استفاده کند. روش‌های اصلی این‌ها هستند:

۱.۳.۲ اثبات مستقیم و غیرمستقیم

اثبات مستقیم بر قاعده‌ای به نام modus ponens («روش تأیید») تکیه دارد: اگر p و $p \to q$ ثابت شده باشند، q هم ثابت شده است. **اثبات غیرمستقیم** بر قاعده modus tollens («روش نفی») تکیه دارد: اگر $\neg q$ و $p \to q$ ثابت شده باشند، $\neg p$ هم ثابت شده است.

$$\frac{p \to q \quad p}{\therefore\ q} \qquad \frac{p \to q \quad \neg q}{\therefore\ \neg p}$$

یک کاربرد رایج روش غیرمستقیم **عکس نقیض** است: به جای اثبات $p \to q$، گزاره هم‌ارز $\neg q \to \neg p$ را ثابت می‌کنیم. مثلاً برای اثبات «اگر n^2 زوج باشد، n زوج است» کافی است نشان دهیم «اگر n فرد باشد، n^2 فرد است»: اگر $n = 2k + 1$ آنگاه $n^2 = 2(2k^2 + 2k) + 1$ فرد است.

۲.۳.۲ اثبات ساختنی

وقتی می‌خواهیم وجود شیئی را ثابت کنیم ($\exists x\, P(x)$)، می‌توانیم نشان دهیم آن شیء را چطور بسازیم: یعنی a را بسازیم و ثابت کنیم $P(a)$ درست است. در این درس این روش بسیار پرکاربرد است: برای اثبات منظم بودن یک زبان، ماشینی می‌سازیم که آن را بپذیرد.

قضیه ۱۱.۲

برای هر عدد زوج $n > 2$ یک گراف 3-منتظم (گرافی که درجه همه رأس‌هایش ۳ است) با n رأس وجود دارد.

اثبات. گراف $G=(V,E)$ را با $V=\{0,1,\dots,n-1\}$ و یال‌های زیر می‌سازیم:

$$E=\underbrace{\{\{i,i+1\}\mid 0\le i\le n-2\}\cup\{\{n-1,0\}\}}_{\text{یک دور}}\cup\underbrace{\{\{i,i+n/2\}\mid 0\le i\le n/2-1\}}_{\text{قطرها}}.$$

دسته اول یال‌ها رأس‌ها را روی یک دور قرار می‌دهد، پس هر رأس از این دسته دقیقاً ۲ یال دارد. دسته دوم هر رأس را به رأس روبه‌رویش روی دور وصل می‌کند. چون n زوج است، $n/2$ عدد صحیح است و هر رأس دقیقاً یک رأس روبه‌رو دارد. دقیق‌تر: رأس $i<n/2$ با یال $\{i,i+n/2\}$ و رأس $j\ge n/2$ با یال $\{j-n/2,j\}$ وصل می‌شود، پس هر رأس دقیقاً یک قطر دارد. چون $n>2$، داریم $n/2\ge 2$؛ پس رأس روبه‌رو هیچ‌وقت همسایه رأس روی دور نیست و یال تکراری ساخته نمی‌شود. در نتیجه درجه همه رأس‌ها $2+1=3$ است.

■

۳.۳.۲ برهان خلف

در برهان خلف فرض می‌کنیم گزاره نادرست است و نشان می‌دهیم این فرض به نتیجه‌ای نادرست (**تناقض**) می‌رسد. پس فرض باید غلط باشد. مثال روزمره: علی می‌بیند مریم کاملاً خشک از بیرون آمده است. اگر بیرون باران می‌بارید (فرض نادرست)، مریم خیس بود (نتیجه نادرست). پس باران نمی‌بارد.

قضیه ۱۲.۲

$\sqrt{2}$ گنگ است.

اثبات. با برهان خلف فرض کنید $\sqrt{2}$ گویا باشد. پس $\sqrt{2}=m/n$ برای اعداد صحیح m,n. می‌توانیم کسر را ساده کنیم تا m و n نسبت به هم اول باشند؛ در نتیجه دست‌کم یکی از آن‌ها فرد است. با ضرب دو طرف در n و به توان دو رساندن داریم

$$n\sqrt{2}=m \quad\Longrightarrow\quad 2n^2=m^2.$$

پس m^2 زوج است و در نتیجه m زوج است (مثال عکس نقیض بالا). بنویسید $m=2k$. با جایگذاری: $2n^2=4k^2$، یعنی $n^2=2k^2$. پس n هم زوج است. اما m و n هر دو زوج شدند، در حالی که فرض کرده بودیم دست‌کم یکی فرد است. تناقض.

■

۴.۳.۲ استقرا

استقرا روشی برای اثبات یک ویژگی برای همه عضوهای یک مجموعه نامتناهی مثل $\mathbb{N}$ است. اثبات دو قدم دارد:

- **پایه:** ویژگی را برای کوچک‌ترین عضو ثابت می‌کنیم، مثلاً $P(1)$. (لازم نیست حتماً از ۱ شروع کنیم.)
- **گام استقرا:** فرض می‌کنیم $P(k)$ درست است (**فرض استقرا**) و ثابت می‌کنیم $P(k+1)$ هم درست است.

در **استقرای قوی** فرض استقرا این است که ویژگی برای همه مقادیر کوچک‌تر درست است: $\bigwedge_{i\in\{1,\dots,k\}} P(i)\to P(k+1)$.

مثال ۱۳.۲ (فرمول اقساط وام). فرض کنید P مبلغ وام، P_t بدهی باقی‌مانده بعد از ماه tام ($P_0 = P$)، I نرخ سود سالانه، $M = 1 + I/12$ ضریب سود ماهانه و Y قسط ماهانه باشد. هر ماه بدهی به خاطر سود در M ضرب و به اندازه قسط کم می‌شود: $P_t = MP_{t-1} - Y$.

قضیه ۱۴.۲

برای هر $t \geq 0$: $P_t = PM^t - Y\left(\frac{M^t - 1}{M - 1}\right)$.

اثبات. با استقرا روی t. **پایه** ($t = 0$): $PM^0 - Y\frac{M^0-1}{M-1} = P - 0 = P = P_0$. **گام استقرا:** فرض کنید فرمول برای k درست باشد. آنگاه

$$\begin{aligned} P_{k+1} = P_k M - Y &= \left[PM^k - Y\left(\frac{M^k - 1}{M - 1}\right)\right] M - Y \\ &= PM^{k+1} - Y\left(\frac{M^{k+1} - M}{M - 1}\right) - Y\left(\frac{M - 1}{M - 1}\right) \\ &= PM^{k+1} - Y\left(\frac{M^{k+1} - 1}{M - 1}\right). \end{aligned}$$

■

قضیه ۱۵.۲

یک درخت دودویی به ارتفاع n حداکثر 2^n برگ دارد.

اثبات. فرض کنید $l(n)$ بیشترین تعداد برگ در یک درخت دودویی به ارتفاع n باشد. (ارتفاع، طول بلندترین مسیر از ریشه تا یک برگ است.) **پایه:** درختی به ارتفاع 0 فقط ریشه را دارد، پس $l(0) = 1 \leq 2^0$. **گام استقرا:** فرض کنید $l(j) \leq 2^j$ برای هر $j \leq k$. ریشه یک درخت به ارتفاع $k + 1$ حداکثر دو فرزند دارد و زیردرخت هر فرزند ارتفاعی حداکثر k دارد. برگ‌های درخت همان برگ‌های این زیردرخت‌ها هستند، پس

$$l(k + 1) \leq 2 \cdot 2^k = 2^{k+1}.$$

■

قضیه ۱۶.۲

اگر n خط راست در صفحه داشته باشیم که هر دو خط یکدیگر را قطع کنند و هیچ سه خطی از یک نقطه نگذرند، صفحه به

$$A(n) = \frac{n(n + 1)}{2} + 1$$

ناحیه تقسیم می‌شود.

اثبات. پایه: یک خط صفحه را به $A(1) = 2$ ناحیه تقسیم می‌کند و فرمول هم $\frac{1\cdot 2}{2} + 1 = 2$ می‌دهد.
مشاهده کلیدی: خط $(k + 1)$ام همه k خط قبلی را در k نقطه متمایز قطع می‌کند. این k نقطه، خط جدید را به

$k+1$ تکه تقسیم می‌کنند و هر تکه یک ناحیه قبلی را به دو ناحیه می‌شکند. پس $A(k+1) = A(k) + k + 1$.

گام استقرا: با فرض $A(k) = \frac{k(k+1)}{2} + 1$:

$$A(k+1) = \frac{k(k+1)}{2} + 1 + k + 1 = \frac{k(k+1) + 2(k+1)}{2} + 1 = \frac{(k+1)(k+2)}{2} + 1.$$

■

جمع‌بندی فصل

- الفبا مجموعه‌ای متناهی و ناتهی است؛ رشته دنباله‌ای متناهی از نمادهاست؛ زبان مجموعه‌ای (شاید نامتناهی) از رشته‌هاست.
- عملیات اصلی روی زبان‌ها: اجتماع، اشتراک، متمم، معکوس، الحاق و ستاره.
- ترتیب رشته‌ای (اول کوتاه‌ترها) اجازه می‌دهد همه رشته‌ها را فهرست کنیم.
- روش‌های اثبات: مستقیم، غیرمستقیم (عکس نقیض)، ساختنی، برهان خلف و استقرا.

بخش دوم

زبان‌های منظم

۳ فصل

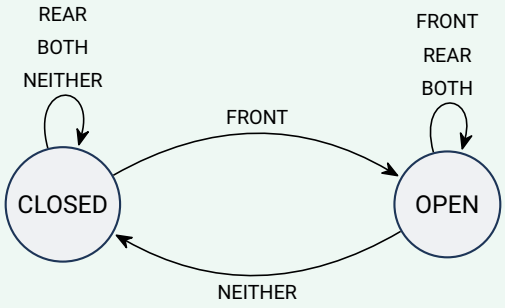
ماشین‌های متناهی قطعی

کامپیوتر واقعی آن‌قدر پیچیده است که نمی‌توان مستقیماً نظریه‌ای ریاضی درباره آن ساخت. به جای آن از **مدل محاسباتی** (Computational Model) استفاده می‌کنیم: یک کامپیوتر ایدئال و ساده‌شده. برای بررسی جنبه‌های مختلف، مدل‌های مختلفی به کار می‌بریم. ساده‌ترین مدل **ماشین متناهی** (Finite Automaton) است: ماشینی که رشته ورودی را می‌خواند و در پایان فقط می‌گوید «می‌پذیرم» یا «رد می‌کنم».

۱.۳ ایده ماشین متناهی

ماشین متناهی کامپیوتری با حافظه **بسیار محدود** است. حافظه آن فقط «حالتی» است که در آن قرار دارد.

مثال ۱.۳ (کنترلر در خودکار). یک در خودکار فروشگاه را در نظر بگیرید که یک حسگر جلوی در و یک حسگر پشت آن دارد. کنترلر دو **حالت** دارد: OPEN و CLOSED، پس فقط به یک بیت حافظه نیاز دارد. ورودی در هر لحظه یکی از چهار مقدار است: FRONT (کسی جلوی در است)، REAR (کسی پشت در است)، BOTH و NEITHER.



همین رفتار را می‌توان با یک **جدول گذار** هم نشان داد:

	NEITHER	FRONT	REAR	BOTH
CLOSED	CLOSED	OPEN	CLOSED	CLOSED
OPEN	CLOSED	OPEN	OPEN	OPEN

کنترلر آسانسور و ترموستات الکترونیکی هم مثال‌های دیگری از ماشین متناهی هستند. ویژگی‌های مشترک همه آن‌ها:

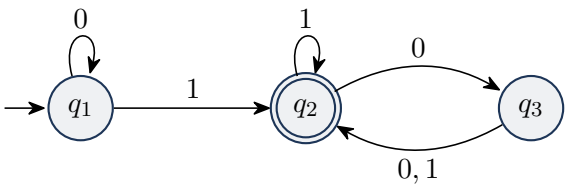
- مجموعه‌ای **متناهی** از حالت‌ها دارند و این حالت‌ها تنها حافظه ماشین‌اند.
- از یک حالت ثابت (**حالت شروع**) شروع می‌کنند.
- ورودی را **نماد به نماد** می‌خوانند و با هر نماد حالتشان را عوض می‌کنند.

- اگر بعد از خواندن کل ورودی در یک **حالت پذیرش** باشند، ورودی را می‌پذیرند.

این مدل ساده نقطه شروع خوبی است: اول آن را دقیق تعریف می‌کنیم، بعد یاد می‌گیریم چطور با آن کار کنیم و در آخر توانایی‌ها و محدودیت‌هایش را بررسی می‌کنیم.

۲.۳ نمودار حالت و تعریف صوری

شکل زیر **نمودار حالت** ماشینی به نام M_1 را نشان می‌دهد:



- هر **حالت** یک دایره با یک نام است. **حالت شروع** با پیکانی که از هیچ‌جا به آن وارد می‌شود و **حالت پذیرش** با دایره دوخطی مشخص شده است.
- هر پیکان یک **گذار** است. ماشین با خواندن نمادی که روی پیکان نوشته شده، در جهت پیکان حرکت می‌کند.
- **ورودی** یک رشته است، مثلاً 1101. ماشین آن را از چپ به راست، یکی‌یکی می‌خواند.
- **خروجی** بعد از خواندن آخرین نماد مشخص می‌شود: اگر ماشین در حالت پذیرش باشد «پذیرش» و گرنه «رد».

مثلاً روی ورودی 1101: از q_1 شروع می‌کنیم، با 1 به q_2 می‌رویم، با 1 در q_2 می‌مانیم، با 0 به q_3 و با 1 دوباره به q_2 می‌رویم. چون q_2 حالت پذیرش است، ورودی پذیرفته می‌شود.

تعریف ۲.۳ (ماشین متناهی قطعی (DFA))

یک **ماشین متناهی قطعی** (Deterministic Finite Automaton) یک پنج‌تایی $(Q, \Sigma, \delta, q_0, F)$ است که در آن

۱. Q مجموعه‌ای متناهی از **حالت‌ها**،

۲. Σ **الفبای** ورودی،

۳. $\delta : Q \times \Sigma \to Q$ **تابع گذار**،

۴. $q_0 \in Q$ **حالت شروع**، و

۵. $F \subseteq Q$ مجموعه **حالت‌های پذیرش** است.

کلمه «قطعی» یعنی: در هر حالت و با هر نماد، **دقیقاً یک** حالت بعدی مشخص وجود دارد. چون δ یک تابع روی کل $Q \times \Sigma$ است، از هر حالت برای هر نماد دقیقاً یک پیکان خارج می‌شود.

مثال ۳.۳ (توصیف صوری M_1).

- $Q = \{q_1, q_2, q_3\}$ و $\Sigma = \{0, 1\}$،
- تابع گذار δ با جدول زیر،
- حالت شروع q_1 و $F = \{q_2\}$.

	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2

تعریف ۴.۳ (زبان یک ماشین)

اگر A مجموعه همه رشته‌هایی باشد که ماشین M می‌پذیرد، می‌گوییم A **زبان ماشین** M است و می‌نویسیم $L(M) = A$. همچنین می‌گوییم M، A را **تشخیص می‌دهد** یا **می‌پذیرد**.

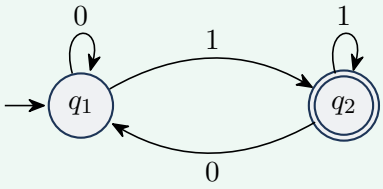
نکته. یک ماشین ممکن است چندین رشته را بپذیرد، اما همیشه **دقیقاً یک زبان** را تشخیص می‌دهد. اگر ماشین هیچ رشته‌ای را نپذیرد، زبانش $\emptyset$ است.

زبان M_1 این است:

$$L(M_1) = \{w \mid w \text{ دست‌کم یک } 1 \text{ دارد و بعد از آخرین } 1 \text{ تعداد زوجی } 0 \text{ آمده است}\}.$$

دلیل: تا وقتی 1 ندیده‌ایم در q_1 می‌مانیم. بعد از دیدن هر 1 به q_2 می‌رویم. از آن به بعد، 0ها ما را بین q_2 و q_3 جابه‌جا می‌کنند (اگر بعد از 0 یک 1 بیاید از q_3 به q_2 برمی‌گردیم، که همان «1 جدید» است). پس در پایان در q_2 هستیم اگر و تنها اگر بعد از آخرین 1 تعداد زوجی 0 آمده باشد.

مثال ۵.۳ (ماشین M_2). ماشین زیر با $Q = \{q_1, q_2\}$، $\Sigma = \{0, 1\}$، حالت شروع q_1 و $F = \{q_2\}$ را در نظر بگیرید:



	0	1
q_1	q_1	q_2
q_2	q_1	q_2

رفتار ماشین روی دو ورودی:

- ورودی 1101: $q_1 \xrightarrow{1} q_2 \xrightarrow{1} q_2 \xrightarrow{0} q_1 \xrightarrow{1} q_2$. ماشین در q_2 تمام می‌کند، پس **می‌پذیرد**.
- ورودی 1010: $q_1 \xrightarrow{1} q_2 \xrightarrow{0} q_1 \xrightarrow{1} q_2 \xrightarrow{0} q_1$. ماشین در q_1 تمام می‌کند، پس **رد می‌کند**.

در واقع حالت ماشین همیشه نشان می‌دهد آخرین نماد خوانده‌شده چه بوده است: q_2 یعنی «آخرین نماد 1 بوده». پس $L(M_2) = \{w \mid w \text{ به } 1 \text{ ختم می‌شود}\}$.

اگر در همین ماشین فقط حالت پذیرش را عوض کنیم و $F = \{q_1\}$ بگذاریم، ماشین M_3 به دست می‌آید با $L(M_3) = \{w \mid w = \varepsilon \text{ یا } w \text{ به } 0 \text{ ختم می‌شود}\}$.

۳.۳ تعریف صوری محاسبه

تا اینجا «پذیرفتن» را به شکل شهودی توضیح دادیم. حالا آن را دقیق تعریف می‌کنیم.

تعریف ۶.۳ (محاسبه DFA)

فرض کنید $M = (Q, \Sigma, \delta, q_0, F)$ یک DFA و $w = w_1 w_2 \cdots w_n$ رشته‌ای روی Σ باشد. M رشته w را **می‌پذیرد** اگر دنباله‌ای از حالت‌ها $r_0, r_1, \ldots, r_n$ در Q با سه شرط زیر وجود داشته باشد:

۱. $r_0 = q_0$ (از حالت شروع شروع می‌کنیم)

۲. $\delta(r_i, w_{i+1}) = r_{i+1}$ برای $i = 0, \ldots, n-1$ (طبق قوانین حرکت می‌کنیم)

۳. $r_n \in F$ (در حالت پذیرش تمام می‌کنیم)

می‌گوییم M زبان A را تشخیص می‌دهد اگر $A = \{w \mid M \text{ رشته } w \text{ را می‌پذیرد}\}$.

مثال ۷.۳. برای ماشین M_2:

Input	States $r_0, \ldots, r_n$	Output
ε	q_1	reject
1	q_1, q_2	accept
000	q_1, q_1, q_1, q_1	reject
011	q_1, q_1, q_2, q_2	accept
1110	q_1, q_2, q_2, q_2, q_1	reject

برای اثبات‌ها، یک ابزار کوچک خیلی کمک می‌کند: **تابع گذار گسترش‌یافته** که به جای یک نماد، یک رشته کامل می‌گیرد.

تعریف ۸.۳ (تابع گذار گسترش‌یافته)

برای DFA M، تابع $\delta^* : Q \times \Sigma^* \to Q$ را به صورت بازگشتی تعریف می‌کنیم:

$$\delta^*(q, \varepsilon) = q, \qquad \delta^*(q, wa) = \delta\big(\delta^*(q, w), a\big) \quad (w \in \Sigma^*,\ a \in \Sigma).$$

یعنی $\delta^*(q, w)$ حالتی است که ماشین با شروع از q و خواندن کل w به آن می‌رسد.

با این نماد، دنباله $r_0, \ldots, r_n$ در تعریف بالا همان $r_i = \delta^*(q_0, w_1 \cdots w_i)$ است. پس

$$M \text{ رشته } w \text{ را می‌پذیرد} \iff \delta^*(q_0, w) \in F.$$

۴.۳ زبان‌های منظم و طراحی DFA

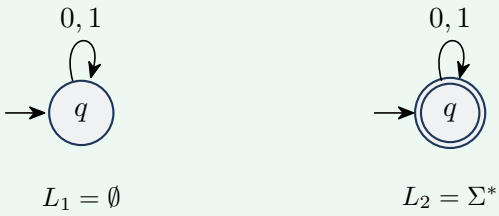
تعریف ۹.۳ (زبان منظم)

زبانی را **منظم** (Regular) می‌نامیم اگر یک ماشین متناهی وجود داشته باشد که آن را تشخیص دهد.

برای اثبات منظم بودن یک زبان کافی است یک DFA برایش بسازیم (اثبات ساختنی). هنگام طراحی، از خودتان بپرسید: «برای تصمیم‌گیری درباره ورودی، چه اطلاعاتی را باید **تا اینجا** به خاطر داشته باشم؟» هر حالت ممکن از این اطلاعات یک حالت ماشین می‌شود.

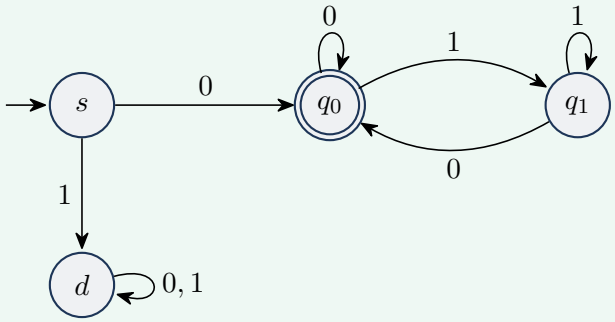
مثال ۱۰.۳ (چهار زبان منظم روی $\Sigma = \{0, 1\}$).

(۱) $L_1 = \emptyset$: یک حالت که پذیرش نیست و با هر نماد در خودش می‌ماند. **(۲)** $L_2 = \Sigma^*$: همان ماشین، اما حالتش پذیرش باشد.

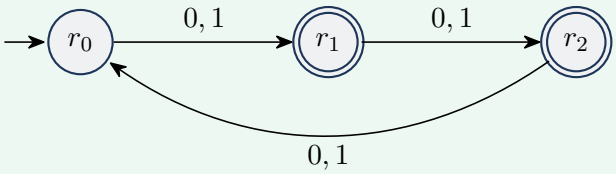


$L_1 = \emptyset$ $L_2 = \Sigma^*$

(۳) $L_3 = \{w \mid w$ با 0 شروع و به 0 ختم می‌شود$\}$: اولین نماد را چک می‌کنیم. اگر 1 بود به یک **حالت تله** (حالتی که هرگز از آن خارج نمی‌شویم) می‌رویم. اگر 0 بود، از آن به بعد فقط باید آخرین نماد را به خاطر بسپاریم:



(۴) $L_4 = \{w \mid |w|$ مضرب ۳ نیست$\}$: فقط باقی‌مانده طول بر ۳ مهم است، پس سه حالت r_0, r_1, r_2 کافی است:



۵.۳ عملیات منظم و بستاری

نظریه محاسبه فقط ساختن ماشین نیست؛ می‌خواهیم ویژگی‌های کلی زبان‌های منظم را هم بشناسیم. یک ابزار مهم، **عملیات** روی زبان‌هاست. همان‌طور که در حساب با عملگرهایی مثل $+$ و $\times$ اعداد را ترکیب می‌کنیم، اینجا زبان‌ها را با عملیات زیر ترکیب می‌کنیم.

تعریف ۱۱.۳ (عملیات منظم)

اگر A و B دو زبان باشند:

- **اجتماع:** $A \cup B = \{x \mid x \in A \text{ یا } x \in B\}$،
- **الحاق:** $A \circ B = \{xy \mid x \in A,\ y \in B\}$،
- **ستاره:** $A^* = \{x_1 x_2 \cdots x_k \mid k \geq 0,\ x_i \in A\}$.

مثال ۱۲.۳. روی $\Sigma = \{a, \ldots, z\}$ فرض کنید $A = \{good, bad\}$ و $B = \{boy, girl\}$. آنگاه

$$A \cup B = \{good, bad, boy, girl\},$$
$$A \circ B = \{goodboy, goodgirl, badboy, badgirl\},$$
$$A^* = \{\varepsilon, good, bad, goodgood, goodbad, badgood, badbad, goodgoodgood, goodgoodbad, \ldots\}.$$

تعریف ۱۳.۳ (بستاری)

یک دسته از اشیاء تحت یک عملیات **بسته** است اگر اعمال آن عملیات روی عضوهای دسته، همیشه عضوی از همان دسته بدهد.

مثلاً اعداد طبیعی تحت ضرب بسته‌اند ولی تحت تقسیم نه ($1/2 \notin \mathbb{N}$). در ادامه ثابت می‌کنیم دسته زبان‌های منظم تحت **متمم**، **اجتماع**، **اشتراک**، **الحاق** و **ستاره** بسته است.

۱.۵.۳ بستاری تحت متمم

قضیه ۱۴.۳

متمم هر زبان منظم، منظم است.

ایده اثبات. چون A منظم است، یک DFA مثل M_1 آن را تشخیص می‌دهد. ماشین M_2 را درست مثل M_1 می‌سازیم، فقط حالت‌های پذیرش و غیرپذیرش را جابه‌جا می‌کنیم. دو ماشین ورودی را دقیقاً یکسان طی می‌کنند، اما هر جا M_1 می‌پذیرد M_2 رد می‌کند و برعکس.

اثبات. فرض کنید $M_1 = (Q, \Sigma, \delta, q_1, F)$ یک DFA با $L(M_1) = A$ باشد. قرار دهید

$$M_2 = (Q, \Sigma, \delta, q_1,\ Q - F).$$

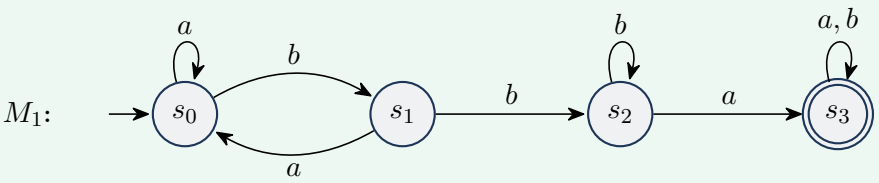
اثبات درستی: دو ماشین حالت‌ها، حالت شروع و تابع گذار یکسانی دارند، پس تابع گذار گسترش‌یافته آن‌ها هم یکسان است. برای هر رشته w فرض کنید $q = \delta^*(q_1, w)$. آنگاه

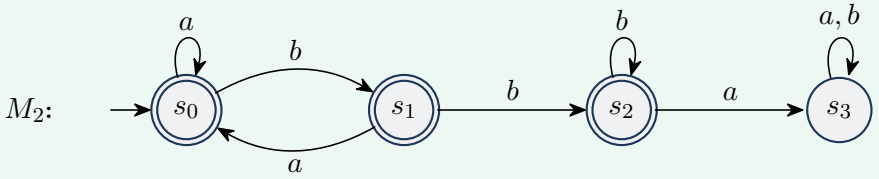
$$w \in L(M_2) \iff q \in Q - F \iff q \notin F \iff w \notin L(M_1) \iff w \in \overline{A}.$$

پس $L(M_2) = \overline{A}$ و $\overline{A}$ منظم است.

■

مثال ۱۵.۳ (وجود زیررشته bba). ماشین M_1 زیر زبان $\{w \mid bba \text{ است } w \text{ زیررشته}\}$ را تشخیص می‌دهد. حالت‌ها نشان می‌دهند چه مقدار از bba را تا اینجا دیده‌ایم. M_2 همان ماشین با حالت‌های پذیرش جابه‌جاشده است و زبان $\{w \mid bba \text{ نیست } w \text{ زیررشته}\}$ را تشخیص می‌دهد:





روی ورودی $aabb$ هر دو ماشین مسیر $s_0 \xrightarrow{a} s_0 \xrightarrow{a} s_0 \xrightarrow{b} s_1 \xrightarrow{b} s_2$ را طی می‌کنند. M_1 رد می‌کند و M_2 می‌پذیرد.

دقت کنید! این ساخت فقط برای ماشین **قطعی** درست است. در ماشین‌های غیرقطعی فصل بعد، جابه‌جا کردن حالت‌های پذیرش در حالت کلی متمم را نمی‌دهد.

۲.۵.۳ بستاری تحت اجتماع و اشتراک

قضیه ۱۶.۳

اگر A_1 و A_2 منظم باشند، $A_1 \cup A_2$ هم منظم است.

ایده اثبات. M_1 و M_2 ماشین‌های A_1 و A_2 هستند. آیا ماشین جدید می‌تواند اول M_1 را روی ورودی اجرا کند و بعد M_2 را؟ نه، چون ماشین متناهی نمی‌تواند نوار ورودی را به عقب برگرداند! راه درست این است که دو ماشین را **هم‌زمان** شبیه‌سازی کنیم: حالت ماشین جدید یک **زوج** است که نشان می‌دهد M_1 و M_2 هر کدام در کدام حالت‌اند. تعداد این زوج‌ها متناهی است ($|Q_1| \times |Q_2|$)، پس ماشین جدید هم متناهی است. به این روش **ساخت ضربی** می‌گوییم.

اثبات. فرض کنید $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ زبان A_1 و $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ زبان A_2 را تشخیص دهند. ماشین $M = (Q, \Sigma, \delta, q_0, F)$ را این‌طور می‌سازیم:

۱. $Q = Q_1 \times Q_2 = \{(r_1, r_2) \mid r_1 \in Q_1,\ r_2 \in Q_2\}$،

۲. $\delta\big((r_1, r_2), a\big) = \big(\delta_1(r_1, a),\ \delta_2(r_2, a)\big)$ برای هر $a \in \Sigma$،

۳. $q_0 = (q_1, q_2)$،

۴. $F = \{(r_1, r_2) \mid r_1 \in F_1 \text{ یا } r_2 \in F_2\}$.

اثبات درستی. اول با استقرا روی طول w نشان می‌دهیم برای هر $w \in \Sigma^*$:

$$\delta^*\big((q_1, q_2), w\big) = \big(\delta_1^*(q_1, w),\ \delta_2^*(q_2, w)\big). \qquad (\star)$$

پایه ($w = \varepsilon$): هر دو طرف برابر (q_1, q_2) هستند. **گام:** فرض کنید $(\star)$ برای w درست باشد و $a \in \Sigma$. اگر $\delta_1^*(q_1, w) = r_1$ و $\delta_2^*(q_2, w) = r_2$، آنگاه

$$\delta^*\big((q_1, q_2), wa\big) = \delta\big((r_1, r_2), a\big) = \big(\delta_1(r_1, a), \delta_2(r_2, a)\big) = \big(\delta_1^*(q_1, wa), \delta_2^*(q_2, wa)\big).$$

حالا با $(\star)$:

$$w \in L(M) \iff \delta_1^*(q_1, w) \in F_1 \text{ یا } \delta_2^*(q_2, w) \in F_2 \iff w \in A_1 \text{ یا } w \in A_2 \iff w \in A_1 \cup A_2.$$

■

نتیجه ۱۷.۳

زبان‌های منظم تحت **اشتراک** و **تفاضل** هم بسته‌اند.

اثبات. در همان ساخت ضربی، اگر حالت‌های پذیرش را $F = F_1 \times F_2$ بگذاریم (یعنی هر دو ماشین بپذیرند)، طبق $(\star)$ ماشین $A_1 \cap A_2$ را تشخیص می‌دهد. برای تفاضل هم $A_1 - A_2 = A_1 \cap \overline{A_2}$ و از بستاری تحت متمم و اشتراک استفاده می‌کنیم.

■

۳.۵.۳ الحاق: نیاز به ابزاری جدید

آیا می‌توانیم بستاری تحت الحاق را هم با همین روش ثابت کنیم؟ ماشین باید رشته را به دو تکه بشکند طوری که M_1 تکه اول و M_2 تکه دوم را بپذیرد. مشکل این است که ماشین **نمی‌داند کجا باید رشته را بشکند**. برای حل این مشکل به تکنیک جدیدی به نام **نامعینی** نیاز داریم که موضوع فصل بعد است.

جمع‌بندی فصل

- DFA پنج‌تایی $(Q, \Sigma, \delta, q_0, F)$ است که در هر حالت با هر نماد دقیقاً یک حرکت دارد.
- M رشته w را می‌پذیرد اگر و تنها اگر $\delta^*(q_0, w) \in F$. زبانی منظم است که یک DFA آن را تشخیص دهد.
- برای طراحی DFA بپرسید «چه اطلاعاتی باید به خاطر بسپارم؟»

- متمم: جابه‌جا کردن F و $Q - F$. اجتماع و اشتراک: ساخت ضربی با حالت‌های زوجی.

۴ فصل

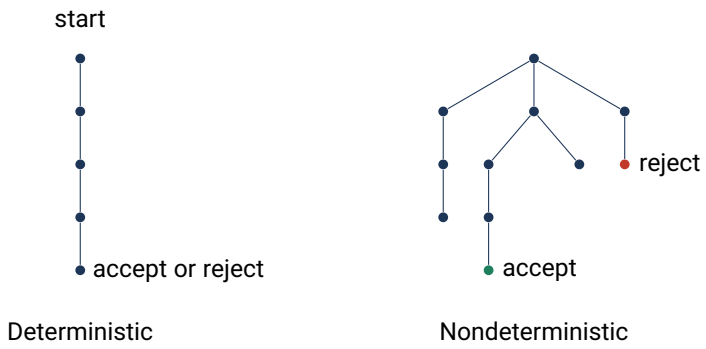
نامعینی و ماشین‌های متناهی غیرقطعی

در ماشین قطعی، حرکت بعدی همیشه به‌طور یکتا مشخص است. در این فصل این شرط را برمی‌داریم و به ماشین اجازه می‌دهیم در یک لحظه چند انتخاب داشته باشد. این کار ماشین را قوی‌تر نمی‌کند (در پایان فصل ثابت می‌کنیم)، اما طراحی ماشین و اثبات قضیه‌ها را خیلی ساده‌تر می‌کند.

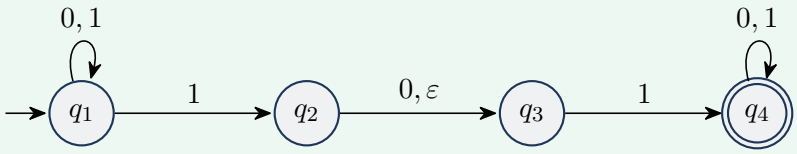
۱.۴ ایده نامعینی

ماشین متناهی غیرقطعی (Nondeterministic Finite Automaton) یا NFA تعمیم DFA است؛ یعنی هر DFA یک NFA هم هست. تفاوت‌ها:

- **چند گذار برای یک نماد:** از یک حالت ممکن است با یک نماد چند پیکان خارج شود. در این صورت ماشین به چند **نسخه** تقسیم می‌شود و هر نسخه یکی از راه‌ها را دنبال می‌کند؛ مثل محاسبه موازی.
- **هیچ گذاری برای یک نماد:** اگر از حالت فعلی با نماد ورودی هیچ پیکانی نباشد، آن نسخه (آن **شاخه** از محاسبه) **می‌میرد**.
- **گذار ε:** پیکانی با برچسب ε اجازه می‌دهد ماشین **بدون خواندن نماد** حالتش را عوض کند. (در واقع ماشین هم در حالت فعلی می‌ماند و هم نسخه‌ای از آن به سر دیگر پیکان می‌رود.)
- **پذیرش:** اگر **دست‌کم یکی** از نسخه‌ها در پایان ورودی در حالت پذیرش باشد، ماشین ورودی را می‌پذیرد.



مثال ۱.۴ (ماشین N_1). ماشین زیر را در نظر بگیرید. از q_1 با نماد 1 دو انتخاب داریم و از q_2 یک گذار ε هست:



محاسبه N_1 روی ورودی 010110 را با نوشتن مجموعه حالت‌های **فعال** (حالت‌هایی که دست‌کم یک نسخه در آن‌هاست) بعد از هر نماد دنبال می‌کنیم:

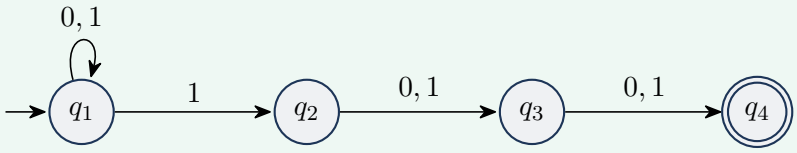
read	–	0	1	0	1	1	0
active	$\{q_1\}$	$\{q_1\}$	$\{q_1, q_2, q_3\}$	$\{q_1, q_3\}$	$\{q_1, q_2, q_3, q_4\}$	$\{q_1, q_2, q_3, q_4\}$	$\{q_1, q_3, q_4\}$

مثلاً بعد از خواندن اولین 1، نسخه‌ای که در q_1 بود دو نسخه می‌شود (یکی در q_1 و یکی در q_2) و نسخه q_2 بلافاصله با گذار ε یک نسخه هم در q_3 می‌سازد. نسخه‌ای که در q_3 است با خواندن 0 می‌میرد، چون از q_3 با 0 پیکانی نیست. چون در پایان q_4 فعال است، N_1 ورودی را می‌پذیرد. در واقع N_1 دقیقاً رشته‌هایی را می‌پذیرد که 101 یا 11 زیررشته آن‌ها باشد.

۱.۱.۴ ماشین غیرقطعی می‌تواند «حدس بزند»

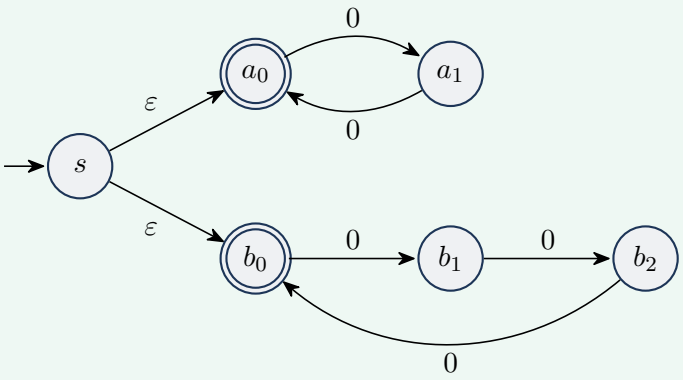
یک راه مفید برای فکر کردن به نامعینی این است: ماشین در لحظه مناسب «حدس می‌زند» چه کند و اگر حدس درستی وجود داشته باشد، ورودی را می‌پذیرد.

مثال ۲.۴ (سومین نماد از آخر). زبان $A = \{w \mid$ سومین نماد w از آخر، 1 است$\}$. ماشین زیر در q_1 می‌ماند و در لحظه‌ای که فکر می‌کند به سومین نماد از آخر رسیده، با خواندن 1 «حدس می‌زند» و به q_2 می‌رود. سپس دقیقاً دو نماد دیگر باید بیاید:

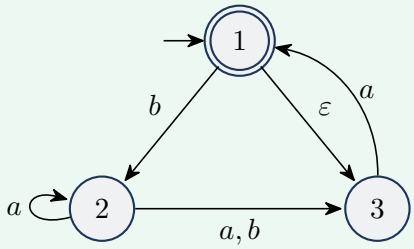


یک DFA برای همین زبان باید سه نماد آخر را به خاطر بسپارد و به $2^3 = 8$ حالت نیاز دارد.

مثال ۳.۴ (مضرب ۲ یا ۳). زبان $A = \{0^{2k} \mid k \geq 0\} \cup \{0^{3k} \mid k \geq 0\}$ روی الفبای $\{0\}$. ماشین در ابتدا با گذار ε «حدس می‌زند» که طول ورودی مضرب ۲ است یا مضرب ۳، و بعد آن را بررسی می‌کند:



مثال ۴.۴ (ماشین N_4).



این ماشین رشته‌های ε، a، $baba$ و baa را می‌پذیرد، اما b، bb و $babba$ را رد می‌کند. مثلاً برای a: از 1 با ε به 3 می‌رویم و با a به 1 برمی‌گردیم که پذیرش است. برای b: تنها راه رفتن به 2 است که پذیرش نیست.

۲.۴ تعریف صوری NFA

برای نوشتن تعریف صوری از دو نماد استفاده می‌کنیم: $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$ و $\mathcal{P}(Q)$ (مجموعه توانی Q). تفاوت اصلی با DFA در تابع گذار است: در NFA، خروجی تابع گذار یک **مجموعه** از حالت‌هاست (که ممکن است تهی باشد) و ورودی آن می‌تواند ε هم باشد.

تعریف ۵.۴ (NFA)

یک ماشین متناهی غیرقطعی یک پنج‌تایی $(Q, \Sigma, \delta, q_0, F)$ است که در آن

۱. Q مجموعه‌ای متناهی از حالت‌ها،

۲. Σ الفبای متناهی،

۳. $\delta : Q \times \Sigma_\varepsilon \to \mathcal{P}(Q)$ تابع گذار،

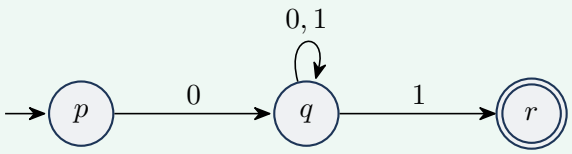
۴. $q_0 \in Q$ حالت شروع، و

۵. $F \subseteq Q$ مجموعه حالت‌های پذیرش است.

مثال ۶.۴ (توصیف صوری N_1). $Q = \{q_1, q_2, q_3, q_4\}$، $\Sigma = \{0, 1\}$، حالت شروع q_1، $F = \{q_4\}$ و تابع گذار:

	0	1	ε
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\emptyset$	$\{q_3\}$
q_3	$\emptyset$	$\{q_4\}$	$\emptyset$
q_4	$\{q_4\}$	$\{q_4\}$	$\emptyset$

مثال ۷.۴ (چند NFA ساده روی $\Sigma = \{0, 1\}$). برای $\emptyset$ یک حالت غیرپذیرش بدون هیچ گذاری کافی است. برای $\{\varepsilon\}$ یک حالت پذیرش بدون هیچ گذاری کافی است: هر نمادی که بیاید، تنها شاخه می‌میرد. برای $\{w \mid w$ با 0 شروع و به 1 ختم می‌شود$\}$:



ماشین روی آخرین 1 «حدس می‌زند» که ورودی تمام شده است. اگر حدس غلط باشد، آن شاخه در r می‌میرد و ضرری ندارد.

تعریف ۸.۴ (محاسبه NFA)

فرض کنید $N = (Q, \Sigma, \delta, q_0, F)$ یک NFA باشد. رشته w را می‌پذیرد اگر بتوان w را به صورت $w = y_1 y_2 \cdots y_m$ نوشت که هر $y_i \in \Sigma_\varepsilon$ (یعنی بین نمادها می‌توان هر تعداد ε گذاشت) و دنباله‌ای از حالت‌ها $r_0, r_1, \ldots, r_m$ در Q با این شرایط وجود داشته باشد:

۱. $r_0 = q_0$،
۲. $r_{i+1} \in \delta(r_i, y_{i+1})$ برای $i = 0, \ldots, m-1$ و،
۳. $r_m \in F$.

تفاوت با تعریف DFA در شرط دوم است: r_{i+1} باید **یکی از** حالت‌های مجاز بعدی باشد، نه تنها حالت بعدی.

۳.۴ هم‌ارزی NFA و DFA

دو ماشین را **هم‌ارز** می‌گوییم اگر یک زبان را تشخیص دهند. شاید فکر کنید NFA قوی‌تر است، اما قضیه زیر نشان می‌دهد این‌طور نیست.

قضیه ۹.۴

هر NFA یک DFA هم‌ارز دارد.

ایده اثبات. یک DFA می‌سازیم که NFA را شبیه‌سازی کند. DFA همان کاری را می‌کند که ما در مثال ۱.۴ با دست کردیم: **مجموعه حالت‌های فعال** را دنبال می‌کند. اگر NFA k حالت داشته باشد، 2^k زیرمجموعه از حالت‌هایش وجود دارد، پس DFA حداکثر 2^k حالت دارد که باز هم متناهی است. به این روش **ساخت زیرمجموعه‌ای** می‌گوییم.

اثبات. فرض کنید $N = (Q, \Sigma, \delta, q_0, F)$ زبان A را تشخیص دهد. ابتدا حالت ساده‌تر را بررسی می‌کنیم.

حالت ۱: N گذار ε ندارد. DFA $M = (Q', \Sigma, \delta', q_0', F')$ را می‌سازیم:

۱. $Q' = \mathcal{P}(Q)$؛ هر حالت M یک مجموعه از حالت‌های N است.

۲. برای $R \in Q'$ و $a \in \Sigma$: $\delta'(R, a) = \{q \in Q \mid q \in \delta(r, a) \text{ برای یک } r \in R\} = \bigcup_{r \in R} \delta(r, a)$.

۳. $q_0' = \{q_0\}$.

۴. $F' = \{R \in Q' \mid R \text{ دارد } N \text{ را یک حالت پذیرش دست‌کم}\}$.

حالت ۲: N گذار ε دارد. برای هر $R \subseteq Q$ تعریف کنید

$$E(R) = \{q \mid q \text{ از یک عضو } R \text{ با صفر یا چند پیکان } \varepsilon \text{ قابل دسترسی است}\}.$$

به $E(R)$ **بستار** ε مجموعه R می‌گوییم. ساخت را این‌طور اصلاح می‌کنیم:

- $\delta'(R, a) = \{q \in Q \mid q \in E(\delta(r, a)) \text{ برای یک } r \in R\}$؛ یعنی بعد از هر حرکت، همه حالت‌هایی را که با ε در دسترس‌اند هم اضافه می‌کنیم.
- $q_0' = E(\{q_0\})$؛ چون قبل از خواندن هر نمادی هم ماشین می‌تواند پیکان‌های ε را طی کند.
- Q' و F' مثل قبل.

اثبات درستی. می‌گوییم «N پس از خواندن w می‌تواند در حالت q باشد» اگر دنباله‌ای مثل تعریف محاسبه NFA برای w وجود داشته باشد که به q ختم شود (بدون شرط $r_m \in F$). با استقرا روی $|w|$ ثابت می‌کنیم:

$$(\star) \qquad \delta'^*(q_0', w) = \{q \mid N \text{ پس از خواندن } w \text{ می‌تواند در } q \text{ باشد}\}.$$

پایه ($w = \varepsilon$): بدون خواندن هیچ نمادی، N فقط با پیکان‌های ε از q_0 حرکت می‌کند، پس می‌تواند دقیقاً در حالت‌های $E(\{q_0\}) = q_0'$ باشد. **گام:** فرض کنید $(\star)$ برای w درست باشد و $R = \delta'^*(q_0', w)$. پس از خواندن wa در q است اگر و تنها اگر پس از w در حالتی مثل $r \in R$ بوده، با a به یک حالت در $\delta(r, a)$ رفته و سپس با صفر یا چند پیکان ε به q رسیده باشد؛ یعنی $q \in E(\delta(r, a))$ برای یک $r \in R$. این دقیقاً تعریف $\delta'(R, a) = \delta'^*(q_0', wa)$ است.

حالا طبق $(\star)$، M رشته w را می‌پذیرد اگر و تنها اگر $\delta'^*(q_0', w)$ شامل یک حالت پذیرش N باشد، یعنی اگر و تنها اگر N بتواند پس از خواندن w در یک حالت پذیرش باشد؛ یعنی N، w را بپذیرد. پس $L(M) = L(N) = A$.

■

نتیجه ۱۰.۴

یک زبان منظم است اگر و تنها اگر یک NFA آن را تشخیص دهد.

اثبات. اگر زبان منظم باشد، یک DFA آن را تشخیص می‌دهد و هر DFA یک NFA است. برعکس، اگر یک NFA آن را تشخیص دهد، طبق قضیه ۳.۴ یک DFA هم آن را تشخیص می‌دهد.

■

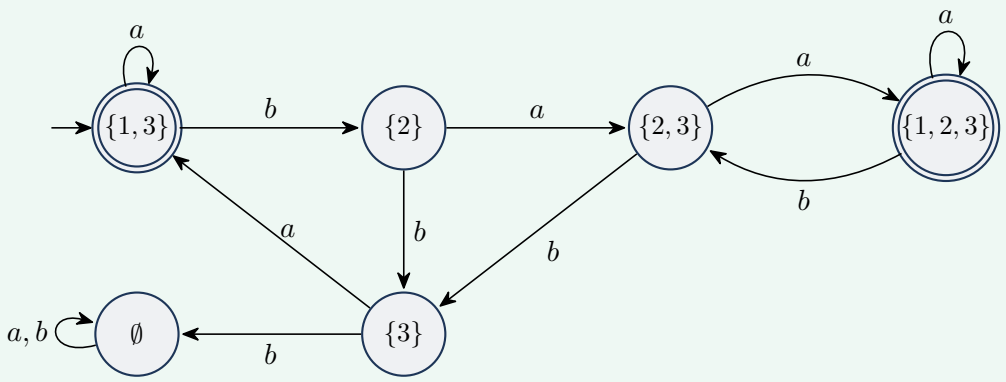
مثال ۱۱.۴ (تبدیل N_4 به DFA). ماشین N_4 از مثال ۱.۱.۴ را با $Q = \{1,2,3\}$ به DFA تبدیل می‌کنیم.

- $Q' = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1,2\}, \{1,3\}, \{2,3\}, \{1,2,3\}\}$.
- حالت شروع: $E(\{1\}) = \{1,3\}$، چون از 1 با ε به 3 می‌رسیم.
- حالت‌های پذیرش: زیرمجموعه‌هایی که شامل 1 هستند: $\{1\}, \{1,2\}, \{1,3\}, \{1,2,3\}$.

جدول گذار (مثلاً $\delta'(\{3\}, a) = E(\{1\}) = \{1,3\}$ و $\delta'(\{2\}, a) = E(\{2,3\}) = \{2,3\}$):

	a	b
$\emptyset$	$\emptyset$	$\emptyset$
$\{1\}$	$\emptyset$	$\{2\}$
$\{2\}$	$\{2,3\}$	$\{3\}$
$\{3\}$	$\{1,3\}$	$\emptyset$
$\{1,2\}$	$\{2,3\}$	$\{2,3\}$
$\{1,3\}$	$\{1,3\}$	$\{2\}$
$\{2,3\}$	$\{1,2,3\}$	$\{3\}$
$\{1,2,3\}$	$\{1,2,3\}$	$\{2,3\}$

حالت‌های $\{1\}$ و $\{1,2\}$ هیچ پیکان ورودی ندارند و از حالت شروع قابل دسترسی نیستند، پس می‌توانیم حذفشان کنیم. نتیجه یک DFA شش‌حالتی است:



نکته. ساخت زیرمجموعه‌ای ممکن است تعداد حالت‌ها را به‌طور نمایی زیاد کند (k حالت به 2^k حالت). مثال «سومین نماد از آخر» نشان می‌دهد که گاهی این افزایش واقعاً لازم است. پس NFA از نظر **قدرت** با DFA برابر است، اما می‌تواند بسیار **کوچک‌تر** باشد.

جمع‌بندی فصل

- NFA می‌تواند برای یک نماد چند انتخاب، هیچ انتخاب یا گذار ε داشته باشد و اگر **یک** شاخه بپذیرد، ورودی پذیرفته می‌شود.
- تابع گذار NFA: $\delta : Q \times \Sigma_\varepsilon \to \mathcal{P}(Q)$.
- ساخت زیرمجموعه‌ای: DFA مجموعه حالت‌های فعال NFA را دنبال می‌کند (همراه با بستار ε).
- NFA و DFA هم‌ارزند؛ هر دو دقیقاً زبان‌های منظم را تشخیص می‌دهند.

۵ فصل

ویژگی‌های بستاری زبان‌های منظم

در فصل ۳ دیدیم که بستاری تحت الحاق با DFA به‌سادگی ثابت نمی‌شود، چون ماشین نمی‌داند رشته را کجا بشکند. با NFA این مشکل حل می‌شود: ماشین «حدس می‌زند» کجا بشکند. در این فصل بستاری تحت سه عمل منظم را با NFA ثابت می‌کنیم و بعد چند عملیات دیگر (معکوس، پیشوند، پسوند و خارج‌قسمت) را بررسی می‌کنیم.

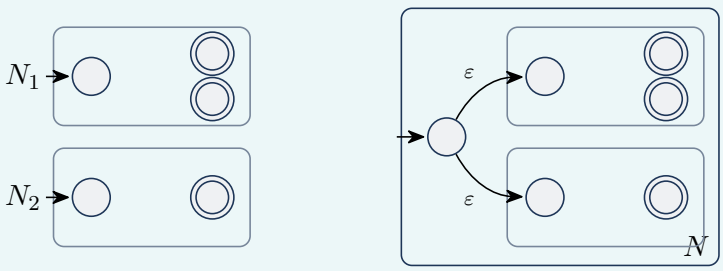
در همه شکل‌های این فصل، یک ماشین را با یک قاب نشان می‌دهیم که حالت شروعش در سمت چپ و حالت‌های پذیرشش در سمت راست است.

۱.۵ اجتماع

قضیه ۱.۵

دسته زبان‌های منظم تحت اجتماع بسته است.

ایده اثبات. فرض کنید NFAهای N_1 و N_2 زبان‌های A_1 و A_2 را تشخیص دهند. ماشین N را با یک **حالت شروع جدید** می‌سازیم که دو پیکان ε به حالت‌های شروع N_1 و N_2 دارد. N در ابتدا «حدس می‌زند» ورودی متعلق به کدام زبان است. (اثبات دیگری با ساخت ضربی در فصل ۳ دیدیم.)



اثبات. فرض کنید $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ زبان A_1 و $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ زبان A_2 را تشخیص دهند و $Q_1 \cap Q_2 = \emptyset$ (در صورت لزوم نام حالت‌ها را عوض می‌کنیم). ماشین $N = (Q, \Sigma, \delta, q_0, F)$ را می‌سازیم:

۱. $Q = \{q_0\} \cup Q_1 \cup Q_2$ که q_0 یک حالت جدید است،

۲. حالت شروع q_0 و $F = F_1 \cup F_2$،

۳. برای $q \in Q$ و $a \in \Sigma_\varepsilon$:

$$\delta(q,a) = \begin{cases} \delta_1(q,a) & q \in Q_1 \\ \delta_2(q,a) & q \in Q_2 \\ \{q_1, q_2\} & q = q_0,\ a = \varepsilon \\ \emptyset & q = q_0,\ a \neq \varepsilon. \end{cases}$$

درستی: هر محاسبه N با یک گذار ε از q_0 به q_1 یا q_2 شروع می‌شود و چون هیچ پیکانی بین Q_1 و Q_2 یا به q_0 نیست، از آن به بعد کاملاً درون N_1 یا کاملاً درون N_2 می‌ماند. پس N رشته w را می‌پذیرد اگر و تنها اگر N_1 یا N_2 آن را بپذیرد، یعنی $L(N) = A_1 \cup A_2$.

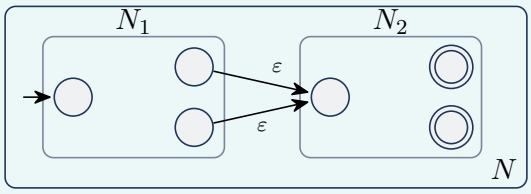
■

۲.۵ الحاق

قضیه ۲.۵

دسته زبان‌های منظم تحت الحاق بسته است.

ایده اثبات. حالت شروع N همان حالت شروع N_1 است. از هر حالت پذیرش N_1 یک پیکان ε به حالت شروع N_2 می‌کشیم و فقط حالت‌های پذیرش N_2 را پذیرش نگه می‌داریم. هر وقت N_1 به حالت پذیرش برسد، N می‌تواند «حدس بزند» که تکه اول تمام شده و به N_2 برود.



اثبات. با همان نمادها، $N = (Q_1 \cup Q_2, \Sigma, \delta, q_1, F_2)$ را می‌سازیم که در آن برای $q \in Q$ و $a \in \Sigma_\varepsilon$:

$$\delta(q,a) = \begin{cases} \delta_1(q,a) & q \in Q_1,\ q \notin F_1 \\ \delta_1(q,a) & q \in F_1,\ a \neq \varepsilon \\ \delta_1(q,a) \cup \{q_2\} & q \in F_1,\ a = \varepsilon \\ \delta_2(q,a) & q \in Q_2. \end{cases}$$

درستی: اگر $w = xy$ با $x \in A_1$ و $y \in A_2$، آنگاه N می‌تواند x را مثل N_1 بخواند و به یک حالت پذیرش N_1 برسد، با ε به q_2 برود و y را مثل N_2 بخواند و بپذیرد. برعکس، هر محاسبه پذیرنده N از $q_1 \in Q_1$ شروع و در $F_2 \subseteq Q_2$ تمام می‌شود. تنها راه رفتن از Q_1 به Q_2 پیکان‌های ε از F_1 به q_2 است و هیچ راه برگشتی نیست. پس محاسبه دقیقاً یک بار از این پیکان‌ها رد می‌شود. بخشی از ورودی که قبل از آن خوانده شده (x) توسط N_1 پذیرفته می‌شود

و بقیه (y) توسط N_2. پس $w = xy \in A_1 \circ A_2$.

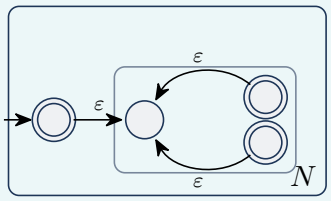
■

۳.۵ ستاره

قضیه ۳.۵

دسته زبان‌های منظم تحت ستاره بسته است.

ایده اثبات. N باید رشته‌ای را بپذیرد که از چند تکه پشت سر هم تشکیل شده که هر تکه را N_1 می‌پذیرد. پس از هر حالت پذیرش N_1 یک پیکان ε به حالت شروع N_1 برمی‌گردانیم تا تکه بعدی شروع شود. همچنین چون $\varepsilon \in A_1^*$، یک **حالت شروع جدید** اضافه می‌کنیم که خودش پذیرش است و با ε به حالت شروع قبلی می‌رود.



اثبات. فرض کنید $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ زبان A_1 را تشخیص دهد. ماشین $N = (\{q_0\} \cup Q_1, \Sigma, \delta, q_0, \{q_0\} \cup F_1)$ را با حالت جدید q_0 می‌سازیم که در آن

$$\delta(q,a) = \begin{cases} \delta_1(q,a) & q \in Q_1,\ q \notin F_1 \\ \delta_1(q,a) & q \in F_1,\ a \neq \varepsilon \\ \delta_1(q,a) \cup \{q_1\} & q \in F_1,\ a = \varepsilon \\ \{q_1\} & q = q_0,\ a = \varepsilon \\ \emptyset & q = q_0,\ a \neq \varepsilon. \end{cases}$$

درستی: ε را N می‌پذیرد چون q_0 پذیرش است. اگر $w = x_1 \cdots x_k$ با $k \geq 1$ و $x_i \in A_1$، N از q_0 با ε به q_1 می‌رود، x_1 را مثل N_1 می‌خواند و به F_1 می‌رسد، با ε به q_1 برمی‌گردد و همین کار را برای $x_2, \ldots, x_k$ تکرار می‌کند. برعکس، هر محاسبه پذیرنده روی رشته ناتهی از q_0 به q_1 می‌رود؛ هر بار که از پیکان‌های ε برگشتی استفاده می‌کند، تکه‌ای از ورودی که از آخرین ورود به q_1 خوانده شده توسط N_1 پذیرفته شده است. پس ورودی به تکه‌هایی از A_1 شکسته می‌شود و $w \in A_1^*$.

■

دقت کنید! چرا حالت شروع جدید لازم است؟ شاید فکر کنید کافی است حالت شروع قدیمی q_1 را پذیرش کنیم. اما اگر پیکانی به q_1 برگردد، ممکن است رشته‌های ناخواسته پذیرفته شوند. مثلاً اگر N_1 زبان a^*b را با یک حلقه a روی q_1 تشخیص دهد، پذیرش کردن q_1 باعث می‌شود رشته a هم پذیرفته شود، در حالی که $a \notin (a^*b)^*$.

نتیجه ۴.۵

دسته زبان‌های منظم تحت اجتماع، الحاق، ستاره، متمم، اشتراک و تفاضل بسته است.

۴.۵ معکوس

قضیه ۵.۵

اگر A منظم باشد، A^R هم منظم است.

اثبات. فرض کنید DFA $M = (Q, \Sigma, \delta, q_0, F)$ زبان A را تشخیص دهد. NFA N را این‌طور می‌سازیم: همه پیکان‌های M را **برعکس** می‌کنیم، یک حالت شروع جدید s اضافه می‌کنیم که با پیکان‌های ε به همه حالت‌های F می‌رود، و تنها حالت پذیرش را q_0 قرار می‌دهیم. به‌طور دقیق، برای $q \in Q$ و $a \in \Sigma$:

$$\delta_N(q, a) = \{p \in Q \mid \delta(p, a) = q\}, \qquad \delta_N(s, \varepsilon) = F.$$

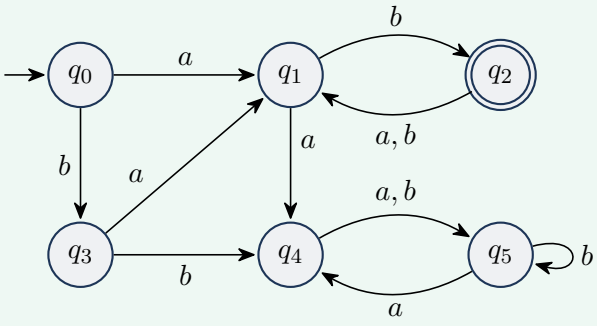
مسیر $s \xrightarrow{\varepsilon} r_n \xrightarrow{w_n} \cdots \xrightarrow{w_1} q_0$ در N وجود داشته باشد. پس $w \in A$ اگر و تنها اگر N رشته w^R را بپذیرد، یعنی $L(N) = A^R$. ■

۵.۵ پیشوند، پسوند و خارج‌قسمت

چند عملیات مفید دیگر روی زبان‌ها:

- **پیشوند:** $Prefix(A) = \{w \mid w$ پیشوند رشته‌ای در A است$\}$.
- **پسوند:** $Suffix(A) = \{w \mid w$ پسوند رشته‌ای در A است$\}$.
- **خارج‌قسمت راست** A بر رشته $u \in \Sigma^*$: همه رشته‌هایی که اگر u به انتهایشان اضافه شود عضو A می‌شوند: $Au^{-1} = \{w \in \Sigma^* \mid wu \in A\}$.
- **خارج‌قسمت چپ** A بر رشته u: $u^{-1}A = \{w \in \Sigma^* \mid uw \in A\}$.

مثال ۶.۵. DFA زیر را در نظر بگیرید:



چند رشته در $L(M)$: ab، bab، $abab$، $abbb$، $babab$، $babbb$. چند رشته در $Prefix(L(M))$: ε، a، b، ab، ba، bab.
دقت کنید q_4 و q_5 حالت‌هایی هستند که دیگر هرگز به حالت پذیرش نمی‌رسند؛ پس مثلاً $aa \notin Prefix(L(M))$.

قضیه ۷.۵

دسته زبان‌های منظم تحت پیشوند بسته است.

ایده اثبات. رشته w پیشوند یک عضو L است اگر و تنها اگر بعد از خواندن w در حالتی باشیم که از آن **بتوان** به یک حالت پذیرش رسید. پس کافی است همه حالت‌هایی را که به یک حالت پذیرش راه دارند، پذیرش کنیم.

اثبات. فرض کنید DFA $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ زبان L را تشخیص دهد. $M_2 = (Q_1, \Sigma, \delta_1, q_1, F_2)$ را با

$$F_2 = \{q \in Q_1 \mid \text{در نمودار } M_1 \text{ از } q \text{ به یک حالت } F_1 \text{ مسیری (شاید به طول صفر) هست}\}$$

می‌سازیم. (F_2 را با یک جست‌وجوی ساده در گراف حالت‌ها پیدا می‌کنیم.)
$L(M_2) \subseteq Prefix(L)$: فرض کنید w ماشین M_2 را به $q \in F_2$ ببرد. طبق تعریف F_2 مسیری از q به حالتی در F_1 هست. اگر x رشته برچسب‌های این مسیر باشد، $\delta_1^*(q_1, wx) \in F_1$، پس $wx \in L$ و $w \in Prefix(L)$.
$Prefix(L) \subseteq L(M_2)$: اگر $w \in Prefix(L)$، رشته‌ای مثل x هست که $wx \in L$. فرض کنید $q = \delta_1^*(q_1, w)$.
خواندن x از q به حالتی در F_1 می‌رسد، پس $q \in F_2$ و $w \in L(M_2)$.

■

قضیه ۸.۵

دسته زبان‌های منظم تحت پسوند بسته است.

ایده اثبات. w پسوند یک عضو L است اگر رشته‌ای مثل x باشد که $xw \in L$. خواندن x ماشین را به یکی از حالت‌های **قابل دسترس** می‌برد. پس یک NFA می‌سازیم که در ابتدا «حدس بزند» x ماشین را به کدام حالت قابل دسترس برده است و از آنجا w را بخواند.

اثبات. فرض کنید DFA $M = (Q, \Sigma, \delta, q_0, F)$ زبان L را تشخیص دهد و $R = \{\delta^*(q_0, x) \mid x \in \Sigma^*\}$ مجموعه حالت‌های قابل دسترس از q_0 باشد (با جست‌وجو در گراف به دست می‌آید). NFA N را با حالت شروع جدید s، پیکان‌های ε از s به همه حالت‌های R، همان گذارهای M و همان F می‌سازیم. آنگاه

$$w \in L(N) \iff \exists q \in R:\ \delta^*(q, w) \in F \iff \exists x:\ \delta^*(q_0, xw) \in F$$
$$\iff \exists x:\ xw \in L \iff w \in Suffix(L).$$

(راه دیگر: $Suffix(L) = \big(Prefix(L^R)\big)^R$ و از بستاری تحت معکوس و پیشوند استفاده کنیم.)

■

قضیه ۹.۵

دسته زبان‌های منظم تحت خارج‌قسمت چپ و راست بر یک رشته بسته است.

اثبات. فرض کنید DFA $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ زبان A را تشخیص دهد و $u \in \Sigma^*$.

خارج‌قسمت چپ: $M_2 = (Q_1, \Sigma, \delta_1, q_2, F_1)$ را می‌سازیم که در آن $q_2 = \delta_1^*(q_1, u)$ حالتی است که M_1 بعد از خواندن u در آن است. یعنی فقط حالت شروع را جابه‌جا می‌کنیم. آنگاه

$$x \in L(M_2) \iff \delta_1^*(q_2, x) \in F_1 \iff \delta_1^*(q_1, ux) \in F_1 \iff ux \in A \iff x \in u^{-1}A.$$

خارج‌قسمت راست: این بار حالت شروع را نگه می‌داریم و حالت‌های پذیرش را عوض می‌کنیم: $F' = \{q \in Q_1 \mid \delta_1^*(q, u) \in F_1\}$، یعنی حالت‌هایی که اگر از آن‌ها u را بخوانیم پذیرفته می‌شویم. آنگاه

$$w \in L(M') \iff \delta_1^*(q_1, w) \in F' \iff \delta_1^*(q_1, wu) \in F_1 \iff wu \in A \iff w \in Au^{-1}.$$

■

جمع‌بندی فصل

- با NFA بستاری تحت اجتماع (حالت شروع جدید)، الحاق (پیکان ε از پذیرش‌های N_1 به شروع N_2) و ستاره (پیکان ε برگشتی و شروع جدید پذیرش) ثابت شد.
- معکوس: برعکس کردن پیکان‌ها، شروع و پذیرش را جابه‌جا کردن.
- پیشوند: پذیرش کردن هر حالتی که به F راه دارد. پسوند: شروع از هر حالت قابل دسترس.
- خارج‌قسمت چپ: جابه‌جا کردن حالت شروع. خارج‌قسمت راست: عوض کردن حالت‌های پذیرش.

فصل ۶

عبارات منظم

در حساب با عملگرهای $+$ و $\times$ عبارت‌هایی مثل $(5+3)\times 4$ می‌سازیم که مقدارشان یک عدد است. به همین شکل، با عملیات منظم عبارت‌هایی می‌سازیم که مقدارشان یک **زبان** است. به این عبارت‌ها **عبارات منظم** (Regular Expressions) می‌گوییم. عبارات منظم روشی فشرده و قدرتمند برای توصیف الگوها هستند و در برنامه‌نویسی (مثلاً جست‌وجوی متن و ساخت کامپایلر) کاربرد زیادی دارند.

۱.۶ تعریف و مثال

مثلاً عبارت $(0\cup 1)0^*$ کوتاه‌شده $(\{0\}\cup\{1\})\circ\{0\}^*$ است: مجموعه همه رشته‌هایی که با یک 0 یا یک 1 شروع می‌شوند و بعد هر تعداد 0 دارند. چند قرارداد:

- نماد الحاق $\circ$ معمولاً نوشته نمی‌شود.
- **اولویت عملگرها:** اول ستاره، بعد الحاق، بعد اجتماع (مثل توان، ضرب و جمع در حساب). پس $01^*\cup 1$ یعنی $\big(0(1^*)\big)\cup 1$.
- اگر $\Sigma=\{0,1\}$، خود Σ را به عنوان عبارت $(0\cup 1)$ به کار می‌بریم. پس Σ همه رشته‌های به طول یک، Σ^* همه رشته‌ها، $0\Sigma^*$ رشته‌هایی که با 0 شروع می‌شوند و Σ^*1 رشته‌هایی که به 1 ختم می‌شوند را توصیف می‌کند.

تعریف ۱.۶ (عبارت منظم)

R یک **عبارت منظم** است اگر یکی از این‌ها باشد:

۱. a برای یک $a\in\Sigma$؛ زبان آن $\{a\}$ است.

۲. ε؛ زبان آن $\{\varepsilon\}$ است که فقط یک رشته (رشته تهی) دارد.

۳. $\emptyset$؛ زبان آن تهی است و هیچ رشته‌ای ندارد.

۴. $(R_1\cup R_2)$، $(R_1\circ R_2)$ یا (R_1^*) که R_1 و R_2 عبارت منظم‌اند.

زبان توصیف‌شده با R را با $L(R)$ نشان می‌دهیم. همچنین $R^+=RR^*$ و R^k الحاق k نسخه از R است.

تعریف دوری یا استقرایی؟ ممکن است به نظر برسد عبارت منظم را با خودش تعریف کرده‌ایم (تعریف دوری). اما این‌طور نیست: R_1 و R_2 همیشه **کوچک‌تر** از R هستند. پس این یک **تعریف استقرایی** است که از عبارت‌های پایه (موارد ۱ تا ۳) شروع می‌کند و عبارت‌های بزرگ‌تر را از کوچک‌ترها می‌سازد.

مثال ۲.۶. روی $\Sigma=\{0,1\}$:

0^*10^*	$\{w \mid w$ contains a single $1\}$
$\Sigma^*1\Sigma^*$	$\{w \mid w$ has at least one $1\}$
$\Sigma^*001\Sigma^*$	$\{w \mid w$ contains 001 as a substring$\}$
$1^*(01^+)^*$	$\{w \mid$ every 0 in w is followed by at least one $1\}$
$(\Sigma\Sigma)^*$	$\{w \mid \lvert w\rvert$ is even$\}$
$(\Sigma\Sigma\Sigma)^*$	$\{w \mid \lvert w\rvert$ is a multiple of $3\}$
$01 \cup 10$	$\{01, 10\}$
$0\Sigma^*0 \cup 1\Sigma^*1 \cup 0 \cup 1$	$\{w \mid w$ starts and ends with the same symbol$\}$
$(0 \cup \varepsilon)1^*$	$01^* \cup 1^*$
$(0 \cup \varepsilon)(1 \cup \varepsilon)$	$\{\varepsilon, 0, 1, 01\}$
$1^*\emptyset$	$\emptyset$
$\emptyset^*$	$\{\varepsilon\}$

دو مورد آخر ارزش توضیح دارند. الحاق هر زبانی با $\emptyset$ تهی است، چون برای ساختن رشته xy باید y را از زبان تهی برداریم که ممکن نیست. اما $\emptyset^* = \{\varepsilon\}$، چون ستاره همیشه الحاق **صفر** رشته (یعنی ε) را دارد.

۱.۱.۶ اتحادهای ساده

برای هر عبارت منظم R:

$$R \cup \emptyset = R, \qquad R \circ \varepsilon = R.$$

اما دو عبارت شبیه به این‌ها در حالت کلی درست **نیستند**:

- $R \cup \varepsilon \neq R$: اگر $R = 0$ آنگاه $L(R) = \{0\}$ ولی $L(R \cup \varepsilon) = \{0, \varepsilon\}$.
- $R \circ \emptyset \neq R$: اگر $R = 0$ آنگاه $L(R) = \{0\}$ ولی $L(R \circ \emptyset) = \emptyset$.

مثال ۳.۶ (کاربرد در کامپایلرها). بخش «تحلیل لغوی» کامپایلر کد برنامه را به **توکن**‌ها (مثل نام متغیر و عدد) می‌شکند و معمولاً از روی عبارات منظم به‌طور خودکار ساخته می‌شود. مثلاً اگر $D = \{0, 1, \ldots, 9\}$ مجموعه ارقام باشد، عددهای اعشاری علامت‌دار با این عبارت توصیف می‌شوند:

$$(+ \cup - \cup \varepsilon)\,(D^+ \cup D^+.D^* \cup D^*.D^+).$$

رشته‌های 72، 3.14159، $+7.$ و $-.01$ همه با این عبارت تطبیق دارند. (به اولویت عملگرها دقت کنید.)

۲.۶ هم‌ارزی عبارات منظم و ماشین‌های متناهی

قضیه ۴.۶

یک زبان منظم است اگر و تنها اگر یک عبارت منظم آن را توصیف کند.

این قضیه دو جهت دارد که هر کدام را در یک لم ثابت می‌کنیم.

۱.۲.۶ از عبارت منظم به NFA

لم ۵.۶

اگر یک عبارت منظم زبانی را توصیف کند، آن زبان منظم است.

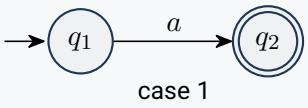
ایده اثبات. عبارت منظم R را به یک NFA تبدیل می‌کنیم. چون NFA و DFA هم‌ارزند، این کافی است. تبدیل را روی ساختار استقرایی تعریف عبارت منظم انجام می‌دهیم: برای عبارت‌های پایه ماشین می‌سازیم و برای عبارت‌های مرکب از ساخت‌های بستاری فصل ۵ استفاده می‌کنیم.

اثبات. شش حالت تعریف عبارت منظم را بررسی می‌کنیم:

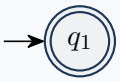
۱. $R = a$ برای $a \in \Sigma$، پس $L(R) = \{a\}$. ماشین $N = (\{q_1, q_2\}, \Sigma, \delta, q_1, \{q_2\})$ با $\delta(q_1, a) = \{q_2\}$ و $\delta(r, b) = \emptyset$ برای بقیه حالت‌ها.

۲. $R = \varepsilon$، پس $L(R) = \{\varepsilon\}$. ماشین $N = (\{q_1\}, \Sigma, \delta, q_1, \{q_1\})$ با $\delta(q_1, b) = \emptyset$ برای هر b.

۳. $R = \emptyset$. ماشین $N = (\{q_1\}, \Sigma, \delta, q_1, \emptyset)$ با $\delta(q_1, b) = \emptyset$ برای هر b.



case 1



case 2



case 3

۴. $R = R_1 \cup R_2$،

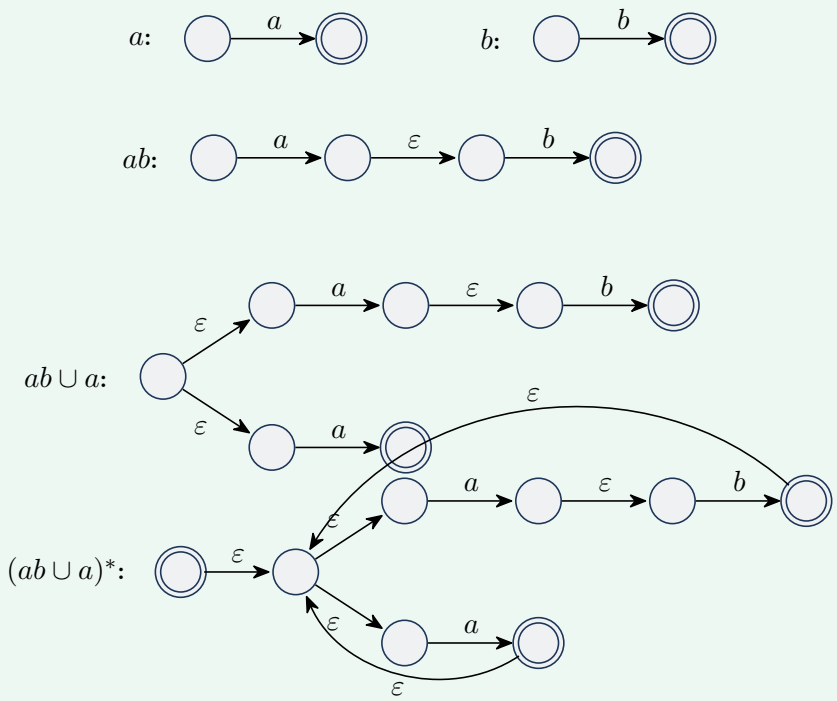
۵. $R = R_1 \circ R_2$،

۶. $R = R_1^*$.

در سه حالت آخر، طبق فرض استقرا برای R_1 و R_2 (که کوچک‌ترند) NFA داریم و با ساخت‌های اثبات بستاری تحت اجتماع، الحاق و ستاره (فصل ۵) یک NFA برای R می‌سازیم.

■

مثال ۶.۶ (تبدیل $(ab \cup a)^*$ به NFA). ماشین را از درون به بیرون می‌سازیم:



در شکل آخر، حالت سمت چپ حالت شروع جدیدِ ساخت ستاره است.

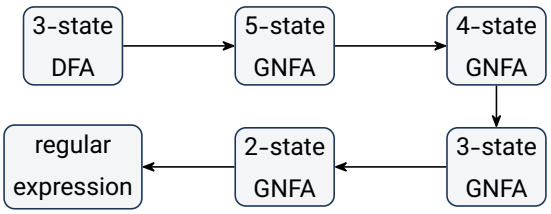
۲.۲.۶ از DFA به عبارت منظم

لم ۷.۶

اگر زبانی منظم باشد، یک عبارت منظم آن را توصیف می‌کند.

برای این جهت از نوع جدیدی از ماشین استفاده می‌کنیم: **ماشین متناهی غیرقطعی تعمیم‌یافته** (Generalized NFA) یا GNFA. در GNFA برچسب پیکان‌ها می‌تواند **هر عبارت منظمی** باشد، نه فقط یک نماد یا ε. ماشین با رد شدن از یک پیکان، یک **بلوک** از نمادهای ورودی را می‌خواند که با برچسب آن پیکان تطبیق دارد.

نقشه اثبات: DFA را به یک GNFA تبدیل می‌کنیم و بعد حالت‌های GNFA را یکی‌یکی حذف می‌کنیم و برچسب‌ها را به‌روز می‌کنیم، تا فقط دو حالت بماند. برچسب تنها پیکان باقی‌مانده همان عبارت منظم مورد نظر است.



برای ساده شدن کار، GNFAها را به یک **شکل خاص** محدود می‌کنیم:

- یک حالت شروع و یک حالت پذیرش دارد و این دو با هم فرق دارند.
- از حالت شروع به هر حالت دیگر پیکان هست، اما هیچ پیکانی وارد حالت شروع نمی‌شود.

- از هر حالت دیگر به حالت پذیرش پیکان هست، اما هیچ پیکانی از حالت پذیرش خارج نمی‌شود.
- بین هر دو حالت دیگر، در هر جهت دقیقاً یک پیکان هست و هر حالت یک حلقه روی خودش دارد.

تعریف ۸.۶ (GNFA)

یک GNFA پنج‌تایی $(Q, \Sigma, \delta, q_{\text{start}}, q_{\text{accept}})$ است که در آن Q مجموعه متناهی حالت‌ها، Σ الفبای ورودی، q_{start} حالت شروع، q_{accept} حالت پذیرش و

$$\delta : \big(Q - \{q_{\text{accept}}\}\big) \times \big(Q - \{q_{\text{start}}\}\big) \to \mathcal{R}$$

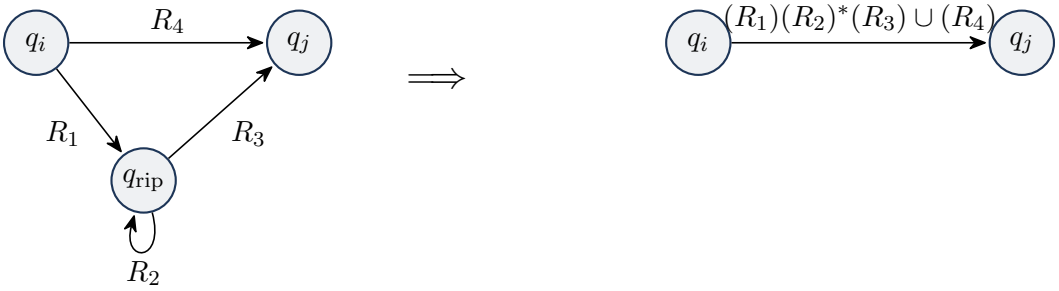
تابع گذار است که $\mathcal{R}$ مجموعه همه عبارات منظم روی Σ است. یعنی $\delta(q_i, q_j)$ برچسب پیکان از q_i به q_j است.

GNFA رشته $w \in \Sigma^*$ را می‌پذیرد اگر بتوان نوشت $w = w_1 w_2 \cdots w_k$ (هر $w_i \in \Sigma^*$) و دنباله‌ای از حالت‌ها $q_0, q_1, \ldots, q_k$ وجود داشته باشد که $q_0 = q_{\text{start}}$، $q_k = q_{\text{accept}}$ و برای هر i، $w_i \in L\big(\delta(q_{i-1}, q_i)\big)$.

تبدیل DFA به GNFA با شکل خاص:

- یک حالت شروع جدید با پیکان ε به حالت شروع قدیمی اضافه می‌کنیم.
- یک حالت پذیرش جدید با پیکان‌های ε از همه حالت‌های پذیرش قدیمی اضافه می‌کنیم.
- اگر بین دو حالت چند پیکان (یا یک پیکان با چند برچسب) بود، آن‌ها را با اجتماع یکی می‌کنیم.
- بین حالت‌هایی که در DFA پیکانی نداشتند، پیکانی با برچسب $\emptyset$ می‌گذاریم. (پیکان $\emptyset$ هرگز قابل استفاده نیست، پس زبان تغییر نمی‌کند.)

حذف یک حالت: یک حالت q_{rip} غیر از شروع و پذیرش را انتخاب و حذف می‌کنیم. برای جبران، برچسب هر پیکان $q_i \to q_j$ را طوری عوض می‌کنیم که همه رشته‌هایی را توصیف کند که ماشین را **یا مستقیماً یا از طریق** q_{rip} از q_i به q_j می‌برند:



Convert(G) = «روی ورودی GNFA G»

۱. فرض کنید k تعداد حالت‌های G باشد.

۲. اگر $k = 2$، آنگاه G فقط یک حالت شروع، یک حالت پذیرش و یک پیکان بین آن‌ها با برچسب R دارد. R را برگردان.

۳. اگر $k > 2$، یک $q_{\text{rip}} \in Q - \{q_{\text{start}}, q_{\text{accept}}\}$ انتخاب کن و GNFA $G' = (Q', \Sigma, \delta', q_{\text{start}}, q_{\text{accept}})$ را بساز که $Q' = Q - \{q_{\text{rip}}\}$ و برای هر $q_i \in Q' - \{q_{\text{accept}}\}$ و $q_j \in Q' - \{q_{\text{start}}\}$:

$$\delta'(q_i, q_j) = (R_1)(R_2)^*(R_3) \cup (R_4),$$

که $R_1 = \delta(q_i, q_{\text{rip}})$، $R_2 = \delta(q_{\text{rip}}, q_{\text{rip}})$، $R_3 = \delta(q_{\text{rip}}, q_j)$ و $R_4 = \delta(q_i, q_j)$.

۴. Convert(G') را برگردان.

لم ۹.۶

برای هر GNFA G، عبارت Convert(G) با G هم‌ارز است.

اثبات. با استقرا روی تعداد حالت‌های G، یعنی k.

پایه ($k = 2$): G فقط یک پیکان از q_{start} به q_{accept} با برچسب R دارد. پس G دقیقاً رشته‌های $L(R)$ را می‌پذیرد و Convert همان R را برمی‌گرداند.

گام: فرض کنید ادعا برای $k-1$ حالت درست باشد و G دارای $k > 2$ حالت باشد. کافی است نشان دهیم G و G' هم‌ارزند، چون طبق فرض استقرا Convert(G') با G' هم‌ارز است.

هر رشته‌ای که G بپذیرد، G' هم می‌پذیرد: فرض کنید G رشته w را با دنباله حالت‌های $q_{\text{start}}, q_1, q_2, \ldots, q_{\text{accept}}$ بپذیرد. اگر q_{rip} در این دنباله نباشد، G' هم با همین دنباله w را می‌پذیرد، چون هر برچسب جدید شامل برچسب قدیمی (به عنوان یکی از دو طرف اجتماع، یعنی R_4) است. اگر q_{rip} در دنباله باشد، هر دسته از q_{rip}های پشت سر هم را حذف کنید. اگر این دسته بین q_i و q_j قرار داشته باشد، بخشی از ورودی که از q_i تا q_j خوانده شده با $R_1 R_2^* R_3$ تطبیق دارد (یک بار R_1، چند بار R_2 و یک بار R_3). پس G' می‌تواند همین بخش را با پیکان جدید $q_i \to q_j$ بخواند و w را بپذیرد.

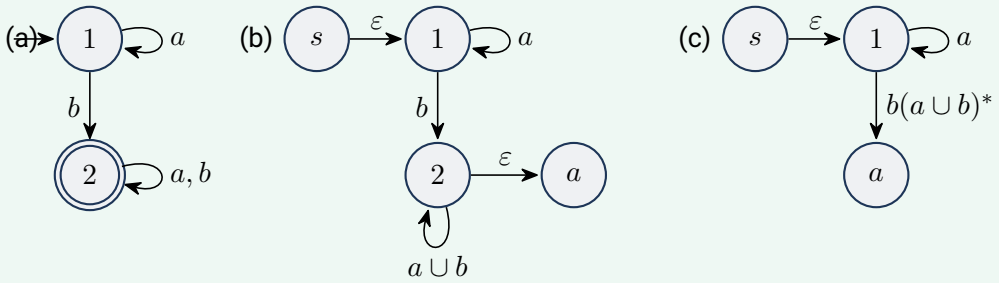
هر رشته‌ای که G' بپذیرد، G هم می‌پذیرد: هر پیکان $q_i \to q_j$ در G' رشته‌هایی را می‌خواند که آن‌ها را یا مستقیماً (با R_4) یا از طریق q_{rip} (با $R_1 R_2^* R_3$) از q_i به q_j می‌خواند. پس هر محاسبه پذیرنده G' را می‌توان به یک محاسبه پذیرنده G تبدیل کرد.

بنابراین G و G' هم‌ارزند و حکم ثابت است.

■

با این لم، اثبات لم ۲.۲.۶ کامل است: برای زبان منظم A یک DFA می‌گیریم، آن را به GNFA تبدیل می‌کنیم و Convert را اجرا می‌کنیم. و قضیه ۲.۶ از دو لم ۱.۲.۶ و ۲.۲.۶ نتیجه می‌شود.

مثال ۱۰.۶. DFA دوحالته زیر را به عبارت منظم تبدیل می‌کنیم (پیکان‌های $\emptyset$ را نمی‌کشیم):



در (b) دو حالت جدید s (شروع) و a (پذیرش) اضافه شده‌اند. در (c) حالت 2 را حذف کرده‌ایم: پیکان جدید $1 \to a$ برچسب $b(a \cup b)^* \varepsilon \cup \emptyset = b(a \cup b)^*$ دارد. در آخر با حذف حالت 1، پیکان $s \to a$ برچسب

$$\varepsilon\, a^*\, b(a \cup b)^* \cup \emptyset = a^* b(a \cup b)^*$$

را می‌گیرد. پس زبان این DFA برابر $a^* b(a \cup b)^*$ است: رشته‌هایی که دست‌کم یک b دارند.

جمع‌بندی فصل

- عبارت منظم از a، ε و $\emptyset$ با اجتماع، الحاق و ستاره ساخته می‌شود. اولویت: ستاره، الحاق، اجتماع.
- $R \cup \emptyset = R$ و $R\varepsilon = R$، اما $R \cup \varepsilon$ و $R\emptyset$ در حالت کلی برابر R نیستند. $\emptyset^* = \{\varepsilon\}$.
- تبدیل عبارت منظم به NFA: ساخت استقرایی با ساخت‌های بستاری.
- تبدیل DFA به عبارت منظم: ساختن GNFA و حذف حالت‌ها با فرمول $(R_1)(R_2)^*(R_3) \cup (R_4)$.
- عبارات منظم، DFAها و NFAها دقیقاً یک دسته زبان را توصیف می‌کنند.

۷ فصل

گرامرهای منظم

تا اینجا دو روش برای توصیف زبان‌های منظم دیدیم: ماشین‌ها (DFA و NFA) که رشته‌ها را **تشخیص** می‌دهند و عبارات منظم که الگوی رشته‌ها را **توصیف** می‌کنند. در این فصل روش سومی می‌بینیم: **گرامر**، که رشته‌های زبان را **تولید** می‌کند. گرامرها در فصل ۹ با جزئیات بیشتری بررسی می‌شوند؛ اینجا فقط آن‌قدر که لازم است معرفی‌شان می‌کنیم.

۱.۷ گرامر چیست؟

تعریف ۱.۷ (گرامر)

یک **گرامر** (Grammar) چهارتایی $G = (V, T, S, P)$ است که در آن V مجموعه متناهی **متغیرها**، T مجموعه متناهی **پایانه‌ها** (همان الفبا) با $V \cap T = \emptyset$، $S \in V$ **متغیر شروع** و P مجموعه متناهی **قاعده‌ها** (productions) است. هر قاعده شکل $A \to x$ دارد، یعنی «A را می‌توان با x جایگزین کرد».

اگر uAv رشته‌ای از متغیرها و پایانه‌ها باشد و قاعده $A \to x$ وجود داشته باشد، می‌نویسیم $uAv \Rightarrow uxv$ (یک گام **اشتقاق**). اگر با صفر یا چند گام از u به v برسیم، می‌نویسیم $u \overset{*}{\Rightarrow} v$. زبان گرامر همه رشته‌های **فقط از پایانه‌ها** است که از S مشتق می‌شوند:

$$L(G) = \{ w \in T^* \mid S \overset{*}{\Rightarrow} w \}.$$

۲.۷ گرامرهای خطی راست و چپ

تعریف ۲.۷ (گرامر خطی راست و خطی چپ)

گرامر $G = (V, T, S, P)$ **خطی راست** است اگر همه قاعده‌هایش به یکی از این دو شکل باشند:

$$A \to xB, \qquad A \to x, \qquad (A, B \in V,\ x \in T^*).$$

و **خطی چپ** است اگر همه قاعده‌هایش به یکی از این دو شکل باشند:

$$A \to Bx, \qquad A \to x.$$

گرامری را که خطی راست یا خطی چپ باشد، **گرامر منظم** (Regular Grammar) می‌نامیم.

یعنی در گرامر خطی راست هر قاعده حداکثر یک متغیر دارد و آن متغیر در **انتهای راست** قاعده است. در نتیجه هر رشته‌ای که در طول اشتقاق ساخته می‌شود شکل $ab\cdots cD$ دارد: چند پایانه و حداکثر یک متغیر در انتها.

مثال ۳.۷. گرامر خطی راست G_1 با قاعده‌های $S \to abS \mid a$ (یعنی دو قاعده $S \to abS$ و $S \to a$). یک اشتقاق:

$$S \Rightarrow abS \Rightarrow ababS \Rightarrow ababa.$$

زبان این گرامر با عبارت منظم $(ab)^*a$ توصیف می‌شود.

مثال ۴.۷. گرامر خطی چپ G_2:

$$S \to S_1ab, \qquad S_1 \to S_1ab \mid S_2, \qquad S_2 \to a.$$

یک اشتقاق: $S \Rightarrow S_1ab \Rightarrow S_1abab \Rightarrow S_2abab \Rightarrow aababab$. زبان این گرامر $aab(ab)^*$ است.

شکل ساده. هر قاعده خطی راست را می‌توان با چند قاعده که هر کدام **حداکثر یک پایانه** دارند جایگزین کرد. کافی است متغیرهای کمکی جدید اضافه کنیم:

$$A \to abcB \quad \rightsquigarrow \quad A \to aB_1,\ B_1 \to bB_2,\ B_2 \to cB,$$
$$A \to abc \quad \rightsquigarrow \quad A \to aB_1,\ B_1 \to bB_2,\ B_2 \to c.$$

زبان گرامر تغییر نمی‌کند، چون متغیرهای جدید فقط در همین زنجیره به کار می‌روند.

تعریف ۵.۷ (گرامر خطی)

گرامری **خطی** است اگر سمت چپ هر قاعده دقیقاً یک متغیر و سمت راست آن **حداکثر یک** متغیر داشته باشد، بدون هیچ محدودیتی بر جای آن متغیر.

هر گرامر منظم خطی است، اما عکس آن درست نیست. مثلاً گرامر خطی G_3:

$$S \to A, \qquad A \to aB \mid \varepsilon, \qquad B \to Ab$$

زبان $\{a^nb^n \mid n \geq 0\}$ را تولید می‌کند ($A \Rightarrow aB \Rightarrow aAb$) که در فصل بعد ثابت می‌کنیم منظم نیست. ترکیب قاعده‌های خطی راست و خطی چپ در یک گرامر همین مشکل را ایجاد می‌کند.

۳.۷ هر گرامر خطی راست زبانی منظم تولید می‌کند

قضیه ۶.۷

اگر $G = (V, T, S, P)$ خطی راست باشد، $L(G)$ منظم است.

ایده اثبات. هر صورت میانی اشتقاق شکل $ab\cdots c\,D$ دارد و با به کار بردن قاعده $D \to dE$ به $ab\cdots cd\,E$ تبدیل می‌شود. این دقیقاً شبیه کار یک NFA است: **متغیر** نقش **حالت** را دارد و پیشوند پایانه‌ای نقش **بخشی از ورودی که تا اینجا خوانده شده**.

اثبات. طبق نکته بالا فرض می‌کنیم هر قاعده حداکثر یک پایانه دارد. متغیرها را $V = \{V_0, V_1, \dots\}$ بنامید که $S = V_0$. پس هر قاعده به شکل $V_i \to vV_k$ یا $V_n \to v$ است که $v \in T \cup \{\varepsilon\}$. NFA زیر را می‌سازیم:

$$M = \big(V \cup \{V_f\},\ T,\ \delta,\ V_0,\ \{V_f\}\big)$$

که V_f یک حالت جدید است و گذارها از روی قاعده‌ها ساخته می‌شوند:

قاعده	گذار
$V_i \to vV_k$	$\delta(V_i, v)$ شامل V_k است
$V_n \to v$	$\delta(V_n, v)$ شامل V_f است

(اگر $v = \varepsilon$ باشد، گذار یک پیکان ε است.)

درستی: اگر $w \in L(G)$، اشتقاقی به شکل

$$V_0 \Rightarrow v_1 V_{i_1} \Rightarrow v_1 v_2 V_{i_2} \Rightarrow \cdots \Rightarrow v_1 \cdots v_{m-1} V_{i_{m-1}} \Rightarrow v_1 \cdots v_m = w$$

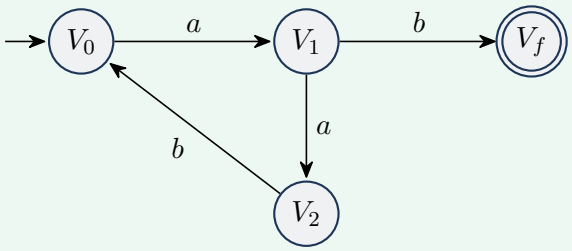
وجود دارد. طبق ساخت، M می‌تواند مسیر $V_0 \xrightarrow{v_1} V_{i_1} \xrightarrow{v_2} \cdots \xrightarrow{v_{m-1}} V_{i_{m-1}} \xrightarrow{v_m} V_f$ را طی کند، پس w را می‌پذیرد. برعکس، هر مسیر پذیرنده M از V_0 به V_f روی w، دنباله‌ای از گذارهاست که هر کدام از یک قاعده آمده؛ با به کار بردن همان قاعده‌ها به همان ترتیب، اشتقاقی برای w در G به دست می‌آید. پس $L(M) = L(G)$.

■

مثال ۷.۷. برای گرامر $V_0 \to aV_1,\ V_1 \to abV_0 \mid b$ ابتدا قاعده $V_1 \to abV_0$ را با متغیر کمکی V_2 می‌شکنیم:

$$V_0 \to aV_1, \qquad V_1 \to aV_2 \mid b, \qquad V_2 \to bV_0.$$

NFA حاصل:



زبان این ماشین (و گرامر) با عبارت منظم $(aab)^*ab$ توصیف می‌شود.

۴.۷ هر زبان منظم یک گرامر خطی راست دارد

قضیه ۸.۷

اگر L زبانی منظم روی Σ باشد، گرامر خطی راستی مثل G وجود دارد که $L = L(G)$.

اثبات. فرض کنید DFA $M = (Q, \Sigma, \delta, q_0, F)$ زبان L را بپذیرد. گرامر $G = (V, \Sigma, S, P)$ را با $V = Q$ و $S = q_0$ می‌سازیم. یعنی هر حالت یک متغیر می‌شود. قاعده‌ها:

- برای هر گذار $\delta(q_i, a) = q_k$، قاعده $q_i \to a q_k$؛
- برای هر حالت پذیرش $q_k \in F$، قاعده $q_k \to \varepsilon$.

این گرامر خطی راست است.

درستی: فرض کنید $w = a_1 a_2 \cdots a_n \in L$. پس M دنباله‌ای از حالت‌ها دارد:

$$\delta(q_0, a_1) = r_1,\ \ \delta(r_1, a_2) = r_2,\ \ \ldots,\ \ \delta(r_{n-1}, a_n) = r_n \in F.$$

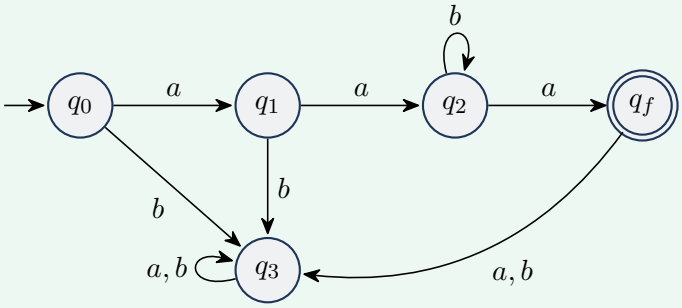
طبق ساخت، اشتقاق زیر در G ممکن است:

$$q_0 \Rightarrow a_1 r_1 \Rightarrow a_1 a_2 r_2 \Rightarrow \cdots \Rightarrow a_1 \cdots a_n r_n \Rightarrow a_1 \cdots a_n,$$

که گام آخر از قاعده $r_n \to \varepsilon$ (چون $r_n \in F$) استفاده می‌کند. برعکس، هر اشتقاق در G شکل بالا را دارد (چون تنها راه حذف متغیر قاعده $q_k \to \varepsilon$ است و فقط برای حالت‌های پذیرش وجود دارد)، پس دنباله‌ای پذیرنده از حالت‌ها در M مشخص می‌کند. بنابراین $L(G) = L$.

■

مثال ۹.۷. یک گرامر خطی راست برای aab^*a. ابتدا یک DFA می‌سازیم (q_3 حالت تله است):



قاعده‌های گرامر از روی گذارهای این ماشین:

$$\begin{aligned}
&q_0 \to a q_1, \quad q_0 \to b q_3, \quad q_1 \to a q_2, \quad q_1 \to b q_3,\\
&q_2 \to a q_f, \quad q_2 \to b q_2, \quad q_f \to a q_3, \quad q_f \to b q_3,\\
&q_3 \to a q_3, \quad q_3 \to b q_3, \quad q_f \to \varepsilon.
\end{aligned}$$

پذیرش aaa با مسیر $q_0 \xrightarrow{a} q_1 \xrightarrow{a} q_2 \xrightarrow{a} q_f$ متناظر است با اشتقاق

$$q_0 \Rightarrow a q_1 \Rightarrow a a q_2 \Rightarrow a a a q_f \Rightarrow aaa.$$

(قاعده‌های q_3 هرگز به رشته پایانه‌ای نمی‌رسند و می‌توان آن‌ها را حذف کرد.)

۵.۷ گرامرهای خطی چپ

قضیه ۱۰.۷

زبان L منظم است اگر و تنها اگر گرامر خطی چپی مثل G وجود داشته باشد که $L = L(G)$.

ایده اثبات. گرامر خطی چپ رشته را «از راست به چپ» می‌سازد. اگر سمت راست همه قاعده‌ها را برعکس کنیم، یک گرامر خطی راست به دست می‌آید که **معکوس** همان زبان را تولید می‌کند. سپس از بستاری زبان‌های منظم تحت معکوس (قضیه ۴.۵) استفاده می‌کنیم.

اثبات. برای گرامر خطی چپ G، گرامر $\widehat{G}$ را با همان متغیرها و قاعده‌های زیر می‌سازیم:

$$A \to Bv \;\leftrightarrow\; A \to v^R B, \qquad\qquad A \to v \;\leftrightarrow\; A \to v^R.$$

$\widehat{G}$ خطی راست است. همچنین این تبدیل دوطرفه است: از هر گرامر خطی راست هم با همین قاعده یک گرامر خطی چپ به دست می‌آید.

ادعا: $L(G) = L(\widehat{G})^R$. هر صورت میانی در اشتقاق G شکل Bx و در $\widehat{G}$ شکل $x^R B$ دارد. با استقرا روی تعداد گام‌ها، $S \overset{*}{\Rightarrow}_G Bx$ اگر و تنها اگر $S \overset{*}{\Rightarrow}_{\widehat{G}} x^R B$: اگر $Bx \Rightarrow_G Cvx$ با قاعده $B \to Cv$، آنگاه $x^R B \Rightarrow_{\widehat{G}} x^R v^R C = (vx)^R C$ با قاعده متناظر $B \to v^R C$؛ و گام پایانی $B \to v$ هم متناظر $B \to v^R$ است. پس $S \overset{*}{\Rightarrow}_G w$ اگر و تنها اگر $S \overset{*}{\Rightarrow}_{\widehat{G}} w^R$.

جهت رفت: اگر G خطی چپ باشد، $\widehat{G}$ خطی راست است، پس طبق قضیه ۳.۷ زبان $L(\widehat{G})$ منظم است. طبق بستاری تحت معکوس، $L(\widehat{G})^R = L(G)$ هم منظم است.

جهت برگشت: اگر L منظم باشد، L^R هم منظم است و طبق قضیه ۴.۷ گرامر خطی راستی مثل $\widehat{G}$ با $L(\widehat{G}) = L^R$ وجود دارد. گرامر خطی چپ متناظر با آن، G، زبان $L(\widehat{G})^R = (L^R)^R = L$ را تولید می‌کند.

■

نتیجه ۱۱.۷

یک زبان منظم است اگر و تنها اگر یک گرامر منظم (خطی راست یا خطی چپ) آن را تولید کند.

جمع‌بندی فصل

- گرامر (V, T, S, P) رشته‌های زبان را با اشتقاق از متغیر شروع تولید می‌کند.
- خطی راست: $A \to xB \mid x$؛ خطی چپ: $A \to Bx \mid x$. گرامر خطی که هر دو نوع قاعده را ترکیب کند ممکن است زبان نامنظم بسازد.
- متغیر $\leftrightarrow$ حالت: گرامر خطی راست و NFA به‌سادگی به هم تبدیل می‌شوند.
- گرامر خطی چپ معکوس زبان را به شکل خطی راست تولید می‌کند؛ با بستاری تحت معکوس، آن هم دقیقاً زبان‌های منظم را می‌سازد.

۸ فصل

زبان‌های نامنظم و لم تزریق

ماشین‌های متناهی حافظه محدودی دارند، پس نمی‌توانند هر زبانی را تشخیص دهند. در این فصل یاد می‌گیریم چطور ثابت کنیم یک زبان منظم **نیست**. ابزار اصلی **لم تزریق** (Pumping Lemma) است و ابزار کمکی، ویژگی‌های بستاری که در فصل‌های قبل ثابت کردیم.

۱.۸ محدودیت ماشین‌های متناهی

زبان $B = \{0^n 1^n \mid n \geq 0\}$ را در نظر بگیرید. ماشینی که B را تشخیص دهد باید تعداد 0ها را به خاطر بسپارد تا بعد آن را با تعداد 1ها مقایسه کند. اما تعداد 0ها می‌تواند هر عددی باشد و ماشین فقط تعداد متناهی حالت دارد. پس به نظر می‌رسد هیچ ماشین متناهی‌ای نمی‌تواند B را تشخیص دهد.

آیا این استدلال شهودی کافی است؟ نه! به این دو زبان روی $\{0,1\}$ نگاه کنید:

$$C = \{w \mid w \text{ دارد } 1 \text{ و } 0 \text{ تعداد برابری}\},$$
$$D = \{w \mid w \text{ دارد } 10 \text{ و } 01 \text{ تعداد برابری زیررشته}\}.$$

در هر دو به نظر می‌رسد باید «شمردن» انجام شود. اما C منظم نیست (همین فصل ثابت می‌کنیم)، در حالی که D **منظم است**! دلیلش این است که در هر رشته، هر جا 0ها به 1 تبدیل می‌شوند یک 01 و هر جا 1ها به 0 تبدیل می‌شوند یک 10 داریم. این دو نوع تغییر یکی در میان اتفاق می‌افتند، پس تعدادشان حداکثر یکی فرق دارد و برابرند اگر و تنها اگر رشته با همان نمادی تمام شود که با آن شروع شده (یا تهی باشد). پس

$$D = L(\varepsilon \cup 0 \cup 1 \cup 0\Sigma^*0 \cup 1\Sigma^*1).$$

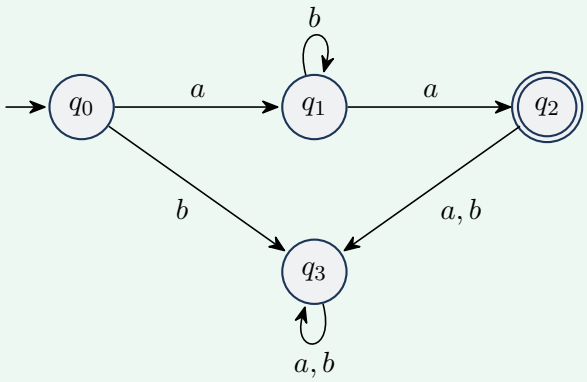
نتیجه: شهود گاهی گمراه‌کننده است و به یک **اثبات دقیق** نیاز داریم.

۲.۸ لم تزریق

لم تزریق می‌گوید همه زبان‌های منظم یک ویژگی خاص دارند:

ایده اصلی. اگر رشته‌ای از یک زبان منظم از «طول تزریق» بلندتر باشد، بخشی از آن را می‌توان هر چند بار که بخواهیم تکرار کرد (یا حذف کرد) و رشته حاصل باز هم در زبان می‌ماند.

مثال ۱.۸ (شهود). DFA زیر زبان ab^*a را تشخیص می‌دهد (q_3 حالت تله است):



رشته‌های زبان: aa، aba، $abba$، $abbba$، ... هر رشته به طول دست‌کم ۳ مثل $abbbba$ را می‌توان به شکل $x = a$، $y = b$ و $z = bbba$ شکست و برای هر $i \geq 0$ داریم $xy^iz = ab^ibbba \in L$. دلیل: y دقیقاً روی حلقه q_1 خوانده می‌شود و دور زدن چندباره روی این حلقه (یا دور نزدن) ماشین را در همان حالت نگه می‌دارد.

قضیه ۲.۸ (لم تزریق برای زبان‌های منظم)

برای هر زبان منظم A عددی مثل p (**طول تزریق**) وجود دارد به طوری که هر رشته $s \in A$ با $|s| \geq p$ را می‌توان به شکل $s = xyz$ نوشت که

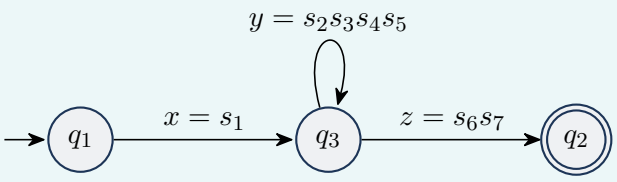
۱. برای هر $i \geq 0$: $xy^iz \in A$،
۲. $|y| > 0$، و
۳. $|xy| \leq p$.

یادآوری: $y^0 = \varepsilon$، پس $xy^0z = xz$. شرط ۲ می‌گوید y تهی نیست (وگرنه قضیه بدیهی می‌شد) و شرط ۳ می‌گوید y در p نماد اول s قرار دارد.

ایده اثبات. فرض کنید DFA M زبان A را تشخیص دهد و ۵ حالت داشته باشد. اگر M رشته‌ای به طول ۷ را بخواند، از ۸ حالت عبور می‌کند (حالت شروع و ۷ حالت بعدی). طبق **اصل لانه کبوتری** در ۶ حالت اول (۶ عدد، ۵ لانه) یک حالت تکراری هست؛ مثلاً q_3:

$$s = \ s_1\ s_2\ s_3\ s_4\ s_5\ s_6\ s_7, \qquad q_1 \xrightarrow{s_1} \underline{q_3} \xrightarrow{s_2} q_2 \xrightarrow{s_3} q_4 \xrightarrow{s_4} q_5 \xrightarrow{s_5} \underline{q_3} \xrightarrow{s_6} q_5 \xrightarrow{s_7} q_2.$$

پس $x = s_1$، $y = s_2s_3s_4s_5$ (بخشی که ماشین را از q_3 به خود q_3 برمی‌گرداند) و $z = s_6s_7$. چون y یک «دور» است، می‌توان آن را هر چند بار تکرار یا حذف کرد و ماشین باز هم در همان حالت پایانی تمام می‌کند.



اثبات. فرض کنید DFA $M = (Q, \Sigma, \delta, q_1, F)$ زبان A را تشخیص دهد و $p = |Q|$. فرض کنید $s = s_1 s_2 \cdots s_n \in A$ و $n \geq p$. حالت‌هایی که M هنگام پردازش s از آن‌ها عبور می‌کند را $r_1, r_2, \ldots, r_{n+1}$ بنامید؛ یعنی $r_1 = q_1$ و $r_{i+1} = \delta(r_i, s_i)$. چون $s \in A$، داریم $r_{n+1} \in F$.

در $p+1$ حالت اول این دنباله $(r_1, \ldots, r_{p+1})$ دست‌کم یک حالت تکراری هست، چون فقط p حالت متفاوت داریم (اصل لانه کبوتری). فرض کنید r_j و r_l اولین دو ظهور یک حالت تکراری باشند $(j < l)$. چون r_l در میان $p+1$ حالت اول است، $l \leq p+1$. قرار دهید

$$x = s_1 \cdots s_{j-1}, \qquad y = s_j \cdots s_{l-1}, \qquad z = s_l \cdots s_n.$$

- x ماشین را از r_1 به r_j، y آن را از r_j به $r_l = r_j$ و z آن را از r_l به $r_{n+1} \in F$ می‌برد. چون y از r_j به خود r_j برمی‌گردد، $xy^i z$ هم برای هر $i \geq 0$ ماشین را به r_{n+1} می‌برد. پس $xy^i z \in A$.
- چون $j \neq l$، $|y| = l - j > 0$.
- چون $l \leq p+1$، $|xy| = l - 1 \leq p$.

■

۳.۸ روش استفاده از لم تزریق

لم تزریق را با برهان خلف به کار می‌بریم:

۱. فرض کنید زبان B منظم است. پس طول تزریقی مثل p دارد.

۲. رشته‌ای مثل $s \in B$ با $|s| \geq p$ پیدا کنید (معمولاً بر حسب p).

۳. نشان دهید **هر** تقسیم $s = xyz$ که شرط‌های ۲ و ۳ را داشته باشد، یک i دارد که $xy^i z \notin B$.

۴. این با لم تزریق تناقض دارد، پس B منظم نیست.

این روش را می‌توان مثل یک **بازی** دید: حریف p را انتخاب می‌کند، شما s را انتخاب می‌کنید، حریف تقسیم xyz را انتخاب می‌کند و شما i را. اگر همیشه بتوانید i را طوری انتخاب کنید که $xy^i z$ از زبان خارج شود، زبان منظم نیست. (این همان نقیض سورهای فصل ۱ است.) انتخاب s مناسب معمولاً به خلاقیت نیاز دارد.

دقت کنید! لم تزریق می‌گوید هر زبان منظم این ویژگی را دارد، اما **عکس آن درست نیست**: زبانی که شرایط لم را دارد ممکن است باز هم منظم نباشد (مثالش را در بخش ۶.۸ می‌بینیم). پس لم تزریق فقط برای اثبات **نامنظم بودن** به کار می‌رود، نه منظم بودن.

۴.۸ مثال‌ها

مثال ۳.۸ $(0^n 1^n)$. $B = \{0^n 1^n \mid n \geq 0\}$ منظم نیست.

فرض کنید B منظم باشد و p طول تزریق آن. رشته $s = 0^p 1^p \in B$ را انتخاب کنید. هر تقسیم $s = xyz$ را در نظر بگیرید. سه حالت برای y هست:

۱. y فقط 0 دارد: $xyyz$ تعداد 0 بیشتری از 1 دارد، پس در B نیست.

۲. y فقط 1 دارد: $xyyz$ تعداد 1 بیشتری دارد، پس در B نیست.

۳. y هم 0 و هم 1 دارد: $xyyz$ ممکن است تعداد برابری 0 و 1 داشته باشد، اما ترتیبشان به هم می‌خورد (یک 1 قبل از یک 0 می‌آید)، پس در B نیست.

در همه حالت‌ها $xy^2z \notin B$؛ تناقض. (با شرط $|xy| \leq p$ می‌شد حالت‌های ۲ و ۳ را از ابتدا کنار گذاشت، چون y باید در p نماد اول، یعنی فقط بین 0ها باشد.)

مثال ۴.۸ (رشته تکراری). $F = \{ww \mid w \in \{0,1\}^*\}$ منظم نیست.

فرض کنید F منظم باشد با طول تزریق p. رشته $s = 0^p 1\, 0^p 1 \in F$ را انتخاب کنید. چون $|xy| \leq p$، هم x و هم y فقط از 0های اول تشکیل شده‌اند: $x = 0^m$ و $y = 0^k$ با $m + k \leq p$ و $k \geq 1$. آنگاه $xy^0z = 0^{p-k}1\,0^p1$. فرض کنید این رشته به شکل uu باشد. چون xz دقیقاً دو 1 دارد و به 1 ختم می‌شود، هر نیمه باید دقیقاً یک 1 داشته باشد و به 1 ختم شود. پس نیمه اول $u = 0^{p-k}1$ و نیمه دوم $u = 0^p1$ است، که برابر نیستند چون $k \geq 1$. پس $xz \notin F$؛ تناقض.

(اسلایدهای درس از $s = 0^p1^p0^p1^p$ استفاده کرده‌اند که آن هم کار می‌کند. دقت کنید که $s = 0^p0^p$ انتخاب بدی است، چون می‌توان آن را تزریق کرد: $y = 00$.)

مثال ۵.۸ (پالیندروم‌های زوج). $L = \{ww^R \mid w \in \{a,b\}^*\}$ منظم نیست.

فرض کنید L منظم باشد با طول تزریق p. رشته $s = a^pb^pb^pa^p = a^pb^{2p}a^p \in L$ را انتخاب کنید. چون $|xy| \leq p$، $y = a^k$ با $1 \leq k \leq p$ و فقط از aهای اول است. آنگاه

$$xy^2z = a^{p+k}b^{2p}a^p.$$

این رشته در L نیست، چون اگر پالیندروم زوج باشد باید تعداد aهای ابتدا و انتهایش برابر باشد، اما $p + k \neq p$. تناقض.

مثال ۶.۸ (جمع اندیس‌ها). $L = \{a^nb^lc^{n+l} \mid n, l \geq 0\}$ منظم نیست.

فرض کنید L منظم باشد با طول تزریق p. رشته $s = a^pb^pc^{2p} \in L$ را انتخاب کنید. باز هم $y = a^k$ با $1 \leq k \leq p$. با $i = 0$:

$$xz = a^{p-k}b^pc^{2p}.$$

برای اینکه این رشته در L باشد باید $2p = (p-k) + p$، یعنی $k = 0$، که ممکن نیست. تناقض.

مثال ۷.۸ (فاکتوریل). $L = \{a^{n!} \mid n \geq 0\}$ منظم نیست.

فرض کنید L منظم باشد با طول تزریق p. می‌توانیم فرض کنیم $p \geq 2$ (اگر لم برای p برقرار باشد، برای هر عدد بزرگ‌تر هم برقرار است). رشته $s = a^{p!} \in L$ را انتخاب کنید. پس $y = a^k$ با $1 \leq k \leq p$ و

$$xy^2z = a^{p!+k}.$$

اما $p! < p! + k \leq p! + p < p! + p \cdot p! = (p+1)!$. (نامساوی آخر چون $p! \geq 2$ و پس $p < p \cdot p!$.) پس $p! + k$ اکیداً بین دو فاکتوریل متوالی $p!$ و $(p+1)!$ است و خودش فاکتوریل نیست. یعنی $xy^2z \notin L$؛ تناقض.

۵.۸ استفاده از ویژگی‌های بستاری

راه دیگر اثبات نامنظم بودن، استفاده از بستاری است:

۱. فرض کنید زبان داده‌شده منظم است.

۲. یک عملیات که زبان‌های منظم تحت آن بسته‌اند (اجتماع، الحاق، ستاره، متمم، اشتراک، معکوس) را، احتمالاً همراه با یک زبان منظم دیگر، روی آن اعمال کنید.

۳. نشان دهید نتیجه منظم نیست (مثلاً یک زبان شناخته‌شده نامنظم است). این تناقض است.

مثال ۸.۸ (تعداد برابر 0 و 1). $C = \{w \mid w$ دارد تعداد برابری 0 و 1$\}$ منظم نیست.

روش ۱ (لم تزریق): با $s = 0^p1^p \in C$، شرط $|xy| \leq p$ نتیجه می‌دهد y فقط 0 دارد، پس $xyyz$ تعداد 0 بیشتری دارد و در C نیست.

روش ۲ (بستاری): اگر C منظم بود، $C \cap 0^*1^*$ هم منظم بود (اشتراک دو زبان منظم). اما $C \cap 0^*1^* = \{0^n1^n \mid n \geq 0\}$ که در مثال ۴.۸ دیدیم منظم نیست. تناقض.

مثال ۹.۸ (دو نیمه متفاوت). $D = \{xy \mid x, y \in \{0,1\}^*,\ |x| = |y|,\ x \neq y\}$ منظم نیست.

فرض کنید D منظم باشد. پس $\overline{D}$ هم منظم است. رشته‌ای به طول فرد در D نیست، پس $\overline{D} = \{ww \mid w \in \{0,1\}^*\} \cup \{u \mid |u|$ فرد است$\}$. با اشتراک با زبان منظم $(\Sigma\Sigma)^*$ (رشته‌های با طول زوج):

$$\overline{D} \cap (\Sigma\Sigma)^* = \{ww \mid w \in \{0,1\}^*\}$$

که منظم نیست. تناقض.

مثال ۱۰.۸. $G = \{0^m1^n \mid m \neq n\}$ منظم نیست: اگر بود، $\overline{G} \cap 0^*1^* = \{0^n1^n\}$ هم منظم بود. به همین ترتیب $H = \{w \in \{0,1\}^* \mid$ تعداد 0 و 1 در w برابر نیست$\}$ منظم نیست، چون $\overline{H} = C$.

۶.۸ یک زبان تزریق‌پذیر که منظم نیست

مثال ۱۱.۸. زبان $E = \{a^kb^mc^n \mid k, m, n \geq 0\ m = n$ آنگاه $k = 1\}$ و اگر را در نظر بگیرید.

E **شرایط لم تزریق را دارد.** طول تزریق را $p = 2$ بگیرید و $s \in E$ را با $|s| \geq 2$ در نظر بگیرید. در همه حالت‌ها $x = \varepsilon$ است:

- $k = 0$ و $m = 0$: $s = c^n$ با $y = c$. $xy^iz = c^{n+i-1} \in E$.
- $k = 0$ و $m > 0$: $s = b^mc^n$ با $y = b$. $xy^iz = b^{m+i-1}c^n \in E$.
- $k = 1$ (پس $m = n$): $s = ab^nc^n$ با $y = a$. $xy^iz = a^ib^nc^n$. برای $i = 1$ همان s است و برای $i \neq 1$ تعداد aها 1 نیست و شرطی لازم نیست. پس در E است.
- $k = 2$: $s = aab^mc^n$ با $y = aa$. $xy^iz = a^{2i}b^mc^n \in E$، چون $2i$ هرگز برابر ۱ نیست.
- $k \geq 3$: با $y = a$. $xy^iz = a^{k+i-1}b^mc^n \in E$، چون $k + i - 1 \geq 2$.

در همه حالت‌ها $|xy| \leq 2$ و $|y| > 0$. پس هر رشته به طول دست‌کم ۲ تزریق‌پذیر است.
اما E منظم نیست. اگر بود، $F = E \cap ab^*c^* = \{ab^n c^n \mid n \geq 0\}$ هم منظم بود. F را با لم تزریق بررسی می‌کنیم: با $s = ab^p c^p$ و هر تقسیم با $|xy| \leq p$ و $|y| > 0$: اگر y در a باشد، xy^0z با a شروع نمی‌شود؛ اگر نباشد، y فقط b دارد و xy^0z تعداد برابری b و c ندارد. پس F منظم نیست و در نتیجه E هم منظم نیست.

۷.۸ ویژگی‌های بستاری زبان‌های نامنظم

قضیه ۱۲.۸

اگر L نامنظم باشد، $\overline{L}$ و L^R هم نامنظم‌اند.

اثبات. با برهان خلف. اگر $\overline{L}$ منظم بود، چون زبان‌های منظم تحت متمم بسته‌اند، $\overline{\overline{L}} = L$ هم منظم بود؛ تناقض.
به همین ترتیب اگر L^R منظم بود، طبق بستاری تحت معکوس $(L^R)^R = L$ منظم بود؛ تناقض.
■

قضیه ۱۳.۸

دسته زبان‌های نامنظم تحت اجتماع، اشتراک و ستاره **بسته نیست**.

اثبات. برای هر مورد دو زبان نامنظم پیدا می‌کنیم که نتیجه عملیات روی آن‌ها منظم باشد. فرض کنید $A = \{0^n 1^n \mid n \geq 0\}$ که نامنظم است؛ طبق قضیه قبل $\overline{A}$ هم نامنظم است.

- **اجتماع:** $A \cup \overline{A} = \Sigma^*$ منظم است.
- **اشتراک:** $A \cap \overline{A} = \emptyset$ منظم است.
- **ستاره:** قرار دهید $L = A \cup \{0,1\}$. L نامنظم است: اگر منظم بود، $L - \{0,1\} = A$ هم منظم بود (چون $0,1 \notin A$ و $\{0,1\}$ منظم است). اما $L^* \supseteq \{0,1\}^*$، پس $L^* = \Sigma^*$ منظم است.

■

نکته. «بسته نبودن» یعنی نتیجه **همیشه** در دسته نمی‌ماند، نه اینکه هیچ‌وقت نمی‌ماند. مثلاً $A \cup A = A$ نامنظم است.

جمع‌بندی فصل

- لم تزریق: در هر زبان منظم، رشته‌های بلند بخشی دارند (y، ناتهی و در p نماد اول) که می‌توان آن را تکرار یا حذف کرد. اثبات: اصل لانه کبوتری روی حالت‌های DFA.
- روش استفاده: برهان خلف؛ s را خوب انتخاب کنید و برای **هر** تقسیم مجاز یک i بد پیدا کنید.
- عکس لم تزریق درست نیست؛ زبان تزریق‌پذیر هم می‌تواند نامنظم باشد.
- بستاری هم ابزار قوی‌ای است: با اشتراک با زبانی مثل 0^*1^* مسئله را به یک زبان نامنظم شناخته‌شده برگردانید.

- زبان‌های نامنظم تحت متمم و معکوس بسته‌اند، اما تحت اجتماع، اشتراک و ستاره نه.

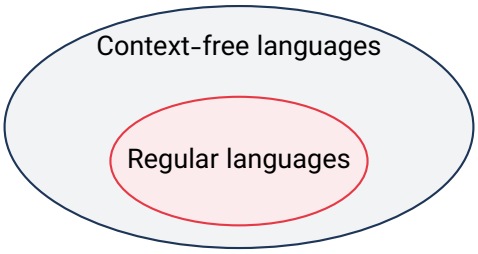
بخش سوم

زبان‌های مستقل از متن

۹ فصل

گرامرهای مستقل از متن

دیدیم که زبان‌هایی مثل $\{0^n1^n\}$ منظم نیستند. در این بخش ابزار قوی‌تری معرفی می‌کنیم: **گرامرهای مستقل از متن** (Context-Free Grammars). این گرامرها **زبان‌های مستقل از متن** (Context-Free Languages) را تولید می‌کنند که شامل همه زبان‌های منظم و زبان‌های دیگری مثل $\{0^n1^n\}$ هستند. کاربرد مهم آن‌ها در طراحی زبان‌های برنامه‌نویسی و ساخت **تجزیه‌گر** (parser) کامپایلرهاست: ابزارهای ساخت تجزیه‌گر، گرامر مستقل از متن زبان را می‌گیرند. در فصل‌های بعد ماشین معادل این گرامرها، یعنی **ماشین پشته‌ای**، را هم می‌بینیم.



۱.۹ گرامر و اشتقاق

یک گرامر از این بخش‌ها تشکیل شده است:

- مجموعه‌ای از **قاعده‌ها** (قواعد جایگزینی). هر قاعده یک خط است: یک متغیر، یک پیکان و یک رشته از متغیرها و پایانه‌ها.
- **متغیرها**، که معمولاً با حروف بزرگ نشان داده می‌شوند.
- **پایانه‌ها**، که شبیه الفبا هستند و معمولاً با حروف کوچک، اعداد یا نمادهای خاص نشان داده می‌شوند.
- یک **متغیر شروع**، که معمولاً سمت چپ اولین قاعده است.

مثال ۱.۹ (گرامر G_1).

$$A \to 0A1, \qquad A \to B, \qquad B \to \#.$$

متغیرها A و B، پایانه‌ها 0، 1 و $\#$ و متغیر شروع A است.

گرامر همه رشته‌های زبانش را با **اشتقاق** (Derivation) تولید می‌کند:

۱. متغیر شروع را بنویسید.

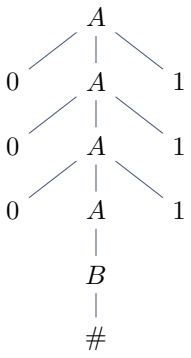
۲. یک متغیر از رشته فعلی و قاعده‌ای که با آن متغیر شروع می‌شود را انتخاب کنید و متغیر را با سمت راست قاعده جایگزین کنید.

۳. تا وقتی متغیری باقی مانده، به گام ۲ برگردید.

مثلاً اشتقاق رشته $000\#111$ در G_1:

$$A \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 000A111 \Rightarrow 000B111 \Rightarrow 000\#111.$$

همین اطلاعات را می‌توان با یک **درخت تجزیه** (Parse Tree) نشان داد. ریشه متغیر شروع است، هر متغیر فرزندانی دارد که سمت راست قاعده به‌کاررفته هستند و برگ‌ها، اگر از چپ به راست خوانده شوند، رشته مشتق‌شده را می‌سازند:



مجموعه همه رشته‌هایی که یک گرامر تولید می‌کند، **زبان گرامر** است. مثلاً $L(G_1) = \{0^n\#1^n \mid n \geq 0\}$. چند قاعده با سمت چپ یکسان را با | («یا») خلاصه می‌کنیم: $A \to 0A1 \mid B$.

مثال ۲.۹ (بخشی از زبان انگلیسی). گرامر G_2 بخش کوچکی از زبان انگلیسی را توصیف می‌کند (متغیرها را بین ⟨ ⟩ می‌نویسیم):

$$\begin{aligned}
\langle \text{SENTENCE} \rangle &\to \langle \text{NOUN-PHRASE} \rangle \langle \text{VERB-PHRASE} \rangle \\
\langle \text{NOUN-PHRASE} \rangle &\to \langle \text{CMPLX-NOUN} \rangle \mid \langle \text{CMPLX-NOUN} \rangle \langle \text{PREP-PHRASE} \rangle \\
\langle \text{VERB-PHRASE} \rangle &\to \langle \text{CMPLX-VERB} \rangle \mid \langle \text{CMPLX-VERB} \rangle \langle \text{PREP-PHRASE} \rangle \\
\langle \text{PREP-PHRASE} \rangle &\to \langle \text{PREP} \rangle \langle \text{CMPLX-NOUN} \rangle \\
\langle \text{CMPLX-NOUN} \rangle &\to \langle \text{ARTICLE} \rangle \langle \text{NOUN} \rangle \\
\langle \text{CMPLX-VERB} \rangle &\to \langle \text{VERB} \rangle \mid \langle \text{VERB} \rangle \langle \text{NOUN-PHRASE} \rangle \\
\langle \text{ARTICLE} \rangle &\to \text{a} \mid \text{the} \\
\langle \text{NOUN} \rangle &\to \text{boy} \mid \text{girl} \mid \text{flower} \\
\langle \text{VERB} \rangle &\to \text{touches} \mid \text{likes} \mid \text{sees} \\
\langle \text{PREP} \rangle &\to \text{with}
\end{aligned}$$

یک اشتقاق برای جمله «a boy sees»:

$$\begin{aligned}
\langle \text{SENTENCE} \rangle &\Rightarrow \langle \text{NOUN-PHRASE} \rangle \langle \text{VERB-PHRASE} \rangle \Rightarrow \langle \text{CMPLX-NOUN} \rangle \langle \text{VERB-PHRASE} \rangle \\
&\Rightarrow \langle \text{ARTICLE} \rangle \langle \text{NOUN} \rangle \langle \text{VERB-PHRASE} \rangle \Rightarrow \text{a } \langle \text{NOUN} \rangle \langle \text{VERB-PHRASE} \rangle \\
&\Rightarrow \text{a boy } \langle \text{VERB-PHRASE} \rangle \Rightarrow \text{a boy } \langle \text{CMPLX-VERB} \rangle \Rightarrow \text{a boy } \langle \text{VERB} \rangle \Rightarrow \text{a boy sees}
\end{aligned}$$

۲.۹ تعریف صوری

تعریف ۳.۹ (گرامر مستقل از متن)

یک **گرامر مستقل از متن** (CFG) یک چهارتایی (V, Σ, R, S) است که در آن

۱. V مجموعه‌ای متناهی از **متغیرها**،

۲. Σ مجموعه‌ای متناهی از **پایانه‌ها** است و $V \cap \Sigma = \emptyset$،

۳. R مجموعه‌ای متناهی از **قاعده‌ها** است؛ هر قاعده یک متغیر و یک رشته از متغیرها و پایانه‌هاست، یعنی $R \subseteq V \times (V \cup \Sigma)^*$،

۴. $S \in V$ **متغیر شروع** است.

«مستقل از متن» یعنی سمت چپ هر قاعده **فقط یک متغیر** است. پس هر وقت متغیر A را ببینیم، می‌توانیم آن را بدون توجه به اطرافش (متنی که در آن قرار دارد) جایگزین کنیم.

تعریف ۴.۹ (اشتقاق و زبان)

اگر u, v, w رشته‌هایی از متغیرها و پایانه‌ها باشند و $A \to w$ یک قاعده باشد، می‌گوییم uAv **نتیجه می‌دهد** uwv و می‌نویسیم $uAv \Rightarrow uwv$. اگر $u = v$ یا دنباله‌ای $u \Rightarrow u_1 \Rightarrow \cdots \Rightarrow u_k \Rightarrow v$ وجود داشته باشد، می‌گوییم u **مشتق می‌کند** v و می‌نویسیم $u \overset{*}{\Rightarrow} v$. زبان گرامر:

$$L(G) = \{w \in \Sigma^* \mid S \overset{*}{\Rightarrow} w\}.$$

مثال ۵.۹. گرامر $S \to aSb \mid \varepsilon$ را در نظر بگیرید. اجزای آن: $V = \{S\}$، $\Sigma = \{a, b\}$، $R = \{S \to aSb,\ S \to \varepsilon\}$ و متغیر شروع S. این گرامر زبان $\{a^n b^n \mid n \geq 0\}$ را تولید می‌کند که منظم نیست. پس **زبان‌های مستقل از متن از زبان‌های منظم بیشترند**.

نکته. هر گرامر خطی راست (فصل ۷) یک گرامر مستقل از متن است، چون سمت چپ قاعده‌هایش یک متغیر است. پس **هر زبان منظم مستقل از متن است**.

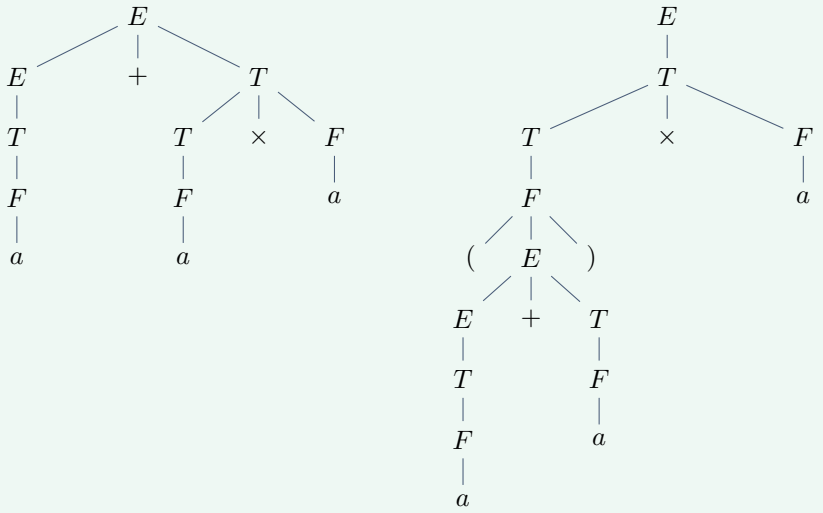
مثال ۶.۹ (پرانتزهای متوازن). $S \to aSb \mid SS \mid \varepsilon$ زبان همه رشته‌های پرانتزهای درست تودرتو را تولید می‌کند، اگر a را «(» و b را «)» بخوانیم: $S \to (S) \mid SS \mid \varepsilon$. مثلاً $(()())$.

مثال ۷.۹ (پالیندروم‌های زوج). $G_p:\ S \to aSa \mid bSb \mid \varepsilon$ زبان $L(G_p) = \{ww^R \mid w \in \{a, b\}^*\}$ را تولید می‌کند. هر قاعده یک نماد را هم‌زمان در ابتدا و انتها می‌گذارد.

مثال ۸.۹ (عبارت‌های حسابی). گرامر G_4 بخشی از یک زبان برنامه‌نویسی است:

$$\begin{aligned}
\langle \text{EXPR} \rangle &\rightarrow \langle \text{EXPR} \rangle + \langle \text{TERM} \rangle \mid \langle \text{TERM} \rangle \\
\langle \text{TERM} \rangle &\rightarrow \langle \text{TERM} \rangle \times \langle \text{FACTOR} \rangle \mid \langle \text{FACTOR} \rangle \\
\langle \text{FACTOR} \rangle &\rightarrow (\ \langle \text{EXPR} \rangle\ ) \mid a
\end{aligned}$$

درخت تجزیه **معنی** عبارت را نشان می‌دهد. در شکل‌های زیر E، T و F به جای ⟨EXPR⟩، ⟨TERM⟩ و ⟨FACTOR⟩ آمده‌اند. در $a + a \times a$ ضرب پایین‌تر از جمع است، یعنی اول انجام می‌شود. در $(a + a) \times a$ پرانتز جمع را پایین‌تر می‌برد:



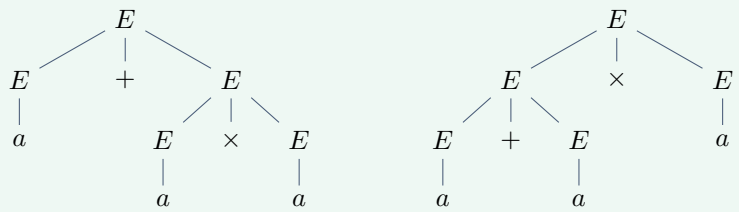
۳.۹ ابهام

گاهی یک گرامر یک رشته را به چند روش تولید می‌کند. به چنین رشته‌ای **مبهم** می‌گوییم. اگر رشته چند **درخت تجزیه** متفاوت داشته باشد، چند **معنی** متفاوت دارد، که در کاربردهایی مثل زبان برنامه‌نویسی مشکل‌ساز است.

مثال ۹.۹ (یک گرامر مبهم).

$$G_5: \quad \langle \text{EXPR} \rangle \rightarrow \langle \text{EXPR} \rangle + \langle \text{EXPR} \rangle \mid \langle \text{EXPR} \rangle \times \langle \text{EXPR} \rangle \mid (\langle \text{EXPR} \rangle) \mid a.$$

رشته $a + a \times a$ در G_5 دو درخت تجزیه دارد: در درخت چپ ضرب زودتر انجام می‌شود ($a + (a \times a)$) و در درخت راست جمع ($(a + a) \times a$):



گرامر G_4 همین زبان را بدون ابهام تولید می‌کند.

دو اشتقاق مختلف ممکن است فقط در **ترتیب** جایگزینی فرق داشته باشند و درخت تجزیه یکسانی بسازند. برای اینکه ابهام را به درخت (ساختار) مربوط کنیم نه به ترتیب، ترتیب را ثابت می‌کنیم:

تعریف ۱۰.۹ (اشتقاق چپ‌ترین و ابهام)

در **اشتقاق چپ‌ترین** در هر گام **چپ‌ترین** متغیر باقی‌مانده جایگزین می‌شود. رشته w **به‌طور مبهم** در گرامر G تولید می‌شود اگر دو اشتقاق چپ‌ترین متفاوت (یا معادلاً دو درخت تجزیه متفاوت) داشته باشد. گرامر G **مبهم** است اگر دست‌کم یک رشته را به‌طور مبهم تولید کند.

مثال ۱۱.۹. دو اشتقاق چپ‌ترین برای $a + a \times a$ در G_5 (E به جای ⟨EXPR⟩):

$$E \Rightarrow E \times E \Rightarrow E + E \times E \Rightarrow a + E \times E \Rightarrow a + a \times E \Rightarrow a + a \times a$$
$$E \Rightarrow E + E \Rightarrow a + E \Rightarrow a + E \times E \Rightarrow a + a \times E \Rightarrow a + a \times a$$

به جای چپ‌ترین می‌توان از **اشتقاق راست‌ترین** هم استفاده کرد (در هر گام راست‌ترین متغیر جایگزین شود)؛ تعریف ابهام تغییر نمی‌کند:

$$E \Rightarrow E + E \Rightarrow E + E \times E \Rightarrow E + E \times a \Rightarrow E + a \times a \Rightarrow a + a \times a$$
$$E \Rightarrow E \times E \Rightarrow E \times a \Rightarrow E + E \times a \Rightarrow E + a \times a \Rightarrow a + a \times a$$

مثال ۱۲.۹ (گرامر انگلیسی مبهم است). جمله «the girl touches the boy with the flower» در G_2 دو درخت تجزیه دارد. در یکی، with the flower بخشی از گروه اسمی the boy with the flower است (پسری که گل دارد). در دیگری بخشی از گروه فعلی است (دختر با گل پسر را لمس می‌کند):

- ⟨VERB⟩⟨NOUN-PHRASE⟩ → ⟨CMPLX-VERB⟩ و ⟨NOUN-PHRASE⟩ → ⟨CMPLX-NOUN⟩⟨PREP-PHRASE⟩؛
- ⟨CMPLX-VERB⟩⟨PREP-PHRASE⟩ → ⟨VERB-PHRASE⟩ و ⟨NOUN-PHRASE⟩ → ⟨CMPLX-NOUN⟩.

مثال ۱۳.۹ (مشکل else آویزان).

$$\begin{aligned}
\langle \text{IF-STMT} \rangle &\to \text{if } \langle \text{EXPR} \rangle \text{ then } \langle \text{STMT} \rangle \\
&\mid \text{if } \langle \text{EXPR} \rangle \text{ then } \langle \text{STMT} \rangle \text{ else } \langle \text{STMT} \rangle \\
\langle \text{EXPR} \rangle &\to E_1 \mid E_2 \\
\langle \text{STMT} \rangle &\to S_1 \mid S_2 \mid \langle \text{IF-STMT} \rangle
\end{aligned}$$

رشته if E_1 then if E_2 then S_1 else S_2 دو اشتقاق چپ‌ترین دارد. در اولی else به if درونی تعلق دارد (قاعده اول برای if بیرونی و قاعده دوم برای درونی) و در دومی به if بیرونی (برعکس). این دو برنامه رفتار متفاوتی دارند. زبان‌های برنامه‌نویسی معمولاً با یک قانون اضافه («else به نزدیک‌ترین if تعلق دارد») این ابهام را رفع می‌کنند.

تعریف ۱۴.۹ (زبان ذاتاً مبهم)

زبانی که **هیچ** گرامر غیرمبهمی برای آن وجود نداشته باشد **ذاتاً مبهم** است.

مثلاً زبان $\{a^i b^j c^k \mid i = j \text{ یا } j = k\}$ ذاتاً مبهم است: هر گرامری برای آن، دست‌کم یکی از رشته‌های $a^n b^n c^n$ را به دو روش تولید می‌کند (یک بار به عنوان $i = j$ و یک بار به عنوان $j = k$). اثبات دقیق این ادعا دشوار است و در این درس نمی‌آید.

نکته. تشخیص اینکه یک گرامر مستقل از متن مبهم است یا یک زبان ذاتاً مبهم است، **تصمیم‌ناپذیر** است: هیچ الگوریتمی وجود ندارد که برای هر گرامری در زمان متناهی جواب درست «بله» یا «نه» بدهد. (معنای دقیق تصمیم‌ناپذیری را در بخش چهارم می‌بینیم.)

جمع‌بندی فصل

- CFG چهارتایی (V, Σ, R, S) است و سمت چپ هر قاعده یک متغیر است.
- اشتقاق ($\Rightarrow$، $\overset{*}{\Rightarrow}$) و درخت تجزیه دو روش نمایش تولید یک رشته‌اند. $L(G) = \{w \in \Sigma^* \mid S \overset{*}{\Rightarrow} w\}$.
- زبان‌های منظم زیرمجموعه سره زبان‌های مستقل از متن‌اند ($\{a^n b^n\}$ مستقل از متن است ولی منظم نیست).
- گرامر مبهم یک رشته را با دو اشتقاق چپ‌ترین متفاوت تولید می‌کند. بعضی زبان‌ها ذاتاً مبهم‌اند و تشخیص ابهام الگوریتم ندارد.

۱۰ فصل

ساده‌سازی گرامرها و فرم‌های نرمال

گرامرهای مستقل از متن می‌توانند شکل‌های بسیار متنوعی داشته باشند. برای اثبات قضیه‌ها و ساختن الگوریتم‌ها (مثلاً الگوریتم CYK در فصل بعد) خیلی بهتر است گرامر شکل ساده و استانداردی داشته باشد. در این فصل نشان می‌دهیم سه نوع قاعده را همیشه می‌توان حذف کرد: قاعده‌های **بی‌فایده**، **قاعده‌های** ε و **قاعده‌های واحد**. سپس دو **فرم نرمال** معروف را معرفی می‌کنیم: فرم نرمال چامسکی و فرم نرمال گرایباخ.

> **نکته.** «ساده‌سازی» لزوماً به معنای کم شدن تعداد قاعده‌ها نیست؛ گاهی تعداد قاعده‌ها زیاد هم می‌شود. هدف این است که قاعده‌ها **شکل** ساده‌تری داشته باشند.

در این فصل گرامر را با نمادگذاری $G = (V, T, S, P)$ می‌نویسیم (T پایانه‌ها و P قاعده‌ها).

۱.۱۰ یک قاعده جایگزینی مفید

قضیه ۱.۱۰

فرض کنید گرامر $G = (V, T, S, P)$ قاعده‌های زیر را داشته باشد:

$$(*)\ \ A \to x_1 B x_2, \qquad (**)\ \ B \to y_1 \mid y_2 \mid \cdots \mid y_n,$$

که $A \neq B$ و $(**)$ همه قاعده‌هایی است که B در سمت چپشان است. اگر $\widehat{G} = (V, T, S, \widehat{P})$ را با حذف قاعده $(*)$ و اضافه کردن قاعده‌های

$$A \to x_1 y_1 x_2 \mid x_1 y_2 x_2 \mid \cdots \mid x_1 y_n x_2$$

بسازیم، آنگاه $L(\widehat{G}) = L(G)$.

اثبات. $L(\widehat{G}) \subseteq L(G)$: هر قاعده جدید $A \to x_1 y_i x_2$ را می‌توان در G با دو گام شبیه‌سازی کرد: $A \Rightarrow x_1 B x_2 \Rightarrow x_1 y_i x_2$. پس هر اشتقاق در $\widehat{G}$ را می‌توان به یک اشتقاق در G تبدیل کرد.

$L(G) \subseteq L(\widehat{G})$: اشتقاقی از w در G را در نظر بگیرید. اگر از $(*)$ استفاده نکند، در $\widehat{G}$ هم معتبر است. اگر استفاده کند، متغیر B که با آن ساخته شده بالاخره باید با یکی از y_iها جایگزین شود (چون w متغیری ندارد و $(**)$ تنها راه جایگزینی B است). ترتیب گام‌ها در گرامر مستقل از متن مهم نیست (جایگزینی‌های مستقل را می‌توان جابه‌جا کرد)، پس می‌توانیم جایگزینی این B را درست بعد از به کار بردن $(*)$ انجام دهیم. این دو گام با یک گام $A \to x_1 y_i x_2$ در $\widehat{G}$ برابرند. با تکرار این کار برای هر استفاده از $(*)$، اشتقاقی برای w در $\widehat{G}$ به دست می‌آید.

■

۲.۱۰ حذف متغیرها و قاعده‌های بی‌فایده

تعریف ۲.۱۰ (متغیر مفید و بی‌فایده)

متغیر A در گرامر $G = (V, T, S, P)$ **مفید** است اگر دست‌کم یک $w \in L(G)$ وجود داشته باشد که

$$S \overset{*}{\Rightarrow} xAy \overset{*}{\Rightarrow} w \qquad (x, y \in (V \cup T)^*).$$

یعنی A در اشتقاق دست‌کم یک رشته پایانه‌ای شرکت کند. متغیری که مفید نیست **بی‌فایده** است و قاعده‌ای که یک متغیر بی‌فایده دارد **قاعده بی‌فایده** است.

یک متغیر به دو دلیل ممکن است بی‌فایده باشد:

۱. از متغیر شروع **قابل دسترسی نیست** (هیچ اشتقاقی از S به آن نمی‌رسد)؛

۲. **هیچ رشته پایانه‌ای تولید نمی‌کند**.

قضیه ۳.۱۰

برای هر گرامر مستقل از متن G (با $L(G) \neq \emptyset$) گرامر هم‌ارزی $\widehat{G}$ وجود دارد که هیچ متغیر یا قاعده بی‌فایده‌ای ندارد.

اثبات. حذف در دو مرحله و **به همین ترتیب** انجام می‌شود.

مرحله ۱: حذف متغیرهایی که رشته پایانه‌ای تولید نمی‌کنند. مجموعه V_1 از متغیرهای «مولد» را این‌طور می‌سازیم: ابتدا هر A که قاعده‌ای مثل $A \to x$ با $x \in T^*$ دارد را در V_1 بگذار. سپس تا وقتی تغییری رخ می‌دهد، هر A که قاعده‌ای مثل $A \to x_1 x_2 \cdots x_k$ دارد که هر x_i یا پایانه است یا در V_1، را به V_1 اضافه کن. این حلقه حداکثر $|V|$ بار تکرار می‌شود. با استقرا روی طول اشتقاق روشن است که $A \in V_1$ اگر و تنها اگر A یک رشته پایانه‌ای تولید کند. همه متغیرهای خارج از V_1 و همه قاعده‌هایی که آن‌ها را دارند حذف می‌کنیم. زبان تغییر نمی‌کند، چون هیچ اشتقاق کاملی از آن متغیرها استفاده نمی‌کند (اگر استفاده می‌کرد، آن متغیر یک رشته پایانه‌ای تولید می‌کرد).

مرحله ۲: حذف متغیرهای غیرقابل دسترس. گراف وابستگی می‌سازیم: رأس‌ها متغیرها هستند و از A به B یال هست اگر قاعده‌ای مثل $A \to xBy$ وجود داشته باشد. هر متغیری که در این گراف از S مسیری به آن نیست، همراه با قاعده‌هایش حذف می‌شود. باز هم زبان تغییر نمی‌کند.

بعد از این دو مرحله هر متغیر باقی‌مانده A از S قابل دسترسی است ($S \overset{*}{\Rightarrow} xAy$) و هر متغیر (از جمله A و متغیرهای درون x و y) یک رشته پایانه‌ای تولید می‌کند. پس برای یک $S \overset{*}{\Rightarrow} xAy \overset{*}{\Rightarrow} w$ برای یک $w \in T^*$، یعنی A مفید است.

■

دقت کنید! ترتیب مراحل مهم است. اگر اول متغیرهای غیرقابل دسترس را حذف کنیم، ممکن است حذف متغیرهای نامولد متغیرهای جدیدی را غیرقابل دسترس کند.

مثال ۴.۱۰.

$$S \to aS \mid A \mid C, \qquad A \to a, \qquad B \to aa, \qquad C \to aCb.$$

مرحله ۱: C هیچ رشته پایانه‌ای تولید نمی‌کند (هر بار C دوباره ظاهر می‌شود). پس C و قاعده‌های $S \to C$ و $C \to aCb$ حذف می‌شوند:

$$S \to aS \mid A, \qquad A \to a, \qquad B \to aa.$$

مرحله ۲: در گراف وابستگی یال‌های $S \to S$ و $S \to A$ را داریم و هیچ یالی به B نمی‌رسد. پس B حذف می‌شود:

$$S \to aS \mid A, \qquad A \to a.$$

۳.۱۰ حذف قاعده‌های ε

تعریف ۵.۱۰ (قاعده ε و متغیر تهی‌پذیر)

قاعده‌ای به شکل $A \to \varepsilon$ یک **قاعده** ε است. متغیر A که برای آن $A \overset{*}{\Rightarrow} \varepsilon$ ممکن باشد **تهی‌پذیر** (nullable) نام دارد.

قضیه ۶.۱۰

برای هر گرامر مستقل از متن G که $\varepsilon \notin L(G)$، گرامر هم‌ارزی $\widehat{G}$ بدون قاعده ε وجود دارد.

اثبات. **گام ۱: یافتن متغیرهای تهی‌پذیر.** مجموعه V_N را این‌طور می‌سازیم:

۱. برای هر قاعده $A \to \varepsilon$، A را در V_N بگذار.

۲. تا وقتی تغییری رخ می‌دهد: برای هر قاعده $B \to A_1 A_2 \cdots A_n$ که همه $A_1, \ldots, A_n$ در V_N باشند، B را به V_N اضافه کن.

گام ۲: ساختن قاعده‌های جدید. هر قاعده به شکل $A \to x_1 x_2 \cdots x_m$ با $m \geq 1$ (که $x_i \in V \cup T$) را در $\widehat{P}$ می‌گذاریم و همراه آن، **همه نسخه‌هایی** را که از حذف هر ترکیبی از متغیرهای تهی‌پذیر به دست می‌آیند. تنها استثنا: اگر همه x_iها تهی‌پذیر باشند، نسخه‌ای که همه حذف شده‌اند (یعنی $A \to \varepsilon$) را اضافه نمی‌کنیم. قاعده‌های ε اصلی هم کنار گذاشته می‌شوند.

درستی. هر قاعده جدید در G قابل شبیه‌سازی است: متغیرهای حذف‌شده تهی‌پذیرند و در G می‌توان آن‌ها را به ε تبدیل کرد؛ پس $L(\widehat{G}) \subseteq L(G)$. برعکس، یک درخت تجزیه برای $w \in L(G)$ (که $w \neq \varepsilon$) را در نظر بگیرید. هر زیردرختی که فقط ε تولید می‌کند را حذف کنید و در گره پدر آن، به جای قاعده اصلی از نسخه‌ای استفاده کنید که آن متغیر حذف شده است. این نسخه در $\widehat{P}$ هست، چون آن متغیر تهی‌پذیر است و چون آن گره پدر چیزی ناتهی تولید می‌کند، همه فرزندانش حذف نشده‌اند. نتیجه یک درخت تجزیه معتبر در $\widehat{G}$ برای w است. پس $L(G) \subseteq L(\widehat{G})$.
■

مثال ۷.۱۰.

$$S \to ABaC, \quad A \to BC, \quad B \to b \mid \varepsilon, \quad C \to D \mid \varepsilon, \quad D \to d.$$

متغیرهای تهی‌پذیر: ابتدا B و C (قاعده ε دارند)، سپس A (چون $A \to BC$ و هر دو تهی‌پذیرند). پس $V_N = \{A, B, C\}$. گرامر جدید:

$$S \to ABaC \mid BaC \mid AaC \mid ABa \mid aC \mid Aa \mid Ba \mid a,$$
$$A \to B \mid C \mid BC, \qquad B \to b, \qquad C \to D, \qquad D \to d.$$

۴.۱۰ حذف قاعده‌های واحد

تعریف ۸.۱۰ (قاعده واحد)

قاعده‌ای به شکل $A \to B$ که $A, B \in V$ یک **قاعده واحد** است.

قضیه ۹.۱۰

برای هر گرامر مستقل از متن G بدون قاعده ε، گرامر هم‌ارزی $\widehat{G}$ بدون قاعده واحد وجود دارد.

اثبات.

۱. قاعده‌های به شکل $A \to A$ را حذف کنید؛ آن‌ها هیچ اثری ندارند.

۲. برای هر متغیر A همه متغیرهای B را پیدا کنید که $A \overset{*}{\Rightarrow} B$ فقط با قاعده‌های واحد. برای این کار گراف وابستگی می‌سازیم که رأس‌هایش متغیرها هستند و برای هر قاعده واحد $C \to D$ یالی از C به D دارد. $A \overset{*}{\Rightarrow} B$ اگر و تنها اگر مسیری از A به B باشد.

۳. همه قاعده‌های غیرواحد P را در $\widehat{P}$ بگذارید.

۴. برای هر زوج $A \overset{*}{\Rightarrow} B$، اگر $B \to y_1 \mid \cdots \mid y_n$ همه قاعده‌های غیرواحد B در $\widehat{P}$ باشند، قاعده‌های $A \to y_1 \mid \cdots \mid y_n$ را به $\widehat{P}$ اضافه کنید.

درستی. هر قاعده جدید $A \to y_i$ در G با $A \overset{*}{\Rightarrow} B \Rightarrow y_i$ قابل شبیه‌سازی است. برعکس، در هر اشتقاق چپ‌ترین در G، هر دنباله از قاعده‌های واحد پشت سر هم مثل $A \Rightarrow B_1 \Rightarrow \cdots \Rightarrow B_k \Rightarrow y$ (که آخری غیرواحد است) با یک گام $A \to y$ در $\widehat{G}$ جایگزین می‌شود. پس دو گرامر یک زبان را تولید می‌کنند.

■

مثال ۱۰.۱۰.

$$S \to Aa \mid B, \qquad B \to A \mid bb, \qquad A \to a \mid bc \mid B.$$

گراف وابستگی قاعده‌های واحد یال‌های $S \to B$، $B \to A$ و $A \to B$ را دارد. پس $S \overset{*}{\Rightarrow} A$، $S \overset{*}{\Rightarrow} B$، $A \overset{*}{\Rightarrow} B$ و $B \overset{*}{\Rightarrow} A$. قاعده‌های غیرواحد:

$$S \to Aa, \qquad B \to bb, \qquad A \to a \mid bc.$$

حالا قاعده‌های جدید را اضافه می‌کنیم:

- از $S \overset{*}{\Rightarrow} A$: $S \to a \mid bc$،

- از $S \overset{*}{\Rightarrow} B$: $S \to bb$،
- از $A \overset{*}{\Rightarrow} B$: $A \to bb$،
- از $B \overset{*}{\Rightarrow} A$: $B \to a \mid bc$.

نتیجه:

$$S \to Aa \mid a \mid bc \mid bb, \qquad A \to a \mid bc \mid bb, \qquad B \to bb \mid a \mid bc.$$

(B حالا غیرقابل دسترس و بی‌فایده است و می‌توان آن را حذف کرد.)

قضیه ۱۱.۱۰

اگر L زبانی مستقل از متن باشد و $\varepsilon \notin L$، گرامری برای L وجود دارد که قاعده بی‌فایده، قاعده ε و قاعده واحد ندارد.

اثبات. سه الگوریتم بالا را به این ترتیب اجرا می‌کنیم: (۱) حذف قاعده‌های ε، (۲) حذف قاعده‌های واحد، (۳) حذف قاعده‌های بی‌فایده. این ترتیب درست کار می‌کند چون حذف قاعده‌های واحد قاعده ε جدیدی نمی‌سازد و حذف قاعده‌های بی‌فایده نه قاعده ε می‌سازد و نه قاعده واحد (فقط قاعده‌ها را حذف می‌کند).

■

۵.۱۰ فرم نرمال چامسکی

تعریف ۱۲.۱۰ (فرم نرمال چامسکی (CNF))

گرامر مستقل از متن در **فرم نرمال چامسکی** (Chomsky Normal Form) است اگر هر قاعده‌اش به یکی از این دو شکل باشد:

$$A \to BC, \qquad A \to a,$$

که a پایانه است و A, B, C متغیرند و B و C متغیر شروع نیستند. علاوه بر این، قاعده $S \to \varepsilon$ (برای متغیر شروع S) هم مجاز است.

قضیه ۱۳.۱۰

هر زبان مستقل از متن را یک گرامر در فرم نرمال چامسکی تولید می‌کند.

ایده اثبات. هر گرامر را با چند گام به CNF تبدیل می‌کنیم: متغیر شروع جدید اضافه می‌کنیم، قاعده‌های ε و واحد را حذف می‌کنیم و قاعده‌های باقی‌مانده را به شکل درست درمی‌آوریم.

اثبات. فرض کنید G گرامری برای زبان باشد.

۱. **متغیر شروع جدید:** متغیر S_0 و قاعده $S_0 \to S$ را اضافه کنید. حالا متغیر شروع هرگز در سمت راست قاعده‌ای ظاهر نمی‌شود.

۲. **حذف قاعده‌های ε:** هر قاعده $A \to \varepsilon$ با $A \neq S_0$ را حذف کنید. برای هر ظهور A در سمت راست یک قاعده،

نسخه‌ای از آن قاعده را که آن ظهور حذف شده اضافه کنید (برای همه ترکیب‌ها؛ مثلاً از $R \to uAvAw$ نسخه‌های $R \to uAvw$، $R \to uvAw$ و $R \to uvw$ را می‌گیریم). اگر $R \to A$ داشتیم، $R \to \varepsilon$ اضافه می‌شود، مگر اینکه قبلاً $R \to \varepsilon$ را حذف کرده باشیم. این کار را تا وقتی قاعده ε (جز احتمالاً $S_0 \to \varepsilon$) باقی است تکرار کنید.

۳. **حذف قاعده‌های واحد:** هر قاعده $A \to B$ را حذف کنید و برای هر قاعده $B \to u$، قاعده $A \to u$ را اضافه کنید (مگر اینکه $A \to u$ یک قاعده واحد باشد که قبلاً حذف شده). تکرار کنید تا قاعده واحدی نماند.

۴. **شکستن قاعده‌های بلند:** هر قاعده $A \to u_1 u_2 \cdots u_k$ با $k \geq 3$ را با زنجیره‌ای از قاعده‌ها با متغیرهای جدید جایگزین کنید:

$$A \to u_1 A_1, \quad A_1 \to u_2 A_2, \quad \ldots, \quad A_{k-2} \to u_{k-1} u_k.$$

۵. **جدا کردن پایانه‌ها:** در هر قاعده با دو نماد سمت راست، هر پایانه u_i را با متغیر جدید U_i جایگزین کنید و قاعده $U_i \to u_i$ را اضافه کنید.

هر گام زبان را حفظ می‌کند (برای گام‌های ۲ و ۳ مثل اثبات‌های بالا، و گام‌های ۴ و ۵ همان قضیه جایگزینی ۱.۱۰ به صورت برعکس هستند). در پایان هر قاعده شکل $A \to BC$، $A \to a$ یا $S_0 \to \varepsilon$ دارد.

■

مثال ۱۴.۱۰ (تبدیل به CNF). گرامر زیر را به CNF تبدیل می‌کنیم:

$$S \to ASA \mid aB, \qquad A \to B \mid S, \qquad B \to b \mid \varepsilon.$$

۱. متغیر شروع جدید:

$$S_0 \to S, \quad S \to ASA \mid aB, \quad A \to B \mid S, \quad B \to b \mid \varepsilon.$$

۲. حذف $B \to \varepsilon$: از $S \to aB$ قاعده $S \to a$ و از $A \to B$ قاعده $A \to \varepsilon$ به دست می‌آید:

$$S_0 \to S, \quad S \to ASA \mid aB \mid a, \quad A \to B \mid S \mid \varepsilon, \quad B \to b.$$

حذف $A \to \varepsilon$: از $S \to ASA$ نسخه‌های SA، AS و S ساخته می‌شوند:

$$S_0 \to S, \quad S \to ASA \mid aB \mid a \mid SA \mid AS \mid S, \quad A \to B \mid S, \quad B \to b.$$

۳. حذف قاعده‌های واحد: $S \to S$ را حذف می‌کنیم. با حذف $S_0 \to S$ همه قاعده‌های S به S_0 اضافه می‌شوند. با حذف $A \to B$ قاعده $A \to b$ و با حذف $A \to S$ همه قاعده‌های S به A اضافه می‌شوند:

$$\begin{aligned} &S_0 \to ASA \mid aB \mid a \mid SA \mid AS, \qquad S \to ASA \mid aB \mid a \mid SA \mid AS, \\ &A \to b \mid ASA \mid aB \mid a \mid SA \mid AS, \qquad B \to b. \end{aligned}$$

۴ و ۵. شکل نهایی: ASA را با AA_1 و قاعده جدید $A_1 \to SA$ جایگزین می‌کنیم و در aB پایانه a را با متغیر U و قاعده $U \to a$:

$$\begin{aligned} &S_0 \to AA_1 \mid UB \mid a \mid SA \mid AS, \qquad S \to AA_1 \mid UB \mid a \mid SA \mid AS, \\ &A \to b \mid AA_1 \mid UB \mid a \mid SA \mid AS, \qquad A_1 \to SA, \qquad U \to a, \qquad B \to b. \end{aligned}$$

۶.۱۰ فرم نرمال گرایباخ

در فرم نرمال گرایباخ محدودیت روی **جای** پایانه‌ها و متغیرهاست: هر قاعده دقیقاً با یک پایانه شروع می‌شود.

تعریف ۱۵.۱۰ (فرم نرمال گرایباخ (GNF))

گرامر مستقل از متن در **فرم نرمال گرایباخ** (Greibach Normal Form) است اگر همه قاعده‌هایش به شکل

$$A \to ax \qquad (a \in T,\ x \in V^*)$$

باشند.

قضیه ۱۶.۱۰

برای هر گرامر مستقل از متن G با $\varepsilon \notin L(G)$، گرامر هم‌ارزی در فرم نرمال گرایباخ وجود دارد.

به‌طور کلی تبدیل یک گرامر به GNF و اثبات اینکه این کار همیشه ممکن است ساده نیست و در این درس به آن نمی‌پردازیم. اما در مثال‌های ساده، قضیه جایگزینی ۱.۱۰ معمولاً کافی است.

مثال ۱۷.۱۰. گرامر

$$S \to AB, \qquad A \to aA \mid bB \mid b, \qquad B \to b$$

در GNF نیست، چون $S \to AB$ با متغیر شروع می‌شود. با قضیه جایگزینی، A را در $S \to AB$ با سمت راست قاعده‌هایش جایگزین می‌کنیم:

$$S \to aAB \mid bBB \mid bB, \qquad A \to aA \mid bB \mid b, \qquad B \to b.$$

این گرامر در GNF است.

نکته. در GNF هر گام اشتقاق دقیقاً یک پایانه تولید می‌کند. پس اشتقاق یک رشته به طول n دقیقاً n گام دارد. در فصل بعد از این ایده برای تجزیه سریع استفاده می‌کنیم.

جمع‌بندی فصل

- قضیه جایگزینی: می‌توان یک متغیر در سمت راست را با همه سمت‌راست‌های قاعده‌هایش جایگزین کرد.
- حذف بی‌فایده‌ها: اول متغیرهای ناتولد، بعد متغیرهای غیرقابل دسترس.
- حذف قاعده‌های ε با یافتن متغیرهای تهی‌پذیر؛ حذف قاعده‌های واحد با گراف وابستگی.
- CNF: فقط $A \to BC$ و $A \to a$ (و $S \to \varepsilon$). هر CFL یک گرامر CNF دارد.
- GNF: فقط $A \to ax$ با $x \in V^*$.

۱۱ فصل

عضویت، تجزیه و الگوریتم CYK

وقتی یک گرامر داریم، طبیعی‌ترین سؤال این است: «آیا رشته w در زبان این گرامر هست؟» و اگر هست، «چطور تولید می‌شود؟» کامپایلر دقیقاً همین کار را با کد برنامه انجام می‌دهد. در این فصل چند الگوریتم برای این دو سؤال می‌بینیم.

۱.۱۱ صورت جمله‌ای، عضویت و تجزیه

تعریف ۱.۱۱ (صورت جمله‌ای)

اگر $w \in L(G)$ و $S \Rightarrow w_1 \Rightarrow \cdots \Rightarrow w_n \Rightarrow w$ یک اشتقاق برای w باشد، به w یک **جمله** و به رشته‌های میانی $S, w_1, \ldots, w_n$ که هم متغیر و هم پایانه دارند **صورت جمله‌ای** (Sentential Form) می‌گوییم.

مثال ۲.۱۱. در گرامر $S \to aSb \mid \varepsilon$ داریم $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$، پس $S \overset{*}{\Rightarrow} aabb$. رشته $aabb$ یک جمله از زبان است و $aaSbb$ یک صورت جمله‌ای.

برای گرامر G و رشته w دو مسئله داریم:

- **عضویت:** آیا $w \in L(G)$؟ به الگوریتمی که این را مشخص کند **الگوریتم عضویت** می‌گوییم.
- **تجزیه** (parsing): اگر $w \in L(G)$، دنباله قاعده‌هایی را پیدا کن که w را تولید می‌کنند.

۲.۱۱ تجزیه با جست‌وجوی کامل

ساده‌ترین روش، امتحان کردن همه اشتقاق‌هاست (**تجزیه با جست‌وجوی کامل** یا **force brute**): در دور اول همه قاعده‌های $S \to x$ را به کار می‌بریم. در هر دور بعدی، چپ‌ترین متغیر هر صورت جمله‌ای را با همه قاعده‌های ممکن جایگزین می‌کنیم. اگر $w \in L(G)$، اشتقاق چپ‌ترینی با طول متناهی دارد، پس این روش بالاخره آن را پیدا می‌کند.

مثال ۳.۱۱. $w = aabb$ و گرامر $S \to SS \mid aSb \mid bSa \mid \varepsilon$. **دور اول:** $S \Rightarrow SS$، $S \Rightarrow aSb$، $S \Rightarrow bSa$ و $S \Rightarrow \varepsilon$. دو صورت آخر به درد نمی‌خورند (w با b شروع نمی‌شود و $\varepsilon \neq w$)، پس کنار می‌روند. **دور دوم:**

$$S \Rightarrow SS \Rightarrow SSS, \quad S \Rightarrow SS \Rightarrow aSbS, \quad S \Rightarrow SS \Rightarrow bSaS, \quad S \Rightarrow SS \Rightarrow S,$$
$$S \Rightarrow aSb \Rightarrow aSSb, \quad S \Rightarrow aSb \Rightarrow aaSbb, \quad S \Rightarrow aSb \Rightarrow abSab, \quad S \Rightarrow aSb \Rightarrow ab.$$

در دور بعد از $aaSbb$ با $S \to \varepsilon$ به w می‌رسیم.

این روش دو مشکل جدی دارد: **ناکارآمد** است و ممکن است برای رشته‌ای که در زبان نیست (مثلاً $w = abb$) **هرگز متوقف**

نشود. مشکل توقف از قاعده‌های $A \to \varepsilon$ و $A \to B$ می‌آید، چون با آن‌ها طول صورت جمله‌ای می‌تواند کم شود یا ثابت بماند.

مثال ۴.۱۱. همان زبان (بدون رشته تهی) را گرامر زیر بدون قاعده ε و قاعده واحد تولید می‌کند:

$$S \to SS \mid aSb \mid bSa \mid ab \mid ba.$$

برای هر $w \in \{a,b\}^+$ جست‌وجوی کامل حداکثر بعد از $|w|$ دور تمام می‌شود، چون در هر دور طول صورت جمله‌ای دست‌کم یکی زیاد می‌شود. بعد از $|w|$ دور یا اشتقاقی پیدا شده یا می‌دانیم $w \notin L(G)$.

قضیه ۵.۱۱

فرض کنید $G = (V, T, S, P)$ گرامر مستقل از متنی بدون قاعده‌های $A \to \varepsilon$ و $A \to B$ باشد. آنگاه جست‌وجوی کامل را می‌توان به الگوریتمی تبدیل کرد که برای هر $w \in T^*$ یا اشتقاقی برای w پیدا می‌کند یا اعلام می‌کند که اشتقاقی وجود ندارد.

اثبات. در هر گام اشتقاق دست‌کم یکی از این دو اتفاق می‌افتد: طول صورت جمله‌ای زیاد می‌شود (قاعده با سمت راست بلندتر از ۱) یا تعداد پایانه‌ها زیاد می‌شود (قاعده $A \to a$). هیچ‌کدام از این دو مقدار هرگز کم نمی‌شود، چون قاعده ε نداریم. همچنین در اشتقاق w، نه طول صورت جمله‌ای و نه تعداد پایانه‌ها نمی‌تواند از $|w|$ بیشتر شود. پس اشتقاق w حداکثر $2|w|$ گام دارد. الگوریتم همه اشتقاق‌های چپ‌ترین با حداکثر $2|w|$ گام را امتحان می‌کند و اگر هیچ‌کدام w را نساخت، جواب «نه» می‌دهد.

■

هزینه: اگر فقط اشتقاق‌های چپ‌ترین را بشماریم، بعد از دور اول حداکثر $|P|$ صورت جمله‌ای داریم، بعد از دور دوم حداکثر $|P|^2$ و همین‌طور تا آخر. پس تعداد کل صورت‌های جمله‌ای حداکثر

$$M = |P| + |P|^2 + \cdots + |P|^{2|w|} = O\big(|P|^{2|w|+1}\big)$$

است که با طول رشته **به‌طور نمایی** رشد می‌کند. در عمل هم این روش بسیار کند است.

قضیه ۶.۱۱

برای هر گرامر مستقل از متن، الگوریتمی وجود دارد که هر $w \in L(G)$ را در تعداد گامی متناسب با $|w|^3$ تجزیه می‌کند.

این قضیه را در بخش ۴.۱۱ با الگوریتم CYK ثابت می‌کنیم. با این حال $|w|^3$ هم برای یک برنامه چند هزار خطی زیاد است. آرزو داریم الگوریتمی **خطی** (با زمان متناسب با $|w|$) داشته باشیم. برای گرامرهای مستقل از متن در حالت کلی چنین روشی نمی‌شناسیم، اما برای حالت‌های خاص و مهمی وجود دارد.

۳.۱۱ گرامرهای ساده

تعریف ۷.۱۱ (گرامر ساده (s-grammar))

گرامر مستقل از متن $G = (V, T, S, P)$ یک **گرامر ساده** است اگر همه قاعده‌هایش به شکل $A \to ax$ ($A \in V$، $a \in T$، $x \in V^*$) باشند و هر زوج (A, a) حداکثر در یک قاعده ظاهر شود.

مثال ۸.۱۱. گرامر $S \to aS \mid bSS \mid aSS \mid c$ ساده است. اما $S \to aS \mid bSS \mid c$ ساده نیست، چون زوج (S, a) در دو قاعده $S \to aS$ و $S \to aSS$ آمده است.

در گرامر ساده، تجزیه هر $w = a_1 a_2 \cdots a_n$ زمانی متناسب با $|w|$ می‌گیرد. برای شروع تنها یک انتخاب داریم: قاعده‌ای به شکل $S \to a_1 x$. پس اشتقاق با $S \Rightarrow a_1 A_1 \cdots A_m$ شروع می‌شود. برای جایگزینی A_1 هم تنها یک قاعده به شکل $A_1 \to a_2 x'$ وجود دارد و همین‌طور تا آخر. هر گام دقیقاً یک پایانه تولید می‌کند و هیچ انتخابی نداریم، پس کل کار در حداکثر $|w|$ گام تمام می‌شود.

۴.۱۱ الگوریتم CYK

الگوریتم CYK (کوک، یانگر و کاسامی) یک الگوریتم عضویت و تجزیه با زمان $O(n^3)$ است. این الگوریتم فقط روی گرامرهای در **فرم نرمال چامسکی** کار می‌کند (که طبق قضیه ۵.۱۰ محدودیتی نیست) و یک مسئله را به دنباله‌ای از مسئله‌های کوچک‌تر می‌شکند (**برنامه‌نویسی پویا**).

فرض کنید $G = (V, T, S, P)$ در CNF و $w = a_1 a_2 \cdots a_n$ باشد. زیررشته‌ها و مجموعه‌های زیر را تعریف می‌کنیم:

$$w_{ij} = a_i \cdots a_j, \qquad V_{ij} = \{A \in V \mid A \overset{*}{\Rightarrow} w_{ij}\}.$$

یعنی V_{ij} همه متغیرهایی است که زیررشته w_{ij} را تولید می‌کنند. پس

$$w \in L(G) \iff S \in V_{1n}.$$

محاسبه V_{ij}:

- **زیررشته‌های تک‌نمادی:** $A \in V_{ii}$ اگر و تنها اگر قاعده $A \to a_i$ از G را داشته باشد (چون در CNF تنها راه تولید یک پایانه همین است).
- **زیررشته‌های بلندتر** ($j > i$): A رشته w_{ij} را تولید می‌کند اگر و تنها اگر قاعده‌ای مثل $A \to BC$ داشته باشد و w_{ij} را بتوان در نقطه‌ای مثل k شکست که B قسمت اول و C قسمت دوم را تولید کند:

$$V_{ij} = \bigcup_{k \in \{i, \ldots, j-1\}} \{A \mid A \to BC,\ B \in V_{ik},\ C \in V_{k+1,j}\}.$$

این مجموعه‌ها را به ترتیب طول محاسبه می‌کنیم: اول $V_{11}, V_{22}, \ldots, V_{nn}$، بعد $V_{12}, V_{23}, \ldots, V_{n-1,n}$، بعد $V_{13}, V_{24}, \ldots$ و الی آخر. وقتی V_{ij} را حساب می‌کنیم، همه مجموعه‌های زیررشته‌های کوتاه‌تر آماده‌اند.

درستی با استقرا روی طول $j - i$ ثابت می‌شود: برای طول ۱ مستقیم از تعریف CNF، و برای طول بیشتر، اولین گام هر اشتقاق از A باید $A \Rightarrow BC$ باشد و B و C دو تکه پیوسته از w_{ij} را تولید می‌کنند، که طبق فرض استقرا در V_{ik} و $V_{k+1,j}$ ثبت شده‌اند.

اگر برای هر عضو V_{ij} به خاطر بسپاریم از کدام k و کدام قاعده آمده، الگوریتم **تجزیه** هم انجام می‌دهد: از $S \in V_{1n}$ به عقب برمی‌گردیم و درخت تجزیه را می‌سازیم.

مثال ۹.۱۱. رشته $w = aabbb$ و گرامر

$$S \to AB, \qquad A \to BB \mid a, \qquad B \to AB \mid b.$$

- $V_{11} = V_{22} = \{A\}$ (قاعده $A \to a$) و $V_{33} = V_{44} = V_{55} = \{B\}$.
- V_{12}: تنها شکست $a|a$ است و AA سمت راست هیچ قاعده‌ای نیست، پس $V_{12} = \emptyset$.
- V_{23}: شکست $a|b$ با $A \in V_{22}$ و $B \in V_{33}$؛ قاعده‌های با سمت راست AB: $S \to AB$ و $B \to AB$. پس $V_{23} = \{S, B\}$.
- V_{34}: شکست $b|b$ با BB: $V_{34} = \{A\}$، و به همین شکل $V_{45} = \{A\}$.

با ادامه همین کار جدول کامل به دست می‌آید:

$V_{11} = \{A\}$	$V_{22} = \{A\}$	$V_{33} = \{B\}$	$V_{44} = \{B\}$	$V_{55} = \{B\}$
$V_{12} = \emptyset$	$V_{23} = \{S, B\}$	$V_{34} = \{A\}$	$V_{45} = \{A\}$	
$V_{13} = \{S, B\}$	$V_{24} = \{A\}$	$V_{35} = \{S, B\}$		
$V_{14} = \{A\}$	$V_{25} = \{S, B\}$			
$V_{15} = \{S, B\}$				

مثلاً برای V_{15} شکست $a|abbb$ را با $A \in V_{11}$ و $B \in V_{25}$ داریم که S و B را می‌دهد. چون $S \in V_{15}$، نتیجه می‌گیریم $aabbb \in L(G)$.

تحلیل زمان: دقیقاً $\frac{n(n+1)}{2}$ مجموعه V_{ij} باید محاسبه شود و محاسبه هر کدام حداکثر n نقطه شکست k را بررسی می‌کند (برای هر شکست، کار با تعداد قاعده‌ها متناسب است که به n بستگی ندارد). پس کل زمان $O(n^3)$ است.

جمع‌بندی فصل

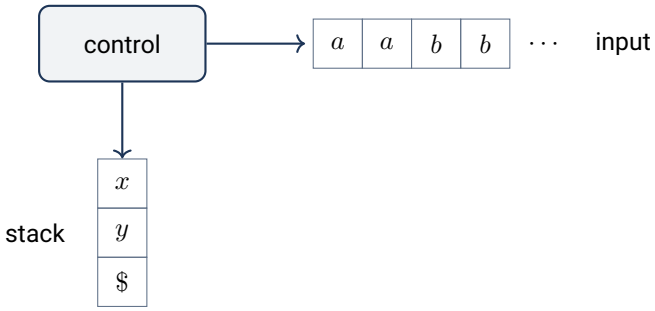
- عضویت: آیا $w \in L(G)$؟ تجزیه: پیدا کردن اشتقاق w.
- جست‌وجوی کامل ساده است اما نمایی است و بدون حذف قاعده‌های ε و واحد ممکن است متوقف نشود.
- در گرامرهای ساده (s-grammar) تجزیه در زمان خطی انجام می‌شود.
- CYK برای گرامرهای CNF: V_{ij} متغیرهای مولد w_{ij} است، به ترتیب طول محاسبه می‌شود و $w \in L(G) \iff S \in V_{1n}$. زمان: $O(n^3)$.

۱۲ فصل

ماشین‌های پشته‌ای

در این فصل مدل محاسباتی جدیدی می‌بینیم: **ماشین پشته‌ای** (Pushdown Automaton) یا PDA. این ماشین شبیه NFA است، اما یک حافظه اضافه به شکل **پشته** دارد. ثابت می‌کنیم قدرت ماشین‌های پشته‌ای دقیقاً با گرامرهای مستقل از متن برابر است. پس برای اثبات مستقل از متن بودن یک زبان دو راه داریم: یا گرامری بسازیم که آن را **تولید** کند، یا ماشین پشته‌ای که آن را **تشخیص** دهد.

۱.۱۲ ایده ماشین پشته‌ای

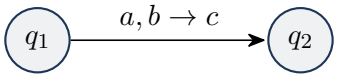


- **بخش کنترل:** حالت‌ها و گذارها، مثل NFA.
- **ورودی:** رشته‌ای که نماد به نماد از چپ به راست خوانده می‌شود.
- **پشته:** ماشین می‌تواند نمادی را روی پشته بگذارد (**push**) یا نماد بالای پشته را بردارد (**pop**). فقط بالای پشته قابل دسترسی است (آخرین ورودی، اولین خروجی). پشته می‌تواند هر تعداد نماد را نگه دارد؛ یعنی حافظه **نامحدود** اما با دسترسی محدود.

با همین حافظه ماشین می‌تواند $\{0^n 1^n\}$ را تشخیص دهد: هر 0 را روی پشته می‌گذارد و به ازای هر 1 یک 0 برمی‌دارد.

۱.۱.۱۲ گذارها

برچسب یک گذار در PDA شکل $a, b \to c$ دارد:



یعنی: نماد a را از ورودی بخوان، نماد b را از بالای پشته بردار و نماد c را روی پشته بگذار. هر کدام از a، b و c می‌تواند ε باشد:

- $a, b \to c$: **جایگزینی** بالای پشته؛ b برداشته و c گذاشته می‌شود.
- $a, \varepsilon \to c$: **push**؛ چیزی برداشته نمی‌شود و c گذاشته می‌شود.
- $a, b \to \varepsilon$: **pop**؛ b برداشته می‌شود و چیزی گذاشته نمی‌شود.
- $a, \varepsilon \to \varepsilon$: پشته **تغییری نمی‌کند**.
- $\varepsilon, b \to c$: بدون خواندن ورودی فقط پشته عوض می‌شود.

اگر ماشین بخواهد b را بردارد ولی بالای پشته b نباشد (یا پشته خالی باشد)، آن گذار قابل اجرا نیست و آن شاخه از محاسبه می‌میرد.

نکته. ماشین‌های پشته‌ای می‌توانند غیرقطعی باشند (از یک حالت با یک ورودی و یک نماد پشته چند انتخاب داشته باشند، یا گذار ε در کنار گذار با ورودی داشته باشند). برخلاف ماشین‌های متناهی، اینجا نامعینی قدرت را زیاد می‌کند: PDAهای قطعی از PDAهای غیرقطعی ضعیف‌ترند (فصل ۱۵). منظور ما از PDA همیشه نوع غیرقطعی است.

۲.۱۲ تعریف صوری

تعریف ۱.۱۲ (ماشین پشته‌ای)

یک ماشین پشته‌ای شش‌تایی $(Q, \Sigma, \Gamma, \delta, q_0, F)$ است که Q، Σ، Γ و F مجموعه‌های متناهی‌اند و

۱. Q مجموعه حالت‌ها،

۲. Σ الفبای ورودی،

۳. Γ **الفبای پشته**،

۴. $\delta : Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \to \mathcal{P}(Q \times \Gamma_\varepsilon)$ تابع گذار،

۵. $q_0 \in Q$ حالت شروع، و

۶. $F \subseteq Q$ مجموعه حالت‌های پذیرش است.

اگر $(r, c) \in \delta(q, a, b)$، یعنی در حالت q با خواندن a و برداشتن b از پشته، ماشین می‌تواند به r برود و c را روی پشته بگذارد.

تعریف ۲.۱۲ (محاسبه PDA)

ماشین پشته‌ای $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ رشته w را **می‌پذیرد** اگر بتوان نوشت $w = w_1 w_2 \cdots w_m$ که $w_i \in \Sigma_\varepsilon$، و دنباله‌ای از حالت‌ها $r_0, r_1, \ldots, r_m \in Q$ و دنباله‌ای از محتواهای پشته $s_0, s_1, \ldots, s_m \in \Gamma^*$ وجود داشته باشد که

۱. $r_0 = q_0$ و $s_0 = \varepsilon$ (شروع از حالت شروع با پشته خالی)

۲. برای $i = 0, \ldots, m-1$: $(r_{i+1}, b) \in \delta(r_i, w_{i+1}, a)$ که $s_i = at$ و $s_{i+1} = bt$ برای $a, b \in \Gamma_\varepsilon$ و $t \in \Gamma^*$ (حرکت درست)

۳. $r_m \in F$ (پایان در حالت پذیرش)

در شرط ۲، a نمادی است که برداشته می‌شود، b نمادی است که گذاشته می‌شود و t بقیه پشته است که دست نمی‌خورد.

۳.۱۲ یک مثال

مثال ۳.۱۲ (ماشین پشته‌ای برای $\{0^n 1^n \mid n \geq 0\}$). ایده:

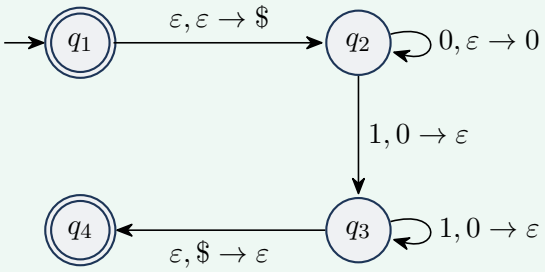
۱. هر 0 که خوانده شد را روی پشته بگذار.

۲. با خواندن اولین 1، به ازای هر 1 یک 0 از پشته بردار.

۳. اگر بعد از 1ها دوباره 0 آمد، رد کن.

۴. اگر ورودی تمام شد و پشته خالی بود، بپذیر؛ اگر پشته خالی نبود یا پشته زودتر از ورودی خالی شد، رد کن.

PDA در تعریف رسمی راهی برای فهمیدن خالی بودن پشته ندارد. ترفند رایج این است که در ابتدا یک نماد خاص $\$$ روی پشته بگذاریم؛ هر وقت دوباره $\$$ را دیدیم، یعنی پشته خالی است.



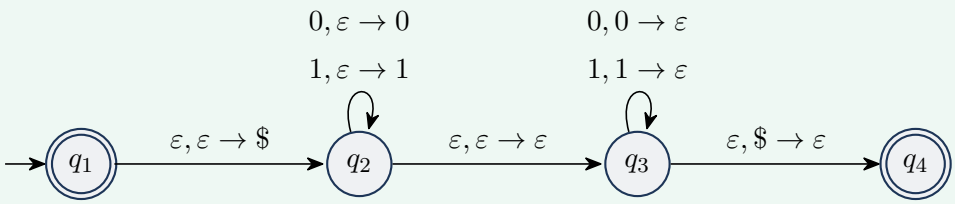
اجرای ماشین روی 000111 (بالای پشته در سمت چپ نوشته شده):

state	q_1	q_2	q_2	q_2	q_2	q_3	q_3	q_3	q_4
read	–	ε	0	0	0	1	1	1	ε
stack	ε	$\$$	$0\$$	$00\$$	$000\$$	$00\$$	$0\$$	$\$$	ε

توصیف صوری: $Q = \{q_1, q_2, q_3, q_4\}$، $\Sigma = \{0, 1\}$، $\Gamma = \{0, \$\}$، حالت شروع q_1 و $F = \{q_1, q_4\}$ (q_1 پذیرش است تا رشته تهی پذیرفته شود). تابع گذار (خانه‌های خالی یعنی $\emptyset$):

input:	0			1			ε		
stack:	0	$\$$	ε	0	$\$$	ε	0	$\$$	ε
q_1									$\{(q_2, \$)\}$
q_2			$\{(q_2, 0)\}$	$\{(q_3, \varepsilon)\}$					
q_3				$\{(q_3, \varepsilon)\}$				$\{(q_4, \varepsilon)\}$	
q_4									

مثال ۴.۱۲ (پالیندروم‌های زوج). برای $\{ww^R \mid w \in \{0,1\}^*\}$ ماشین نمادها را روی پشته می‌گذارد و در جایی «حدس می‌زند» که به وسط رشته رسیده است. از آن به بعد هر نماد ورودی را با بالای پشته مقایسه می‌کند:



اگر حدس درست باشد (و رشته پالیندروم باشد) یک شاخه می‌پذیرد. هیچ ماشین **قطعی**‌ای نمی‌تواند وسط رشته را تشخیص دهد.

۴.۱۲ هم‌ارزی ماشین‌های پشته‌ای و گرامرهای مستقل از متن

قضیه ۵.۱۲

یک زبان مستقل از متن است اگر و تنها اگر یک ماشین پشته‌ای آن را تشخیص دهد.

این قضیه دو جهت دارد که هر کدام را در یک لم ثابت می‌کنیم.

۱.۴.۱۲ از گرامر به ماشین پشته‌ای

لم ۶.۱۲

اگر یک زبان مستقل از متن باشد، یک ماشین پشته‌ای آن را تشخیص می‌دهد.

ایده اثبات. گرامر G برای زبان را داریم. ماشین P را می‌سازیم که ورودی w را می‌پذیرد اگر G آن را تولید کند. P با استفاده از نامعینی یک اشتقاق چپ‌ترین از w را «حدس می‌زند» و آن را روی پشته شبیه‌سازی می‌کند. پشته **بقیه صورت جمله‌ای** را نگه می‌دارد: پایانه‌های ابتدای صورت جمله‌ای با ورودی مقایسه و برداشته می‌شوند و اولین متغیر روی پشته با سمت راست یکی از قاعده‌هایش جایگزین می‌شود.

نحوه کار P:

۱. نماد $\$$ و سپس متغیر شروع را روی پشته بگذار.

۲. این مراحل را تکرار کن:

(آ) اگر بالای پشته یک متغیر A است، به‌طور غیرقطعی یکی از قاعده‌های A را انتخاب کن و A را با سمت راست آن قاعده جایگزین کن.

(ب) اگر بالای پشته یک پایانه a است، نماد بعدی ورودی را بخوان. اگر برابر a بود آن را از پشته بردار و ادامه بده؛ وگرنه این شاخه رد می‌شود.

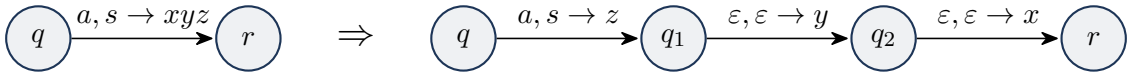
(ج) اگر بالای پشته $\$$ است، به حالت پذیرش برو. اگر کل ورودی خوانده شده باشد، ورودی پذیرفته می‌شود.

یک نمادگذاری کوتاه: گذاشتن یک **رشته** $u = u_1 \cdots u_l$ روی پشته در یک گام را با $a, s \to u$ نشان می‌دهیم (طوری که

u_1 بالای پشته قرار گیرد). این کار با حالت‌های میانی جدید $q_1, \ldots, q_{l-1}$ قابل پیاده‌سازی است:

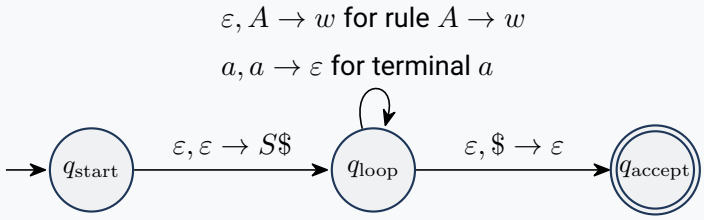
$$(q_1, u_l) \in \delta(q, a, s), \quad \delta(q_1, \varepsilon, \varepsilon) = \{(q_2, u_{l-1})\}, \quad \ldots, \quad \delta(q_{l-1}, \varepsilon, \varepsilon) = \{(r, u_1)\}.$$

یعنی رشته را از آخر به اول، یکی‌یکی push می‌کنیم.



اثبات. فرض کنید $G = (V, \Sigma, R, S)$ گرامر زبان باشد. ماشین $P = (Q, \Sigma, \Gamma, \delta, q_{\text{start}}, \{q_{\text{accept}}\})$ را می‌سازیم که $Q = \{q_{\text{start}}, q_{\text{loop}}, q_{\text{accept}}\} \cup E$ (E حالت‌های میانی نمادگذاری کوتاه)، $\Gamma = V \cup \Sigma \cup \{\$\}$ و

$$\begin{aligned}
&\delta(q_{\text{start}}, \varepsilon, \varepsilon) = \{(q_{\text{loop}}, S\$)\},\\
&\delta(q_{\text{loop}}, \varepsilon, A) = \{(q_{\text{loop}}, w) \mid A \to w \text{ است } R \text{ یک قاعده در}\},\\
&\delta(q_{\text{loop}}, a, a) = \{(q_{\text{loop}}, \varepsilon)\} \quad a \text{ برای هر پایانه},\\
&\delta(q_{\text{loop}}, \varepsilon, \$) = \{(q_{\text{accept}}, \varepsilon)\}.
\end{aligned}$$



درستی: در هر لحظه، رشته‌ای که تا اینجا از ورودی خوانده شده و پس از آن محتوای پشته (بدون $)، با هم یک صورت جمله‌ای از یک اشتقاق چپ‌ترین را تشکیل می‌دهند. گذار نوع دوم یک گام اشتقاق را روی چپ‌ترین متغیر (که بالای پشته است) انجام می‌دهد و گذار نوع سوم پایانه‌های ابتدایی را با ورودی تطبیق می‌دهد. پس P وقتی به $ می‌رسد و کل ورودی را خوانده، یک اشتقاق کامل $S \overset{*}{\Rightarrow} w$ را دنبال کرده است. برعکس، برای هر اشتقاق چپ‌ترین $S \overset{*}{\Rightarrow} w$، شاخه‌ای از P که همان قاعده‌ها را به همان ترتیب انتخاب کند، w را می‌پذیرد. پس $L(P) = L(G)$.

■

نتیجه ۷.۱۲

هر زبان منظم مستقل از متن است.

اثبات. هر NFA یک ماشین پشته‌ای است که از پشته‌اش استفاده نمی‌کند. پس هر زبان منظم را یک PDA تشخیص می‌دهد و طبق قضیه ۴.۱۲ مستقل از متن است.

■

۲.۴.۱۲ از ماشین پشته‌ای به گرامر

لم ۸.۱۲

اگر یک ماشین پشته‌ای زبانی را تشخیص دهد، آن زبان مستقل از متن است.

برای ساده شدن اثبات، فرض می‌کنیم ماشین P سه ویژگی دارد:

۱. دقیقاً یک حالت پذیرش، q_{accept}، دارد.

۲. قبل از پذیرش، پشته‌اش را خالی می‌کند.

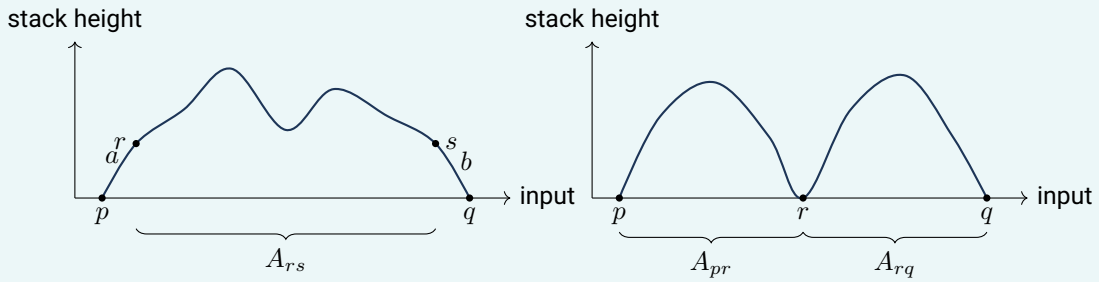
۳. هر گذار یا دقیقاً یک نماد push می‌کند یا دقیقاً یک نماد pop می‌کند، نه هر دو و نه هیچ‌کدام.

این فرض‌ها محدودیتی ایجاد نمی‌کنند: برای ۱، از همه حالت‌های پذیرش با $\varepsilon, \varepsilon \to \varepsilon$ به یک حالت پذیرش جدید می‌رویم. برای ۲، در ابتدا $ را push می‌کنیم و قبل از رفتن به حالت پذیرش با چند گذار ε پشته را تا $ خالی می‌کنیم. برای ۳، هر گذار $a, b \to c$ با $b, c \neq \varepsilon$ را با دو گذار (اول pop و بعد push، با یک حالت میانی) و هر گذار $a, \varepsilon \to \varepsilon$ را با دو گذار (push) یک نماد دلخواه و سپس pop همان) جایگزین می‌کنیم.

ایده اثبات. برای هر زوج حالت p, q از P یک متغیر A_{pq} در گرامر می‌گذاریم. A_{pq} همه رشته‌هایی را تولید می‌کند که می‌توانند P را از حالت p **با پشته خالی** به حالت q **با پشته خالی** ببرند. (این رشته‌ها پشته را از p تا q دست‌نخورده باقی می‌گذارند؛ یعنی هر چه روی پشته باشد، در q دوباره همان است.)

فرض کنید x چنین رشته‌ای باشد. طبق ویژگی ۳، اولین حرکت P روی x یک push است (پشته خالی است و چیزی برای pop نیست) و آخرین حرکت یک pop (چون پشته در پایان خالی است). دو حالت پیش می‌آید:

- **نمادی که در آخر pop می‌شود همان نمادی است که اول push شده.** پس پشته فقط در ابتدا و انتها خالی است. اگر نماد خوانده‌شده در حرکت اول a و در حرکت آخر b باشد، r حالت بعد از حرکت اول و s حالت قبل از حرکت آخر باشد، قاعده $A_{pq} \to aA_{rs}b$ را می‌گذاریم.
- **در غیر این صورت،** نماد اول قبل از پایان pop شده، پس پشته در نقطه‌ای میانی مثل حالت r خالی می‌شود. قاعده $A_{pq} \to A_{pr}A_{rq}$ را می‌گذاریم.



اثبات. فرض کنید $P = (Q, \Sigma, \Gamma, \delta, q_0, \{q_{\text{accept}}\})$ سه ویژگی بالا را داشته باشد. گرامر G متغیرهای $\{A_{pq} \mid p, q \in Q\}$ و متغیر شروع $A_{q_0,q_{\text{accept}}}$ دارد. قاعده‌های G:

۱. برای هر $p, q, r, s \in Q$، $u \in \Gamma$ و $a, b \in \Sigma_\varepsilon$: اگر $\delta(p, a, \varepsilon)$ شامل (r, u) باشد و $\delta(s, b, u)$ شامل (q, ε)،

قاعده $A_{pq} \to aA_{rs}b$ را بگذار.

۲. برای هر $p, q, r \in Q$، قاعده $A_{pq} \to A_{pr}A_{rq}$ را بگذار.

۳. برای هر $p \in Q$، قاعده $A_{pp} \to \varepsilon$ را بگذار.

درستی ساخت از دو ادعای زیر نتیجه می‌شود: A_{pq} رشته x را تولید می‌کند اگر و تنها اگر x بتواند P را از p با پشته خالی به q با پشته خالی ببرد. با $p = q_0$ و $q = q_{\text{accept}}$ داریم $L(G) = L(P)$ (طبق ویژگی ۲، پذیرش یعنی رسیدن به q_{accept} با پشته خالی).

ادعای ۱. اگر $A_{pq} \overset{*}{\Rightarrow} x$، آنگاه x می‌تواند P را از p با پشته خالی به q با پشته خالی ببرد.

با استقرا روی تعداد گام‌های اشتقاق. **پایه** (یک گام): تنها قاعده‌هایی که سمت راستشان متغیر ندارد $A_{pp} \to \varepsilon$ هستند، و رشته ε ماشین را از p با پشته خالی به خود p با پشته خالی می‌برد. **گام:** فرض کنید ادعا برای اشتقاق‌های حداکثر k گامی درست باشد و $A_{pq} \overset{*}{\Rightarrow} x$ در $k+1$ گام. گام اول یا $A_{pq} \Rightarrow aA_{rs}b$ است یا $A_{pq} \Rightarrow A_{pr}A_{rq}$.

- اگر $A_{pq} \Rightarrow aA_{rs}b$، بنویسید $x = ayb$ که $A_{rs} \overset{*}{\Rightarrow} y$ در k گام. طبق فرض استقرا y ماشین را از r با پشته خالی به s با پشته خالی می‌برد. طبق ساخت، $\delta(p, a, \varepsilon)$ شامل (r, u) و $\delta(s, b, u)$ شامل (q, ε) است. پس با خواندن a، ماشین از p به r می‌رود و u را push می‌کند؛ سپس y را می‌خواند و به s می‌رسد در حالی که u زیر پشته دست‌نخورده مانده؛ سپس با خواندن b و pop کردن u به q با پشته خالی می‌رسد.
- اگر $A_{pq} \Rightarrow A_{pr}A_{rq}$، بنویسید $x = yz$ که $A_{pr} \overset{*}{\Rightarrow} y$ و $A_{rq} \overset{*}{\Rightarrow} z$، هر کدام حداکثر در k گام. طبق فرض استقرا y ماشین را از p به r و z آن را از r به q می‌برد، هر دو با پشته خالی در ابتدا و انتها. پس x ماشین را از p به q با پشته خالی می‌برد.

ادعای ۲. اگر x بتواند P را از p با پشته خالی به q با پشته خالی ببرد، آنگاه $A_{pq} \overset{*}{\Rightarrow} x$.

با استقرا روی تعداد گام‌های محاسبه. **پایه** (صفر گام): بدون هیچ حرکتی ماشین در p می‌ماند و چیزی نمی‌خواند، پس $q = p$ و $x = \varepsilon$. قاعده $A_{pp} \to \varepsilon$ در G هست. **گام:** فرض کنید ادعا برای محاسبه‌های حداکثر k گامی درست باشد و محاسبه‌ای $k+1$ گامی روی x از p به q با پشته خالی داریم. یا (۱) پشته فقط در ابتدا و انتهای محاسبه خالی است، یا (۲) در جای دیگری هم خالی می‌شود.

- **حالت ۱:** نماد u که در حرکت اول push شده، همان نمادی است که در حرکت آخر pop می‌شود. فرض کنید a و b نمادهای خوانده‌شده در حرکت اول و آخر، r حالت بعد از حرکت اول و s حالت قبل از حرکت آخر باشند. پس $\delta(p, a, \varepsilon)$ شامل (r, u) و $\delta(s, b, u)$ شامل (q, ε) است و قاعده $A_{pq} \to aA_{rs}b$ در G هست. بنویسید $x = ayb$. بخش میانی محاسبه، y را از r تا s می‌خواند و در طول آن u ته پشته دست‌نخورده می‌ماند؛ پس y ماشین را از r با پشته خالی به s با پشته خالی هم می‌برد، آن هم در $k-1$ گام. طبق فرض استقرا $A_{rs} \overset{*}{\Rightarrow} y$، پس $A_{pq} \overset{*}{\Rightarrow} x$.
- **حالت ۲:** فرض کنید r حالتی باشد که پشته در آن (در میانه محاسبه) خالی می‌شود و $x = yz$ که y قبل و z بعد از آن خوانده شده‌اند. هر دو بخش حداکثر k گام دارند و با پشته خالی شروع و تمام می‌شوند. طبق فرض استقرا $A_{pr} \overset{*}{\Rightarrow} y$ و $A_{rq} \overset{*}{\Rightarrow} z$. چون قاعده $A_{pq} \to A_{pr}A_{rq}$ در G هست، $A_{pq} \overset{*}{\Rightarrow} x$.

■

با دو لم ۱.۴.۱۲ و ۲.۴.۱۲ اثبات قضیه ۴.۱۲ کامل است.

جمع‌بندی فصل

- PDA = NFA + پشته. گذار $a, b \to c$: بخوان a، pop کن b، push کن c (هر کدام می‌تواند ε باشد).
- تابع گذار: $\delta : Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \to \mathcal{P}(Q \times \Gamma_\varepsilon)$. ترفند \$ برای تشخیص پشته خالی.
- گرامر به PDA: پشته بقیه صورت جمله‌ای چپ‌ترین را نگه می‌دارد.
- PDA به گرامر: متغیر A_{pq} رشته‌هایی را تولید می‌کند که ماشین را با پشته خالی از p به q می‌برند؛ قاعده‌های $aA_{rs}b$، $A_{pr}A_{rq}$ و $A_{pp} \to \varepsilon$.
- زبان مستقل از متن $\iff$ قابل تشخیص با PDA. هر زبان منظم مستقل از متن است.

فصل ۱۳

ویژگی‌های بستاری زبان‌های مستقل از متن

در فصل ۵ دیدیم زبان‌های منظم تحت تقریباً همه عملیات معمول بسته‌اند. زبان‌های مستقل از متن تحت بسیاری از عملیات بسته‌اند، اما نه همه. در این فصل هر دو طرف را بررسی می‌کنیم. در اثبات‌ها از گرامرها یا ماشین‌های پشته‌ای استفاده می‌کنیم، هر کدام که ساده‌تر باشد.

در همه اثبات‌ها فرض می‌کنیم متغیرهای دو گرامر مختلف با هم مشترک نیستند (در صورت لزوم نامشان را عوض می‌کنیم).

۱.۱۳ اجتماع، الحاق و ستاره

قضیه ۱.۱۳

زبان‌های مستقل از متن تحت **اجتماع** بسته‌اند.

اثبات. فرض کنید $G_1 = (V_1, \Sigma, R_1, S_1)$ زبان A و $G_2 = (V_2, \Sigma, R_2, S_2)$ زبان B را تولید کنند. گرامر (V, Σ, R, S) را با متغیر شروع جدید S می‌سازیم:

$$V = V_1 \cup V_2 \cup \{S\}, \qquad R = R_1 \cup R_2 \cup \{S \to S_1 \mid S_2\}.$$

اگر $w \in A$، آنگاه $S_1 \overset{*}{\Rightarrow} w$ و پس $S \Rightarrow S_1 \overset{*}{\Rightarrow} w$. اگر $w \in B$ هم به همین شکل $S \Rightarrow S_2 \overset{*}{\Rightarrow} w$. برعکس، اگر $w \in L(G)$، اولین گام اشتقاق یا $S \Rightarrow S_1$ است یا $S \Rightarrow S_2$؛ چون متغیرهای دو گرامر از هم جدا هستند، بقیه اشتقاق کاملاً در G_1 یا کاملاً در G_2 است. پس $w \in A \cup B$.

■

قضیه ۲.۱۳

زبان‌های مستقل از متن تحت **الحاق** بسته‌اند.

اثبات. با همان نمادها، $R = R_1 \cup R_2 \cup \{S \to S_1 S_2\}$. هر رشته‌ای که G تولید کند به شکل xy است که $S_1 \overset{*}{\Rightarrow} x$ و $S_2 \overset{*}{\Rightarrow} y$ (چون S_1 و S_2 متغیرهای مشترکی ندارند و مستقل از هم جایگزین می‌شوند). برعکس، برای هر $x \in A$ و $y \in B$ داریم $S \Rightarrow S_1 S_2 \overset{*}{\Rightarrow} x S_2 \overset{*}{\Rightarrow} xy$. پس $L(G) = A \circ B$.

■

قضیه ۳.۱۳

زبان‌های مستقل از متن تحت **ستاره** بسته‌اند.

اثبات. اگر $G_1 = (V_1, \Sigma, R_1, S_1)$ زبان A را تولید کند، قرار دهید $V = V_1 \cup \{S\}$ و $R = R_1 \cup \{S \to SS_1 \mid \varepsilon\}$. با k بار استفاده از $S \to SS_1$ و یک بار $S \to \varepsilon$ داریم $S \overset{*}{\Rightarrow} S_1 S_1 \cdots S_1$ (k بار) و هر S_1 مستقلاً یک رشته از A تولید می‌کند. پس $L(G) = \{x_1 \cdots x_k \mid k \geq 0,\ x_i \in A\} = A^*$.

■

۲.۱۳ پیشوند

قضیه ۴.۱۳

زبان‌های مستقل از متن تحت **پیشوند** بسته‌اند.

ایده اثبات. گرامری در فرم نرمال چامسکی برای L بگیرید. درخت تجزیه یک رشته w و یک پیشوند آن را در نظر بگیرید. «خط برش» پیشوند، برگ‌ها را به دو دسته چپ و راست تقسیم می‌کند و متغیرهای درخت سه نوع می‌شوند:

- $\langle A, L\rangle$: همه برگ‌های زیردرخت A در پیشوند هستند (**چپ** خط برش)؛
- $\langle A, S\rangle$: زیردرخت A **روی خط برش** است؛ بخشی از برگ‌هایش در پیشوند هستند. این متغیرها مسیری از ریشه به پایین می‌سازند (**مسیر برش**)؛
- $\langle A, R\rangle$: هیچ برگی از زیردرخت A در پیشوند نیست (**راست** خط برش).

گرامر جدید درخت را با این برچسب‌ها می‌سازد: متغیرهای «چپ» مثل قبل رشته تولید می‌کنند، متغیرهای «راست» با ε جایگزین می‌شوند و متغیرهای «روی برش» مسیر برش را دنبال می‌کنند.

اثبات. اگر $L = \emptyset$ آنگاه $Prefix(L) = \emptyset$ و حکم بدیهی است. فرض کنید $L \neq \emptyset$ و $G = (V, \Sigma, R, S)$ گرامری در CNF و بدون متغیر بی‌فایده برای L باشد. پس قاعده‌ها سه شکل دارند: $S \to \varepsilon$، $A \to BC$ و $A \to t$. گرامر $G' = (V', \Sigma, R', S')$ را می‌سازیم:

$$V' = \{\langle A, D\rangle \mid A \in V,\ D \in \{L, S, R\}\}, \qquad S' = \langle S, S\rangle,$$

و قاعده‌های R':

قاعده در G	قاعده‌های جدید در G'
($L \neq \emptyset$ چون)	$\langle S, S\rangle \to \varepsilon$
$A \to BC$	$\langle A, L\rangle \to \langle B, L\rangle\langle C, L\rangle$ $\langle A, S\rangle \to \langle B, L\rangle\langle C, S\rangle \mid \langle B, S\rangle\langle C, R\rangle$ $\langle A, R\rangle \to \langle B, R\rangle\langle C, R\rangle$
$A \to t$	$\langle A, L\rangle \to t, \quad \langle A, S\rangle \to t, \quad \langle A, R\rangle \to \varepsilon$

با استقرا روی طول اشتقاق ساده است که (۱) $\langle A, L\rangle \overset{*}{\Rightarrow} x$ اگر و تنها اگر $A \overset{*}{\Rightarrow} x$، و (۲) $\langle A, R\rangle \overset{*}{\Rightarrow} \varepsilon$ و این متغیر

چیز دیگری تولید نمی‌کند (چون هر متغیر G رشته‌ای تولید می‌کند، $\langle A, R\rangle$ همیشه به ε می‌رسد). ادعای اصلی:

$$\langle A, S\rangle \overset{*}{\Rightarrow} x \iff x \text{ است } A \text{ پیشوند ناتهی یک رشته از زبان}.$$

جهت رفت: اگر گام اول $\langle A, S\rangle \to t$ باشد، $x = t$ که خودش رشته‌ای از A است. اگر $\langle B, L\rangle\langle C, S\rangle$ باشد، $x = yz$ که $B \overset{*}{\Rightarrow} y$ و طبق استقرا z پیشوند ناتهی یک رشته z' از C است؛ پس x پیشوند yz' است که A آن را تولید می‌کند. اگر $\langle B, S\rangle\langle C, R\rangle$ باشد، x پیشوند ناتهی رشته‌ای y' از B است و A رشته $y'z'$ را (برای هر z' که C تولید می‌کند) تولید می‌کند. **جهت برگشت:** فرض کنید x پیشوند ناتهی رشته w با $A \overset{*}{\Rightarrow} w$ باشد. اگر $A \to t$، $x = t$. اگر $A \to BC$ با $B \overset{*}{\Rightarrow} y$، $C \overset{*}{\Rightarrow} z$ و $w = yz$: اگر x داخل y تمام شود (یا دقیقاً در انتهای آن)، x پیشوند ناتهی y است و از $\langle B, S\rangle\langle C, R\rangle$ استفاده می‌کنیم؛ وگرنه $x = yz_1$ که z_1 پیشوند ناتهی z است و از $\langle B, L\rangle\langle C, S\rangle$ استفاده می‌کنیم.

با $A = S$ و قاعده $\langle S, S\rangle \to \varepsilon$ (برای پیشوند تهی) نتیجه می‌گیریم $L(G') = Prefix(L)$.

■

مثال ۵.۱۳. اگر $w = w_1w_2\cdots w_n \in L$ و $S \overset{*}{\Rightarrow} A_1A_2\cdots A_n$ با $A_i \Rightarrow w_i$، آنگاه برای هر i:

$$\langle S, S\rangle \overset{*}{\Rightarrow} \langle A_1, L\rangle\cdots\langle A_{i-1}, L\rangle\langle A_i, S\rangle\langle A_{i+1}, R\rangle\cdots\langle A_n, R\rangle \overset{*}{\Rightarrow} w_1w_2\cdots w_i.$$

۳.۱۳ معکوس و پسوند

قضیه ۶.۱۳

زبان‌های مستقل از متن تحت **معکوس** بسته‌اند.

اثبات. فرض کنید $G = (V, \Sigma, R, S)$ زبان B را تولید کند. گرامر G' را با همان متغیرها و قاعده‌های

$$R' = \{A \to u^R \mid A \to u \text{ است } R \text{ قاعده‌ای در}\}$$

می‌سازیم. با $A \overset{n}{\Rightarrow} u$ نشان می‌دهیم A در دقیقاً n گام u را مشتق می‌کند. با استقرا روی n ثابت می‌کنیم برای هر $A \in V$ و $u \in (V \cup \Sigma)^*$:

$$A \overset{n}{\Rightarrow}_G u \iff A \overset{n}{\Rightarrow}_{G'} u^R.$$

پایه ($n = 0$): $u = A = A^R$، پس دو طرف برابرند. **گام:** فرض کنید حکم برای $n - 1$ درست باشد. اگر $A \overset{n}{\Rightarrow}_G u$، آخرین گام یک متغیر C را با قاعده $C \to y$ جایگزین کرده است؛ یعنی $u = xyz$ و $A \overset{n-1}{\Rightarrow}_G xCz$. طبق فرض استقرا $A \overset{n-1}{\Rightarrow}_{G'} (xCz)^R = z^RCx^R$. طبق ساخت، $C \to y^R$ در G' هست. پس

$$A \overset{n}{\Rightarrow}_{G'} z^Ry^Rx^R = (xyz)^R = u^R.$$

جهت برگشت هم دقیقاً به همین شکل است (چون G از G' با همان تبدیل به دست می‌آید). با $A = S$ و $u = w \in \Sigma^*$ داریم $S \overset{*}{\Rightarrow}_G w \iff S \overset{*}{\Rightarrow}_{G'} w^R$، پس $L(G') = B^R$.

■

قضیه ۷.۱۳

زبان‌های مستقل از متن تحت **پسوند** بسته‌اند.

اثبات. w پسوند یک رشته از A است اگر و تنها اگر w^R پیشوند یک رشته از A^R باشد. پس $Suffix(A) = \big(Prefix(A^R)\big)^R$ و از بستاری تحت معکوس و پیشوند نتیجه می‌گیریم $Suffix(A)$ مستقل از متن است.
■

۴.۱۳ اشتراک با یک زبان منظم

قضیه ۸.۱۳

اگر A مستقل از متن و B منظم باشد، $A \cap B$ مستقل از متن است.

ایده اثبات. یک PDA برای A و یک DFA برای B را **هم‌زمان** اجرا می‌کنیم (مثل ساخت ضربی فصل ۳). این کار ممکن است چون DFA پشته‌ای ندارد و فقط یک پشته لازم داریم.

اثبات. فرض کنید PDA $M_1 = (Q_1, \Sigma, \Gamma, \delta_1, q_1, F_1)$ زبان A و DFA $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ زبان B را تشخیص دهند. PDA $M = (Q_1 \times Q_2, \Sigma, \Gamma, \delta, (q_1, q_2), F_1 \times F_2)$ را می‌سازیم که برای هر $a \in \Sigma_\varepsilon$ و $b, c \in \Gamma_\varepsilon$:

$$\delta\big((q,r), a, b\big) = \{((s,t), c) \mid (s,c) \in \delta_1(q, a, b) \text{ و } t = \delta_2(r, a)\},$$

با این قرارداد که اگر $a = \varepsilon$، $t = r$ (DFA بدون خواندن ورودی حرکت نمی‌کند). وقتی M روی w اجرا می‌شود، پشته و مؤلفه اول حالتش دقیقاً مثل M_1 تغییر می‌کنند و مؤلفه دوم مثل M_2. پس M رشته w را می‌پذیرد اگر و تنها اگر یک شاخه M_1 آن را بپذیرد و M_2 هم آن را بپذیرد؛ یعنی $L(M) = A \cap B$.
■

۵.۱۳ عملیاتی که تحت آن‌ها بسته نیستند

تا اینجا بستاری زبان‌های مستقل از متن را تحت اجتماع، الحاق، ستاره، پیشوند، معکوس، پسوند و اشتراک با زبان منظم دیدیم. اما:

قضیه ۹.۱۳

زبان‌های مستقل از متن تحت **اشتراک** بسته نیستند.

اثبات. دو زبان زیر مستقل از متن‌اند:

$$A = \{a^m b^m c^n \mid m, n \geq 0\}, \qquad B = \{a^m b^n c^n \mid m, n \geq 0\}.$$

مثلاً A با گرامر $S \to XC$، $X \to aXb \mid \varepsilon$، $C \to cC \mid \varepsilon$ تولید می‌شود و B به شکل مشابه. اما

$$A \cap B = \{a^n b^n c^n \mid n \geq 0\}$$

که در فصل بعد (مثال ۲.۱۴) ثابت می‌کنیم مستقل از متن نیست.

■

نتیجه ۱۰.۱۳

زبان‌های مستقل از متن تحت **متمم** بسته نیستند.

اثبات. اگر بسته بودند، چون تحت اجتماع بسته‌اند، طبق قانون دمورگان $A \cap B = \overline{\overline{A} \cup \overline{B}}$ تحت اشتراک هم بسته بودند، که با قضیه ۵.۱۳ تناقض دارد.

■

جمع‌بندی فصل

- **بسته است:** اجتماع ($S \to S_1 \mid S_2$)، الحاق ($S \to S_1 S_2$)، ستاره ($S \to SS_1 \mid \varepsilon$)، پیشوند (متغیرهای $\langle A, L/S/R \rangle$ روی گرامر CNF)، معکوس (برعکس کردن سمت راست قاعده‌ها)، پسوند و اشتراک با زبان منظم (ضرب PDA و DFA).
- **بسته نیست:** اشتراک ($\{a^m b^m c^n\} \cap \{a^m b^n c^n\}$) و در نتیجه متمم.

۱۴ فصل

لم تزریق برای زبان‌های مستقل از متن

همان‌طور که لم تزریق فصل ۸ برای اثبات نامنظم بودن به کار می‌رفت، نسخه‌ای از آن برای زبان‌های مستقل از متن هم وجود دارد. این بار رشته به **پنج** بخش تقسیم می‌شود و بخش‌های دوم و چهارم **با هم** تکرار می‌شوند.

۱.۱۴ صورت لم

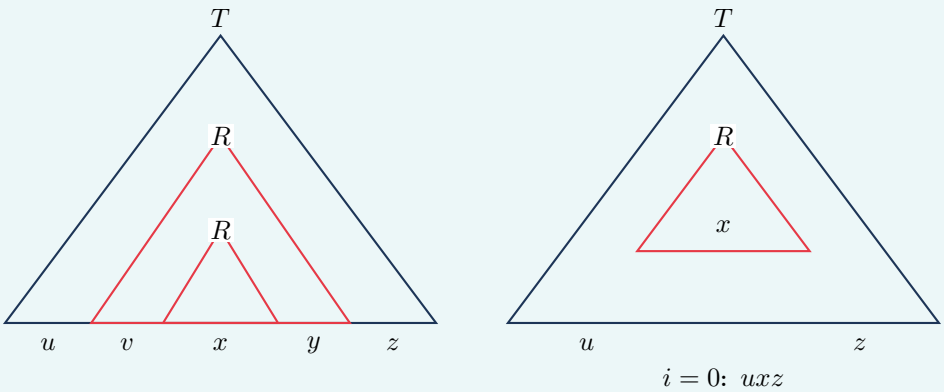
قضیه ۱.۱۴ (لم تزریق برای زبان‌های مستقل از متن)

اگر A زبانی مستقل از متن باشد، عددی مثل p (طول تزریق) وجود دارد به طوری که هر $s \in A$ با $|s| \geq p$ را می‌توان به شکل $s = uvxyz$ نوشت که

۱. برای هر $i \geq 0$: $uv^ixy^iz \in A$،
۲. $|vy| > 0$، و
۳. $|vxy| \leq p$.

شرط ۲ می‌گوید دست‌کم یکی از v و y ناتهی است و شرط ۳ می‌گوید v، x و y روی هم حداکثر p نماد دارند (اما این بخش می‌تواند در هر جای s باشد، نه فقط ابتدای آن).

ایده اثبات. فرض کنید G گرامری برای A باشد و $s \in A$ رشته‌ای خیلی بلند. درخت تجزیه s باید خیلی بلند باشد، پس مسیری طولانی از ریشه به یک برگ دارد. چون تعداد متغیرها متناهی است، طبق اصل لانه کبوتری یک متغیر مثل R روی این مسیر تکرار می‌شود. زیردرخت R بالایی رشته vxy و زیردرخت R پایینی رشته x را تولید می‌کند. حالا می‌توانیم زیردرخت R پایینی را با زیردرخت R بالایی جایگزین کنیم (و این کار را تکرار کنیم) تا برای $i > 1$ ساخته شود، یا زیردرخت بالایی را با پایینی جایگزین کنیم تا uxz ($i = 0$) به دست آید. همه این درخت‌ها درخت تجزیه معتبرند:



اثبات. فرض کنید $G = (V, \Sigma, R, S)$ گرامری برای A باشد و b بیشترین تعداد نماد در سمت راست یک قاعده (فرض کنید $b \geq 2$). در هر درخت تجزیه هر گره حداکثر b فرزند دارد. پس حداکثر b برگ در فاصله یک گام از ریشه، حداکثر b^2 برگ در فاصله دو گام و حداکثر b^h برگ در فاصله h گام هستند. در نتیجه اگر ارتفاع درخت حداکثر h باشد، رشته تولیدشده حداکثر b^h نماد دارد. برعکس، درخت تجزیه رشته‌ای به طول دست‌کم $b^h + 1$ ارتفاع دست‌کم $h + 1$ دارد.

قرار دهید $p = b^{|V|+1}$. فرض کنید $s \in A$ و $|s| \geq p$. چون $p = b^{|V|+1} > b^{|V|}$، ارتفاع هر درخت تجزیه s دست‌کم $|V| + 1$ است. از میان درخت‌های تجزیه s، درخت τ را با **کمترین تعداد گره** انتخاب کنید. بلندترین مسیر τ از ریشه به یک برگ طول دست‌کم $|V| + 1$ دارد، پس دست‌کم $|V| + 2$ گره دارد: یک پایانه (برگ) و دست‌کم $|V| + 1$ متغیر. طبق اصل لانه کبوتری، در میان **پایین‌ترین** $|V| + 1$ متغیر این مسیر، یک متغیر R تکرار شده است. دو ظهور آن را R بالایی و R پایینی بنامید و $s = uvxyz$ را این‌طور تعریف کنید: زیردرخت R پایینی x را تولید می‌کند و زیردرخت R بالایی vxy را.

۱. هر دو زیردرخت ریشه R دارند، پس می‌توان یکی را جای دیگری گذاشت. جایگزین کردن زیردرخت پایینی با بالایی (و تکرار) درخت‌هایی برای uv^ixy^iz با $i > 1$ می‌سازد، و جایگزین کردن زیردرخت بالایی با پایینی درختی برای uxz. پس $uv^ixy^iz \in A$ برای هر $i \geq 0$.

۲. فرض کنید v و y هر دو تهی باشند. آنگاه با جایگزین کردن زیردرخت R بالایی با زیردرخت R پایینی، درختی به دست می‌آید که همچنان s را تولید می‌کند اما گره‌های کمتری دارد. این با انتخاب τ به عنوان کوچک‌ترین درخت تناقض دارد. پس $|vy| > 0$.

۳. R بالایی در میان پایین‌ترین $|V| + 1$ متغیر بلندترین مسیر است. پس بلندترین مسیر زیردرخت آن حداکثر $|V|+1$ یال دارد (اگر بیشتر بود، مسیر کل درخت هم بلندتر می‌شد). بنابراین این زیردرخت حداکثر $p = b^{|V|+1}$ برگ دارد، یعنی $|vxy| \leq p$.

■

دقت کنید! مثل حالت منظم، این لم فقط شرط **لازم** است: هر زبان مستقل از متن شرایط آن را دارد، اما زبانی که شرایط را دارد ممکن است مستقل از متن نباشد.

۲.۱۴ مثال‌ها

روش استفاده مثل فصل ۸ است: فرض می‌کنیم زبان مستقل از متن است، رشته مناسب s را انتخاب می‌کنیم و نشان می‌دهیم **هر** تقسیم $s = uvxyz$ با شرط‌های ۲ و ۳، یک i دارد که uv^ixy^iz از زبان بیرون می‌افتد.

مثال ۲.۱۴ $(a^nb^nc^n)$. $B = \{a^nb^nc^n \mid n \geq 0\}$ مستقل از متن نیست.

فرض کنید B مستقل از متن باشد و p طول تزریق آن. رشته $s = a^pb^pc^p \in B$ را انتخاب کنید. هر تقسیم $s = uvxyz$ با $|vy| > 0$ را در نظر بگیرید:

۱. **هر کدام از v و y فقط یک نوع نماد دارد.** در این صورت v و y روی هم حداکثر دو نوع از سه نماد a، b و c را دارند. پس در uv^2xy^2z تعداد دست‌کم یک نماد زیاد شده و تعداد دست‌کم یک نماد ثابت مانده است. پس تعداد aها، bها و cها برابر نیست و $uv^2xy^2z \notin B$.

۲. v **یا** y **بیش از یک نوع نماد دارد.** در این صورت در uv^2xy^2z نمادها به ترتیب درست نیستند (مثلاً اگر $v = ab$، رشته شامل $abab$ است). هیچ رشته‌ای در B زیررشته‌ای مثل ba، ca یا cb ندارد، پس $uv^2xy^2z \notin B$.

هر دو حالت به تناقض می‌رسند، پس B مستقل از متن نیست. (اینجا حتی به شرط ۳ هم نیازی نشد.)

مثال ۳.۱۴ (رشته تکراری). $D = \{ww \mid w \in \{0,1\}^*\}$ مستقل از متن نیست.

فرض کنید p طول تزریق باشد و $s = 0^p1^p0^p1^p \in D$. (دقت کنید $0^p1\,0^p1$ که در فصل ۸ به کار رفت اینجا جواب نمی‌دهد، چون با $v = 0$ و $y = 0$ در دو طرف 1 اول قابل تزریق است.) هر تقسیم $s = uvxyz$ با شرط‌های ۲ و ۳ را در نظر بگیرید. چون $|vxy| \le p$، بخش vxy حداکثر دو بلوک مجاور از چهار بلوک 0^p، 1^p، 0^p، 1^p را می‌پوشاند.

- اگر vxy کاملاً در نیمه اول s باشد، با $i = 2$ طول رشته کمی زیاد می‌شود و نیمه دوم رشته جدید با 1 شروع می‌شود (چون بخشی از بلوک‌های اول به نیمه دوم منتقل شده)، در حالی که نیمه اولش با 0 شروع می‌شود. پس رشته به شکل ww نیست. حالت کاملاً در نیمه دوم هم مشابه است: نیمه اول رشته جدید به 0 ختم می‌شود و نیمه دوم به 1.
- اگر vxy از وسط s رد شود، در بلوک‌های 1^p0^p میانی است. با $i = 0$ رشته $0^p1^j0^k1^p$ به دست می‌آید که دست‌کم یکی از j و k کمتر از p است. چنین رشته‌ای به شکل ww نیست، چون اگر بود هر نیمه باید 0^p و 1^p کامل داشته باشد.

پس D مستقل از متن نیست. (اگر uv^ixy^iz طول فرد داشته باشد، بدیهی است که در D نیست؛ تحلیل بالا برای طول زوج است.)

نکته. این دو مثال نشان می‌دهند ماشین پشته‌ای چه کاری را **نمی‌تواند** انجام دهد: پشته فقط یک «شمارش» یا یک «مقایسه آینه‌ای» را می‌تواند پیگیری کند (a^nb^n و ww^R مستقل از متن‌اند)، اما نه دو شمارش هم‌زمان ($a^nb^nc^n$) یا مقایسه در یک جهت (ww).

جمع‌بندی فصل

- لم تزریق برای CFLها: $s = uvxyz$ با $uv^ixy^iz \in A$، $|vy| > 0$ و $|vxy| \le p$.
- اثبات: درخت تجزیه بلند، تکرار یک متغیر روی بلندترین مسیر، جایگزینی زیردرخت‌ها. طول تزریق $p = b^{|V|+1}$.
- $\{a^nb^nc^n\}$ و $\{ww\}$ مستقل از متن نیستند.

۱۵ فصل

ماشین‌های پشته‌ای قطعی و گرامرهای LL

در این فصل اول دو ویژگی تصمیم‌پذیر گرامرهای مستقل از متن را می‌بینیم و بعد به ماشین‌های پشته‌ای **قطعی** می‌پردازیم. نشان می‌دهیم که برخلاف ماشین‌های متناهی، اینجا نامعینی واقعاً قدرت اضافه می‌کند. در پایان گرامرهای LL را معرفی می‌کنیم که پایه تجزیه‌گرهای سریع در کامپایلرها هستند.

۱.۱۵ چند ویژگی تصمیم‌پذیر

قضیه ۱.۱۵

برای گرامر مستقل از متن $G = (V, T, S, P)$ الگوریتمی وجود دارد که تعیین می‌کند $L(G)$ تهی است یا نه.

اثبات. برای سادگی فرض کنید $\varepsilon \notin L(G)$ (در غیر این صورت زبان تهی نیست و تغییر کوچکی در استدلال لازم است). الگوریتم حذف متغیرهای بی‌فایده (فصل ۱۰) را اجرا کنید. اگر S بی‌فایده تشخیص داده شد، $L(G)$ تهی است؛ وگرنه $L(G)$ دست‌کم یک عضو دارد. در واقع کافی است مرحله اول آن الگوریتم را اجرا کنیم: اگر $L(G) \neq \emptyset$ و تنها اگر S یک رشته پایانه‌ای تولید کند.

■

مسئله عضویت ($w \in L(G)$؟) هم با الگوریتم CYK (فصل ۱۱) تصمیم‌پذیر است. اما همه سؤال‌ها درباره گرامرها جواب الگوریتمی ندارند:

نکته. هیچ الگوریتمی وجود ندارد که تعیین کند **دو گرامر مستقل از متن یک زبان را تولید می‌کنند یا نه**. این را در فصل ۲۰ ثابت می‌کنیم.

۲.۱۵ ماشین پشته‌ای قطعی

اسلایدهای این بخش از کتاب لینز هستند که ماشین پشته‌ای را کمی متفاوت از سیپسر تعریف می‌کند. در تعریف لینز ماشین $M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$ یک **نماد شروع پشته** z دارد (در ابتدا پشته فقط z را دارد)، در هر حرکت **همیشه** نماد بالای پشته را برمی‌دارد و می‌تواند یک **رشته** را جای آن بگذارد:

$$\delta : Q \times (\Sigma \cup \{\lambda\}) \times \Gamma \to \text{زیرمجموعه‌های متناهی } Q \times \Gamma^*.$$

(لینز رشته تهی را با λ نشان می‌دهد.) اگر $(q', u) \in \delta(q, a, b)$، ماشین در حالت q با خواندن a و دیدن b بالای پشته، b را برمی‌دارد، رشته u را جای آن می‌گذارد (اولین نماد u بالای پشته) و به q' می‌رود. این دو تعریف هم‌ارزند.

تعریف ۲.۱۵ (ماشین پشته‌ای قطعی (DPDA))

ماشین پشته‌ای $M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$ **قطعی** است اگر برای هر $q \in Q$، $a \in \Sigma \cup \{\lambda\}$ و $b \in \Gamma$:

۱. $\delta(q, a, b)$ حداکثر یک عضو داشته باشد، و

۲. اگر $\delta(q, \lambda, b)$ ناتهی باشد، $\delta(q, c, b)$ برای هر $c \in \Sigma$ تهی باشد.

شرط اول می‌گوید با یک ورودی و یک نماد پشته حداکثر یک انتخاب داریم. شرط دوم می‌گوید اگر حرکت λ ممکن است، حرکت با خواندن ورودی ممکن نیست. پس در هر پیکربندی حداکثر یک حرکت ممکن است. (برخلاف DFA، ممکن است در پیکربندی‌ای هیچ حرکتی نباشد.)

تعریف ۳.۱۵ (زبان مستقل از متن قطعی)

زبان L **مستقل از متن قطعی** (DCFL) است اگر و تنها اگر یک DPDA مثل M وجود داشته باشد که $L = L(M)$.

مثال ۴.۱۵. زبان $L = \{a^n b^n \mid n \geq 0\}$ مستقل از متن قطعی است. ماشین

$$M = (\{q_0, q_1, q_2\}, \{a, b\}, \{0, 1\}, \delta, q_0, 0, \{q_0\})$$

با

$$\begin{aligned} &\delta(q_0, a, 0) = \{(q_1, 10)\}, \quad \delta(q_1, a, 1) = \{(q_1, 11)\}, \\ &\delta(q_1, b, 1) = \{(q_2, \lambda)\}, \quad \delta(q_2, b, 1) = \{(q_2, \lambda)\}, \\ &\delta(q_2, \lambda, 0) = \{(q_0, \lambda)\} \end{aligned}$$

این زبان را می‌پذیرد. نماد 0 همان نماد شروع پشته است و نقش علامت ته پشته را دارد. هر a یک 1 اضافه می‌کند و هر b یک 1 برمی‌دارد. وقتی دوباره به 0 برسیم، به حالت پذیرش q_0 می‌رویم. این ماشین هر دو شرط تعریف را دارد، پس قطعی است.

۳.۱۵ نامعینی در ماشین‌های پشته‌ای قدرت اضافه می‌کند

قضیه ۵.۱۵

زبان‌های مستقل از متنی وجود دارند که قطعی نیستند. مثلاً

$$L = L_1 \cup L_2, \qquad L_1 = \{a^n b^n \mid n \geq 0\}, \quad L_2 = \{a^n b^{2n} \mid n \geq 0\}$$

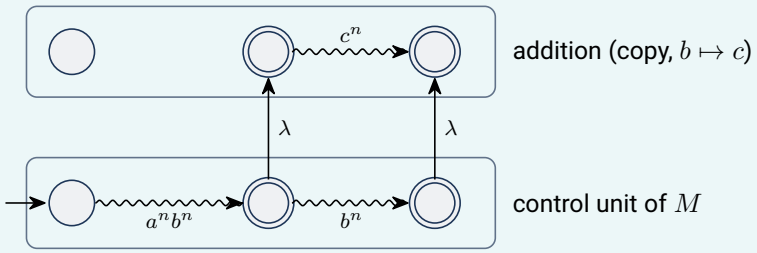
مستقل از متن است، اما مستقل از متن قطعی نیست.

مستقل از متن بودن L روشن است: اجتماع دو زبان مستقل از متن است. شهود اینکه چرا قطعی نیست: در ابتدای خواندن رشته هیچ اطلاعاتی نداریم که بفهمیم رشته از L_1 است یا از L_2. پس ماشین قطعی نمی‌داند بعد از دیدن n تا b باید بپذیرد و تمام کند یا ادامه دهد.

ایده اثبات. فرض کنید یک DPDA مثل M برای L وجود داشته باشد. نشان می‌دهیم آنگاه زبان

$$\widehat{L} = L \cup \{a^n b^n c^n \mid n \geq 1\}$$

مستقل از متن است، که ممکن نیست. ایده: چون M قطعی است، روی ورودی $a^n b^{2n}$ بعد از خواندن $a^n b^n$ دقیقاً در همان پیکربندی‌ای است که روی ورودی $a^n b^n$ (که در آن می‌پذیرد) می‌رسید. یک کپی از M می‌سازیم که در آن هر b به c تبدیل شده است. ماشین جدید $\widehat{M}$ وقتی M به حالت پذیرش رسید، می‌تواند به کپی برود و c^n را به جای b^n دوم بخواند.



اثبات. فرض کنید DPDA $M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$ با $Q = \{q_0, q_1, \ldots, q_n\}$ زبان L را بپذیرد. ماشین $\widehat{M} = (\widehat{Q}, \Sigma \cup \{c\}, \Gamma, \delta \cup \widehat{\delta}, q_0, z, \widehat{F})$ را می‌سازیم:

$$\widehat{Q} = Q \cup \{\widehat{q}_0, \widehat{q}_1, \ldots, \widehat{q}_n\}, \qquad \widehat{F} = F \cup \{\widehat{q}_i \mid q_i \in F\},$$

- برای هر $q_f \in F$ و $s \in \Gamma$: $\widehat{\delta}(q_f, \lambda, s) = \{(\widehat{q}_f, s)\}$ (پرش به کپی بدون تغییر پشته)؛
- برای هر $\delta(q_i, b, s) = \{(q_j, u)\}$: $\widehat{\delta}(\widehat{q}_i, c, s) = \{(\widehat{q}_j, u)\}$ (کپی گذارهای b با c)؛
- برای هر $\delta(q_i, \lambda, s) = \{(q_j, u)\}$: $\widehat{\delta}(\widehat{q}_i, \lambda, s) = \{(\widehat{q}_j, u)\}$.

($\widehat{M}$ لازم نیست قطعی باشد؛ فقط یک ماشین پشته‌ای است.) نماد $\vdash^*$ یعنی «با صفر یا چند حرکت به این پیکربندی می‌رسد» و پیکربندی را با سه‌تایی (حالت، بقیه ورودی، محتوای پشته) نشان می‌دهیم.

رشته‌های $a^n b^n c^n$ ($\widehat{M}$ و همه رشته‌های L) را می‌پذیرد. هر رشته L را $\widehat{M}$ مثل M می‌پذیرد. حالا چون $a^n b^n \in L$:

$$(q_0, a^n b^n, z) \vdash_M^* (q_i, \lambda, u), \qquad q_i \in F.$$

چون M قطعی است، روی $a^n b^{2n}$ هم دقیقاً همین حرکت‌ها را انجام می‌دهد: $(q_0, a^n b^{2n}, z) \vdash_M^* (q_i, b^n, u)$. و چون $a^n b^{2n} \in L$، ادامه محاسبه به پذیرش می‌رسد:

$$(q_i, b^n, u) \vdash_M^* (q_j, \lambda, u_1), \qquad q_j \in F.$$

طبق ساخت، کپی همین حرکت‌ها را با c به جای b انجام می‌دهد: $(\widehat{q}_i, c^n, u) \vdash_{\widehat{M}}^* (\widehat{q}_j, \lambda, u_1)$. پس

$$(q_0, a^n b^n c^n, z) \vdash_{\widehat{M}}^* (q_i, c^n, u) \vdash_{\widehat{M}} (\widehat{q}_i, c^n, u) \vdash_{\widehat{M}}^* (\widehat{q}_j, \lambda, u_1),$$

و چون $\widehat{q}_j \in \widehat{F}$، رشته $a^n b^n c^n$ پذیرفته می‌شود.

$\widehat{M}$ رشته دیگری را نمی‌پذیرد. (این بخش در اسلایدها به عنوان تمرین آمده بود.) فرض کنید $\widehat{M}$ رشته w را بپذیرد. اگر محاسبه پذیرنده هرگز به کپی نرود، w را M پذیرفته و $w \in L$. در غیر این صورت، تنها راه رفتن به کپی، حرکت

λ از یک $q_i \in F$ است. پس $w = xv$ که $(q_0, x, z) \vdash_M^* (q_i, \lambda, u)$ با $q_i \in F$؛ یعنی M رشته x را می‌پذیرد و $x \in L$. در کپی فقط گذارهای c و λ وجود دارد، پس $v = c^m$ و کپی، M را روی b^m از پیکربندی (q_i, b^m, u) تا یک حالت پذیرش شبیه‌سازی می‌کند. چون M قطعی است، این یعنی M رشته xb^m را هم می‌پذیرد؛ پس $xb^m \in L$.

حالا اگر $m = 0$، $w = x \in L$. اگر $m \geq 1$: $x \in L$ پس $x = a^n b^n$ یا $x = a^n b^{2n}$.

- اگر $x = a^n b^{2n}$، آنگاه $xb^m = a^n b^{2n+m}$ که در L نیست (برای $n \geq 1$ چون $2n + m \notin \{n, 2n\}$، و برای $n = 0$ چون $b^m \notin L$). پس این حالت ممکن نیست.
- اگر $x = a^n b^n$، آنگاه $a^n b^{n+m} \in L$ ایجاب می‌کند $n + m = 2n$، یعنی $m = n \geq 1$. پس $w = a^n b^n c^n$.

در نتیجه $L(\widehat{M}) = \widehat{L} = L \cup \{a^n b^n c^n \mid n \geq 1\}$.

تناقض. $\widehat{L}$ زبان یک ماشین پشته‌ای است، پس مستقل از متن است. طبق قضیه ۴.۱۳ اشتراک آن با زبان منظم $a^* b^* c^+$ هم مستقل از متن است. اما

$$\widehat{L} \cap a^* b^* c^+ = \{a^n b^n c^n \mid n \geq 1\}$$

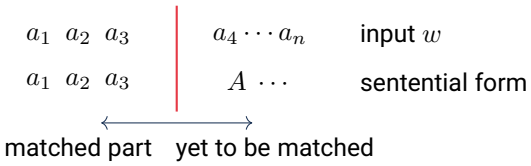
که طبق مثال ۲.۱۴ (با همان استدلال) مستقل از متن نیست. پس فرض وجود M غلط است و L مستقل از متن قطعی نیست.

■

۴.۱۵ گرامرهای زبان‌های مستقل از متن قطعی

زبان‌های مستقل از متن قطعی را می‌توان **سریع** تجزیه کرد: گذارهای یک DPDA را می‌توان بدون عقب‌گرد و بدون موازی‌کاری دنبال کرد (فقط باید حواسمان به گذارهای λ باشد). برای کامپایلرها به **گرامرهای قطعی** نیاز داریم.

فرض کنید در حال **تجزیه بالا به پایین** هستیم، یعنی دنبال اشتقاق چپ‌ترین یک جمله. ورودی w را از چپ به راست می‌خوانیم و یک صورت جمله‌ای می‌سازیم که پیشوند پایانه‌ای آن با بخش خوانده‌شده w یکی است:



سؤال اصلی: برای جایگزینی متغیر A از کدام قاعده استفاده کنیم؟ در **گرامرهای ساده** (فصل ۱۱) در هر گام فقط یک قاعده ممکن است (قاعده‌ای که با نماد فعلی ورودی شروع می‌شود)، اما این گرامرها برای کاربردهای واقعی بیش از حد محدودند.

تعریف ۶.۱۵ (گرامر LL)

در یک گرامر LL، ورودی از چپ به راست خوانده می‌شود (L اول) و اشتقاق چپ‌ترین ساخته می‌شود (L دوم). در هر گام می‌توان با دیدن نماد فعلی ورودی و تعداد متناهی نماد بعدی، قاعده درست را به‌طور یکتا پیش‌بینی کرد. در یک گرامر LL(k) قاعده درست با دیدن نماد فعلی و $k-1$ نماد بعدی (**پیش‌بینی** یا lookahead) به‌طور یکتا مشخص می‌شود.

هر گرامر ساده یک گرامر LL است.

مثال ۷.۱۵. گرامر $S \to aSb \mid ab$ ساده نیست (زوج (S, a) در دو قاعده آمده)، اما LL است: برای انتخاب قاعده به دو نماد متوالی ورودی نگاه می‌کنیم. اگر اولی a و دومی b بود، قاعده $S \to ab$ و در غیر این صورت $S \to aSb$ را به کار می‌بریم.

مثال ۸.۱۵. گرامر پرانتزهای متوازن

$$S \to SS \mid aSb \mid ab$$

برای **هیچ** k یک گرامر LL(k) نیست. با دیدن هر تعداد متناهی نماد نمی‌توان فهمید کدام قاعده را به کار ببریم. مثلاً دو رشته زیر را در نظر بگیرید (a را «(» و b را «)» بخوانید):

$$\underbrace{((((((\cdots(}_{>k}\ (())\ \cdots))))))) \qquad \underbrace{((((((\cdots(}_{>k}\ \cdots))))))\ ()$$

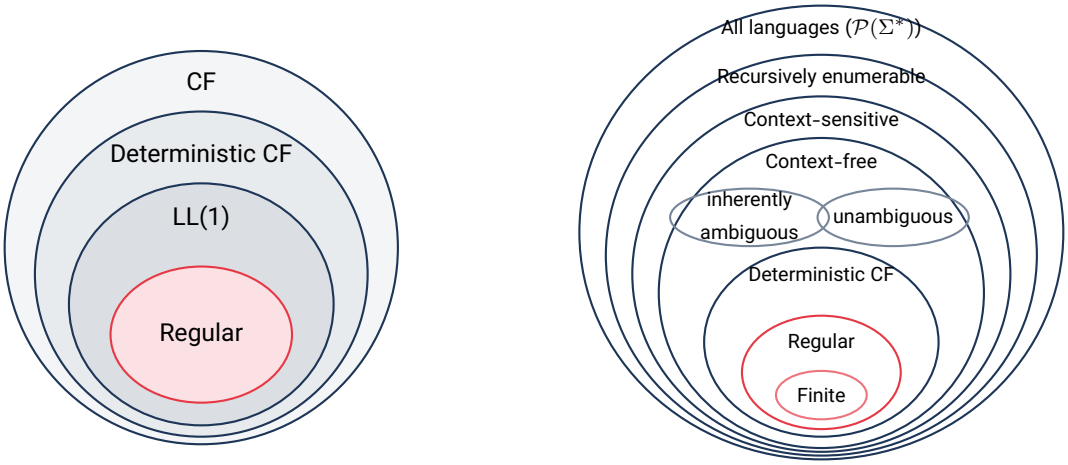
هر دو با بیش از k پرانتز باز شروع می‌شوند، اما برای اولی در گام اول باید $S \to aSb$ و برای دومی $S \to SS$ را به کار برد. با این حال همین زبان یک گرامر LL هم‌ارز دارد:

$$S_0 \to aSbS, \qquad S \to aSbS \mid \lambda.$$

در این گرامر S با دیدن a قاعده اول و در غیر این صورت λ را انتخاب می‌کند.

۵.۱۵ تصویر کلی

رابطه دسته‌های زبانی که تا اینجا دیدیم:



(در شکل راست، زبان‌های **حساس به متن** دسته‌ای بین مستقل از متن و بازگشتی شمارش‌پذیر هستند که در این درس به آن‌ها نمی‌پردازیم؛ زبان‌های بازگشتی شمارش‌پذیر را در بخش بعد می‌بینیم.)

جمع‌بندی فصل

- تهی بودن زبان یک CFG و عضویت در آن تصمیم‌پذیرند؛ هم‌ارزی دو CFG تصمیم‌پذیر نیست.
- DPDA: در هر پیکربندی حداکثر یک حرکت. DCFLها زیرمجموعه سره CFLها هستند.
- $\{a^n b^n\} \cup \{a^n b^{2n}\}$ مستقل از متن است ولی قطعی نیست؛ اثبات با ساختن ماشینی برای $\widehat{L}$ که شامل $a^n b^n c^n$ است.
- گرامر LL(k): قاعده با دیدن k نماد به‌طور یکتا انتخاب می‌شود. هر گرامر ساده LL است.

بخش چهارم

محاسبه‌پذیری

فصل ۱۶

ماشین تورینگ

تا اینجا سه مدل محاسباتی دیدیم:

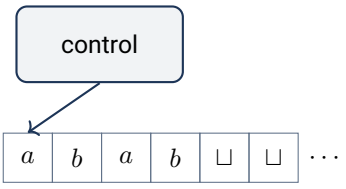
- DFA: قطعی، بدون حافظه موقت؛
- NFA: غیرقطعی، بدون حافظه موقت؛
- PDA: غیرقطعی، با یک پشته نامحدود.

در این فصل به مدلی می‌رسیم که **مدل یک کامپیوتر همه‌منظوره** است: **ماشین تورینگ** (Turing Machine) که آلن تورینگ در سال ۱۹۳۶ پیشنهاد کرد. ماشین تورینگ قطعی است و یک نوار نامحدود (از یک طرف) دارد که می‌تواند روی آن بنویسد و بخواند. هر کاری که یک کامپیوتر واقعی انجام دهد، ماشین تورینگ هم می‌تواند انجام دهد.

۱.۱۶ ماشین تورینگ چیست؟

ماشین تورینگ این اجزا را دارد:

- یک مجموعه متناهی از حالت‌ها با یک حالت شروع و دو حالت خاص: **پذیرش** و **رد**.
- یک **نوار** که از یک طرف نامحدود است و به خانه‌ها تقسیم شده. در ابتدا رشته ورودی در خانه‌های اول نوار است و بقیه خانه‌ها **خالی** (␣) هستند.
- یک **هد** که روی نوار قرار دارد، نماد زیرش را می‌خواند، می‌تواند آن را عوض کند و یک خانه به چپ یا راست حرکت کند.



تفاوت‌ها با ماشین‌های متناهی:

۱. ماشین تورینگ هم می‌تواند روی نوار **بنویسد** و هم از آن بخواند.

۲. هد می‌تواند هم به **چپ** و هم به **راست** حرکت کند.

۳. نوار **نامحدود** است.

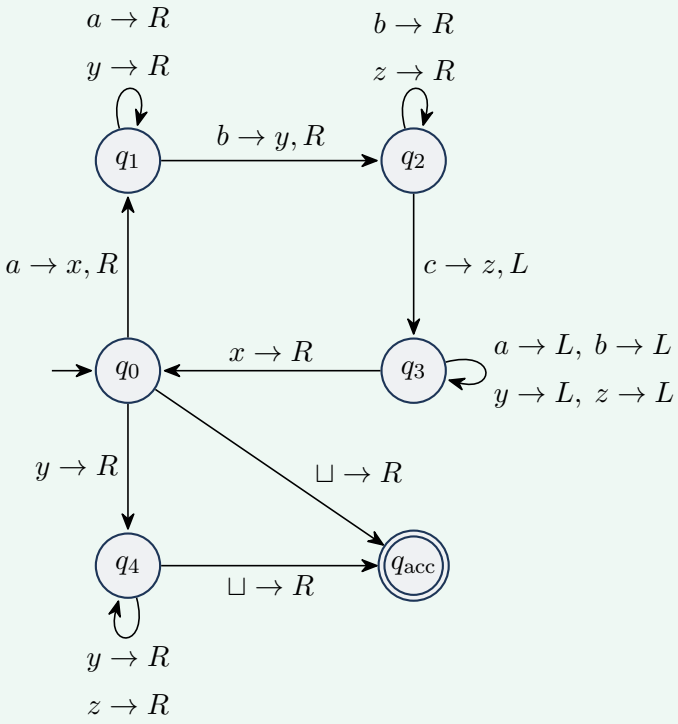
۴. حالت‌های پذیرش و رد **فوراً** اثر می‌کنند: به محض ورود به آن‌ها ماشین متوقف می‌شود (لازم نیست ورودی تمام شده باشد).

نمایش گذارها: برچسب $a \to b, R$ روی پیکانی از q_1 به q_2 یعنی: «اگر در q_1 هستی و هد نماد a را می‌خواند، آن را با b عوض کن، هد را یک خانه به راست ببر و به q_2 برو». برچسب $a \to R$ کوتاه‌شده $a \to a, R$ است (نماد عوض نمی‌شود) و $a \to L$ هم به همین شکل.

۲.۱۶ مثال: سه دسته نماد برابر

در فصل ۱۴ دیدیم $A = \{a^n b^n c^n \mid n \geq 0\}$ مستقل از متن نیست. حالا یک ماشین تورینگ برای آن می‌سازیم.

مثال ۱.۱۶ (ماشین تورینگ برای $a^n b^n c^n$). فرض کنید ورودی آن‌قدر طولانی است که نمی‌توانید همه‌اش را به خاطر بسپارید، اما اجازه دارید روی ورودی جلو و عقب بروید و روی آن علامت بگذارید. ماشین در هر دور یک a، یک b و یک c را با هم جفت می‌کند. برای اینکه بداند کدام نمادها قبلاً بررسی شده‌اند، آن‌ها را با x، y و z جایگزین می‌کند. اگر همه نمادها جایگزین شدند، همه چیز جفت شده و ماشین می‌پذیرد؛ اگر جایی ناهماهنگی پیدا کرد، رد می‌کند.



گذارهایی که رسم نشده‌اند به حالت رد می‌روند. q_1 به دنبال اولین b علامت‌نخورده می‌گردد، q_2 به دنبال اولین c و q_3 هد را به اولین نماد علامت‌نخورده برمی‌گرداند. اگر در q_0 به جای a یک y ببینیم، همه aها جفت شده‌اند و q_4 بررسی می‌کند که دیگر b یا c علامت‌نخورده‌ای نمانده باشد.

۳.۱۶ تعریف صوری

تعریف ۲.۱۶ (ماشین تورینگ)

یک ماشین تورینگ هفت‌تایی $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ است که در آن Q، Σ و Γ مجموعه‌های متناهی‌اند و

۱. Q مجموعه حالت‌ها،

۲. Σ الفبای ورودی است و نماد خالی را ندارد ($\sqcup \notin \Sigma$)،

۳. Γ الفبای نوار است، $\sqcup \in \Gamma$ و $\Sigma \subseteq \Gamma$،

۴. $\delta : (Q - \{q_{\text{accept}}, q_{\text{reject}}\}) \times \Gamma \to Q \times \Gamma \times \{L, R\}$ تابع گذار،

۵. $q_0 \in Q$ حالت شروع،

۶. $q_{\text{accept}} \in Q$ تنها حالت پذیرش، و

۷. $q_{\text{reject}} \in Q$ تنها حالت رد است و $q_{\text{accept}} \neq q_{\text{reject}}$.

اگر $\delta(q, s) = (r, t, D)$، یعنی: در حالت q وقتی هد نماد s را می‌خواند، به حالت r برو، s را با t عوض کن و هد را در جهت $D \in \{L, R\}$ حرکت بده. چون دامنه δ همه حالت‌ها جز پذیرش و رد است، از هر حالت دیگر برای هر نماد نوار دقیقاً یک گذار وجود دارد و از حالت‌های پذیرش و رد هیچ گذاری نیست؛ ماشین با ورود به آن‌ها متوقف می‌شود.

نکته. در عمل تقریباً هیچ‌وقت توصیف صوری کامل یک ماشین تورینگ را نمی‌نویسیم، چون خیلی بزرگ می‌شود. بیشتر توصیف‌های **سطح بالا** می‌دهیم. اما باید به خاطر داشت هر توصیف سطح بالا در واقع خلاصه‌ای از یک توصیف صوری است که با حوصله می‌توان آن را کامل نوشت.

۴.۱۶ محاسبه ماشین تورینگ

- ماشین ورودی $w = w_1 \cdots w_n \in \Sigma^*$ را در n خانه اول نوار می‌گیرد و بقیه نوار خالی است. هد روی چپ‌ترین خانه است.
- چون $\sqcup \notin \Sigma$، اولین $\sqcup$ روی نوار انتهای ورودی را نشان می‌دهد.
- محاسبه طبق تابع گذار پیش می‌رود. اگر ماشین بخواهد هد را از انتهای چپ نوار به چپ ببرد، هد سر جایش می‌ماند.
- محاسبه تا ورود به حالت پذیرش یا رد ادامه می‌یابد و آنجا متوقف می‌شود. اگر هیچ‌کدام رخ ندهد، ماشین **برای همیشه** ادامه می‌دهد.

تعریف ۳.۱۶ (پیکربندی)

در طول محاسبه سه چیز تغییر می‌کند: حالت فعلی، محتوای نوار و مکان هد. به ترکیب این سه یک **پیکربندی** (Configuration) می‌گوییم و آن را به شکل $u\,q\,v$ می‌نویسیم: حالت فعلی q است، محتوای نوار uv (و بعد از آن فقط $\sqcup$) و هد روی اولین نماد v قرار دارد.

مثال ۴.۱۶. در اجرای ماشین مثال قبل روی $aabbcc$، پیکربندی $xxy\,q_1\,bzc$ یعنی نوار $xxybzc$ است، ماشین در q_1 است و هد روی b (چهارمین خانه) قرار دارد.

تعریف ۵.۱۶ (حرکت یک‌گامی)

پیکربندی C_1 پیکربندی C_2 را **نتیجه می‌دهد** اگر ماشین بتواند در یک گام از C_1 به C_2 برود. برای $a, b, c \in \Gamma$، $u, v \in \Gamma^*$ و $q_i, q_j \in Q$:

- **حرکت به چپ:** اگر $\delta(q_i, b) = (q_j, c, L)$، آنگاه $ua\,q_i\,bv$ نتیجه می‌دهد $u\,q_j\,acv$. در انتهای چپ نوار، $q_i\,bv$ نتیجه می‌دهد $q_j\,cv$.
- **حرکت به راست:** اگر $\delta(q_i, b) = (q_j, c, R)$، آنگاه $ua\,q_i\,bv$ نتیجه می‌دهد $uac\,q_j\,v$. در انتهای چپ هم $q_i\,bv$ نتیجه می‌دهد $c\,q_j\,v$.
- **انتهای راست:** پیکربندی $ua\,q_i$ همان $ua\,q_i \sqcup$ است؛ یعنی فرض می‌کنیم هد روی یک خانه خالی است و قوانین بالا را به کار می‌بریم.

تعریف ۶.۱۶ (پذیرش)

- **پیکربندی شروع** روی ورودی w برابر $q_0\,w$ است.
- در **پیکربندی پذیرش** حالت q_{accept} و در **پیکربندی رد** حالت q_{reject} است. این دو **پیکربندی توقف** هستند و پیکربندی دیگری را نتیجه نمی‌دهند.

ماشین M ورودی w را **می‌پذیرد** اگر دنباله‌ای از پیکربندی‌ها $C_1, C_2, \ldots, C_k$ وجود داشته باشد که (۱) C_1 پیکربندی شروع M روی w باشد، (۲) هر C_i نتیجه دهد C_{i+1} و (۳) C_k یک پیکربندی پذیرش باشد.

مثال ۷.۱۶. اجرای ماشین $a^nb^nc^n$ روی $aabbcc$ (با $\vdash$ برای «نتیجه می‌دهد» و $\vdash^*$ برای چند گام):

$$
\begin{aligned}
&q_0aabbcc \vdash xq_1abbcc \vdash xaq_1bbcc \vdash xayq_2bcc \vdash xaybq_2cc \vdash xayq_3bzc \\
&\vdash^* q_3xaybzc \vdash xq_0aybzc \vdash xxq_1ybzc \vdash xxyq_1bzc \vdash xxyyq_2zc \\
&\vdash xxyyzq_2c \vdash xxyyq_3zz \vdash^* xq_3xyyzz \vdash xxq_0yyzz \vdash xxyq_4yzz \\
&\vdash^* xxyyzzq_4\sqcup \vdash xxyyzz\sqcup\, q_{\text{acc}}.
\end{aligned}
$$

۵.۱۶ زبان‌های تشخیص‌پذیر و تصمیم‌پذیر

تعریف ۸.۱۶ (تشخیص‌پذیر)

مجموعه رشته‌هایی که M می‌پذیرد، **زبان** M است و با $L(M)$ نشان داده می‌شود. زبانی را **تورینگ-تشخیص‌پذیر** (Turing-recognizable) (یا **بازگشتی شمارش‌پذیر** (Recursively Enumerable)) می‌نامیم اگر یک ماشین تورینگ آن را تشخیص دهد.

اجرای یک ماشین تورینگ روی یک ورودی سه نتیجه ممکن دارد: **پذیرش**، **رد** یا **حلقه** (هرگز متوقف نمی‌شود). ماشین می‌تواند یک ورودی را با ورود به q_{reject} یا با حلقه زدن «نپذیرد». مشکل این است که از بیرون نمی‌توان فهمید ماشینی که

هنوز متوقف نشده، در حلقه افتاده یا فقط کارش طول می‌کشد.

تعریف ۹.۱۶ (تصمیم‌گیرنده و تصمیم‌پذیر)

ماشین تورینگی که روی **همه** ورودی‌ها متوقف شود یک **تصمیم‌گیرنده** (Decider) است. تصمیم‌گیرنده‌ای که زبانی را تشخیص دهد، آن زبان را **تصمیم می‌گیرد**. زبانی **تصمیم‌پذیر** (Decidable) (یا **بازگشتی**) است اگر یک ماشین تورینگ آن را تصمیم بگیرد.

هر زبان تصمیم‌پذیر تورینگ-تشخیص‌پذیر است. در فصل ۱۹ می‌بینیم که زبانی وجود دارد که تشخیص‌پذیر هست اما تصمیم‌پذیر نیست.

نکته. تصمیم‌پذیری **ویژگی زبان** است، نه ویژگی یک ماشین خاص. فرض کنید تصمیم‌گیرنده M زبان L را تصمیم بگیرد. می‌توانیم M را طوری تغییر دهیم که به جای رد کردن، حلقه بزند: یک حالت جدید q_{loop} اضافه کنیم، همه گذارهایی را که به q_{reject} می‌روند به q_{loop} ببریم و در q_{loop} با هر نمادی به راست حرکت کنیم. ماشین جدید دیگر تصمیم‌گیرنده نیست، اما زبانش همان L است. L همچنان تصمیم‌پذیر است، چون **یک** تصمیم‌گیرنده (یعنی M) دارد.

قضیه ۱۰.۱۶

هر زبان منظم تصمیم‌پذیر است.

اثبات. فرض کنید DFA $M = (Q, \Sigma, \delta, q_0, F)$ زبان منظم A را تشخیص دهد. ماشین تورینگ $M' = (Q', \Sigma, \Gamma, \delta', q_0, q_{\text{accept}}, q_{\text{reject}})$ را با $Q' = Q \cup \{q_{\text{accept}}, q_{\text{reject}}\}$ و $\Gamma = \Sigma \cup \{\sqcup\}$ می‌سازیم:

$$\delta'(q,t) = \begin{cases} (\delta(q,t),\ t,\ R) & t \in \Sigma \\ (q_{\text{accept}},\ \sqcup,\ R) & t = \sqcup \text{ و } q \in F \\ (q_{\text{reject}},\ \sqcup,\ R) & t = \sqcup \text{ و } q \notin F. \end{cases}$$

M' ورودی را از چپ به راست می‌خواند و حالتش را دقیقاً مثل M عوض می‌کند. وقتی به اولین $\sqcup$ (انتهای ورودی) می‌رسد، بر اساس اینکه M در حالت پذیرش است یا نه، می‌پذیرد یا رد می‌کند. پس $L(M') = L(M) = A$. همچنین M' بعد از هر نماد به راست می‌رود و با رسیدن به $\sqcup$ متوقف می‌شود، پس روی هر ورودی متوقف می‌شود و تصمیم‌گیرنده است.

■

۶.۱۶ مثال: توان‌های دو

مثال ۱۱.۱۶. ماشین تورینگ M_2 زبان $A = \{0^{2^n} \mid n \geq 0\}$ (رشته‌هایی از 0 که طولشان توانی از ۲ است) را تصمیم می‌گیرد.

توصیف سطح بالا. روی ورودی w:

۱. نوار را از چپ به راست طی کن و یکی در میان 0ها را خط بزن (تعداد 0ها نصف می‌شود).

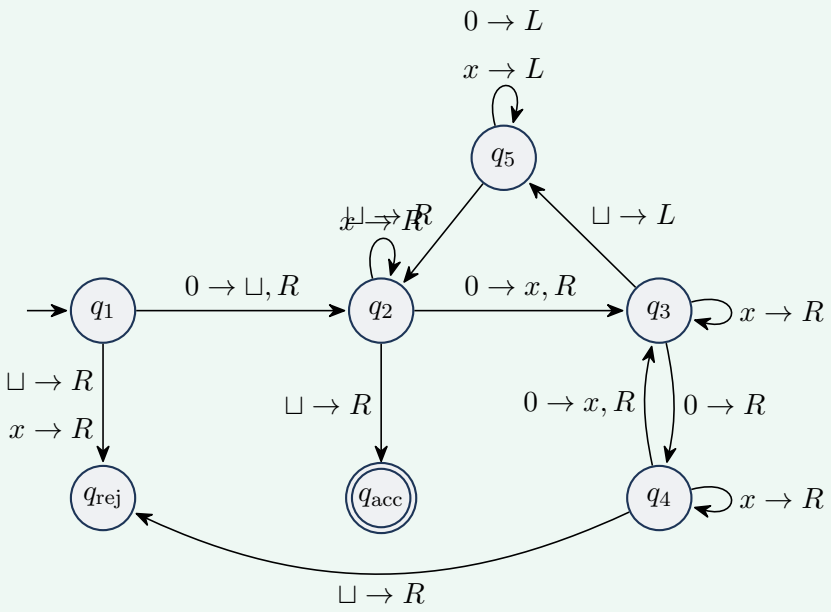
۲. اگر در مرحله ۱ نوار فقط یک 0 داشت، **بپذیر**.

۳. اگر در مرحله ۱ نوار بیش از یک 0 داشت و تعداد 0ها فرد بود، **رد کن**؛ چون عدد فرد بزرگ‌تر از ۱ توانی از ۲ نیست. (ماشین با حالتش زوج یا فرد بودن تعداد 0های دیده‌شده را به خاطر می‌سپارد.)

۴. هد را به انتهای چپ نوار برگردان.

۵. به مرحله ۱ برو.

توصیف صوری. $M_2 = (Q, \Sigma, \Gamma, \delta, q_1, q_{\text{accept}}, q_{\text{reject}})$ با $Q = \{q_1, q_2, q_3, q_4, q_5, q_{\text{accept}}, q_{\text{reject}}\}$، $\Sigma = \{0\}$، $\Gamma = \{0, x, \sqcup\}$ و تابع گذار زیر:



در q_1 اولین 0 با $\sqcup$ جایگزین می‌شود تا انتهای چپ نوار علامت‌گذاری شود. q_3 و q_4 زوج و فرد بودن را دنبال می‌کنند و q_5 هد را به ابتدای نوار برمی‌گرداند. نمونه اجرا روی 0000:

$$q_1 0000 \vdash \sqcup q_2 000 \vdash \sqcup x q_3 00 \vdash \sqcup x 0 q_4 0 \vdash \sqcup x 0 x q_3 \sqcup \vdash \sqcup x 0 q_5 x \sqcup$$
$$\vdash \sqcup x q_5 0 x \sqcup \vdash \sqcup q_5 x 0 x \sqcup \vdash q_5 \sqcup x 0 x \sqcup \vdash \sqcup q_2 x 0 x \sqcup \vdash \sqcup x q_2 0 x \sqcup$$
$$\vdash \sqcup x x q_3 x \sqcup \vdash \sqcup x x x q_3 \sqcup \vdash \sqcup x x q_5 x \sqcup \vdash \sqcup x q_5 x x \sqcup \vdash \sqcup q_5 x x x \sqcup$$
$$\vdash q_5 \sqcup x x x \sqcup \vdash \sqcup q_2 x x x \sqcup \vdash \sqcup x q_2 x x \sqcup \vdash \sqcup x x q_2 x \sqcup \vdash \sqcup x x x q_2 \sqcup \vdash \sqcup x x x \sqcup q_{\text{acc}}.$$

جمع‌بندی فصل

- ماشین تورینگ: نوار نامحدود خواندنی/نوشتنی، هد دوطرفه، حالت‌های پذیرش و رد با اثر فوری.
- $\delta : (Q - \{q_{\text{accept}}, q_{\text{reject}}\}) \times \Gamma \to Q \times \Gamma \times \{L, R\}$. پیکربندی uqv وضعیت کامل ماشین است.
- سه نتیجه: پذیرش، رد، حلقه. تشخیص‌پذیر: یک ماشین می‌پذیردش. تصمیم‌پذیر: یک ماشین که همیشه متوقف می‌شود آن را می‌پذیرد.
- هر زبان منظم تصمیم‌پذیر است و $\{a^n b^n c^n\}$ و $\{0^{2^n}\}$ هم تصمیم‌پذیرند.

۱۷ فصل

گونه‌های ماشین تورینگ

تعریف‌های دیگری هم برای ماشین تورینگ وجود دارد: ماشین با چند نوار، ماشین غیرقطعی، ماشینی که می‌تواند سر جایش بماند و غیره. نکته جالب این است که **همه این گونه‌ها قدرت یکسانی دارند**؛ یعنی دقیقاً همان زبان‌ها را تشخیص می‌دهند. به این ویژگی **استحکام** (robustness) مدل می‌گوییم: ماشین تورینگ در برابر تغییرات معقول در تعریف، به‌طور شگفت‌انگیزی پایدار است.

۱.۱۷ هم‌ارزی و ماشین با حرکت «ماندن»

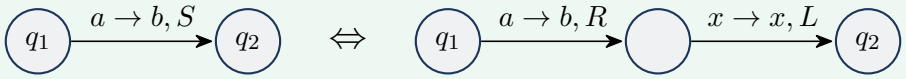
دو مدل را **هم‌ارز** می‌گوییم اگر دقیقاً یک دسته زبان را تشخیص دهند. کلید اثبات هم‌ارزی گونه‌های ماشین تورینگ این است:

نکته. برای نشان دادن هم‌ارزی دو مدل، کافی است نشان دهیم هر کدام می‌تواند دیگری را **شبیه‌سازی** کند.

مثال ۱.۱۷ (حرکت «ماندن»). فرض کنید به ماشین تورینگ اجازه دهیم هد را حرکت ندهد:

$$\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R, S\}.$$

این گونه قدرت بیشتری ندارد. هر گذار S (stay put) را می‌توان با دو گذار جایگزین کرد: یک حرکت به راست به یک حالت جدید و سپس یک حرکت به چپ بدون تغییر نماد:



(گذار دوم برای هر نماد $x \in \Gamma$ تعریف می‌شود.) برعکس، ماشین معمولی حالت خاصی از این گونه است.

۲.۱۷ ماشین تورینگ چندنواره

ماشین تورینگ چندنواره چند نوار دارد که هر کدام هد خواندن و نوشتن مستقل خودش را دارد. در ابتدا ورودی روی نوار ۱ است و بقیه نوارها خالی‌اند. تابع گذار برای ماشین k-نواره:

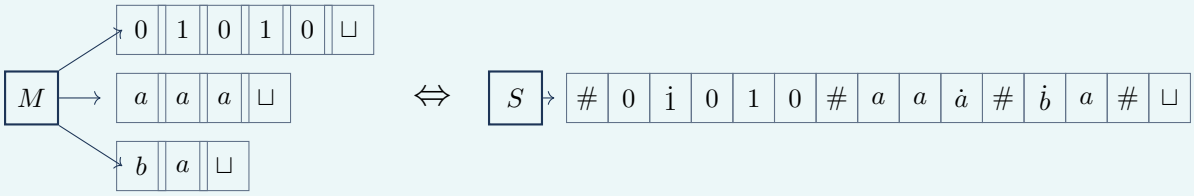
$$\delta : Q \times \Gamma^k \to Q \times \Gamma^k \times \{L, R, S\}^k, \qquad \delta(q_i, a_1, \ldots, a_k) = (q_j, b_1, \ldots, b_k, d_1, \ldots, d_k).$$

یعنی اگر در حالت q_i هدهای ۱ تا k نمادهای a_1 تا a_k را بخوانند، ماشین به q_j می‌رود، b_1 تا b_k را روی نوارها می‌نویسد و هر هد را طبق d_1 تا d_k حرکت می‌دهد.

قضیه ۲.۱۷

هر ماشین تورینگ چندنواره یک ماشین تورینگ تک‌نواره هم‌ارز دارد.

ایده اثبات. ماشین k-نواره M را با ماشین تک‌نواره S شبیه‌سازی می‌کنیم. محتوای k نوار را پشت سر هم روی یک نوار می‌نویسد و آن‌ها را با نماد $\#$ از هم جدا می‌کند. برای اینکه بداند هد هر نوار کجاست، روی نماد زیر هر هد یک **نقطه** می‌گذارد (برای هر نماد $a \in \Gamma$ نسخه نقطه‌دار $\dot{a}$ را به الفبای نوار اضافه می‌کند). این‌ها **هدهای مجازی** هستند.



اثبات. فرض کنید M یک ماشین k-نواره باشد. ماشین تک‌نواره S روی ورودی $w = w_1 \cdots w_n$ این کارها را انجام می‌دهد:

۱. نوار را طوری آماده کن که هر k نوار M را نشان دهد:

$$\# \dot{w}_1 w_2 \cdots w_n \# \dot{\sqcup} \# \dot{\sqcup} \# \cdots \#.$$

۲. برای شبیه‌سازی یک حرکت M، نوار را از اولین $\#$ (انتهای چپ) تا $(k+1)$امین $\#$ (انتهای راست) طی کن تا نمادهای زیر هدهای مجازی را بفهمی. (این نمادها را در حالت ماشین به خاطر می‌سپاری؛ تعدادشان متناهی است.)

۳. یک بار دیگر نوار را طی کن و هر نوار مجازی را طبق تابع گذار M به‌روز کن: نماد زیر هد را عوض کن و نقطه را به خانه کناری منتقل کن.

۴. اگر هد مجازی‌ای روی یک $\#$ رفت (یعنی M هد آن نوار را به بخش خالی و قبلاً ندیده‌شده نوار برده)، یک $\sqcup$ در آن خانه بنویس و همه محتوای بعد از آن را یک خانه به راست جابه‌جا کن. سپس شبیه‌سازی را ادامه بده.

۵. اگر M به حالت پذیرش یا رد رفت، S هم همان کار را بکند.

S در هر لحظه کل وضعیت M (حالت، محتوای همه نوارها و مکان همه هدها) را نگه می‌دارد و هر حرکت M را با چند حرکت شبیه‌سازی می‌کند. پس S دقیقاً همان رشته‌هایی را می‌پذیرد (و رد می‌کند) که M می‌پذیرد (و رد می‌کند)، و روی هر ورودی متوقف می‌شود اگر و تنها اگر M متوقف شود.

■

نتیجه ۳.۱۷

یک زبان تورینگ-تشخیص‌پذیر است اگر و تنها اگر یک ماشین تورینگ چندنواره آن را تشخیص دهد.

اثبات. ماشین تک‌نواره حالت خاص ماشین چندنواره است (با $k = 1$). جهت دیگر همان قضیه ۲.۱۷ است. ■

۳.۱۷ ماشین تورینگ غیرقطعی

در ماشین تورینگ غیرقطعی (NTM) در هر لحظه ممکن است چند حرکت ممکن باشد:

$$\delta : Q \times \Gamma \to \mathcal{P}(Q \times \Gamma \times \{L, R\}).$$

محاسبه یک NTM یک **درخت** است که هر شاخه‌اش یکی از امکان‌ها را دنبال می‌کند. اگر **دست‌کم یک شاخه** به حالت پذیرش برسد، ورودی پذیرفته می‌شود.

قضیه ۴.۱۷

هر ماشین تورینگ غیرقطعی یک ماشین تورینگ قطعی هم‌ارز دارد.

ایده اثبات. ماشین قطعی D همه شاخه‌های محاسبه NTM N را امتحان می‌کند و اگر یکی از آن‌ها به پذیرش برسد، می‌پذیرد. یعنی درخت محاسبه N را جست‌وجو می‌کند. جست‌وجوی **عمق-اول** ایده خوبی نیست، چون ممکن است در یک شاخه نامتناهی گیر کند و هرگز به شاخه پذیرنده‌ای که در جای دیگر درخت است نرسد. پس D درخت را **سطح-به-سطح** (breadth-first) جست‌وجو می‌کند: قبل از رفتن به عمق $d + 1$، همه گره‌های عمق d را بررسی می‌کند.

اثبات. D سه نوار دارد (طبق قضیه ۲.۱۷ این مجاز است):

- **نوار ۱** ورودی را نگه می‌دارد و هرگز تغییر نمی‌کند.
- **نوار ۲** یک کپی از نوار N را در یک شاخه از محاسبه نگه می‌دارد.
- **نوار ۳** مکان D در درخت محاسبه N را نگه می‌دارد.

آدرس‌دهی گره‌ها: فرض کنید هر پیکربندی N حداکثر b فرزند در درخت داشته باشد (b بیشترین تعداد انتخاب در تابع گذار N است). قرار دهید $\Gamma_b = \{1, 2, \ldots, b\}$. به هر گره درخت یک **آدرس** می‌دهیم که رشته‌ای روی Γ_b است: مثلاً 231 یعنی از ریشه انتخاب دوم، سپس انتخاب سوم و سپس انتخاب اول را دنبال کن. آدرس ریشه ε است. بعضی رشته‌ها ممکن است آدرس معتبری نباشند (اگر پیکربندی آن تعداد انتخاب نداشته باشد).

D این‌طور کار می‌کند:

۱. در ابتدا نوار ۱ ورودی w را دارد و نوارهای ۲ و ۳ خالی‌اند.

۲. نوار ۱ را روی نوار ۲ کپی کن و رشته روی نوار ۳ را ε قرار بده.

۳. با نوار ۲، N را روی ورودی w در یک شاخه شبیه‌سازی کن. قبل از هر گام N، نماد بعدی نوار ۳ را ببین تا بفهمی کدام انتخاب را دنبال کنی.

   (آ) اگر نمادهای نوار ۳ تمام شد یا انتخاب نامعتبر بود، به مرحله ۴ برو.

   (ب) اگر به پیکربندی رد رسیدی، به مرحله ۴ برو.

   (ج) اگر به پیکربندی پذیرش رسیدی، **بپذیر**.

۴. رشته روی نوار ۳ را با رشته بعدی در ترتیب رشته‌ای (فصل ۲) جایگزین کن و با رفتن به مرحله ۲ شاخه بعدی را شبیه‌سازی کن.

چون ترتیب رشته‌ای اول همه رشته‌های کوتاه‌تر را می‌آورد، D گره‌ها را سطح به سطح بررسی می‌کند. اگر N ورودی را بپذیرد، یک گره پذیرش در عمق متناهی d هست و D بالاخره به آدرس آن می‌رسد و می‌پذیرد. اگر N نپذیرد، هیچ شاخه‌ای به پذیرش نمی‌رسد و D هم نمی‌پذیرد (یا رد می‌کند یا تا ابد ادامه می‌دهد). پس $L(D) = L(N)$.

■

نتیجه ۵.۱۷

یک زبان تورینگ-تشخیص‌پذیر است اگر و تنها اگر یک ماشین تورینگ غیرقطعی آن را تشخیص دهد.

یک NTM را **تصمیم‌گیرنده** می‌گوییم اگر روی هر ورودی **همه** شاخه‌هایش متوقف شوند.

نتیجه ۶.۱۷

یک زبان تصمیم‌پذیر است اگر و تنها اگر یک ماشین تورینگ غیرقطعی آن را تصمیم بگیرد.

اثبات. جهت برگشت نیاز به یک تغییر کوچک در D دارد. اگر همه شاخه‌های N متوقف شوند، درخت محاسبه (که هر گره حداکثر b فرزند دارد) متناهی است. D را طوری تغییر دهید که اگر همه گره‌های یک عمق مشخص یا نامعتبر بودند یا رد شده بودند، **رد کند**. چون درخت متناهی است، این اتفاق بالاخره می‌افتد (مگر اینکه زودتر پذیرش پیدا شود). پس D همیشه متوقف می‌شود.

■

۴.۱۷ شمارنده‌ها

تعریف ۷.۱۷ (شمارنده)

شمارنده (Enumerator) یک ماشین تورینگ است که به یک **چاپگر** وصل است و می‌تواند از آن به عنوان دستگاه خروجی استفاده کند و رشته‌هایی را چاپ کند. زبانی که شمارنده E **شمارش می‌کند**، مجموعه همه رشته‌هایی است که E بالاخره چاپ می‌کند (به هر ترتیبی و احتمالاً با تکرار). شمارنده ورودی ندارد و ممکن است هرگز متوقف نشود.

نام «بازگشتی شمارش‌پذیر» از همین گونه ماشین تورینگ آمده است.

قضیه ۸.۱۷

یک زبان تورینگ-تشخیص‌پذیر است اگر و تنها اگر یک شمارنده آن را شمارش کند.

اثبات. **جهت ۱: از شمارنده به ماشین تشخیص‌دهنده.** فرض کنید شمارنده E زبان A را شمارش کند. ماشین M روی ورودی w:

۱. E را اجرا کن. هر بار که E رشته‌ای چاپ کرد، آن را با w مقایسه کن.

۲. اگر w در خروجی E ظاهر شد، **بپذیر**؛ وگرنه به اجرای E ادامه بده.

M دقیقاً رشته‌هایی را می‌پذیرد که E چاپ می‌کند، پس $L(M) = A$.

جهت ۲: از ماشین تشخیص‌دهنده به شمارنده. فرض کنید M زبان A را تشخیص دهد و $s_1, s_2, s_3, \ldots$ فهرست همه رشته‌های Σ^* (مثلاً به ترتیب رشته‌ای) باشد. شمارنده E:

۱. برای $i = 1, 2, 3, \ldots$ تکرار کن:

۲. M را روی هر یک از ورودی‌های $s_1, s_2, \ldots, s_i$ به اندازه i گام اجرا کن.

۳. هر رشته‌ای که پذیرفته شد را چاپ کن.

چرا M را مستقیماً روی s_1، بعد s_2 و ... اجرا نمی‌کنیم؟ چون ممکن است M روی s_1 حلقه بزند و هرگز به s_2 نرسیم. با محدود کردن تعداد گام‌ها و بزرگ کردن تدریجی آن، هیچ ورودی‌ای باعث گیر کردن نمی‌شود. اگر M رشته s را بپذیرد، $s = s_j$ برای یک j و M آن را در K گام می‌پذیرد. پس E در دور $i = \max(j, K)$ (و همه دورهای بعد) s را چاپ می‌کند. همچنین E هیچ رشته‌ای را که M نمی‌پذیرد چاپ نمی‌کند. پس E دقیقاً A را شمارش می‌کند.

■

۵.۱۷ مدل‌های دیگر

مدل‌های دیگر زیادی برای محاسبه همه‌منظوره پیشنهاد شده‌اند. بعضی خیلی شبیه ماشین تورینگ‌اند و بعضی کاملاً متفاوت. اما همه یک ویژگی اصلی مشترک دارند: **دسترسی نامحدود به حافظه نامحدود**. وقتی این مدل‌ها شرایط معقولی داشته باشند (مثلاً در یک گام فقط مقدار متناهی کار انجام دهند)، همه با هم هم‌ارزند.

این وضعیت شبیه زبان‌های برنامه‌نویسی است: پایتون، جاوا و ... ظاهر متفاوتی دارند، اما هر الگوریتمی را که بتوان در یکی نوشت، در بقیه هم می‌توان نوشت. همه آن‌ها **یک دسته الگوریتم** را توصیف می‌کنند.

جمع‌بندی فصل

- برای اثبات هم‌ارزی دو مدل، نشان دهید هر کدام دیگری را شبیه‌سازی می‌کند.
- هم‌ارزی ماشین چندنواره و تک‌نواره: نوارها با $\#$ جدا و هدها با نقطه مشخص می‌شوند.
- هم‌ارزی NTM و DTM: جست‌وجوی سطح-به-سطح درخت محاسبه با سه نوار و آدرس‌های روی Γ_b.
- هم‌ارزی شمارنده و ماشین تشخیص‌دهنده: اجرای ماشین روی $s_1, \ldots, s_i$ به اندازه i گام.
- همه مدل‌های معقول محاسبه همه‌منظوره هم‌ارزند.

۱۸ فصل

تز چرچ–تورینگ و زبان‌های تصمیم‌پذیر

۱.۱۸ الگوریتم چیست؟

به زبان ساده، **الگوریتم** مجموعه‌ای از دستورهای ساده برای انجام یک کار است (به آن روال یا دستور کار هم می‌گویند). الگوریتم‌ها قدمت زیادی دارند؛ مثلاً الگوریتم پیدا کردن اعداد اول یا الگوریتم بزرگ‌ترین مقسوم‌علیه مشترک (اقلیدس). اما با وجود این تاریخ طولانی، خود مفهوم الگوریتم تا قرن بیستم به‌طور دقیق تعریف نشده بود. چرا به تعریف دقیق نیاز داریم؟ داستان زیر جواب را نشان می‌دهد.

۱.۱.۱۸ مسئله دهم هیلبرت

تعریف ۱.۱۸ (چندجمله‌ای و ریشه صحیح)

یک **چندجمله‌ای** مجموع چند **جمله** است و هر جمله حاصل‌ضرب چند متغیر در یک عدد ثابت (**ضریب**) است. مثلاً $6 \cdot x \cdot x \cdot x \cdot y \cdot z \cdot z = 6x^3yz^2$ جمله‌ای با ضریب ۶ است و

$$6x^3yz^2 + 3xy^2 - x^3 - 10$$

یک چندجمله‌ای روی متغیرهای x، y و z. یک **ریشه** مقداردهی‌ای به متغیرهاست که چندجمله‌ای را صفر کند (مثلاً $x = 5$، $y = 3$، $z = 0$ برای چندجمله‌ای بالا). ریشه‌ای که همه مقدارهایش عدد صحیح باشند **ریشه صحیح** است.

بعضی چندجمله‌ای‌ها ریشه صحیح دارند و بعضی ندارند. دیوید هیلبرت در کنگره بین‌المللی ریاضی‌دانان در پاریس (سال ۱۹۰۰) ۲۳ مسئله را به عنوان چالش قرن بیستم مطرح کرد. مسئله دهم این بود:

> **مسئله دهم هیلبرت.** روندی پیدا کنید که با تعداد متناهی عمل (یعنی یک **الگوریتم**) مشخص کند آیا یک چندجمله‌ای ریشه صحیح دارد یا نه.

هیلبرت ظاهراً فرض می‌کرد چنین الگوریتمی وجود دارد و فقط باید پیدایش کرد. امروز می‌دانیم **هیچ الگوریتمی برای این کار وجود ندارد**. اما اثبات «وجود نداشتن الگوریتم» با یک تصور شهودی از الگوریتم ممکن نیست؛ برای آن به تعریف دقیق نیاز داریم.

۲.۱.۱۸ تز چرچ–تورینگ

در سال ۱۹۳۶ دو تعریف دقیق برای الگوریتم ارائه شد: آلونزو چرچ با **حساب لامبدا** و آلن تورینگ با **ماشین تورینگ**. این دو تعریف هم‌ارز از آب درآمدند. رابطه میان مفهوم شهودی و تعریف دقیق را **تز چرچ–تورینگ** (Church–Turing Thesis) می‌نامیم:

تز چرچ–تورینگ. مفهوم شهودی الگوریتم = الگوریتم‌های ماشین تورینگ

این یک «تز» است، نه قضیه؛ هرگز اثبات نشده (و چون یک طرفش مفهومی شهودی است، اثبات ریاضی هم ندارد). اما شواهد زیادی به نفع آن هست: هر مدل محاسباتی واقع‌بینانه‌ای که تا امروز پیشنهاد شده، با ماشین تورینگ هم‌ارز بوده است. اگر دستگاهی بتواند به سؤال‌هایی فراتر از توان ماشین تورینگ جواب دهد، به آن **پیشگو** (oracle) می‌گوییم.

۳.۱.۱۸ مسئله هیلبرت به زبان این درس

مسئله دهم هیلبرت یعنی: آیا مجموعه زیر تصمیم‌پذیر است؟

$$D = \{p \mid p \text{ یک چندجمله‌ای با ریشه صحیح است}\}.$$

جواب منفی است: D تصمیم‌پذیر نیست، اما تورینگ-تشخیص‌پذیر است.

مثال ۲.۱۸ (حالت یک‌متغیره). $D_1 = \{p \mid p \text{ چندجمله‌ای روی متغیر } x \text{ با ریشه صحیح است}\}$ تشخیص‌پذیر است:

> M_1 = «روی ورودی $\langle p \rangle$ که p چندجمله‌ای روی x است»
>
> ۱. p را به ترتیب برای $x = 0, 1, -1, 2, -2, 3, -3, \ldots$ محاسبه کن. اگر در هر نقطه مقدار p صفر شد، **بپذیر**.

اگر p ریشه صحیح داشته باشد، M_1 بالاخره آن را پیدا می‌کند و می‌پذیرد. اگر نداشته باشد، M_1 تا ابد ادامه می‌دهد. پس M_1 تشخیص‌دهنده است، نه تصمیم‌گیرنده.

برای حالت چندمتغیره هم ماشینی مشابه می‌سازیم که همه مقداردهی‌های ممکن متغیرها را (به ترتیبی که همه را پوشش دهد) امتحان کند؛ پس D هم تشخیص‌پذیر است.

آیا M_1 را می‌توان به تصمیم‌گیرنده تبدیل کرد؟ **بله**. ریشه‌های یک چندجمله‌ای تک‌متغیره همیشه بین

$$\pm k \frac{c_{\max}}{c_1}$$

قرار دارند که k تعداد جمله‌ها، $c_{\max}$ ضریب با بیشترین قدرمطلق و c_1 ضریب جمله با بیشترین درجه است. (برای x بزرگ‌تر از این مقدار، جمله اول از مجموع بقیه بزرگ‌تر است و چندجمله‌ای نمی‌تواند صفر شود.) پس کافی است فقط مقدارهای این بازه را امتحان کنیم و اگر ریشه‌ای پیدا نشد، رد کنیم. پس D_1 تصمیم‌پذیر است. اما برای حالت چندمتغیره، **قضیه ماتیاسویچ** نشان می‌دهد چنین کرانی نمی‌توان پیدا کرد و D تصمیم‌پذیر نیست.

۲.۱۸ سطوح توصیف ماشین تورینگ

ماشین تورینگ را در سه سطح می‌توان توصیف کرد:

۱. **توصیف صوری:** همه هفت جزء ماشین (حالت‌ها، تابع گذار و ...) کامل نوشته می‌شوند.

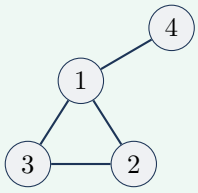
۲. **توصیف پیاده‌سازی:** با کلمات توضیح می‌دهیم هد چطور حرکت می‌کند و داده‌ها چطور روی نوار ذخیره می‌شوند، بدون جزئیات حالت‌ها.

۳. **توصیف سطح بالا:** فقط الگوریتم را توضیح می‌دهیم و جزئیات پیاده‌سازی را نادیده می‌گیریم.

از این به بعد بیشتر از توصیف سطح بالا استفاده می‌کنیم.

ورودی همیشه یک رشته است. اگر بخواهیم شیئی غیر از رشته (مثل چندجمله‌ای، گراف، گرامر، ماشین یا ترکیبی از این‌ها) به ماشین بدهیم، آن را به شکل یک رشته **کدگذاری** می‌کنیم. کدگذاری شیء O را با $\langle O \rangle$ و کدگذاری چند شیء $O_1, \ldots, O_k$ را با $\langle O_1, O_2, \ldots, O_k \rangle$ نشان می‌دهیم. ماشین تورینگ کدگذاری را رمزگشایی می‌کند و اگر ورودی کدگذاری معتبری نبود، رد می‌کند (این بررسی را معمولاً به‌طور ضمنی فرض می‌کنیم).

مثال ۳.۱۸ (گراف‌های همبند). $A = \{\langle G \rangle \mid G$ {یک گراف بی‌جهت همبند است}$\}$. یک گراف و کدگذاری آن (فهرست رأس‌ها و سپس فهرست یال‌ها):



$\langle G \rangle =$ `(1,2,3,4)((1,2),(2,3),(3,1),(1,4))`

M = «روی ورودی $\langle G \rangle$، کدگذاری گراف G»

۱. اولین رأس G را انتخاب و علامت‌گذاری کن.

۲. مرحله زیر را تا وقتی رأس جدیدی علامت نخورد تکرار کن:

۳. هر رأس G را که با یک یال به یک رأس علامت‌خورده وصل است، علامت بزن.

۴. همه رأس‌ها را بررسی کن. اگر همه علامت خورده بودند **بپذیر**؛ وگرنه **رد کن**.

۳.۱۸ زبان‌های تصمیم‌پذیر درباره زبان‌های منظم

در این بخش و بخش بعد مسئله‌هایی را می‌بینیم که **الگوریتم دارند**. این مثال‌ها کمک می‌کنند در فصل‌های بعد مسئله‌های بدون الگوریتم را بهتر درک کنیم. بعضی از آن‌ها کاربرد عملی هم دارند؛ مثلاً اینکه آیا یک گرامر مستقل از متن رشته‌ای را تولید می‌کند، به کامپایل برنامه‌ها مربوط است.

برای بیان مسئله‌های محاسباتی از **زبان** استفاده می‌کنیم: مسئله «آیا DFA B رشته w را می‌پذیرد؟» معادل این است که بپرسیم آیا $\langle B, w \rangle$ عضو زبان زیر است:

$$A_{\text{DFA}} = \{\langle B, w \rangle \mid B \text{ {یک DFA است که } } w \text{ را می‌پذیرد}\}.$$

تصمیم‌پذیر بودن این زبان یعنی مسئله محاسباتی تصمیم‌پذیر است.

قضیه ۴.۱۸

A_{DFA} تصمیم‌پذیر است.

اثبات.

> **M = «روی ورودی $\langle B, w\rangle$ که B یک DFA و w یک رشته است»**
>
> ۱. B را روی w شبیه‌سازی کن.
>
> ۲. اگر شبیه‌سازی در یک حالت پذیرش تمام شد، **بپذیر**؛ وگرنه **رد کن**.

جزئیات پیاده‌سازی: $\langle B\rangle$ پنج جزء DFA را فهرست می‌کند. M حالت فعلی B و مکان فعلی در w را روی نوار نگه می‌دارد. از q_0 شروع می‌کند، هر بار یک نماد w را می‌خواند و با نگاه کردن به تابع گذار حالت را عوض می‌کند. وقتی w تمام شد، بررسی می‌کند حالت فعلی پذیرش است یا نه. شبیه‌سازی دقیقاً $|w|$ گام دارد، پس M همیشه متوقف می‌شود.

■

قضیه ۵.۱۸

$A_{\text{NFA}} = \{\langle B, w\rangle \mid B$ یک NFA است که w را می‌پذیرد$\}$ تصمیم‌پذیر است.

اثبات. ایده جدید: از یک الگوریتم قبلی به عنوان زیرروال استفاده می‌کنیم.

> **N = «روی ورودی $\langle B, w\rangle$ که B یک NFA است»**
>
> ۱. B را با روش ساخت زیرمجموعه‌ای (قضیه ۳.۴) به DFA هم‌ارز C تبدیل کن.
>
> ۲. ماشین M قضیه قبل را روی $\langle C, w\rangle$ اجرا کن.
>
> ۳. اگر M پذیرفت **بپذیر**؛ وگرنه **رد کن**.

ساخت زیرمجموعه‌ای یک الگوریتم با تعداد گام متناهی است و M تصمیم‌گیرنده است، پس N هم همیشه متوقف می‌شود.

■

قضیه ۶.۱۸

$A_{\text{REX}} = \{\langle R, w\rangle \mid R$ یک عبارت منظم است که w را تولید می‌کند$\}$ تصمیم‌پذیر است.

اثبات.

> **P = «روی ورودی $\langle R, w\rangle$»**
>
> ۱. عبارت منظم R را با روش لم ۱.۲.۶ به NFA هم‌ارز A تبدیل کن.
>
> ۲. N را روی $\langle A, w\rangle$ اجرا کن و همان جواب را بده.

■

قضیه ۷.۱۸

$E_{\text{DFA}} = \{\langle A \rangle \mid A \text{ یک DFA است و } L(A) = \emptyset\}$ تصمیم‌پذیر است.

اثبات. زبان DFA تهی نیست اگر و تنها اگر از حالت شروع به یک حالت پذیرش مسیری باشد.

T = «روی ورودی $\langle A \rangle$ که A یک DFA است»

۱. حالت شروع A را علامت بزن.

۲. تا وقتی حالت جدیدی علامت نخورد تکرار کن:

۳. هر حالتی را که از یک حالت علامت‌خورده پیکانی به آن وارد می‌شود، علامت بزن.

۴. اگر هیچ حالت پذیرشی علامت نخورده، **بپذیر**؛ وگرنه **رد کن**.

حلقه حداکثر $|Q|$ بار تکرار می‌شود، پس T همیشه متوقف می‌شود.

■

قضیه ۸.۱۸

$EQ_{\text{DFA}} = \{\langle A, B \rangle \mid A, B \text{ دو DFA هستند و } L(A) = L(B)\}$ تصمیم‌پذیر است.

اثبات. DFA C را طوری می‌سازیم که **تفاضل متقارن** $L(A)$ و $L(B)$ را بپذیرد؛ یعنی رشته‌هایی که دقیقاً در یکی از دو زبان هستند:

$$L(C) = \big(L(A) \cap \overline{L(B)}\big) \cup \big(\overline{L(A)} \cap L(B)\big).$$

$L(C) = \emptyset$ اگر و تنها اگر $L(A) = L(B)$. DFA C را می‌توان با ساخت‌های بستاری تحت متمم، اشتراک و اجتماع (فصل ۳) ساخت.

F = «روی ورودی $\langle A, B \rangle$ که A و B دو DFA هستند»

۱. DFA C را مانند بالا بساز.

۲. ماشین T قضیه قبل را روی $\langle C \rangle$ اجرا کن.

۳. اگر T پذیرفت **بپذیر**؛ وگرنه **رد کن**.

■

۴.۱۸ زبان‌های تصمیم‌پذیر درباره زبان‌های مستقل از متن

قضیه ۹.۱۸

$A_{\text{CFG}} = \{\langle G, w \rangle \mid G \text{ یک CFG است که } w \text{ را تولید می‌کند}\}$ تصمیم‌پذیر است.

ایده ۱ (کار نمی‌کند): همه اشتقاق‌ها را یکی‌یکی امتحان کنیم. اگر G رشته w را تولید کند بالاخره پیدا می‌شود، اما اگر تولید نکند ماشین تا ابد ادامه می‌دهد. این یک تشخیص‌دهنده است، نه تصمیم‌گیرنده.

ایده ۲: اگر G در فرم نرمال چامسکی باشد، **هر** اشتقاق رشته‌ای به طول $n \geq 1$ دقیقاً $2n-1$ گام دارد. دلیل: هر گام با قاعده $A \to BC$ تعداد متغیرها را یکی زیاد می‌کند و هر گام با قاعده $A \to a$ یک متغیر را به پایانه تبدیل می‌کند. از یک متغیر شروع می‌کنیم و به n پایانه می‌رسیم، پس دقیقاً $n-1$ گام از نوع اول و n گام از نوع دوم داریم. پس کافی است فقط اشتقاق‌هایی با $2n-1$ گام را بررسی کنیم، که تعدادشان متناهی است.

اثبات.

> **S = «روی ورودی $\langle G, w \rangle$»**
>
> ۱. G را به یک گرامر هم‌ارز در فرم نرمال چامسکی تبدیل کن.
>
> ۲. همه اشتقاق‌های $2n-1$ گامی را فهرست کن که $n = |w|$. (اگر $n = 0$، همه اشتقاق‌های یک‌گامی را فهرست کن.)
>
> ۳. اگر یکی از این اشتقاق‌ها w را تولید کرد، **بپذیر**؛ وگرنه **رد کن**.

تبدیل به CNF الگوریتم دارد و تعداد اشتقاق‌های با طول ثابت متناهی است، پس S همیشه متوقف می‌شود. (الگوریتم CYK فصل ۱۱ همین کار را بسیار سریع‌تر انجام می‌دهد.)

■

قضیه ۱۰.۱۸

$E_{\text{CFG}} = \{\langle G \rangle \mid G \text{ یک CFG است و } L(G) = \emptyset\}$ تصمیم‌پذیر است.

ایده ۱ (کار نمی‌کند): همه wها را یکی‌یکی با S قضیه قبل امتحان کنیم. تعداد wها نامتناهی است و اگر زبان تهی باشد، ماشین هرگز متوقف نمی‌شود.

اثبات. به جای آن، برای هر متغیر بررسی می‌کنیم که آیا می‌تواند رشته‌ای از پایانه‌ها تولید کند.

> **R = «روی ورودی $\langle G \rangle$ که G یک CFG است»**
>
> ۱. همه پایانه‌های G را علامت بزن.
>
> ۲. تا وقتی متغیر جدیدی علامت نخورد تکرار کن:
>
> ۳. هر متغیر A را علامت بزن اگر G قاعده‌ای مثل $A \to U_1 U_2 \cdots U_k$ داشته باشد که همه $U_1, \ldots, U_k$ قبلاً علامت خورده باشند.
>
> ۴. اگر متغیر شروع علامت نخورده، **بپذیر**؛ وگرنه **رد کن**.

یک متغیر علامت می‌خورد اگر و تنها اگر رشته‌ای از پایانه‌ها تولید کند (با استقرا روی تعداد دورها و طول اشتقاق). حلقه حداکثر $|V|$ بار تکرار می‌شود.

■

قضیه ۱۱.۱۸

$EQ_{\text{CFG}} = \{\langle G, H \rangle \mid G, H \text{ دو CFG هستند و } L(G) = L(H)\}$ تصمیم‌پذیر **نیست**.

روشی که برای EQ_{DFA} به کار رفت اینجا جواب نمی‌دهد، چون زبان‌های مستقل از متن تحت متمم و اشتراک بسته نیستند. اثبات تصمیم‌ناپذیری را در فصل ۲۰ می‌بینیم.

قضیه ۱۲.۱۸

هر زبان مستقل از متن تصمیم‌پذیر است.

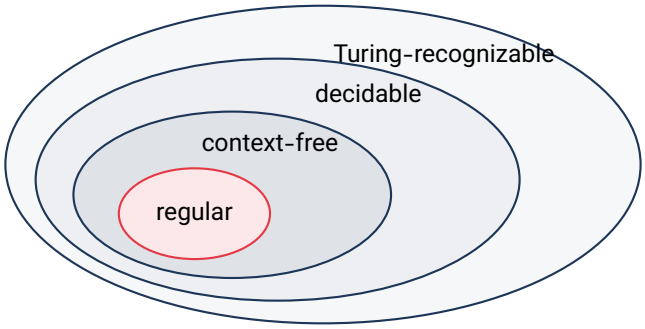
ایده ۱ (کار نمی‌کند): PDA زبان را با یک ماشین تورینگ شبیه‌سازی کنیم. ماشین تورینگ می‌تواند پشته را روی نوارش شبیه‌سازی کند، اما بعضی شاخه‌های PDA ممکن است تا ابد روی پشته بنویسند و بخوانند بدون اینکه متوقف شوند. پس ماشین تورینگ حاصل تصمیم‌گیرنده نخواهد بود.

اثبات. فرض کنید A مستقل از متن و G یک گرامر برای آن باشد. یک نسخه از G را درون ماشین M_G قرار می‌دهیم:

> M_G = «روی ورودی w»
>
> ۱. ماشین S قضیه ۴.۱۸ را روی $\langle G, w\rangle$ اجرا کن.
>
> ۲. اگر S پذیرفت **بپذیر**؛ وگرنه **رد کن**.

■

رابطه دسته‌های زبان‌هایی که تا اینجا دیدیم:



جمع‌بندی فصل

- تز چرچ–تورینگ: مفهوم شهودی الگوریتم همان ماشین تورینگ است.
- مسئله دهم هیلبرت (ریشه صحیح چندجمله‌ای) تشخیص‌پذیر است اما تصمیم‌پذیر نیست.
- ورودی ماشین همیشه رشته است؛ اشیاء را با $\langle\cdot\rangle$ کدگذاری می‌کنیم.
- تصمیم‌پذیر: A_{DFA}، A_{NFA}، A_{REX}، E_{DFA}، EQ_{DFA}، A_{CFG}، E_{CFG} و هر زبان مستقل از متن. تصمیم‌ناپذیر: EQ_{CFG}.

۱۹ فصل

تصمیم‌ناپذیری

آیا مسئله‌های واقعی و ملموسی وجود دارند که هیچ کامپیوتری نتواند حلشان کند؟ بله. مثلاً **درستی‌سنجی برنامه**: یک برنامه و یک توصیف دقیق از کاری که باید انجام دهد داریم و می‌خواهیم مطمئن شویم برنامه دقیقاً همان کار را می‌کند. با اینکه هم برنامه و هم توصیف کاملاً دقیق‌اند، در حالت کلی هیچ الگوریتمی نمی‌تواند این را بررسی کند. در این فصل اولین مسئله تصمیم‌ناپذیر را پیدا می‌کنیم و روش اثبات تصمیم‌ناپذیری را یاد می‌گیریم.

۱.۱۹ مسئله پذیرش برای ماشین تورینگ

$A_{\text{TM}} = \{\langle M, w\rangle \mid M$ یک ماشین تورینگ است که w را می‌پذیرد$\}$.

قضیه ۱.۱۹

A_{TM} تصمیم‌ناپذیر است.

قبل از اثبات، ببینیم A_{TM} **تشخیص‌پذیر** است:

U = «روی ورودی $\langle M, w\rangle$ که M ماشین تورینگ و w رشته است»

۱. M را روی ورودی w شبیه‌سازی کن.

۲. اگر M به حالت پذیرش رسید **بپذیر**؛ اگر به حالت رد رسید **رد کن**.

اگر M روی w حلقه بزند، U هم روی $\langle M, w\rangle$ حلقه می‌زند. پس U تصمیم‌گیرنده نیست. U نمی‌تواند تشخیص دهد که M متوقف نخواهد شد؛ به‌زودی می‌بینیم این کار **غیرممکن** است.

U نمونه‌ای از **ماشین تورینگ جهانی** (Universal Turing Machine) است (تورینگ، ۱۹۳۶): ماشینی که می‌تواند هر ماشین تورینگ دیگری را از روی توصیفش شبیه‌سازی کند. این همان ایده کامپیوترهای «برنامه‌پذیر» امروزی است که برنامه را به عنوان داده می‌گیرند.

۲.۱۹ روش قطری‌سازی

برای اثبات قضیه ۱.۱۹ از روش **قطری‌سازی** استفاده می‌کنیم که کانتور در سال ۱۸۷۳ ابداع کرد. کانتور می‌خواست اندازه مجموعه‌های نامتناهی را مقایسه کند. نمی‌توان آن‌ها را شمرد، اما می‌توان سعی کرد عضوهایشان را با هم **جفت** کرد (فصل ۱).

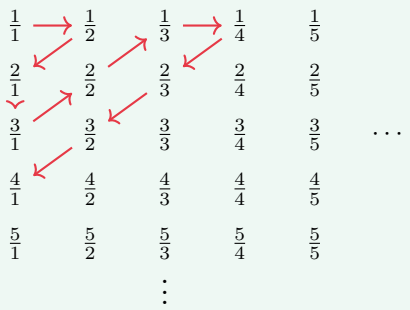
یادآوری: تابع $f : A \to B$ **یک‌به‌یک** است اگر $a \neq b \to f(a) \neq f(b)$، **پوشا** است اگر $\forall b \in B\ \exists a \in A :\ f(a) = b$، و **تناظر** (دوسویی) است اگر هر دو باشد. A و B **هم‌اندازه** هستند اگر تناظری بین آن‌ها باشد.

مثال ۲.۱۹. مجموعه اعداد طبیعی $\mathbb{N}$ و مجموعه اعداد طبیعی زوج $\mathcal{E}$ هم‌اندازه‌اند، چون $f(n) = 2n$ یک تناظر بین آن‌هاست. این عجیب به نظر می‌رسد، چون $\mathcal{E}$ زیرمجموعه سره $\mathbb{N}$ است! اما برای مجموعه‌های نامتناهی، «جزء» می‌تواند با «کل» هم‌اندازه باشد.

تعریف ۳.۱۹ (شمارا)

مجموعه A **شماراست** اگر متناهی باشد یا با $\mathbb{N}$ هم‌اندازه باشد.

مثال ۴.۱۹ (اعداد گویای مثبت). مجموعه $\mathcal{Q} = \{\frac{m}{n} \mid m, n \in \mathbb{N}\}$ شماراست. همه کسرها را در یک جدول نامتناهی می‌نویسیم: سطر i کسرهایی با صورت i و ستون j کسرهایی با مخرج j. سپس جدول را روی **قطرهای فرعی** پیمایش می‌کنیم و کسرهای تکراری (مثل $\frac{2}{2} = \frac{1}{1}$) را رد می‌کنیم:



هر کسر بالاخره روی یکی از قطرها قرار می‌گیرد و هر قطر متناهی است، پس همه کسرها در فهرست می‌آیند:

$$\frac{1}{1}, \frac{1}{2}, \frac{2}{1}, \frac{3}{1}, \frac{1}{3}, \frac{1}{4}, \frac{2}{3}, \frac{3}{2}, \frac{4}{1}, \dots$$

پس آیا همه مجموعه‌های نامتناهی هم‌اندازه‌اند؟ نه. بعضی مجموعه‌ها آن‌قدر بزرگ‌اند که با $\mathbb{N}$ تناظری ندارند. به آن‌ها **ناشمارا** می‌گوییم.

قضیه ۵.۱۹

مجموعه اعداد حقیقی $\mathcal{R}$ ناشماراست.

اثبات. با برهان خلف فرض کنید تناظری مثل $f : \mathbb{N} \to \mathcal{R}$ وجود دارد. پس می‌توان همه اعداد حقیقی را در یک فهرست نوشت: $f(1), f(2), f(3), \dots$. عددی مثل $x \in (0, 1)$ می‌سازیم که در این فهرست **نیست:** رقم iام بعد از ممیز x را طوری انتخاب می‌کنیم که با رقم iام بعد از ممیز $f(i)$ فرق داشته باشد. مثلاً:

n	$f(n)$
1	$3.\underline{1}4159\dots$
2	$55.5\underline{5}555\dots$
3	$0.12\underline{3}45\dots$
4	$0.500\underline{0}0\dots$
$\vdots$	$\vdots$

$$x = 0.4641\dots$$

(رقم اول x با ۱، رقم دوم با ۵، رقم سوم با ۳ و رقم چهارم با ۰ فرق دارد.) برای جلوگیری از مشکل نمایش‌های دوگانه

مثل $0.1999\ldots = 0.2000\ldots$، رقم‌های ۰ و ۹ را هرگز برای x انتخاب نمی‌کنیم. حالا x با هر $f(n)$ دست‌کم در رقم nام فرق دارد، پس $x \neq f(n)$ برای هر n. یعنی f پوشا نیست؛ تناقض.

■

۳.۱۹ زبان‌هایی که تشخیص‌پذیر نیستند

نتیجه قضیه ۲.۱۹ را این‌طور به کار می‌بریم: نشان می‌دهیم ماشین‌های تورینگ شمارا هستند، اما زبان‌ها ناشمارا هستند. چون هر ماشین فقط یک زبان را تشخیص می‌دهد، زبان‌هایی می‌مانند که هیچ ماشینی تشخیصشان نمی‌دهد.

نتیجه ۶.۱۹

زبان‌هایی وجود دارند که تورینگ-تشخیص‌پذیر نیستند.

اثبات. **مجموعه همه رشته‌ها شماراست.** برای هر الفبای Σ، رشته‌های Σ^* را به ترتیب رشته‌ای فهرست می‌کنیم: اول رشته به طول صفر، بعد رشته‌های به طول یک، بعد طول دو و ... هر رشته جای مشخصی در این فهرست دارد.
مجموعه همه ماشین‌های تورینگ شماراست. هر ماشین تورینگ M یک کدگذاری $\langle M \rangle$ دارد که یک رشته است. اگر از فهرست همه رشته‌ها، رشته‌هایی را که کدگذاری معتبر یک ماشین نیستند حذف کنیم، فهرستی از همه ماشین‌ها به دست می‌آید.
مجموعه همه زبان‌ها ناشماراست. ابتدا نشان می‌دهیم مجموعه $\mathcal{B}$ همه **دنباله‌های دودویی نامتناهی** ناشماراست. استدلال دقیقاً مثل قضیه ۲.۱۹ است: اگر $\mathcal{B}$ شمارا بود، دنباله‌ها را فهرست می‌کردیم و دنباله جدیدی می‌ساختیم که بیت iامش با بیت iام دنباله iام فرق دارد:

$$\begin{array}{ll} \underline{0}00000\ldots & \\ 0\underline{1}0100\ldots & \\ 10\underline{0}100\ldots & \Longrightarrow \quad 1011\ldots \\ 111\underline{0}00\ldots & \\ \vdots & \end{array}$$

این دنباله در فهرست نیست؛ تناقض.
حالا تناظری بین مجموعه همه زبان‌ها $\mathcal{L}$ و $\mathcal{B}$ می‌سازیم. اگر $\Sigma^* = \{s_1, s_2, s_3, \ldots\}$ (به ترتیب رشته‌ای)، هر زبان A یک **دنباله مشخصه** χ_A دارد که بیت iامش ۱ است اگر $s_i \in A$ و ۰ است اگر $s_i \notin A$:

$$\begin{array}{rccccccccccc} \Sigma^* = \{ & \varepsilon, & 0, & 1, & 00, & 01, & 10, & 11, & 000, & 001, & \cdots \} \\ A = \{ & & 0, & & 00, & 01, & & & 000, & 001, & \cdots \} \\ \chi_A = & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & \cdots \end{array}$$

تابع $f : \mathcal{L} \to \mathcal{B}$ با $f(A) = \chi_A$ یک‌به‌یک و پوشاست (هر زبان یعنی یک انتخاب دودویی برای هر رشته). چون $\mathcal{B}$ ناشماراست، $\mathcal{L}$ هم ناشماراست.
پس تعداد زبان‌ها بیشتر از تعداد ماشین‌هاست و زبانی وجود دارد که هیچ ماشین تورینگی آن را تشخیص نمی‌دهد.

■

۴.۱۹ اثبات تصمیم‌ناپذیری مسئله پذیرش

اثبات (قضیه ۱.۱۹). با برهان خلف فرض کنید H یک تصمیم‌گیرنده برای A_{TM} باشد:

$$H(\langle M, w\rangle) = \begin{cases} \text{accept} & \text{اگر } M \text{ رشته } w \text{ را بپذیرد} \\ \text{reject} & \text{اگر } M \text{ رشته } w \text{ را نپذیرد.} \end{cases}$$

با استفاده از H ماشین جدید D را می‌سازیم. D از H می‌پرسد «آیا M وقتی کدگذاری خودش را به عنوان ورودی بگیرد، آن را می‌پذیرد؟» و دقیقاً **برعکس** عمل می‌کند:

> **D = «روی ورودی $\langle M\rangle$ که M یک ماشین تورینگ است»**
>
> ۱. H را روی ورودی $\langle M, \langle M\rangle\rangle$ اجرا کن.
>
> ۲. برعکس خروجی H عمل کن: اگر H پذیرفت **رد کن** و اگر H رد کرد **بپذیر**.

(اجرای یک برنامه روی کد خودش عجیب نیست؛ مثلاً یک کامپایلر را می‌توان با خودش کامپایل کرد.) پس

$$D(\langle M\rangle) = \begin{cases} \text{accept} & \text{اگر } M \text{ رشته } \langle M\rangle \text{ را نپذیرد} \\ \text{reject} & \text{اگر } M \text{ رشته } \langle M\rangle \text{ را بپذیرد.} \end{cases}$$

حالا D را روی کدگذاری خودش، $\langle D\rangle$، اجرا کنید:

$$D(\langle D\rangle) = \begin{cases} \text{accept} & \text{اگر } D \text{ رشته } \langle D\rangle \text{ را نپذیرد} \\ \text{reject} & \text{اگر } D \text{ رشته } \langle D\rangle \text{ را بپذیرد.} \end{cases}$$

هر کاری D انجام دهد، مجبور است برعکس آن را انجام دهد؛ تناقض. پس نه D و نه H نمی‌توانند وجود داشته باشند.

■

خلاصه اثبات:

- H رشته $\langle M, w\rangle$ را دقیقاً وقتی می‌پذیرد که M رشته w را بپذیرد.
- D رشته $\langle M\rangle$ را دقیقاً وقتی رد می‌کند که M رشته $\langle M\rangle$ را بپذیرد.
- D رشته $\langle D\rangle$ را دقیقاً وقتی رد می‌کند که D رشته $\langle D\rangle$ را بپذیرد. تناقض!

قطری‌سازی کجاست؟. جدولی بکشید که سطرهایش ماشین‌های $M_1, M_2, \ldots$ و ستون‌هایش کدگذاری‌های $\langle M_1\rangle, \langle M_2\rangle, \ldots$ هستند. خانه (i, j) خروجی H روی $\langle M_i, \langle M_j\rangle\rangle$ است. D هم یک ماشین است، پس سطری در این جدول دارد؛ اما طبق تعریف، خروجی D روی $\langle M_i\rangle$ برعکس **قطر** جدول است. وقتی به خانه $(D, \langle D\rangle)$ می‌رسیم، این خانه باید برعکس خودش باشد:

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	$\cdots$	$\langle D \rangle$
M_1	acc	rej	acc	rej		acc
M_2	acc	acc	acc	acc		acc
M_3	rej	rej	rej	rej		rej
M_4	acc	acc	rej	rej		acc
$\vdots$					$\ddots$	
D	rej	rej	acc	acc		?

۵.۱۹ یک زبان تشخیص‌ناپذیر

A_{TM} تشخیص‌پذیر است ولی تصمیم‌پذیر نیست. حالا زبانی مشخص پیدا می‌کنیم که حتی تشخیص‌پذیر هم نیست.
یادآوری: متمم یک زبان، همه رشته‌هایی است که در آن زبان نیستند.

تعریف ۷.۱۹ (هم-تشخیص‌پذیر)

زبانی **هم-تورینگ-تشخیص‌پذیر** (co-Turing-recognizable) است اگر متمم یک زبان تورینگ-تشخیص‌پذیر باشد.

قضیه ۸.۱۹

یک زبان تصمیم‌پذیر است اگر و تنها اگر هم تورینگ-تشخیص‌پذیر باشد و هم هم-تورینگ-تشخیص‌پذیر.

اثبات. **جهت رفت:** اگر A تصمیم‌پذیر باشد، تشخیص‌پذیر هم هست. همچنین $\overline{A}$ هم تصمیم‌پذیر است (کافی است خروجی تصمیم‌گیرنده A را برعکس کنیم) و در نتیجه تشخیص‌پذیر است. پس A هم-تشخیص‌پذیر است.
جهت برگشت: فرض کنید A و $\overline{A}$ هر دو تشخیص‌پذیر باشند؛ M_1 تشخیص‌دهنده A و M_2 تشخیص‌دهنده $\overline{A}$ باشد. تصمیم‌گیرنده M برای A:

> M = «روی ورودی w»
>
> ۱. M_1 و M_2 را روی w **به‌موازات** اجرا کن.
>
> ۲. اگر M_1 پذیرفت **بپذیر**؛ اگر M_2 پذیرفت **رد کن**.

«به‌موازات» یعنی M دو نوار دارد و یکی در میان یک گام از M_1 و یک گام از M_2 را اجرا می‌کند تا یکی از آن‌ها بپذیرد. (اجرای M_1 تا پایان و بعد M_2 کار نمی‌کند، چون ممکن است M_1 حلقه بزند.)
هر رشته w یا در A است یا در $\overline{A}$. پس دقیقاً یکی از M_1 و M_2 بالاخره w را می‌پذیرد و M همیشه متوقف می‌شود. اگر $w \in A$، M_1 می‌پذیرد و M هم می‌پذیرد؛ اگر $w \notin A$، M_2 می‌پذیرد و M رد می‌کند. پس M تصمیم‌گیرنده A است.

■

نتیجه ۹.۱۹

$\overline{A_{\mathsf{TM}}}$ تورینگ-تشخیص‌پذیر نیست.

اثبات. می‌دانیم A_{TM} تشخیص‌پذیر است. اگر $\overline{A_{\mathrm{TM}}}$ هم تشخیص‌پذیر بود، طبق قضیه ۵.۱۹ A_{TM} تصمیم‌پذیر می‌شد. اما طبق قضیه ۱.۱۹ این‌طور نیست. پس $\overline{A_{\mathrm{TM}}}$ تشخیص‌پذیر نیست.

■

جمع‌بندی فصل

- A_{TM} تشخیص‌پذیر است (با ماشین جهانی U) اما تصمیم‌پذیر نیست (قطری‌سازی با ماشین D).
- $\mathbb{N}$، اعداد زوج و اعداد گویا شمارا هستند؛ $\mathcal{R}$ و مجموعه دنباله‌های دودویی نامتناهی ناشمارا هستند.
- ماشین‌ها شمارا و زبان‌ها ناشمارا هستند، پس زبان‌های تشخیص‌ناپذیر وجود دارند.
- تصمیم‌پذیر = تشخیص‌پذیر و هم-تشخیص‌پذیر. پس $\overline{A_{\mathrm{TM}}}$ تشخیص‌پذیر نیست.

۲۰ فصل

کاهش‌پذیری

در فصل قبل با قطری‌سازی ثابت کردیم A_{TM} تصمیم‌ناپذیر است. برای مسئله‌های دیگر لازم نیست هر بار از صفر شروع کنیم. ابزار اصلی اثبات حل‌ناپذیری مسئله‌ها **کاهش** (Reduction) است.

۱.۲۰ ایده کاهش

کاهش یعنی تبدیل یک مسئله به مسئله دیگر، طوری که با داشتن راه حل مسئله دوم بتوان مسئله اول را حل کرد. اگر مسئله A به مسئله B کاهش یابد، می‌توانیم از راه حل B برای حل A استفاده کنیم. چند مثال روزمره:

- پیدا کردن راه در یک شهر به پیدا کردن نقشه شهر کاهش می‌یابد.
- محاسبه مساحت مستطیل به اندازه‌گیری طول و عرض آن کاهش می‌یابد.
- حل دستگاه معادلات خطی به وارون کردن یک ماتریس کاهش می‌یابد.

اگر A به B کاهش یابد، حل A **سخت‌تر از** حل B نیست. برای تصمیم‌پذیری این یعنی:

- اگر B تصمیم‌پذیر باشد، A هم تصمیم‌پذیر است.
- معادلاً: اگر A تصمیم‌ناپذیر باشد، B هم تصمیم‌ناپذیر است.

روش کار در این فصل همیشه همین است: برای اثبات تصمیم‌ناپذیری B، فرض می‌کنیم B تصمیم‌گیرنده‌ای دارد و با آن یک تصمیم‌گیرنده برای مسئله‌ای می‌سازیم که می‌دانیم تصمیم‌ناپذیر است (معمولاً A_{TM}). (کاهش در نظریه پیچیدگی هم نقش مهمی دارد؛ فصل ۲۴.)

۲.۲۰ مسئله توقف

$$HALT_{\text{TM}} = \{\langle M, w\rangle \mid M \text{ یک ماشین تورینگ است که روی ورودی } w \text{ متوقف می‌شود}\}.$$

قضیه ۱.۲۰

$HALT_{\text{TM}}$ تصمیم‌ناپذیر است.

ایده اثبات. با برهان خلف فرض کنید ماشین R زبان $HALT_{\text{TM}}$ را تصمیم بگیرد. با R یک تصمیم‌گیرنده S برای A_{TM} می‌سازیم. S ورودی $\langle M, w\rangle$ را به R می‌دهد؛ R همیشه متوقف می‌شود. اگر R رد کند، یعنی M روی w متوقف نمی‌شود (حلقه می‌زند)، پس M رشته w را نمی‌پذیرد و S رد می‌کند. اگر R بپذیرد، M روی w متوقف می‌شود، پس می‌توانیم بدون ترس از حلقه، M را روی w شبیه‌سازی کنیم و همان جواب را برگردانیم.

اثبات. فرض کنید R تصمیم‌گیرنده $HALT_{\text{TM}}$ باشد. ماشین S:

> S = «روی ورودی $\langle M, w \rangle$»
>
> ۱. R را روی $\langle M, w \rangle$ اجرا کن.
>
> ۲. اگر R رد کرد، **رد کن**.
>
> ۳. اگر R پذیرفت، M را روی w شبیه‌سازی کن تا متوقف شود.
>
> ۴. اگر M پذیرفت **بپذیر**؛ اگر رد کرد **رد کن**.

S همیشه متوقف می‌شود و A_{TM} را تصمیم می‌گیرد، که با قضیه ۱.۱۹ تناقض دارد. پس $HALT_{\text{TM}}$ تصمیم‌ناپذیر است.

■

۳.۲۰ تهی بودن زبان یک ماشین

$$E_{\text{TM}} = \{\langle M \rangle \mid M \text{ یک ماشین تورینگ است و } L(M) = \emptyset\}.$$

قضیه ۲.۲۰

E_{TM} تصمیم‌ناپذیر است.

ایده اثبات. فرض کنید R تصمیم‌گیرنده E_{TM} باشد. می‌خواهیم با آن A_{TM} را تصمیم بگیریم. روی ورودی $\langle M, w \rangle$، ماشین M را طوری تغییر می‌دهیم که **همه رشته‌ها جز** w **را رد کند** و روی w مثل M رفتار کند. زبان ماشین تغییریافته یا $\{w\}$ است (اگر M رشته w را بپذیرد) یا $\emptyset$. پس با پرسیدن از R که «آیا زبان ماشین جدید تهی است؟» می‌فهمیم M رشته w را می‌پذیرد یا نه.

اثبات. فرض کنید R تصمیم‌گیرنده E_{TM} باشد. ماشین تغییریافته:

> M_1 = «روی ورودی x»
>
> ۱. اگر $x \neq w$، **رد کن**.
>
> ۲. اگر $x = w$، M را روی ورودی w اجرا کن و اگر M پذیرفت **بپذیر**.

(M_1 چند حالت جدید و چند بررسی برای مقایسه x با w دارد؛ w درون توصیف M_1 نوشته شده.) حالا:

> S = «روی ورودی $\langle M, w \rangle$»
>
> ۱. با استفاده از توصیف M و w، ماشین M_1 بالا را بساز.
>
> ۲. R را روی $\langle M_1 \rangle$ اجرا کن.
>
> ۳. اگر R پذیرفت **رد کن**؛ اگر R رد کرد **بپذیر**.

دقت کنید S هرگز M_1 را اجرا نمی‌کند؛ فقط توصیفش را می‌سازد و به R می‌دهد. اگر M رشته w را بپذیرد، $L(M_1) = \{w\} \neq \emptyset$ و R رد می‌کند، پس S می‌پذیرد. اگر نپذیرد، $L(M_1) = \emptyset$ و S رد می‌کند. پس S تصمیم‌گیرنده A_{TM} است؛ تناقض.

■

۴.۲۰ آیا زبان یک ماشین منظم است؟

$REGULAR_{\text{TM}} = \{\langle M \rangle \mid M$ یک ماشین تورینگ است و $L(M)$ منظم است$\}$.

قضیه ۳.۲۰

$REGULAR_{\text{TM}}$ تصمیم‌ناپذیر است.

اثبات. فرض کنید R تصمیم‌گیرنده $REGULAR_{\text{TM}}$ باشد. روی ورودی $\langle M, w \rangle$، ماشین M_2 را می‌سازیم که اگر M رشته w را بپذیرد زبانی **منظم** (یعنی Σ^*) و در غیر این صورت زبانی **نامنظم** (یعنی $\{0^n1^n \mid n \geq 0\}$) را تشخیص دهد:

> **M_2 = «روی ورودی x»**
>
> ۱. اگر x به شکل 0^n1^n است، **بپذیر**.
>
> ۲. در غیر این صورت M را روی w اجرا کن و اگر M پذیرفت، **بپذیر**.

یعنی M_2 همه رشته‌های $\{0^n1^n\}$ را همیشه می‌پذیرد و اگر M رشته w را بپذیرد، بقیه رشته‌ها را هم می‌پذیرد:

$$L(M_2) = \begin{cases} \Sigma^* & \text{اگر } M \text{ رشته } w \text{ را بپذیرد} \\ \{0^n1^n \mid n \geq 0\} & \text{در غیر این صورت.} \end{cases}$$

> **S = «روی ورودی $\langle M, w \rangle$»**
>
> ۱. ماشین M_2 بالا را بساز.
>
> ۲. R را روی $\langle M_2 \rangle$ اجرا کن.
>
> ۳. اگر R پذیرفت **بپذیر**؛ اگر رد کرد **رد کن**.

S تصمیم‌گیرنده A_{TM} است؛ تناقض.

■

به همین شکل می‌توان ثابت کرد آزمودن اینکه زبان یک ماشین تورینگ **مستقل از متن**، **تصمیم‌پذیر** یا **متناهی** است، تصمیم‌ناپذیر است. همه این‌ها حالت‌های خاص یک قضیه کلی هستند:

قضیه ۴.۲۰ (رایس)

هر ویژگی **غیربدیهی** از زبان‌هایی که ماشین‌های تورینگ تشخیص می‌دهند، تصمیم‌ناپذیر است.

در اینجا «ویژگی» یعنی مجموعه‌ای P از زبان‌های تشخیص‌پذیر و می‌پرسیم آیا $L(M) \in P$؛ پس اگر $L(M_1) = L(M_2)$ آنگاه M_1 و M_2 هر دو ویژگی را دارند یا هیچ‌کدام. «غیربدیهی» یعنی دست‌کم یک ماشین ویژگی را دارد و دست‌کم یک ماشین ندارد.

اثبات. فرض کنید R تصمیم‌گیرنده $L_P = \{\langle M \rangle \mid L(M) \in P\}$ باشد. بدون کاستن از کلیت فرض کنید $\emptyset \notin P$ (وگرنه همین استدلال را برای ویژگی متمم $\overline{P}$ انجام می‌دهیم؛ تصمیم‌پذیری L_P و $L_{\overline{P}}$ معادل است). چون P غیربدیهی است، ماشینی مثل T با $L(T) \in P$ وجود دارد. روی ورودی $\langle M, w \rangle$ ماشین زیر را می‌سازیم:

> M_w = «روی ورودی x»
>
> ۱. M را روی w اجرا کن. اگر رد کرد **رد کن**؛ اگر پذیرفت به مرحله ۲ برو.
>
> ۲. T را روی x اجرا کن و اگر پذیرفت **بپذیر**.

اگر M رشته w را بپذیرد، $L(M_w) = L(T) \in P$؛ وگرنه $L(M_w) = \emptyset \notin P$. پس با اجرای R روی $\langle M_w \rangle$ می‌فهمیم M رشته w را می‌پذیرد یا نه، و A_{TM} تصمیم‌پذیر می‌شود؛ تناقض.

■

۵.۲۰ هم‌ارزی دو ماشین

$$EQ_{\text{TM}} = \{\langle M_1, M_2 \rangle \mid M_1, M_2 \text{ ماشین تورینگ‌اند و } L(M_1) = L(M_2)\}.$$

قضیه ۵.۲۰

EQ_{TM} تصمیم‌ناپذیر است.

اثبات. این بار از E_{TM} کاهش می‌دهیم. ایده: مسئله تهی بودن حالت خاصی از هم‌ارزی است؛ کافی است ماشین را با ماشینی که همه چیز را رد می‌کند مقایسه کنیم. فرض کنید R تصمیم‌گیرنده EQ_{TM} باشد.

> S = «روی ورودی $\langle M \rangle$»
>
> ۱. R را روی $\langle M, M_1 \rangle$ اجرا کن که M_1 ماشینی است که همه ورودی‌ها را رد می‌کند.
>
> ۲. اگر R پذیرفت **بپذیر**؛ اگر رد کرد **رد کن**.

S دقیقاً وقتی می‌پذیرد که $L(M) = L(M_1) = \emptyset$. پس S تصمیم‌گیرنده E_{TM} است؛ تناقض.

■

۶.۲۰ تاریخچه محاسبه و ماشین‌های خطی‌کران‌دار

تاریخچه محاسبه یکی از مهم‌ترین تکنیک‌های کاهش است و معمولاً وقتی به کار می‌آید که مسئله درباره **وجود** چیزی باشد (مثلاً وجود ریشه صحیح در مسئله دهم هیلبرت).

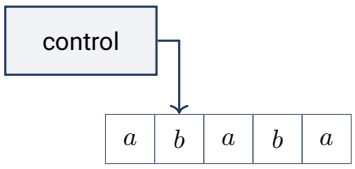
تعریف ۶.۲۰ (تاریخچه محاسبه)

فرض کنید M ماشین تورینگ و w یک ورودی باشد. یک **تاریخچه محاسبه پذیرنده** برای M روی w دنباله‌ای از پیکربندی‌ها $C_1, C_2, \ldots, C_l$ است که C_1 پیکربندی شروع M روی w، C_l یک پیکربندی پذیرش و هر C_{i+1} طبق قوانین M از C_i نتیجه می‌شود. **تاریخچه محاسبه ردکننده** هم مشابه است، با این تفاوت که C_l پیکربندی رد است.

تاریخچه‌های محاسبه دنباله‌های **متناهی** هستند. اگر M روی w متوقف نشود، هیچ تاریخچه پذیرنده یا ردکننده‌ای ندارد. ماشین‌های قطعی روی هر ورودی حداکثر یک تاریخچه محاسبه دارند؛ ماشین‌های غیرقطعی ممکن است چند تاریخچه داشته باشند (برای شاخه‌های مختلف).

تعریف ۷.۲۰ (ماشین خطی‌کران‌دار (LBA))

ماشین خطی‌کران‌دار (Linear Bounded Automaton) نوعی ماشین تورینگ است که هدش اجازه ندارد از بخشی از نوار که ورودی روی آن نوشته شده بیرون برود. اگر ماشین بخواهد هد را از هر کدام از دو انتهای ورودی بیرون ببرد، هد سر جایش می‌ماند (مثل انتهای چپ نوار در ماشین تورینگ معمولی).



حافظه LBA فقط به اندازه ورودی است. با بزرگ کردن الفبای نوار می‌توان حافظه را به اندازه یک **ضریب ثابت** زیاد کرد؛ پس می‌گوییم برای ورودی به طول n، حافظه در دسترس **خطی** بر حسب n است. با این محدودیت هم LBAها قدرت زیادی دارند: تصمیم‌گیرنده‌های A_{DFA}، A_{CFG}، E_{DFA}، E_{CFG} و هر زبان مستقل از متن همه LBA هستند.

لم ۸.۲۰

فرض کنید M یک LBA با q حالت و g نماد در الفبای نوار باشد. برای نواری به طول n، M دقیقاً qng^n پیکربندی متمایز دارد.

اثبات. پیکربندی تصویری از ماشین در میانه محاسبه است و از حالت کنترل، مکان هد و محتوای نوار تشکیل می‌شود. M دارای q حالت است، هد می‌تواند در یکی از n خانه باشد و g^n رشته ممکن از نمادهای نوار می‌تواند روی نوار باشد. حاصلضرب این سه عدد تعداد کل پیکربندی‌هاست.

■

قضیه ۹.۲۰

$A_{\text{LBA}} = \{\langle M, w\rangle \mid M$ یک LBA است که w را می‌پذیرد$\}$ **تصمیم‌پذیر** است.

ایده اثبات. اگر M یک پیکربندی را تکرار کند، از آن به بعد همان مسیر را دوباره و دوباره طی می‌کند، یعنی در حلقه افتاده است. چون M فقط تعداد محدودی پیکربندی دارد، اگر بیش از آن تعداد گام اجرا شود حتماً پیکربندی تکراری داشته است. پس کافی است M را حداکثر qng^n گام اجرا کنیم.

اثبات.

> **L = «روی ورودی $\langle M, w\rangle$ که M یک LBA است»**
>
> ۱. M را روی w به اندازه qng^n گام یا تا وقتی متوقف شود شبیه‌سازی کن ($n = |w|$).
>
> ۲. اگر M متوقف شد، اگر پذیرفته بود **بپذیر** و اگر رد کرده بود **رد کن**. اگر متوقف نشد، **رد کن**.

اگر M در qng^n گام متوقف نشود، طبق لم ۶.۲۰ و اصل لانه کبوتری یک پیکربندی را تکرار کرده و هرگز متوقف نمی‌شود؛ پس w را نمی‌پذیرد و رد کردن درست است.

■

قضیه ۱۰.۲۰

$E_{\text{LBA}} = \{\langle M\rangle \mid M \text{ یک LBA است و } L(M) = \emptyset\}$ تصمیم‌ناپذیر است.

ایده اثبات. از A_{TM} کاهش می‌دهیم. روی ورودی $\langle M, w\rangle$ یک LBA مثل B می‌سازیم که دقیقاً **تاریخچه‌های محاسبه پذیرنده** M **روی** w را می‌پذیرد. اگر M رشته w را بپذیرد، زبان B یک رشته دارد (چون M قطعی است)؛ وگرنه زبان B تهی است. پس با پرسیدن تهی بودن زبان B می‌فهمیم M رشته w را می‌پذیرد یا نه.
B رشته‌ای به شکل زیر را دریافت می‌کند:

$$\# C_1 \# C_2 \# C_3 \# \cdots \# C_l \#$$

و سه شرط را بررسی می‌کند:

۱. C_1 پیکربندی شروع M روی w است.

۲. هر C_{i+1} طبق قوانین M از C_i نتیجه می‌شود.

۳. C_l یک پیکربندی پذیرش M است.

اثبات. ساختن B: بررسی شرط ۱: B رشته C_1 را با $q_0 w_1 w_2 \cdots w_n$ مقایسه می‌کند (w در توصیف B نوشته شده). شرط ۳: B بررسی می‌کند که C_l حالت q_{accept} را دارد. شرط ۲: برای هر i، B بین C_i و C_{i+1} رفت و برگشت می‌کند و نمادهای متناظر را با علامت‌گذاری روی نوار (که همان ورودی است) مقایسه می‌کند. دو پیکربندی متوالی باید جز در اطراف هد یکسان باشند و تغییر اطراف هد باید طبق تابع گذار M باشد. همه این کارها بدون بیرون رفتن از بخش ورودی نوار انجام می‌شود، پس B یک LBA است.
کاهش: فرض کنید R تصمیم‌گیرنده E_{LBA} باشد.

> **S = «روی ورودی $\langle M, w\rangle$»**
>
> ۱. LBA B را از روی M و w بساز.
>
> ۲. R را روی $\langle B\rangle$ اجرا کن.
>
> ۳. اگر R رد کرد **بپذیر**؛ اگر پذیرفت **رد کن**.

اگر M رشته w را بپذیرد، تاریخچه پذیرنده‌ای وجود دارد، $L(B) \neq \emptyset$ و S می‌پذیرد؛ وگرنه $L(B) = \emptyset$ و S رد می‌کند.

پس S تصمیم‌گیرنده A_{TM} است؛ تناقض.

■

۷.۲۰ گرامرهایی که همه رشته‌ها را تولید می‌کنند

$$ALL_{\text{CFG}} = \{\langle G\rangle \mid G \text{ یک CFG است و } L(G) = \Sigma^*\}.$$

قضیه ۱۱.۲۰

ALL_{CFG} تصمیم‌ناپذیر است.

ایده اثبات. باز هم از A_{TM} با تاریخچه محاسبه کاهش می‌دهیم. روی ورودی $\langle M, w\rangle$ یک گرامر G می‌سازیم که:

- اگر M رشته w را نپذیرد، G **همه** رشته‌ها را تولید کند؛
- اگر M رشته w را بپذیرد، G همه رشته‌ها **جز** تاریخچه محاسبه پذیرنده M روی w را تولید کند.

پس $L(G) = \Sigma^*$ اگر و تنها اگر M رشته w را نپذیرد.

اثبات. یک تاریخچه محاسبه پذیرنده را به شکل $\#C_1\#C_2\#\cdots\#C_l\#$ می‌نویسیم. G همه رشته‌هایی را تولید می‌کند که دست‌کم یکی از این شرط‌ها را داشته باشند:

۱. با C_1 (پیکربندی شروع M روی w) شروع نشوند،

۲. به یک پیکربندی پذیرش ختم نشوند، یا

۳. در آن‌ها یک C_i وجود داشته باشد که C_{i+1} به‌درستی از آن طبق قوانین M به دست نیامده باشد.

اگر M رشته w را نپذیرد، هیچ تاریخچه پذیرنده‌ای وجود ندارد، پس **هر** رشته دست‌کم یکی از این شرط‌ها را دارد و G همه رشته‌ها را تولید می‌کند. اگر M رشته w را بپذیرد، تنها رشته‌ای که هیچ‌کدام از شرط‌ها را ندارد همان تاریخچه پذیرنده است.

ساختن مستقیم G دشوار است. به جای آن یک PDA مثل D می‌سازیم و آن را با قضیه ۴.۱۲ به گرامر تبدیل می‌کنیم. D به‌طور غیرقطعی یکی از سه شرط را انتخاب و بررسی می‌کند:

۱. ابتدای ورودی را بررسی می‌کند و اگر C_1 پیکربندی شروع درست نبود، می‌پذیرد.

۲. انتهای ورودی را بررسی می‌کند و اگر آخرین پیکربندی، پیکربندی پذیرش نبود، می‌پذیرد.

۳. به‌طور غیرقطعی یک C_i را انتخاب می‌کند، آن را روی پشته می‌گذارد و هنگام خواندن C_{i+1} آن را از پشته برمی‌دارد و مقایسه می‌کند. اگر ناهماهنگی یا به‌روزرسانی نادرستی (طبق تابع گذار M در اطراف هد) پیدا کرد، می‌پذیرد.

یک مشکل فنی در شرط ۳: وقتی C_i را از پشته برمی‌داریم، به ترتیب معکوس بیرون می‌آید. برای حل این مشکل تاریخچه را طوری می‌نویسیم که پیکربندی‌های زوج **معکوس** باشند:

$$\#\,\overrightarrow{C_1}\,\#\,\overleftarrow{C_2^R}\,\#\,\overrightarrow{C_3}\,\#\,\overleftarrow{C_4^R}\,\#\cdots\#\,C_l\,\#$$

به این ترتیب هر دو پیکربندی متوالی را می‌توان با پشته مقایسه کرد.

حالا اگر R تصمیم‌گیرنده ALL_{CFG} باشد، ماشین S روی $\langle M, w\rangle$ گرامر G را از روی D می‌سازد، R را روی $\langle G\rangle$ اجرا می‌کند و اگر R رد کرد می‌پذیرد و اگر پذیرفت رد می‌کند. S تصمیم‌گیرنده A_{TM} است؛ تناقض.

■

نتیجه ۱۲.۲۰

EQ_{CFG} تصمیم‌ناپذیر است.

اثبات. اگر تصمیم‌گیرنده‌ای برای EQ_{CFG} داشتیم، می‌توانستیم ALL_{CFG} را تصمیم بگیریم: روی ورودی $\langle G\rangle$، گرامر G را با گرامر ساده $H : S \to aS \mid \varepsilon$ (برای هر $a \in \Sigma$) که Σ^* را تولید می‌کند مقایسه می‌کنیم. $\langle G, H\rangle \in EQ_{\text{CFG}}$ اگر و تنها اگر $L(G) = \Sigma^*$.

■

جمع‌بندی فصل

- اگر A به B کاهش یابد و A تصمیم‌ناپذیر باشد، B هم تصمیم‌ناپذیر است.
- تصمیم‌ناپذیر (با کاهش از A_{TM}): $HALT_{\text{TM}}$، E_{TM}، $REGULAR_{\text{TM}}$ و هر ویژگی غیربدیهی زبان ماشین‌ها (رایس). EQ_{TM} با کاهش از E_{TM}.
- LBA تعداد متناهی (qng^n) پیکربندی دارد، پس A_{LBA} تصمیم‌پذیر است؛ اما E_{LBA} تصمیم‌ناپذیر است.
- با تاریخچه محاسبه: E_{LBA} و ALL_{CFG} (و در نتیجه EQ_{CFG}) تصمیم‌ناپذیرند.

فصل ۲۱

مسئله تناظر پست و کاهش نگاشتی

تصمیم‌ناپذیری فقط به مسئله‌هایی درباره ماشین‌ها محدود نیست. در این فصل یک مسئله ساده درباره رشته‌ها می‌بینیم که تصمیم‌ناپذیر است: **مسئله تناظر پست** (Post Correspondence Problem) یا PCP. سپس مفهوم کاهش را با تعریف دقیق **کاهش نگاشتی** صوری می‌کنیم.

۱.۲۱ مسئله تناظر پست

یک **دومینو** دو نیمه دارد که روی هر کدام یک رشته نوشته شده، مثل $\left[\begin{smallmatrix}\mathtt{a}\\\mathtt{ab}\end{smallmatrix}\right]$. یک **مجموعه دومینو** داریم، مثلاً

$$\left\{\left[\begin{smallmatrix}\mathtt{b}\\\mathtt{ca}\end{smallmatrix}\right], \left[\begin{smallmatrix}\mathtt{a}\\\mathtt{ab}\end{smallmatrix}\right], \left[\begin{smallmatrix}\mathtt{ca}\\\mathtt{a}\end{smallmatrix}\right], \left[\begin{smallmatrix}\mathtt{abc}\\\mathtt{c}\end{smallmatrix}\right]\right\}.$$

هدف پیدا کردن یک **تطابق** است: فهرستی از دومینوهای این مجموعه (با تکرار مجاز) که اگر کنار هم بچینیم، رشته بالا با رشته پایین یکی شود. برای مجموعه بالا:

$$\left[\begin{smallmatrix}\mathtt{a}\\\mathtt{ab}\end{smallmatrix}\right]\left[\begin{smallmatrix}\mathtt{b}\\\mathtt{ca}\end{smallmatrix}\right]\left[\begin{smallmatrix}\mathtt{ca}\\\mathtt{a}\end{smallmatrix}\right]\left[\begin{smallmatrix}\mathtt{a}\\\mathtt{ab}\end{smallmatrix}\right]\left[\begin{smallmatrix}\mathtt{abc}\\\mathtt{c}\end{smallmatrix}\right]: \quad \text{بالا} = abcaaabc = \text{پایین}.$$

بعضی مجموعه‌ها تطابق ندارند. مثلاً در $\left\{\left[\begin{smallmatrix}\mathtt{abc}\\\mathtt{ab}\end{smallmatrix}\right], \left[\begin{smallmatrix}\mathtt{ca}\\\mathtt{a}\end{smallmatrix}\right], \left[\begin{smallmatrix}\mathtt{acc}\\\mathtt{ba}\end{smallmatrix}\right]\right\}$ رشته بالای هر دومینو از رشته پایینش بلندتر است، پس رشته بالا همیشه بلندتر از پایین می‌ماند.

تعریف ۱.۲۱ (PCP)

یک نمونه از PCP مجموعه‌ای از دومینوهاست:

$$P = \left\{\left[\frac{t_1}{b_1}\right], \left[\frac{t_2}{b_2}\right], \ldots, \left[\frac{t_k}{b_k}\right]\right\}.$$

یک **تطابق** دنباله‌ای $i_1, i_2, \ldots, i_l$ (با $l \geq 1$) است که $t_{i_1}t_{i_2}\cdots t_{i_l} = b_{i_1}b_{i_2}\cdots b_{i_l}$. مسئله این است که تعیین کنیم P تطابق دارد یا نه:

$$PCP = \{\langle P\rangle \mid P \text{ یک نمونه PCP با یک تطابق است}\}.$$

قضیه ۲.۲۱

PCP تصمیم‌ناپذیر است.

ایده اثبات. از A_{TM} با تاریخچه‌های محاسبه پذیرنده کاهش می‌دهیم. از هر ماشین M و ورودی w یک نمونه P می‌سازیم که تطابق‌هایش دقیقاً تاریخچه‌های محاسبه پذیرنده M روی w هستند. یعنی ساختن یک تطابق، همان **شبیه‌سازی** M روی w است. پس اگر بتوانیم تشخیص دهیم تطابقی هست، می‌فهمیم M رشته w را می‌پذیرد یا

نه.

سه جزئیات فنی:

- فرض می‌کنیم M روی w هرگز سعی نمی‌کند هد را از انتهای چپ نوار بیرون ببرد (M را می‌توان طوری تغییر داد که این‌طور باشد).
- اگر $w = \varepsilon$، به جای آن از $\sqcup$ استفاده می‌کنیم.
- ابتدا مسئله را برای حالتی حل می‌کنیم که تطابق باید **با دومینوی اول** $\left[\frac{t_1}{b_1}\right]$ شروع شود. به این مسئله **PCP اصلاح‌شده** (MPCP) می‌گوییم:

$$MPCP = \{\langle P\rangle \mid P \text{ نمونه‌ای از PCP با تطابقی است که با دومینوی اول شروع می‌شود}\}.$$

اثبات. فرض کنید ماشین R زبان PCP را تصمیم بگیرد. ماشین S را می‌سازیم که A_{TM} را تصمیم بگیرد. فرض کنید $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$. ماشین S یک نمونه P از PCP می‌سازد که تطابق دارد اگر و تنها اگر M رشته w را بپذیرد. برای این کار، S ابتدا یک نمونه P' از MPCP در هفت بخش می‌سازد.

بخش ۱ (پیکربندی اول): دومینوی زیر را به عنوان دومینوی اول در P' بگذار:

$$\left[\begin{matrix}\#\\ \#\mathtt{q}_0\mathtt{w}_1\mathtt{w}_2\cdots\mathtt{w}_n\#\end{matrix}\right].$$

چون تطابق باید با این دومینو شروع شود، رشته پایین با C_1 (پیکربندی شروع) شروع می‌شود. حالا رشته پایین از بالا جلوتر است. برای اینکه بالا به پایین برسد باید دومینوهایی اضافه کنیم که بالایشان C_1 باشد؛ این دومینوها طوری ساخته شده‌اند که پایینشان C_2 (پیکربندی بعدی) را تولید کند. پس تطابق مجبور است گام به گام محاسبه M را شبیه‌سازی کند.

بخش ۲ (حرکت هد به راست): برای هر $a, b \in \Gamma$ و هر $q, r \in Q$ با $q \neq q_{\text{reject}}$:

اگر $\delta(q, a) = (r, b, R)$ دومینوی $\left[\begin{matrix}\mathtt{qa}\\ \mathtt{br}\end{matrix}\right]$ را در P' بگذار.

بخش ۳ (حرکت هد به چپ): برای هر $a, b, c \in \Gamma$ و هر $q, r \in Q$ با $q \neq q_{\text{reject}}$:

اگر $\delta(q, a) = (r, b, L)$ دومینوی $\left[\begin{matrix}\mathtt{cqa}\\ \mathtt{rcb}\end{matrix}\right]$ را در P' بگذار.

بخش ۴ (نمادهای دور از هد): برای هر $a \in \Gamma$ دومینوی $\left[\begin{matrix}\mathtt{a}\\ \mathtt{a}\end{matrix}\right]$ را بگذار. این دومینوها بخش‌هایی از نوار را که تغییر نمی‌کنند کپی می‌کنند.

بخش ۵ (جداکننده و نوار نامحدود): دومینوهای $\left[\begin{matrix}\#\\ \#\end{matrix}\right]$ و $\left[\begin{matrix}\#\\ \sqcup\#\end{matrix}\right]$ را بگذار. اولی جداکننده پیکربندی‌ها را کپی می‌کند و دومی یک خانه خالی به انتهای پیکربندی اضافه می‌کند تا نوار نامحدود شبیه‌سازی شود (وقتی هد به انتهای راست می‌رسد).

مثال ۳.۲۱. فرض کنید $\Gamma = \{0, 1, 2, \sqcup\}$، $w = 0100$، حالت شروع q_0 و $\delta(q_0, 0) = (q_7, 2, R)$. بخش ۱ دومینوی $\left[\begin{smallmatrix} \# \\ \#q_0 0100 \# \end{smallmatrix}\right]$، بخش ۲ دومینوی $\left[\begin{smallmatrix} q_0 0 \\ 2q_7 \end{smallmatrix}\right]$ و بخش ۴ دومینوهای $\left[\begin{smallmatrix} \sqcup \\ \sqcup \end{smallmatrix}\right], \left[\begin{smallmatrix} 0 \\ 0 \end{smallmatrix}\right], \left[\begin{smallmatrix} 1 \\ 1 \end{smallmatrix}\right], \left[\begin{smallmatrix} 2 \\ 2 \end{smallmatrix}\right]$ را می‌سازند. تطابق جزئی:

top	$\#$	$q_0 0$	1	0	0	$\#$
bottom	$\#q_0 0100\#$	$2q_7$	1	0	0	$\#$

حالا رشته بالا $\#q_0 0100\#$ و پایین $\#q_0 0100 \# 2q_7 100\#$ است؛ یعنی پایین یک پیکربندی ($C_2 = 2q_7 100$) جلوتر است. اگر $\delta(q_5, 0) = (q_9, 2, L)$ بود، بخش ۳ دومینوهای $\left[\begin{smallmatrix} 0 q_5 0 \\ q_9 02 \end{smallmatrix}\right]$، $\left[\begin{smallmatrix} 1 q_5 0 \\ q_9 12 \end{smallmatrix}\right]$، $\left[\begin{smallmatrix} 2 q_5 0 \\ q_9 22 \end{smallmatrix}\right]$ و $\left[\begin{smallmatrix} \sqcup q_5 0 \\ q_9 \sqcup 2 \end{smallmatrix}\right]$ را می‌ساخت.

بخش ۶ (پس از پذیرش): وقتی M به q_{accept} می‌رسد، می‌خواهیم رشته بالا به پایین «برسد» تا تطابق کامل شود. برای هر $a \in \Gamma$ دومینوهای

$$\left[\begin{smallmatrix} a\, q_{\text{acc}} \\ q_{\text{acc}} \end{smallmatrix}\right] \qquad \text{و} \qquad \left[\begin{smallmatrix} q_{\text{acc}}\, a \\ q_{\text{acc}} \end{smallmatrix}\right]$$

را بگذار. این دومینوها اجازه می‌دهند q_{accept} نمادهای کنارش را یکی‌یکی «بخورد» تا فقط q_{accept} بماند: مثلاً از $\#21q_{\text{accept}}02\#$ به $\#2q_{\text{accept}}02\#$، سپس $\#2q_{\text{accept}}2\#$ و ... تا $\#q_{\text{accept}}\#$.

بخش ۷ (تکمیل تطابق): دومینوی $\left[\begin{smallmatrix} q_{\text{acc}} \# \# \\ \# \end{smallmatrix}\right]$ را بگذار. وقتی پایین به $\#q_{\text{accept}}\#$ ختم شده، این دومینو بالا و پایین را برابر می‌کند.

با این ساخت، P' تطابقی که با دومینوی اول شروع شود دارد اگر و تنها اگر M رشته w را بپذیرد: هر تطابق باید پیکربندی‌های متوالی یک محاسبه درست M را در پایین بسازد (دومینوها فقط تغییرهای مجاز اطراف هد را اجازه می‌دهند) و تنها راه برابر شدن بالا و پایین، رسیدن به q_{accept} و استفاده از بخش‌های ۶ و ۷ است. برعکس، تاریخچه پذیرنده M روی w دقیقاً یک تطابق می‌سازد.

تبدیل MPCP به PCP: حالا باید شرط «شروع با دومینوی اول» را حذف کنیم. برای رشته $u = u_1 u_2 \cdots u_n$ تعریف کنید:

$$\star u = *u_1 * u_2 * \cdots * u_n, \qquad u\star = u_1 * u_2 * \cdots * u_n *, \qquad \star u \star = *u_1 * u_2 * \cdots * u_n *,$$

که $*$ نماد جدیدی است. اگر $P' = \left\{ \left[\frac{t_1}{b_1}\right], \ldots, \left[\frac{t_k}{b_k}\right] \right\}$، قرار دهید

$$P = \left\{ \left[\frac{\star t_1}{\star b_1 \star}\right], \left[\frac{\star t_1}{b_1 \star}\right], \left[\frac{\star t_2}{b_2 \star}\right], \ldots, \left[\frac{\star t_k}{b_k \star}\right], \left[\frac{* \diamond}{\diamond}\right] \right\}.$$

در P تنها دومینویی که بالا و پایینش با یک نماد یکسان ($*$) شروع می‌شود، اولی است؛ پس هر تطابق P مجبور است با آن شروع شود. در بقیه دومینوها نمادهای بالا و پایین یکی در میان با $*$ جابه‌جا شده‌اند و دومینوی آخر $\left[\frac{* \diamond}{\diamond}\right]$ اجازه می‌دهد $*$ اضافه در انتهای رشته بالا هم جبران شود. با حذف $*$ها هر تطابق P یک تطابق P' (شروع‌شده با دومینوی اول) می‌دهد و برعکس.

پس S روی $\langle M, w \rangle$ نمونه P را می‌سازد و R را روی $\langle P \rangle$ اجرا می‌کند و همان جواب را برمی‌گرداند. S تصمیم‌گیرنده A_{TM} است؛ تناقض.

■

۲.۲۱ کاهش نگاشتی

حالا مفهوم کاهش را دقیق تعریف می‌کنیم. این تعریف به ما اجازه می‌دهد کاهش را دقیق‌تر به کار ببریم، مثلاً برای اثبات **تشخیص‌ناپذیری**، و در نظریه پیچیدگی هم کاربرد دارد. ایده: مسئله A را با یک **تابع محاسبه‌پذیر** که نمونه‌های A را به نمونه‌های B تبدیل می‌کند، به B کاهش می‌دهیم. به این کاهش **کاهش نگاشتی** یا **کاهش چند-به-یک** می‌گوییم.

تعریف ۴.۲۱ (تابع محاسبه‌پذیر)

تابع $f : \Sigma^* \to \Sigma^*$ **محاسبه‌پذیر** است اگر ماشین تورینگی مثل M وجود داشته باشد که روی هر ورودی w متوقف شود و در پایان فقط $f(w)$ روی نوارش باشد.

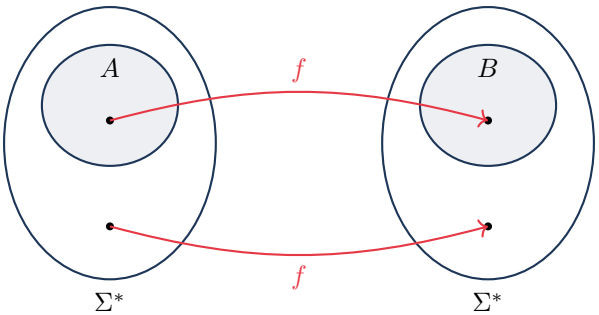
مثلاً عملیات حسابی روی اعداد صحیح ($\langle m, n\rangle \mapsto \langle m+n\rangle$) یا تبدیل توصیف ماشین‌ها ($\langle M\rangle \mapsto \langle M'\rangle$ که M' هرگز هد را از انتهای چپ بیرون نمی‌برد) توابع محاسبه‌پذیرند.

تعریف ۵.۲۱ (کاهش نگاشتی)

زبان A به زبان B **کاهش‌پذیر نگاشتی** (Mapping Reducible) است و می‌نویسیم $A \leq_m B$، اگر تابع محاسبه‌پذیری مثل $f : \Sigma^* \to \Sigma^*$ وجود داشته باشد که برای هر w:

$$w \in A \iff f(w) \in B.$$

به f **کاهش** از A به B می‌گوییم.



یعنی f عضوهای A را به عضوهای B و غیرعضوهای A را به غیرعضوهای B می‌برد؛ پس آزمون عضویت در A به آزمون عضویت در B تبدیل می‌شود.

قضیه ۶.۲۱

اگر $A \leq_m B$ و B تصمیم‌پذیر باشد، A هم تصمیم‌پذیر است.

اثبات. فرض کنید M تصمیم‌گیرنده B و f کاهش از A به B باشد. تصمیم‌گیرنده N برای A:

N = «روی ورودی w»

۱. $f(w)$ را محاسبه کن.

۲. M را روی $f(w)$ اجرا کن و همان خروجی را بده.

اگر $w \in A$ آنگاه $f(w) \in B$ و M می‌پذیرد؛ اگر $w \notin A$ آنگاه $f(w) \notin B$ و M رد می‌کند. هر دو مرحله متوقف می‌شوند، پس N تصمیم‌گیرنده A است.

■

نتیجه ۷.۲۱

اگر $A \leq_m B$ و A تصمیم‌ناپذیر باشد، B هم تصمیم‌ناپذیر است.

مثال ۸.۲۱ (کاهش نگاشتی از A_{TM} به $HALT_{\text{TM}}$). تابع محاسبه‌پذیر $f(\langle M, w\rangle) = \langle M', w'\rangle$ را می‌خواهیم که

$$\langle M, w\rangle \in A_{\text{TM}} \iff \langle M', w'\rangle \in HALT_{\text{TM}}.$$

> F = «روی ورودی $\langle M, w\rangle$»
>
> ۱. ماشین M' زیر را بساز:
> M' = «روی ورودی x: (۱) M را روی x اجرا کن. (۲) اگر M پذیرفت، **بپذیر**. (۳) اگر M رد کرد، وارد حلقه شو.»
> ۲. $\langle M', w\rangle$ را خروجی بده.

M' روی w متوقف می‌شود اگر و تنها اگر M رشته w را بپذیرد (اگر M رد کند، M' حلقه می‌زند و اگر M حلقه بزند، M' هم حلقه می‌زند). اگر ورودی F کدگذاری درستی نداشته باشد، F رشته‌ای خارج از $HALT_{\text{TM}}$ را خروجی می‌دهد.

نکته. اثبات تصمیم‌ناپذیری PCP هم در واقع دو کاهش نگاشتی است: $A_{\text{TM}} \leq_m MPCP \leq_m PCP$. اما همه کاهش‌های فصل قبل نگاشتی نیستند؛ مثلاً در اثبات E_{TM}، خروجی R را برعکس کردیم. آن کاهش در واقع یک کاهش نگاشتی از A_{TM} به $\overline{E_{\text{TM}}}$ است.

قضیه ۹.۲۱

اگر $A \leq_m B$ و B تورینگ-تشخیص‌پذیر باشد، A هم تورینگ-تشخیص‌پذیر است.

اثبات. اثبات مثل قضیه ۲.۲۱ است، با این تفاوت که M و N تشخیص‌دهنده‌اند: N روی w مقدار $f(w)$ را حساب می‌کند و M را روی آن اجرا می‌کند. N رشته w را می‌پذیرد اگر و تنها اگر M رشته $f(w)$ را بپذیرد، یعنی اگر و تنها اگر $w \in A$.

■

نتیجه ۱۰.۲۱

اگر $A \leq_m B$ و A تورینگ-تشخیص‌پذیر نباشد، B هم تورینگ-تشخیص‌پذیر نیست.

دقت کنید که اگر $A \leq_m B$ با تابع f، همان f نشان می‌دهد $\overline{A} \leq_m \overline{B}$ (چون $w \notin A \iff f(w) \notin B$). این نکته را در اثبات زیر به کار می‌بریم.

قضیه ۱۱.۲۱

EQ_{TM} نه تورینگ-تشخیص‌پذیر است و نه هم-تورینگ-تشخیص‌پذیر.

اثبات. EQ_{TM} **تشخیص‌پذیر نیست.** نشان می‌دهیم $A_{\text{TM}} \leq_m \overline{EQ_{\text{TM}}}$. پس $\overline{A_{\text{TM}}} \leq_m EQ_{\text{TM}}$ و چون $\overline{A_{\text{TM}}}$ تشخیص‌پذیر نیست، EQ_{TM} هم نیست. کاهش f:

> F = «روی ورودی $\langle M, w \rangle$»
>
> ۱. دو ماشین M_1 و M_2 را بساز:
>
> M_1 = «روی هر ورودی: **رد کن**.»
>
> M_2 = «روی هر ورودی: M را روی w اجرا کن. اگر پذیرفت، **بپذیر**.»
>
> ۲. $\langle M_1, M_2 \rangle$ را خروجی بده.

اگر M رشته w را بپذیرد، M_2 همه چیز را می‌پذیرد و M_1 و M_2 هم‌ارز نیستند. اگر M رشته w را نپذیرد، M_2 هیچ چیز نمی‌پذیرد و دو ماشین هم‌ارزند. پس $\langle M, w \rangle \in A_{\text{TM}} \iff \langle M_1, M_2 \rangle \in \overline{EQ_{\text{TM}}}$.

EQ_{TM} **هم-تشخیص‌پذیر نیست.** نشان می‌دهیم $A_{\text{TM}} \leq_m EQ_{\text{TM}}$؛ پس $\overline{A_{\text{TM}}} \leq_m \overline{EQ_{\text{TM}}}$ و $\overline{EQ_{\text{TM}}}$ تشخیص‌پذیر نیست. کاهش g همان F است، با این تفاوت که M_1 این بار **همه ورودی‌ها را می‌پذیرد**. اگر M رشته w را بپذیرد، M_2 هم همه چیز را می‌پذیرد و دو ماشین هم‌ارزند؛ وگرنه هم‌ارز نیستند. پس $\langle M, w \rangle \in g(\langle M, w \rangle) \in EQ_{\text{TM}} \iff A_{\text{TM}}$.

■

جمع‌بندی فصل

- PCP: آیا دنباله‌ای از دومینوها هست که رشته بالا و پایین یکی شود؟ تصمیم‌ناپذیر است (تطابق = تاریخچه محاسبه پذیرنده؛ با هفت بخش دومینو و تبدیل MPCP به PCP).
- $A \leq_m B$: تابع محاسبه‌پذیر f با $w \in A \iff f(w) \in B$.
- اگر $A \leq_m B$: تصمیم‌پذیری و تشخیص‌پذیری از B به A منتقل می‌شود، و تصمیم‌ناپذیری و تشخیص‌ناپذیری از A به B.
- EQ_{TM} نه تشخیص‌پذیر است و نه هم-تشخیص‌پذیر.

بخش پنجم

پیچیدگی محاسبه

۲۲ فصل

پیچیدگی زمانی و کلاس P

حتی وقتی مسئله‌ای تصمیم‌پذیر است، ممکن است در عمل حل‌شدنی نباشد، چون الگوریتم آن زمان یا حافظه بیش از حدی لازم دارد. **نظریه پیچیدگی محاسبه** منابع لازم برای حل مسئله‌ها (زمان، حافظه، دیسک، تعداد پیام و ...) را بررسی می‌کند. در این بخش روی **زمان** تمرکز می‌کنیم: یاد می‌گیریم زمان را چطور اندازه بگیریم و مسئله‌ها را بر اساس زمان لازم دسته‌بندی کنیم.

۱.۲۲ اندازه‌گیری زمان

مثال ۱.۲۲. زبان $A = \{0^k 1^k \mid k \geq 0\}$ تصمیم‌پذیر است. یک ماشین تورینگ تک‌نواره برای تصمیم‌گیری آن چقدر زمان لازم دارد؟

«M_1 = روی ورودی w»

۱. نوار را طی کن و اگر یک 0 در سمت راست یک 1 پیدا شد، **رد کن**.

۲. تا وقتی هم 0 و هم 1 روی نوار مانده، تکرار کن:

۳. نوار را طی کن و یک 0 و یک 1 را خط بزن.

۴. اگر بعد از خط خوردن همه 1ها هنوز 0 مانده بود یا برعکس، **رد کن**. وگرنه، اگر نه 0 و نه 1 روی نوار نمانده، **بپذیر**.

زمان اجرا به عوامل زیادی بستگی دارد. برای ساده شدن، فقط آن را تابعی از **طول نمایش ورودی** در نظر می‌گیریم. همچنین معمولاً **تحلیل بدترین حالت** انجام می‌دهیم: بیشترین زمان اجرا روی همه ورودی‌های به طول n. (در **تحلیل حالت متوسط**، میانگین زمان روی همه ورودی‌های به طول n حساب می‌شود.)

تعریف ۲.۲۲ (زمان اجرا)

فرض کنید M یک ماشین تورینگ قطعی باشد که روی همه ورودی‌ها متوقف می‌شود. **زمان اجرا** یا **پیچیدگی زمانی** M تابع $f : \mathbb{N} \to \mathbb{N}$ است که $f(n)$ بیشترین تعداد گامی است که M روی ورودی‌های به طول n برمی‌دارد. می‌گوییم M در زمان $f(n)$ اجرا می‌شود یا M یک ماشین با زمان $f(n)$ است. معمولاً n طول ورودی است.

۱.۱.۲۲ نمادهای O بزرگ و o کوچک

زمان دقیق اجرا معمولاً عبارت پیچیده‌ای است. پس آن را **تخمین** می‌زنیم: با **تحلیل مجانبی** رفتار روی ورودی‌های بزرگ را بررسی می‌کنیم. فقط جمله با بالاترین مرتبه را نگه می‌داریم و ضریب آن و جمله‌های مرتبه پایین‌تر را کنار می‌گذاریم. مثلاً

$$f(n) = 6n^3 + 2n^2 + 20n + 45 \quad \rightsquigarrow \quad n^3,$$

یعنی f به‌طور مجانبی حداکثر n^3 است و می‌نویسیم $f(n) = O(n^3)$.

تعریف ۳.۲۲ (O بزرگ)

فرض کنید $f, g : \mathbb{N} \to \mathbb{R}^+$. می‌گوییم $f(n) = O(g(n))$ اگر اعداد صحیح مثبت c و n_0 وجود داشته باشند که برای هر $n \geq n_0$:

$$f(n) \leq c\, g(n).$$

در این حالت $g(n)$ یک **کران بالا** (یا دقیق‌تر، کران بالای مجانبی) برای $f(n)$ است.

یعنی اگر تفاوت در حد ضریب ثابت را نادیده بگیریم، f کوچک‌تر یا مساوی g است؛ O نماینده یک ثابت پنهان است.

مثال ۴.۲۲.

- $f_1(n) = 5n^3 + 2n^2 + 22n + 6 = O(n^3)$، چون برای $n \geq 10$ داریم $f_1(n) \leq 6n^3$. همچنین $f_1(n) = O(n^4)$، اما $f_1(n)$ از مرتبه $O(n^2)$ **نیست**.
- تغییر پایه لگاریتم مقدار را فقط در یک ضریب ثابت تغییر می‌دهد: $\log_b n = \log_2 n / \log_2 b$. پس در نماد O پایه را نمی‌نویسیم: $f(n) = O(\log n)$.
- نماد O را در عبارت‌های حسابی هم به کار می‌بریم: $f(n) = O(n^2) + O(n) = O(n^2)$.
- O در توان: $f(n) = 2^{O(n)}$ یعنی $f(n) \leq 2^{cn}$ برای یک c. همچنین $f(n) = 2^{O(\log n)}$ یعنی $f(n) \leq n^c$ برای یک c (چون $n = 2^{\log_2 n}$)، و $f(n) = n^{O(1)}$ هم یعنی $f(n) \leq n^c$.
- کران‌های **چندجمله‌ای** شکل n^c دارند ($c > 0$) و کران‌های **نمایی** شکل 2^{n^δ} ($\delta > 0$).

برای اینکه بگوییم یک تابع به‌طور مجانبی **اکیداً** کوچک‌تر از دیگری است، از o کوچک استفاده می‌کنیم. رابطه O و o مثل رابطه $\leq$ و $<$ است.

تعریف ۵.۲۲ (o کوچک)

فرض کنید $f, g : \mathbb{N} \to \mathbb{R}^+$. می‌گوییم $f(n) = o(g(n))$ اگر

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0.$$

به بیان دیگر برای هر عدد حقیقی $c > 0$ عددی مثل n_0 وجود دارد که برای هر $n \geq n_0$: $f(n) < c\, g(n)$.

مثلاً $n^2 = o(n^3)$، اما $f(n)$ هرگز $o(f(n))$ نیست.

۲.۱.۲۲ تحلیل الگوریتم

مثال ۶.۲۲ (تحلیل M_1).

- **مرحله ۱:** یک بار طی کردن نوار و برگشتن: $2n = O(n)$ گام.
- **مراحل ۲ و ۳:** هر بار دو نماد خط می‌خورد، پس حداکثر $n/2$ دور داریم و هر دور $O(n)$ گام دارد: $(n/2)\,O(n) = O(n^2)$.
- **مرحله ۴:** یک بار طی کردن نوار: $O(n)$.

پس زمان اجرای M_1 برابر $O(n) + O(n^2) + O(n) = O(n^2)$ است.

تعریف ۷.۲۲ (کلاس پیچیدگی زمانی)

فرض کنید $t : \mathbb{N} \to \mathbb{R}^+$ یک تابع باشد. **کلاس پیچیدگی زمانی** $\mathrm{TIME}(t(n))$ مجموعه همه زبان‌هایی است که یک ماشین تورینگ با زمان $O(t(n))$ آن‌ها را تصمیم می‌گیرد.

پس $A = \{0^k 1^k \mid k \geq 0\} \in \mathrm{TIME}(n^2)$. آیا می‌توان A را سریع‌تر تصمیم گرفت؟

مثال ۸.۲۲ (الگوریتم $O(n \log n)$).

M_2 = «روی ورودی w»

۱. نوار را طی کن و اگر یک 0 در سمت راست یک 1 پیدا شد، **رد کن**.

۲. تا وقتی هم 0 و هم 1 روی نوار مانده، تکرار کن:

۳. نوار را طی کن و بررسی کن که تعداد کل 0ها و 1های باقی‌مانده زوج است یا فرد. اگر فرد بود، **رد کن**.

۴. دوباره نوار را طی کن و یکی در میان 0ها (با شروع از اولین 0) و یکی در میان 1ها (با شروع از اولین 1) را خط بزن.

۵. اگر هیچ 0 و هیچ 1ی روی نوار نمانده، **بپذیر**؛ وگرنه **رد کن**.

هر دور، تعداد 0ها و تعداد 1ها را نصف می‌کند، پس حداکثر $1 + \log_2 n$ دور داریم و هر دور $O(n)$ گام دارد. کل زمان $O(n \log n)$ است. (درستی: مرحله ۳ در هر دور، زوج یا فرد بودن تعداد 0ها و 1ها را مقایسه می‌کند. پشت سر هم قرار گرفتن این نتیجه‌ها در دورهای متوالی، دقیقاً نمایش دودویی تعداد 0ها و تعداد 1ها را از کم‌ارزش‌ترین بیت مقایسه می‌کند.)

مثال ۹.۲۲ (با ماشین دونواره: $O(n)$).

M_3 = «روی ورودی w (ماشین دونواره)»

۱. نوار ۱ را طی کن و اگر یک 0 در سمت راست یک 1 پیدا شد، **رد کن**.

۲. 0های نوار ۱ را تا اولین 1 طی کن و همزمان آن‌ها را روی نوار ۲ کپی کن.

۳. 1های نوار ۱ را تا انتهای ورودی طی کن؛ به ازای هر 1 یک 0 را روی نوار ۲ خط بزن. اگر همه 0ها قبل از تمام شدن 1ها خط خوردند، **رد کن**.

۴. اگر همه 0ها خط خورده‌اند **بپذیر**؛ اگر 0ی مانده **رد کن**.

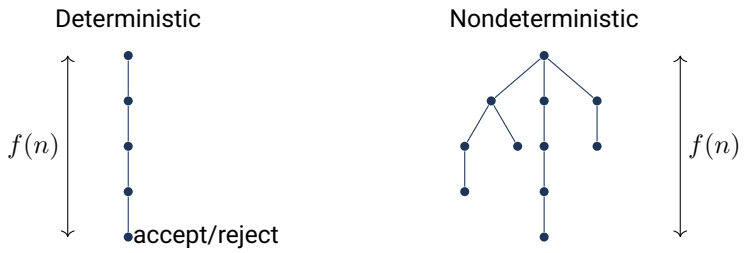
هر مرحله $O(n)$ گام دارد، پس زمان کل $O(n)$ است.

۲.۲۲ رابطه پیچیدگی مدل‌های محاسباتی

طبق تز چرچ–تورینگ همه مدل‌های معقول محاسبه از نظر **تصمیم‌پذیری** هم‌ارزند. اما در نظریه پیچیدگی، انتخاب مدل روی پیچیدگی زمانی زبان اثر دارد؛ مثلاً A را با ماشین تک‌نواره در $O(n \log n)$ و با ماشین دونواره در $O(n)$ تصمیم گرفتیم. سه مدل را مقایسه می‌کنیم: ماشین تک‌نواره، چندنواره و غیرقطعی.

تعریف ۱۰.۲۲ (زمان اجرای ماشین غیرقطعی)

فرض کنید N یک ماشین تورینگ غیرقطعی و تصمیم‌گیرنده باشد. **زمان اجرای** N تابع $f : \mathbb{N} \to \mathbb{N}$ است که $f(n)$ بیشترین تعداد گامی است که N روی **هر شاخه** از محاسبه‌اش روی ورودی‌های به طول n برمی‌دارد.



قضیه ۱۱.۲۲

فرض کنید $t(n) \geq n$. هر ماشین تورینگ چندنواره با زمان $t(n)$ یک ماشین تک‌نواره هم‌ارز با زمان $O(t^2(n))$ دارد.

ایده اثبات. در فصل ۱۷ دیدیم ماشین تک‌نواره چطور ماشین چندنواره را شبیه‌سازی می‌کند. حالا آن شبیه‌سازی را تحلیل می‌کنیم و نشان می‌دهیم شبیه‌سازی هر گام ماشین چندنواره حداکثر $O(t(n))$ گام طول می‌کشد.

اثبات. فرض کنید M یک ماشین k-نواره با زمان $t(n)$ و S شبیه‌ساز تک‌نواره آن (قضیه ۲.۱۷) باشد. M در $t(n)$ گام روی هر نوار حداکثر $t(n)$ خانه را می‌بیند، پس بخش فعال نوار S (که k نوار را کنار هم دارد) طول $O(t(n))$ دارد.

در شبیه‌سازی:

- آماده کردن نوار در ابتدا: $O(n)$ گام.
- برای هر گام M: یک بار طی کردن بخش فعال برای خواندن نمادهای زیر هدها ($O(t(n))$)، یک بار طی کردن برای به‌روزرسانی و حرکت دادن هدها ($O(t(n))$) و حداکثر k بار جابه‌جا کردن محتوا به راست ($O(t(n))$ هر بار). پس هر گام $O(t(n))$.

کل شبیه‌سازی $t(n)$ گام از M را شامل می‌شود: $O(n) + t(n) \times O(t(n)) = O(t^2(n))$، چون $t(n) \geq n$.

■

قضیه ۱۲.۲۲

فرض کنید $t(n) \geq n$. هر ماشین تورینگ غیرقطعی تک‌نواره با زمان $t(n)$ یک ماشین قطعی تک‌نواره هم‌ارز با زمان $2^{O(t(n))}$ دارد.

اثبات. فرض کنید N یک ماشین غیرقطعی با زمان $t(n)$ و D ماشین قطعی‌ای باشد که درخت محاسبه N را سطح-به-سطح جست‌وجو می‌کند (قضیه ۳.۱۷). اگر b بیشترین تعداد انتخاب در تابع گذار N باشد، هر گره درخت حداکثر b فرزند دارد و عمق درخت حداکثر $t(n)$ است. پس تعداد کل گره‌ها $O(b^{t(n)})$ است. D برای رسیدن به هر گره، از ریشه شروع می‌کند و حداکثر $t(n)$ گام برمی‌دارد. پس زمان D (که سه نوار دارد) برابر است با

$$O\big(t(n)\, b^{t(n)}\big) = 2^{O(t(n))}.$$

تبدیل ماشین سه‌نواره D به تک‌نواره، طبق قضیه قبل زمان را حداکثر به توان دو می‌رساند:

$$\big(2^{O(t(n))}\big)^2 = 2^{O(2t(n))} = 2^{O(t(n))}.$$

■

۳.۲۲ کلاس P

دو قضیه بالا یک تفاوت مهم را نشان می‌دهند: تفاوت زمانی بین ماشین تک‌نواره و چندنواره حداکثر **چندجمله‌ای** (توان دوم) است، اما تفاوت بین ماشین قطعی و غیرقطعی (تا جایی که می‌دانیم) **نمایی** است.

- تفاوت چندجمله‌ای کوچک است؛ تفاوت نمایی بزرگ است.
- مثلاً برای $n = 1000$: n^3 یک میلیارد است، اما 2^n از تعداد اتم‌های جهان بیشتر است.
- الگوریتم‌های نمایی به ندرت مفیدند. معمولاً از **جست‌وجوی کامل** (brute force) میان همه جواب‌های ممکن به دست می‌آیند.

همه مدل‌های محاسباتی قطعی معقول **به‌طور چندجمله‌ای هم‌ارز**اند: هر کدام دیگری را با افزایش زمان حداکثر چندجمله‌ای شبیه‌سازی می‌کند. پس روی ویژگی‌هایی از پیچیدگی تمرکز می‌کنیم که با تفاوت‌های چندجمله‌ای عوض نمی‌شوند. به این ترتیب نظریه‌ای داریم که به انتخاب یک مدل خاص وابسته نیست.

تعریف ۱۳.۲۲ (کلاس P)

P کلاس همه زبان‌هایی است که یک ماشین تورینگ قطعی تک‌نواره در **زمان چندجمله‌ای** تصمیمشان می‌گیرد:

$$\mathrm{P} = \bigcup_k \mathrm{TIME}(n^k).$$

- P برای همه مدل‌های محاسباتی‌ای که به‌طور چندجمله‌ای با ماشین تورینگ قطعی تک‌نواره هم‌ارزند، یکسان است. پس P از نظر ریاضی کلاسی **مستحکم** است.
- P تقریباً همان کلاس مسئله‌هایی است که در عمل روی یک کامپیوتر قابل حل‌اند.

قضیه ۱۴.۲۲

مسئله مسیر در گراف جهت‌دار در P است:

$$PATH = \{\langle G, s, t\rangle \mid G \text{ گرافی جهت‌دار است که مسیری جهت‌دار از } s \text{ به } t \text{ دارد}\} \in \mathrm{P}.$$

جست‌وجوی کامل کافی نیست: بررسی همه مسیرهای ممکن در گرافی با m رأس کارآمد نیست؛ تعداد مسیرها تقریباً m^m است.

اثبات. الگوریتم چندجمله‌ای (جست‌وجوی سطح-به-سطح):

> **M = «روی ورودی $\langle G, s, t\rangle$ که G گراف جهت‌دار با رأس‌های s و t است»**
>
> ۱. رأس s را علامت بزن.
>
> ۲. تا وقتی رأس جدیدی علامت نخورد، تکرار کن:
>
> ۳. همه یال‌های G را بررسی کن. اگر یالی مثل (a, b) از یک رأس علامت‌خورده a به یک رأس علامت‌نخورده b پیدا شد، b را علامت بزن.
>
> ۴. اگر t علامت خورده، **بپذیر**؛ وگرنه **رد کن**.

مراحل ۱ و ۴ فقط یک بار اجرا می‌شوند. مرحله ۳ حداکثر m بار اجرا می‌شود (هر بار، جز آخرین، دست‌کم یک رأس علامت می‌خورد). هر مرحله را می‌توان در زمان چندجمله‌ای بر حسب اندازه ورودی پیاده‌سازی کرد. پس M الگوریتمی چندجمله‌ای است.

■

جمع‌بندی فصل

- زمان اجرا: بیشترین تعداد گام روی ورودی‌های به طول n (بدترین حالت).
- $f = O(g)$: $f \le cg$ برای nهای بزرگ. $f = o(g)$: $f/g \to 0$.
- $\mathrm{TIME}(t(n))$: زبان‌های قابل تصمیم در زمان $O(t(n))$. $\{0^k1^k\}$ در $O(n^2)$، $O(n \log n)$ و با دو نوار در $O(n)$.
- تبدیل ماشین چندنواره به تک‌نواره زمان را از t به $O(t^2)$ و تبدیل ماشین غیرقطعی به قطعی آن را به $2^{O(t)}$ می‌رساند.

- $\mathrm{P} = \bigcup_k \mathrm{TIME}(n^k)$؛ مستحکم و نماینده مسئله‌های عملاً قابل حل. $PATH \in \mathrm{P}$.

فصل ۲۳

کلاس NP

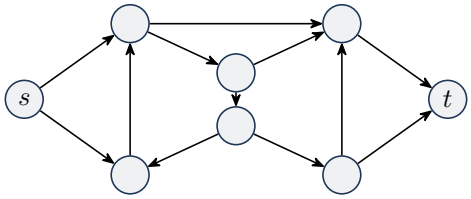
برای مسئله $PATH$ توانستیم با دوری از جست‌وجوی کامل، الگوریتمی چندجمله‌ای پیدا کنیم. اما برای بعضی مسئله‌های دیگر، که بسیاری از آن‌ها جالب و کاربردی هم هستند، تا امروز هیچ الگوریتم چندجمله‌ای‌ای پیدا نشده است.

۱.۲۳ یک مثال: مسیر همیلتونی

تعریف ۱.۲۳ (HAMPATH)

یک **مسیر همیلتونی** در گراف جهت‌دار G مسیری جهت‌دار است که از هر رأس **دقیقاً یک بار** عبور کند.

$HAMPATH = \{\langle G, s, t\rangle \mid G$ گرافی جهت‌دار با یک مسیر همیلتونی از s به t است$\}$.



در این گراف مسیر همیلتونی‌ای از s به t وجود دارد (آن را پیدا کنید!).

ما نمی‌دانیم چرا نتوانسته‌ایم برای بعضی مسئله‌ها الگوریتم چندجمله‌ای پیدا کنیم. شاید چنین الگوریتم‌هایی وجود دارند و هنوز پیدا نشده‌اند؛ شاید هم این مسئله‌ها ذاتاً سخت‌اند. اما یک نکته مهم را می‌دانیم: پیچیدگی بسیاری از این مسئله‌ها به هم **وابسته** است؛ یک الگوریتم چندجمله‌ای برای یکی از آن‌ها می‌تواند کل یک کلاس از مسئله‌ها را حل کند.

این مسئله‌ها یک ویژگی مشترک دارند که **وارسی‌پذیری چندجمله‌ای** نام دارد: پیدا کردن جواب شاید سخت باشد، اما **بررسی** یک جواب پیشنهادی آسان است. مثلاً اگر کسی یک مسیر همیلتونی از s به t به ما بدهد، به‌راحتی (در زمان چندجمله‌ای) می‌توانیم بررسی کنیم که درست است.

۲.۲۳ وارسی‌کننده و کلاس NP

تعریف ۲.۲۳ (وارسی‌کننده)

یک **وارسی‌کننده** (Verifier) برای زبان A الگوریتمی مثل V است که

$$A = \{w \mid V \text{ رشته } \langle w, c\rangle \text{ را برای یک رشته } c \text{ می‌پذیرد}\}.$$

زمان وارسی‌کننده را فقط بر حسب طول w می‌سنجیم؛ پس یک **وارسی‌کننده با زمان چندجمله‌ای** در زمان چندجمله‌ای بر حسب طول w اجرا می‌شود. زبان A **وارسی‌پذیر چندجمله‌ای** است اگر یک وارسی‌کننده با زمان چندجمله‌ای داشته باشد.

- به c یک **گواه** (certificate) یا **اثبات عضویت** در A می‌گوییم.
- چون وارسی‌کننده در زمان چندجمله‌ای اجرا می‌شود، فقط می‌تواند بخشی چندجمله‌ای از گواه را بخواند؛ پس می‌توان فرض کرد طول گواه چندجمله‌ای (بر حسب طول w) است.
- برای $HAMPATH$ گواه همان مسیر همیلتونی از s به t است.

نکته. بعضی مسئله‌ها ممکن است وارسی‌پذیر چندجمله‌ای نباشند. مثلاً متمم مسئله، $\overline{HAMPATH}$: حتی اگر بدانیم گراف مسیر همیلتونی **ندارد**، نمی‌دانیم چطور بدون یک الگوریتم نمایی این را به کسی ثابت کنیم؛ باید **نبودن** چیزی را وارسی کرد.

تعریف ۳.۲۳ (کلاس NP)

NP کلاس زبان‌هایی است که **وارسی‌کننده با زمان چندجمله‌ای** دارند.

نام NP از **زمان چندجمله‌ای غیرقطعی** (Nondeterministic Polynomial time) آمده است. دلیلش این است که روش دیگری برای توصیف همین کلاس وجود دارد: مسئله‌هایی که با یک ماشین تورینگ **غیرقطعی** در زمان چندجمله‌ای حل می‌شوند.

مثال ۴.۲۳ (HAMPATH با ماشین غیرقطعی).

N_1 = «روی ورودی $\langle G, s, t\rangle$ که G گرافی جهت‌دار با رأس‌های s و t است»

۱. فهرستی از m عدد $p_1, \ldots, p_m$ بنویس که m تعداد رأس‌های G است. هر عدد به‌طور غیرقطعی بین 1 و m انتخاب می‌شود.

۲. اگر عدد تکراری در فهرست بود، **رد کن**.

۳. اگر $s \neq p_1$ یا $t \neq p_m$، **رد کن**.

۴. برای هر i بین 1 و $m-1$ بررسی کن که (p_i, p_{i+1}) یالی از G است. اگر نبود **رد کن**؛ اگر همه بودند **بپذیر**.

هر مرحله در زمان چندجمله‌ای اجرا می‌شود (مرحله ۱ در هر شاخه $O(m \log m)$ و بقیه هم چندجمله‌ای)، پس N_1 یک ماشین غیرقطعی با زمان چندجمله‌ای است.

قضیه ۵.۲۳

یک زبان در NP است اگر و تنها اگر یک ماشین تورینگ غیرقطعی با زمان چندجمله‌ای آن را تصمیم بگیرد.

ایده اثبات. یک وارسی‌کننده با زمان چندجمله‌ای را به یک ماشین غیرقطعی با زمان چندجمله‌ای تبدیل می‌کنیم و برعکس. ماشین غیرقطعی وارسی‌کننده را با **حدس زدن گواه** شبیه‌سازی می‌کند. وارسی‌کننده هم ماشین غیرقطعی را با استفاده از **شاخه پذیرنده به عنوان گواه** شبیه‌سازی می‌کند.

اثبات. **از وارسی‌کننده به ماشین غیرقطعی:** فرض کنید $A \in \mathrm{NP}$ و V یک وارسی‌کننده با زمان n^k برای A باشد. ماشین N:

> **N = «روی ورودی w به طول n»**
>
> ۱. به‌طور غیرقطعی رشته‌ای مثل c با طول حداکثر n^k انتخاب کن.
>
> ۲. V را روی ورودی $\langle w, c\rangle$ اجرا کن.
>
> ۳. اگر V پذیرفت **بپذیر**؛ وگرنه **رد کن**.

V در زمان n^k حداکثر n^k نماد از گواه را می‌خواند، پس محدود کردن طول c به n^k چیزی را از دست نمی‌دهد. N رشته w را می‌پذیرد اگر و تنها اگر گواهی مثل c باشد که V بپذیرد، یعنی اگر و تنها اگر $w \in A$. هر شاخه N در زمان چندجمله‌ای تمام می‌شود.

از ماشین غیرقطعی به وارسی‌کننده: فرض کنید ماشین غیرقطعی N در زمان چندجمله‌ای A را تصمیم بگیرد. وارسی‌کننده V:

> **V = «روی ورودی $\langle w, c\rangle$ که w و c رشته هستند»**
>
> ۱. N را روی ورودی w شبیه‌سازی کن و هر نماد c را توصیف انتخاب غیرقطعی در یک گام در نظر بگیر (مثل آدرس‌دهی گره‌ها در اثبات قضیه ۳.۱۷).
>
> ۲. اگر این شاخه از محاسبه N پذیرفت **بپذیر**؛ وگرنه **رد کن**.

اگر $w \in A$، شاخه پذیرنده‌ای وجود دارد و وقتی گواه درست (انتخاب‌های آن شاخه) داده شود، V دقیقاً آن شاخه را اجرا می‌کند و می‌پذیرد. اگر $w \notin A$، هیچ شاخه‌ای نمی‌پذیرد، پس V برای هیچ cای نمی‌پذیرد. V فقط یک شاخه را اجرا می‌کند، پس در زمان چندجمله‌ای است.

■

تعریف ۶.۲۳ (کلاس زمانی غیرقطعی)

$$\mathrm{NTIME}(t(n)) = \{L \mid L \text{ زبانی است که یک ماشین تورینگ غیرقطعی با زمان } O(t(n)) \text{ تصمیمش می‌گیرد}\}.$$

نتیجه ۷.۲۳

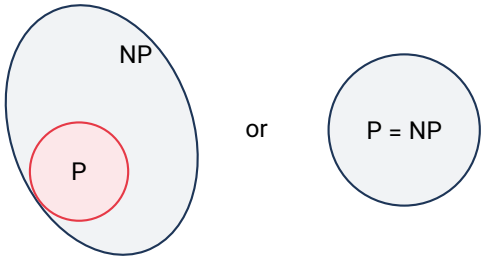
$$\mathrm{NP} = \bigcup_k \mathrm{NTIME}(n^k).$$

۳.۲۳ پرسش P در برابر NP

به زبان ساده:

- P = کلاس زبان‌هایی که عضویت در آن‌ها را می‌توان به‌سرعت **تصمیم گرفت**.
- NP = کلاس زبان‌هایی که عضویت در آن‌ها را می‌توان به‌سرعت **وارسی کرد**.

روشن است که $\mathrm{P} \subseteq \mathrm{NP}$: اگر بتوانیم جواب را سریع پیدا کنیم، بدون نیاز به گواه هم می‌توانیم آن را سریع بررسی کنیم. به نظر می‌رسد وارسی‌پذیری چندجمله‌ای خیلی قوی‌تر از تصمیم‌پذیری چندجمله‌ای باشد. اما هیچ‌کس تا امروز نتوانسته **ثابت کند** مسئله‌ای وجود دارد که وارسی‌کننده چندجمله‌ای دارد ولی تصمیم‌گیرنده چندجمله‌ای ندارد.



پرسش «آیا $\mathrm{P} = \mathrm{NP}$؟» یکی از بزرگ‌ترین مسئله‌های حل‌نشده علوم کامپیوتر نظری است. بیشتر پژوهشگران باور دارند $\mathrm{P} \neq \mathrm{NP}$. این مسئله یکی از هفت «مسئله هزاره» مؤسسه ریاضی کلی است که برای حل هر کدام یک میلیون دلار جایزه گذاشته شده است:

- حدس برچ و سوینرتون-دایر،
- حدس هاج،
- وجود و هموواری جواب معادلات ناویر–استوکس،
- **مسئله P در برابر NP**،
- فرضیه ریمان،
- وجود نظریه یانگ–میلز و شکاف جرم،
- حدس پوانکاره (که تنها مسئله حل‌شده این فهرست است).

جمع‌بندی فصل

- وارسی‌کننده V: $w \in A$ اگر و تنها اگر گواهی c باشد که $\langle w, c \rangle$ رشته V را بپذیرد.
- NP: زبان‌های دارای وارسی‌کننده چندجمله‌ای $=$ زبان‌هایی که ماشین غیرقطعی در زمان چندجمله‌ای تصمیمشان می‌گیرد $= \bigcup_k \mathrm{NTIME}(n^k)$.
- $HAMPATH \in \mathrm{NP}$ (گواه: خود مسیر). $\mathrm{P} \subseteq \mathrm{NP}$ و پرسش $\mathrm{P} = \mathrm{NP}$ هنوز باز است.

فصل ۲۴

تمامیت-NP

استیون کوک و لئونید لوین در اوایل دهه ۱۹۷۰ به‌طور مستقل کشف کردند که بعضی مسئله‌های NP پیچیدگی‌شان به پیچیدگی **کل** کلاس NP وابسته است: اگر برای **یکی** از این مسئله‌ها الگوریتم چندجمله‌ای وجود داشته باشد، **همه** مسئله‌های NP در زمان چندجمله‌ای حل می‌شوند. به این مسئله‌ها **تمام-NP** (NP-complete) می‌گوییم.

این پدیده از دو جهت مهم است:

- **نظری:** پژوهشگری که می‌خواهد ثابت کند $\mathrm{P} = \mathrm{NP}$، فقط کافی است برای یک مسئله تمام-NP الگوریتم چندجمله‌ای پیدا کند.
- **عملی:** تمام-NP بودن یک مسئله شاهد محکمی است که الگوریتم چندجمله‌ای ندارد. پس به جای اتلاف وقت برای پیدا کردن چنین الگوریتمی، بهتر است سراغ راه‌حل‌های تقریبی یا حالت‌های خاص برویم.

۱.۲۴ کاهش‌پذیری در زمان چندجمله‌ای

تعریف ۱.۲۴ (تابع محاسبه‌پذیر در زمان چندجمله‌ای)

تابع $f : \Sigma^* \to \Sigma^*$ **محاسبه‌پذیر در زمان چندجمله‌ای** است اگر یک ماشین تورینگ با زمان چندجمله‌ای مثل M وجود داشته باشد که روی هر ورودی w با فقط $f(w)$ روی نوارش متوقف شود.

تعریف ۲.۲۴ (کاهش چندجمله‌ای)

زبان A **به‌طور چندجمله‌ای کاهش‌پذیر** به زبان B است و می‌نویسیم $A \leq_P B$، اگر تابعی محاسبه‌پذیر در زمان چندجمله‌ای مثل $f : \Sigma^* \to \Sigma^*$ وجود داشته باشد که برای هر w:

$$w \in A \iff f(w) \in B.$$

به f **کاهش چندجمله‌ای** از A به B می‌گوییم.

کاهش چندجمله‌ای همان کاهش نگاشتی فصل ۲۱ است، با این شرط اضافه که تبدیل باید **کارآمد** باشد. این کاهش آزمون عضویت در A را به آزمون عضویت در B تبدیل می‌کند.

قضیه ۳.۲۴

اگر $A \leq_P B$ و $B \in \mathrm{P}$، آنگاه $A \in \mathrm{P}$.

اثبات. فرض کنید M الگوریتمی چندجمله‌ای برای B و f کاهش چندجمله‌ای از A به B باشد. الگوریتم N برای A:

> **N = «روی ورودی w»**
>
> ۱. $f(w)$ را محاسبه کن.
>
> ۲. M را روی $f(w)$ اجرا کن و همان خروجی را بده.

اگر $w \in A$ آنگاه $f(w) \in B$ و M می‌پذیرد؛ اگر $w \notin A$ آنگاه $f(w) \notin B$ و M رد می‌کند. زمان: مرحله ۱ چندجمله‌ای است. چون f در زمان چندجمله‌ای محاسبه می‌شود، طول $f(w)$ هم چندجمله‌ای بر حسب $|w|$ است (ماشین در زمان t حداکثر t نماد می‌نویسد). M روی ورودی‌ای با طول چندجمله‌ای در زمان چندجمله‌ای اجرا می‌شود و ترکیب دو چندجمله‌ای هم چندجمله‌ای است. پس N در زمان چندجمله‌ای اجرا می‌شود.

■

۲.۲۴ تمامیت-NP

تعریف ۴.۲۴ (تمام-NP)

زبان B **تمام-NP** است اگر دو شرط را داشته باشد:

۱. $B \in \mathrm{NP}$، و

۲. برای هر $A \in \mathrm{NP}$: $A \leq_P B$.

قضیه ۵.۲۴

اگر B تمام-NP باشد و $B \in \mathrm{P}$، آنگاه $\mathrm{P} = \mathrm{NP}$.

اثبات. هر $A \in \mathrm{NP}$ طبق شرط ۲ به‌طور چندجمله‌ای به B کاهش می‌یابد و طبق قضیه ۱.۲۴، $A \in \mathrm{P}$. پس $\mathrm{NP} \subseteq \mathrm{P}$ و چون $\mathrm{P} \subseteq \mathrm{NP}$، برابرند.

■

قضیه ۶.۲۴

اگر B تمام-NP باشد و $B \leq_P C$ برای یک $C \in \mathrm{NP}$، آنگاه C هم تمام-NP است.

اثبات. $C \in \mathrm{NP}$ را داریم؛ باید نشان دهیم هر $A \in \mathrm{NP}$ به‌طور چندجمله‌ای به C کاهش می‌یابد. چون B تمام-NP است، $A \leq_P B$ با کاهشی مثل f. همچنین $B \leq_P C$ با کاهشی مثل g. کاهش‌های چندجمله‌ای قابل ترکیب‌اند: تابع $g \circ f$ (اول f، بعد g) در زمان چندجمله‌ای محاسبه می‌شود (چون طول $f(w)$ چندجمله‌ای است) و

$$w \in A \iff f(w) \in B \iff g(f(w)) \in C.$$

پس $A \leq_P C$.

■

این قضیه راه اصلی اثبات تمام-NP بودن است: کافی است یک مسئله تمام-NP شناخته‌شده را به مسئله جدید کاهش دهیم. اما برای شروع به **اولین** مسئله تمام-NP نیاز داریم.

۳.۲۴ مسئله صدق‌پذیری

اولین مسئله ،تمام-NP مسئله **صدق‌پذیری** است. **متغیرهای بولی** مقدار 0 یا 1 می‌گیرند و با عملگرهای AND ($\wedge$)، OR ($\vee$) و NOT (که با خط بالای متغیر نشان می‌دهیم: $\overline{x}$) ترکیب می‌شوند. یک **فرمول بولی** عبارتی از متغیرهای بولی و این عملگرهاست، مثلاً

$$\phi = (\overline{x} \wedge y) \vee (x \wedge \overline{z}).$$

فرمول بولی **صدق‌پذیر** است اگر مقداردهی‌ای از 0 و 1 به متغیرها وجود داشته باشد که مقدار فرمول را 1 کند. مثلاً ϕ با $x = 0$، $y = 1$ و $z = 0$ صدق‌پذیر است.

$$SAT = \{\langle \phi \rangle \mid \phi \text{ است یک فرمول بولی صدق‌پذیر}\}.$$

قضیه ۷.۲۴ (کوک–لوین)

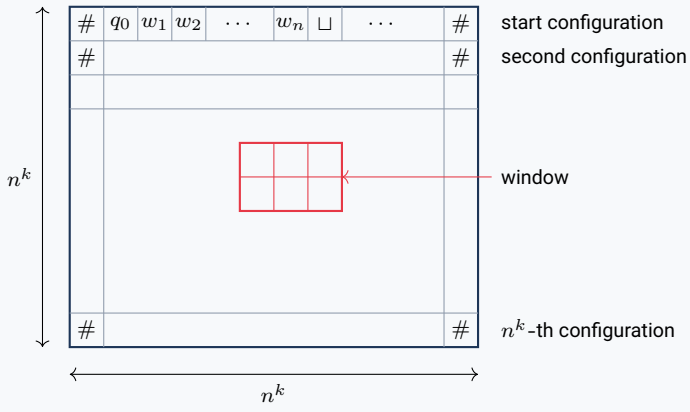
SAT تمام-NP است.

ایده اثبات.

- $SAT \in \text{NP}$ ساده است: گواه یک مقداردهی صدق‌کننده است که در زمان چندجمله‌ای بررسی می‌شود.
- بخش اصلی: باید نشان دهیم **هر** زبان $A \in \text{NP}$ به‌طور چندجمله‌ای به SAT کاهش می‌یابد. فرض کنید N یک ماشین غیرقطعی با زمان چندجمله‌ای برای A باشد. برای هر رشته w یک فرمول بولی ϕ می‌سازیم که **محاسبه N روی w را شبیه‌سازی کند**: اگر N بپذیرد، ϕ صدق‌پذیر است و اگر N رد کند، نیست.

اثبات. $SAT \in \text{NP}$: ماشین غیرقطعی یک مقداردهی به متغیرها حدس می‌زند و فرمول را با آن ارزیابی می‌کند؛ همه این کارها در زمان چندجمله‌ای انجام می‌شود.

ساخت کاهش. فرض کنید N یک ماشین غیرقطعی باشد که A را در زمان n^k تصمیم می‌گیرد (برای سادگی زمان را n^k می‌گیریم؛ با یک ثابت کار تغییر نمی‌کند). یک **جدول** (tableau) برای N روی w، جدولی $n^k \times n^k$ است که سطرهایش پیکربندی‌های یک شاخه از محاسبه N روی w هستند: سطر اول پیکربندی شروع و هر سطر پیکربندی بعدی سطر قبل است. برای راحتی هر سطر را با نماد $\#$ شروع و تمام می‌کنیم.



هر جدول پذیرنده متناظر با یک شاخه پذیرنده از محاسبه N روی w است (جدولی که یکی از سطرهایش پیکربندی پذیرش باشد). پس مسئله «آیا N رشته w را می‌پذیرد؟» معادل این است که «آیا یک جدول پذیرنده برای N روی w وجود دارد؟».

متغیرها. فرض کنید Q و Γ مجموعه حالت‌ها و الفبای نوار N باشند و $C = Q \cup \Gamma \cup \{\#\}$. برای هر خانه (i,j) جدول و هر نماد $s \in C$ یک متغیر بولی $x_{i,j,s}$ تعریف می‌کنیم با این معنی:

$$x_{i,j,s} = 1 \iff \text{خانه } (i,j) \text{ جدول نماد } s \text{ را دارد}.$$

فرمول ϕ از چهار بخش تشکیل می‌شود:

$$\phi = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}.$$

۱. هر خانه دقیقاً یک نماد دارد:

$$\phi_{\text{cell}} = \bigwedge_{1 \le i,j \le n^k} \left[\left(\bigvee_{s \in C} x_{i,j,s} \right) \wedge \left(\bigwedge_{\substack{s,t \in C \\ s \neq t}} (\overline{x_{i,j,s}} \vee \overline{x_{i,j,t}}) \right) \right].$$

بخش اول داخل کروشه می‌گوید «دست‌کم یک نماد در خانه هست» و بخش دوم «بیش از یک نماد نیست». این شرط برای هر خانه جدول برقرار است.

۲. سطر اول پیکربندی شروع است:

$$\phi_{\text{start}} = x_{1,1,\#} \wedge x_{1,2,q_0} \wedge x_{1,3,w_1} \wedge x_{1,4,w_2} \wedge \cdots \wedge x_{1,n+2,w_n} \wedge x_{1,n+3,\sqcup} \wedge \cdots \wedge x_{1,n^k-1,\sqcup} \wedge x_{1,n^k,\#}.$$

۳. جدول پیکربندی پذیرش دارد:

$$\phi_{\text{accept}} = \bigvee_{1 \le i,j \le n^k} x_{i,j,q_{\text{accept}}}.$$

۴. هر سطر به‌طور مجاز از سطر قبل نتیجه می‌شود. این را با بررسی **پنجره‌های** 2×3 از خانه‌ها انجام می‌دهیم. یک پنجره **مجاز** است اگر با تابع گذار N تناقض نداشته باشد. مثلاً اگر $\delta(q_1,a) = \{(q_1,b,R)\}$ و $\delta(q_1,b) = \{(q_2,c,L),(q_2,a,R)\}$، پنجره‌های زیر مجازند:

(a)

a	q_1	b
q_2	a	c

(b)

a	q_1	b
a	a	q_2

(c)

a	a	q_1
a	a	b

(d)

$\#$	b	a
$\#$	b	a

(e)

a	b	a
a	b	q_2

(f)

b	b	b
c	b	b

(در (a) و (b) گذار $\delta(q_1,b)$ به دو روش مختلف اجرا شده است. در (c) حالت q_1 در لبه راست پنجره است و نمادی که می‌خواند بیرون پنجره است؛ اگر آن نماد a باشد، با b عوض می‌شود و هد به راست می‌رود، پس جای q_1 نماد b می‌نشیند؛ در (e) هد از سمت راست وارد پنجره شده؛ در (f) هد کنار پنجره است و نماد زیر آن عوض شده.) و پنجره‌های زیر مجاز نیستند:

(a)

a	b	a
a	a	a

(b)

a	q_1	b
q_2	a	a

(c)

b	q_1	b
q_2	b	q_2

(در (a) نماد وسط بدون حضور هد عوض شده؛ در (b) گذار با تابع گذار نمی‌خواند؛ در (c) دو حالت در یک سطر آمده است.)

می‌توان ثابت کرد که اگر سطر اول پیکربندی شروع باشد و **همه** پنجره‌های جدول مجاز باشند، هر سطر پیکربندی‌ای است که به‌طور مجاز از سطر قبلی‌اش نتیجه می‌شود. (دلیل: هر خانه‌ای در سطر بالا که کنار هد نیست، در پنجره‌ای قرار دارد که آن را به‌طور تغییرنیافته به سطر پایین منتقل می‌کند، و پنجره‌هایی که هد را دارند، دقیقاً مطابق یک گذار N به‌روز می‌شوند.) پنجره (i,j) را پنجره‌ای می‌گیریم که خانه (i,j) وسط سطر بالای آن است:

$$\phi_{\text{move}} = \bigwedge_{1\le i<n^k,\ 1<j<n^k} (\text{پنجره } (i,j) \text{ مجاز است}),$$

که «پنجره (i,j) مجاز است» با فرمول زیر بیان می‌شود:

$$\bigvee_{\substack{a_1,\dots,a_6 \\ \text{پنجره مجاز}}} (x_{i,j-1,a_1} \wedge x_{i,j,a_2} \wedge x_{i,j+1,a_3} \wedge x_{i+1,j-1,a_4} \wedge x_{i+1,j,a_5} \wedge x_{i+1,j+1,a_6}).$$

درستی. اگر N رشته w را بپذیرد، جدول پذیرنده‌ای هست و مقداردهی متناظر با آن همه چهار بخش را صادق می‌کند. برعکس، هر مقداردهی صدق‌کننده ϕ یک جدول را مشخص می‌کند (طبق ϕ_{cell})، که با پیکربندی شروع آغاز می‌شود (ϕ_{start})، هر سطرش به‌طور مجاز از قبلی نتیجه می‌شود (ϕ_{move}) و به پذیرش می‌رسد (ϕ_{accept}). پس N رشته w را می‌پذیرد.

زمان کاهش. اندازه ϕ را تخمین می‌زنیم.

- جدول $n^k \times n^k = n^{2k}$ خانه دارد و هر خانه $l = |C|$ متغیر دارد. l به N بستگی دارد و نه به ورودی، پس ثابت است. تعداد کل متغیرها $O(n^{2k})$ است.
- ϕ_{cell}: برای هر خانه بخشی با اندازه ثابت؛ کل $O(n^{2k})$.
- ϕ_{start}: برای هر خانه سطر اول بخشی ثابت؛ کل $O(n^k)$.
- ϕ_{accept} و ϕ_{move}: برای هر خانه بخشی ثابت (تعداد پنجره‌های مجاز ثابت است)؛ کل $O(n^{2k})$.
- هر متغیر یک اندیس دارد که با $O(\log n)$ نماد نوشته می‌شود؛ پس طول کل ϕ برابر $O(n^{2k}\log n)$ است.

ϕ ساختار بسیار منظمی دارد و می‌توان آن را به‌سادگی در زمانی متناسب با طولش تولید کرد. پس کاهش چندجمله‌ای است و $A \le_P SAT$.

■

۴.۲۴ ۳SAT

تعریف ۸.۲۴ (فرمول cnf)

- یک **لیترال** یک متغیر بولی یا نقیض آن است، مثل x یا $\overline{x}$.
- یک **بند** (clause) چند لیترال است که با $\vee$ به هم وصل شده‌اند، مثل $(x_1 \vee \overline{x_2} \vee \overline{x_3} \vee x_4)$.
- یک فرمول بولی در **فرم نرمال عطفی** (cnf) است اگر از چند بند تشکیل شده باشد که با $\wedge$ به هم وصل شده‌اند:

$$(x_1 \vee \overline{x_2} \vee \overline{x_3} \vee x_4) \wedge (x_3 \vee \overline{x_5} \vee x_6) \wedge (x_3 \vee \overline{x_6}).$$

- یک **فرمول ۳cnf** فرمولی cnf است که هر بندش دقیقاً سه لیترال دارد:

$$(x_1 \vee \overline{x_2} \vee \overline{x_3}) \wedge (x_3 \vee \overline{x_5} \vee x_6) \wedge (x_3 \vee \overline{x_6} \vee x_4) \wedge (x_4 \vee x_5 \vee x_6).$$

$3SAT = \{\langle \phi \rangle \mid \phi$ یک فرمول ۳cnf صدق‌پذیر است$\}$.

قضیه ۹.۲۴

$3SAT$ تمام-NP است.

اثبات. $3SAT \in \mathrm{NP}$ دقیقاً مثل SAT است. برای بخش دوم، کاهش کوک–لوین را طوری تغییر می‌دهیم که مستقیماً یک فرمول ۳cnf بسازد.

گام ۱: رساندن به فرم cnf. ϕ_{cell}، ϕ_{start} و ϕ_{accept} از قبل cnf هستند (ϕ_{accept} یک بند بزرگ است و ϕ_{start} عطف بندهای تک‌لیترالی). ϕ_{move} عطفی از فرمول‌های «پنجره (i, j) مجاز است» است که هر کدام **فصل عطف‌ها** با اندازه ثابت است. هر فرمول با اندازه ثابت را می‌توان با قوانین پخش‌پذیری به cnf با اندازه ثابت (شاید بزرگ‌تر، اما همچنان ثابت و مستقل از n) تبدیل کرد. پس کل فرمول cnf می‌شود و اندازه‌اش فقط در یک ضریب ثابت بزرگ می‌شود.

گام ۲: رساندن هر بند به دقیقاً سه لیترال.

- **بندهای کوتاه:** یک لیترال را تکرار می‌کنیم: $(a_1) \to (a_1 \vee a_1 \vee a_1)$ و $(a_1 \vee a_2) \to (a_1 \vee a_2 \vee a_2)$. این کار معنی بند را تغییر نمی‌دهد.
- **بندهای بلند:** بند $(a_1 \vee a_2 \vee \cdots \vee a_l)$ با $l > 3$ را با متغیرهای کمکی جدید $z_1, \ldots, z_{l-3}$ به زنجیره زیر تبدیل می‌کنیم:

$$(a_1 \vee a_2 \vee z_1) \wedge (\overline{z_1} \vee a_3 \vee z_2) \wedge (\overline{z_2} \vee a_4 \vee z_3) \wedge \cdots \wedge (\overline{z_{l-3}} \vee a_{l-1} \vee a_l).$$

چرا تبدیل بندهای بلند صدق‌پذیری را حفظ می‌کند؟ اگر یک مقداردهی بند اصلی را صادق کند، یک لیترال a_r مقدار 1 دارد. قرار دهید $z_1 = \cdots = z_{r-2} = 1$ و بقیه zها 0 =. آنگاه بندهای قبل از بند شامل a_r با z مثبتشان، بند شامل a_r با خود a_r و بندهای بعد از آن با $\overline{z}$ شان صادق می‌شوند. برعکس، اگر همه a_iها 0 باشند، زنجیره نمی‌تواند صادق باشد: بند اول $z_1 = 1$ را لازم دارد، بند دوم سپس $z_2 = 1$ و ... تا بند آخر که با $z_{l-3} = 1$ و $a_{l-1} = a_l = 0$ کاذب می‌شود. پس زنجیره صدق‌پذیر است اگر و تنها اگر بند اصلی صدق‌پذیر باشد.

هر دو گام در زمان چندجمله‌ای انجام می‌شوند و اندازه فرمول را فقط چندجمله‌ای زیاد می‌کنند. پس هر زبان NP به‌طور چندجمله‌ای به $3SAT$ کاهش می‌یابد و $3SAT$ تمام-NP است.

■

نکته. با داشتن $3SAT$ و قضیه ۲.۲۴، تمام-NP بودن مسئله‌های بسیار زیادی (مثل CLIQUE، VERTEX-COVER، HAMPATH و SUBSET-SUM) با کاهش از $3SAT$ ثابت می‌شود. امروز هزاران مسئله تمام-NP می‌شناسیم.

جمع‌بندی فصل

- $A \leq_P B$: کاهش نگاشتی با تابعی که در زمان چندجمله‌ای محاسبه می‌شود. اگر $B \in \mathrm{P}$ آنگاه $A \in \mathrm{P}$.
- تمام-NP: در NP است و همه NP به آن کاهش می‌یابد. اگر یکی در P باشد، $\mathrm{P} = \mathrm{NP}$.

- کوک–لوین: SAT تمام-NP است؛ جدول $n^k \times n^k$ و فرمول $\phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$ با پنجره‌های 2×3.
- $3SAT$ هم تمام-NP است؛ با تبدیل به cnf و شکستن بندهای بلند با متغیرهای کمکی.

«از آغاز شروع کن و ادامه بده تا به پایان برسی؛ آنجا بایست.»

— لوئیس کارول، آلیس در سرزمین عجایب

واژه‌نامه

معادل انگلیسی اصطلاح‌های اصلی این جزوه، به ترتیب الفبای فارسی.

اشتقاق — Derivation

اشتقاق چپ‌ترین — Leftmost derivation

اصل لانه کبوتری — Pigeonhole principle

الحاق — Concatenation

الفبا — Alphabet

بازگشتی شمارش‌پذیر — Recursively enumerable

بستار اپسیلون — Epsilon closure

بستاری — Closure

بند — Clause

پایانه — Terminal

پسوند — Suffix

پشته — Stack

پنجره — Window

پیچیدگی زمانی — Time complexity

پیش‌بینی — Lookahead

پیشوند — Prefix

پیکربندی — Configuration

تابع گذار — Transition function

تابع محاسبه‌پذیر — Computable function

تاریخچه محاسبه — Computation history

تجزیه — Parsing

تحلیل مجانبی — Asymptotic analysis

تز چرچ–تورینگ — Church–Turing thesis

تصمیم‌پذیر — Decidable

تصمیم‌گیرنده — Decider

تطابق — Match

تورینگ-تشخیص‌پذیر — Turing-recognizable

جدول (در اثبات کوک–لوین) — Tableau

حالت — State

حالت پذیرش — Accept state

حالت شروع — Start state

خارج‌قسمت — Quotient

درخت تجزیه — Parse tree

دنباله مشخصه — Characteristic sequence

ذاتاً مبهم — Inherently ambiguous

رشته — String

رشته تهی — Empty string

زبان — Language

زبان مستقل از متن — Context-free language (CFL)

زبان مستقل از متن قطعی — Deterministic CFL (DCFL)

زبان منظم — Regular language

زمان اجرا — Running time

زمان چندجمله‌ای — Polynomial time

ساخت زیرمجموعه‌ای — Subset construction

ساخت ضربی — Product construction

ستاره کلینی — Kleene star

شمارا — Countable

شمارنده — Enumerator

صدق‌پذیری — Satisfiability

صورت جمله‌ای — Sentential form

طول تزریق — Pumping length

عبارت منظم — Regular expression

عضویت — Membership

عملیات منظم — Regular operations

فرم نرمال چامسکی — Chomsky normal form (CNF)

فرم نرمال عطفی — Conjunctive normal form

فرم نرمال گرایباخ — Greibach normal form (GNF)

قاعده — Rule / Production

قاعده واحد — Unit rule

قطری‌سازی — Diagonalization

کاهش — Reduction

کاهش چندجمله‌ای — Polynomial time reduction

کاهش نگاشتی — Mapping reducibility

کدگذاری — Encoding

گرامر — Grammar

گرامر خطی چپ — Left-linear grammar

گرامر خطی راست Right-linear grammar

گرامر ساده Simple grammar (s-grammar)

گرامر مستقل از متن Context-free grammar (CFG)

گواه Certificate

لم تزریق Pumping lemma

لیترال Literal

ماشین پشته‌ای Pushdown automaton (PDA)

ماشین پشته‌ای قطعی Deterministic PDA (DPDA)

ماشین تورینگ Turing machine

ماشین تورینگ جهانی Universal Turing machine

ماشین چندنواره Multitape Turing machine

ماشین خطی‌کران‌دار Linear bounded automaton (LBA)

ماشین متناهی Finite automaton

ماشین متناهی غیرقطعی Nondeterministic finite automaton (NFA)

ماشین متناهی غیرقطعی تعمیم‌یافته Generalized NFA (GNFA)

ماشین متناهی قطعی Deterministic finite automaton (DFA)

مبهم Ambiguous

متغیر Variable

متغیر بی‌فایده Useless variable

متغیر تهی‌پذیر Nullable variable

مسیر همیلتونی Hamiltonian path

مسئله تناظر پست Post correspondence problem (PCP)

مسئله توقف Halting problem

معکوس Reverse

ناشمارا Uncountable

نوار Tape

وارسی‌کننده Verifier

هد Head

هم-تورینگ-تشخیص‌پذیر Co-Turing-recognizable

تمام-NP NP-complete